

# Inżynieria Systemów Programowalnych

## Część I

dr inż. Miron Kłosowski  
EA 309

[miron.klosowski@pg.edu.pl](mailto:miron.klosowski@pg.edu.pl)

[www.ue.eti.pg.gda.pl/isp](http://www.ue.eti.pg.gda.pl/isp)

# Plan wykładu

- Język VHDL w syntezie układów cyfrowych.
- Budowa logiki programowalnej.
- Układy programowalne SPLD i CPLD.
- Układy programowalne FPGA.
- Synteza bloków funkcjonalnych FPGA.
- Konfiguracja układów FPGA.

# Język VHDL w syntezie układów cyfrowych

# Geneza języka VHDL

- Very High Speed Integrated Circuit Hardware Description Language.
- Symulacja układów cyfrowych  $< 1\mu\text{m}$  (projekt Departamentu Obrony USA).
- Rok 1987 – zatwierdzony standard IEEE1076.
- Rok 1993 – zatwierdzony standard IEEE1164 (stosowany obecnie).

# Zastosowania języka VHDL

- Opis i dokumentacja elektroniczna układów cyfrowych VLSI.
- Integracja różnych metod opisu układów cyfrowych.
- Symulacja funkcjonalna układów cyfrowych VLSI.
- Synteza układów cyfrowych VLSI (ASIC i FPGA).
- Weryfikacja i symulacja czasowa układów cyfrowych VLSI.

# Metody opisu układów cyfrowych

- Schemat ideowy.
- Schemat blokowy.
- Diagram czasowy.
- Graf maszyny stanów.
- Tabela prawdy.
- Równania logiczne.

# Poziomy abstrakcji projektu

| Poziom<br>złożoności | Reprezentacja<br>behawioralna:               | Reprezentacja<br>strukturalna:                       |
|----------------------|----------------------------------------------|------------------------------------------------------|
| System               | Opis w języku<br>naturalnym                  | Jednostka centralna,<br>dysk, antena, ...            |
| Układ<br>scalony     | Zestaw algorytmów<br>(Behavioral Level)      | Mikroprocesor, pamięć,<br>układ I/O, ...             |
| Rejestry             | Przepływ danych<br>(Register Transfer Level) | Rejestry, ALU, ROM,<br>MUX, licznik, ...             |
| Bramki               | Równania logiczne<br>(Data Flow)             | Funktory logiczne: and,<br>or, xor, ... (Gate Level) |
| Układ<br>elektr.     | Równania różniczkowe<br>(Transistor Level)   | Tranzystory, rezystory,<br>kondensatory, ...         |
| Layout,<br>krzem     | Fizyka półprzewodników                       | Geometria struktur, ...                              |

# Definicja interfejsu jednostki projektowej (1)

```
entity multiplexer is
port (
    signal s : in std_logic;
    signal x0,x1 : in std_logic_vector(7 downto 0);
    signal y : out std_logic_vector(7 downto 0)
);
end entity multiplexer;
```

Słowo **signal** jest opcjonalne.

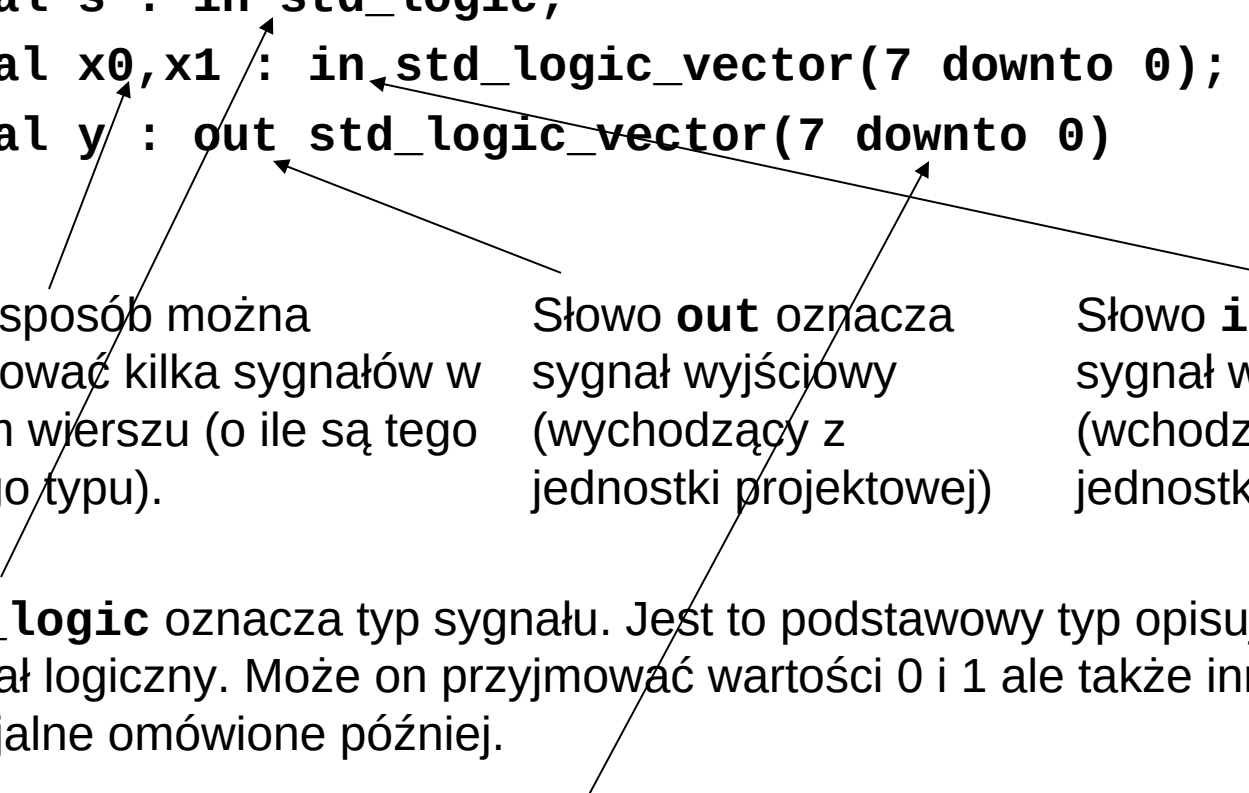
Uwaga na brak średnika!

Każdy projekt musi składać się z co najmniej jednej jednostki projektowej (entity). Jednostek tych może być więcej – zawsze jednak jest tylko jedna jednostka nadrzędna (top-level), znajdująca się na szczycie hierarchii. Definicja port (...) takiej jednostki odnosi się do końcówek I/O układu programowalnego.



# Definicja interfejsu jednostki projektowej (2)

```
signal s : in std_logic;  
signal x0,x1 : in std_logic_vector(7 downto 0);  
signal y : out std_logic_vector(7 downto 0)
```



W ten sposób można zdefiniować kilka sygnałów w jednym wierszu (o ile są tego samego typu).

Słowo **out** oznacza sygnał wyjściowy (wychodzący z jednostki projektowej)

Słowo **in** oznacza sygnał wejściowy (wchodzący do jednostki projektowej)

**std\_logic** oznacza typ sygnału. Jest to podstawowy typ opisujący sygnał logiczny. Może on przyjmować wartości 0 i 1 ale także inne specjalne omówione później.

**std\_logic\_vector(7 downto 0)** oznacza typ sygnału. Jest to wektorowa wersja sygnału **std\_logic** pozwalająca opisywać np. magistrale. Podany jest zakres indeksów wektora (jak widać magistrala ma 8 bitów).

# Definicja architektury jednostki projektowej

```
architecture data_flow of multiplexer is  
begin
```

```
    y <= x1 when ( s = '1' ) else x0;  
end architecture data_flow;
```

Ciało architektury.

Nazwa architektury – często wiele mówi o poziomie abstrakcji realizacji danej architektury.

Każda jednostka projektowa musi zawierać opis sprzętu – zapisuje się go w jednostce **architecture**. Nadawanie osobnych nazw architekturom jest uzasadnione – ponieważ możliwe jest zdefiniowanie kilku architektur dla każdej jednostki projektowej. O tym która z tych architektur jest następnie podana do syntezy czy symulacji decydują tzw. konfiguracje.

Ciało architektury zawiera opis sprzętu który ma sens instrukcji realizowanych współbieżnie ! Kolejność umieszczenia tych instrukcji w ciele nie ma znaczenia !

# Deklaracje użycia bibliotek

```
library IEEE;  
use IEEE.std_logic_1164.all;
```

Deklaracja użycia  
biblioteki IEEE.

Użycie pakietu `std_logic_1164`.

Słowo **all** oznacza użycie  
wszystkich składników pakietu.

Inne często używane biblioteki:

```
IEEE.std_logic_signed.all  
IEEE.std_logic_unsigned.all  
IEEE.std_logic_arith.all  
std.text_io.all  
IEEE.numeric_std.all;
```

} Umożliwiają np. dodawanie `std_logic_vector`

Np: konwersje między typami: `integer` a `std_logic_vector`

Np: obsługa plików tekstowych (do symulacji).

Np: funkcja `std_match` do porównywania wektorów.

Biblioteka zawsze podłączana domyślnie **`std.standard.all`**  
zawiera definicje podstawowych typów takich jak: **`boolean`**, **`bit`**,  
**`character`**, **`string`**, **`integer`**, **`real`**, **`natural`**,  
**`positive`**, **`bit_vector`** i innych.

# Przykładowy projekt (całość)

```
library IEEE;
use IEEE.std_logic_1164.all;

entity multiplexer is
port (
    signal s : in std_logic;
    signal x0,x1 : in std_logic_vector(7 downto 0);
    signal y : out std_logic_vector(7 downto 0)
);
end entity multiplexer;

architecture data_flow of multiplexer is
begin
    y <= x1 when ( s = '1' ) else x0;
end architecture data_flow;
```

# Przypisanie warunkowe

```
y <= x1;
```

zwykle przypisanie współbieżne sygnału

```
y <= x  when s = '1' else  
    '1'  when p = "00" else  
    '0'  when p = "11" else  
    x and z;
```

przypisanie warunkowe (przykład)

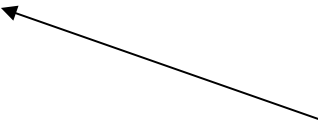
Przypisanie warunkowe pozwala na implementację dowolnych multiplekserów z priorytetowym sprawdzaniem warunków. Warunki są sprawdzane od pierwszego (w przykładzie:  $s = '1'$ ). Pierwszy napotkany **warunek** który jest spełniony (TRUE) powoduje realizację przypisania skojarzonej z nim **wartości**. Istotną cechą tego przypisania jest to że warunki mogą być dowolne (tak jak w przykładzie – dotyczą różnych sygnałów). Należy pamiętać że priorytetowe sprawdzanie warunków niejednokrotnie prowadzi do syntezy dużych struktur logicznych z długą ścieżką krytyczną (a więc opóźnieniem). Kiedy sprawdzanie priorytetowe może być zastąpione „równoczesnym” należy rozważyć stosowanie struktury przypisania selektywnego. Bardzo ważne jest zastosowanie na końcu słowa **else** – dzięki temu unikniemy powstania zatrzasku (kiedy żaden warunek nie jest spełniony w ogóle nie wystąpi przypisanie i sygnał docelowy pozostanie niezmienny – tak działa zatrzask!) Przypisanie warunkowe może być również stosowane do realizacji funkcji logicznych wielu zmiennych.

# Przypisanie selektywne

**with sel select**

```
y <= a when "000" | "001",  
    b when "101",  
    c when "010",  
    d when others;
```

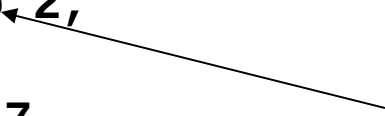
Uwaga na  
przecinki



**with number select**

```
y <= x1 and x2 when 0 to 2,  
    x1 or x2  when 3,  
    x1 nor x2 when 5 | 7,  
    x1 xor x2 when others;
```

Możliwe tylko dla typów  
integer i wyliczeniowych



Przypisanie selektywne funkcjonuje dla pojedynczej zmiennej selekcji która może przyjmować różne wartości. Od tego którą wartość przyjmie zależy realizacja skojarzonego z nią przypisania. Wartości selektorów wymienione po słowie when nie mogą się powtarzać i muszą obejmować wszystkie możliwe kombinacje wartości dla danego typu. Zazwyczaj konieczne jest stosowanie słowa others na końcu ponieważ nawet uwzględnienie wszystkich możliwych binarnych kombinacji selektora nie wyczerpuje wszystkich możliwości (selektor jest zazwyczaj typem `std_logic_vector` – a typ `std_logic` jest typem metalogicznym i może przyjmować aż 9 różnych wartości).

# Symulacja projektu – testbench (1)

Przykładowy TESTBENCH (wzorzec jest zazwyczaj generowany automatycznie):

```
LIBRARY ieee;  
USE ieee.std_logic_1164.ALL;  
USE ieee.std_logic_unsigned.all;  
USE ieee.numeric_std.ALL;
```

```
ENTITY fpgalab1_tb_vhd IS  
END fpgalab1_tb_vhd;
```

```
ARCHITECTURE behavior OF fpgalab1_tb_vhd IS
```

```
-- Component Declaration for the Unit Under Test (UUT)
```

```
COMPONENT projekt1
```

```
PORT(
```

```
    clk : IN std_logic;
```

```
    reset : IN std_logic;
```

```
    sw0 : IN std_logic;
```

```
    sw1 : IN std_logic;
```

```
    sw2 : IN std_logic;
```

```
    sw3 : IN std_logic;
```

```
    an : OUT std_logic_vector(3 downto 0);
```

```
    seg : OUT std_logic_vector(7 downto 0);
```

```
    ld0 : OUT std_logic;
```

```
    ld1 : OUT std_logic
```

```
);
```

```
END COMPONENT;
```

Testbench sam generuje i testuje wszystkie sygnały więc końcówki nie występują.

Symulowany układ (UUT) jest podłączony poprzez mechanizm komponentów (omówiony szczegółowo później). W tym miejscu znajduje się deklaracja nazw, typów i kierunków sygnałów testowanego układu (tych, które po syntezie znajdą się na końcówkach układu scalonego FPGA).

# Symulacja projektu – testbench (2)

```
SIGNAL sw0 : std_logic := '0';  
SIGNAL sw1 : std_logic := '0';  
SIGNAL tst : std_logic := '1';  
SIGNAL trx : std_logic := '1';  
SIGNAL an  : std_logic_vector(3 downto 0);  
SIGNAL seg : std_logic_vector(7 downto 0);  
SIGNAL ld0 : std_logic;  
SIGNAL ld1 : std_logic;
```

Definicje sygnałów używanych do testowania komponentu – podawanych na jego końcówki (sygnały podawane na wejścia UUT powinno się zainicjować wartością początkową).

```
signal clk : std_logic := '0';  
signal clk_p : std_logic := '0';  
signal clk_n : std_logic := '1';  
constant PERIOD : time := 10 ns;  
constant DUTY_CYCLE : real := 0.25;  
signal reset : std_logic := '1';
```

Definicje dodatkowych sygnałów i stałych potrzebnych do testowania. Inicjalizacja tych sygnałów jest zazwyczaj konieczna do prawidłowego działania symulacji.

**BEGIN**

```
-- Instantiate the Unit Under Test (UUT)  
 uut: projekt1 PORT MAP(  
     clk => clk,  
     reset => reset,  
     sw0 => sw0,  
     sw1 => sw1,  
     sw2 => tst,  
     sw3 => trx,  
     an => an,  
     seg => seg,  
     ld0 => ld0,  
     ld1 => ld1
```

Realizacja podłączenia komponentu (UUT - czyli testowanego modułu) wraz z opisem połączeń pomiędzy sygnałami komponentu i sygnałami testbencha.

```
);
```



# Symulacja projektu – testbench (3)

```
-- clk_simple <= not clk_simple after PERIOD/2;
```

Generacja prostego zegara.

```
clk_p <= not clk_p after PERIOD/2;  
clk_n <= not clk_n after PERIOD/2;
```

Generacja zegara różnicowego.

```
clk <= '1' after (PERIOD - (PERIOD * DUTY_CYCLE)) when clk = '0'  
      else '0' after (PERIOD * DUTY_CYCLE);
```

Generacja zegara o dowolnym współczynniku wypełnienia.

```
reset <= '0' after 5 ns;
```

Wyłączenie sygnału reset.

```
tb : PROCESS  
BEGIN
```

Proces – wewnątrz procesu instrukcje wykonywane są sekwencyjnie.

```
-- Wait 10 ns for global reset to finish
```

```
wait for 10 ns;
```

```
-- Place stimulus here
```

```
sw0 <= '1';
```

```
sw1 <= '0';
```

```
wait for 20 ns;
```

```
sw0 <= '0';
```

```
wait for 10 ns;
```

```
sw1 <= '1';
```

```
wait; -- will wait forever
```

```
END PROCESS;
```

Instrukcja opóźnienia.

Instrukcje wprowadzają nowe wartości do sygnałów w sposób sekwencyjny.

Instrukcja wait bezargumentowa – spowoduje zatrzymanie procesu (jej brak spowodowałby ponowne uruchomienie procesu od początku).

# Symulacja projektu – testbench (4)

```
tb1 : PROCESS
variable x : integer;
BEGIN
```

```
  wait for 10 ns;
  loop
```

```
    x := 1;
```

```
    while x < 11 loop
```

```
      tst <= not tst;
```

```
      wait for x*2 ns;
```

```
      x := x + 1;
```

```
      assert x /= 11
```

```
      report "Ostatnia iteracja zakończona"
      severity NOTE;
```

```
    end loop;
```

```
    wait until rising_edge(clk);
```

```
  end loop;
```

```
END PROCESS;
```

```
tb2 : PROCESS
```

```
BEGIN
```

```
  wait on clk_p;
```

```
  trx <= transport clk_p after (PERIOD/2)+3 ns;
```

```
END PROCESS;
```

```
END;
```

Instrukcja pętli nieskończonej (gdyby jej nie było – proces i tak uruchamiałby się ponownie – ale za każdym razem wykonywałoby się 10 ns czekanie).

Instrukcja pętli warunkowej.

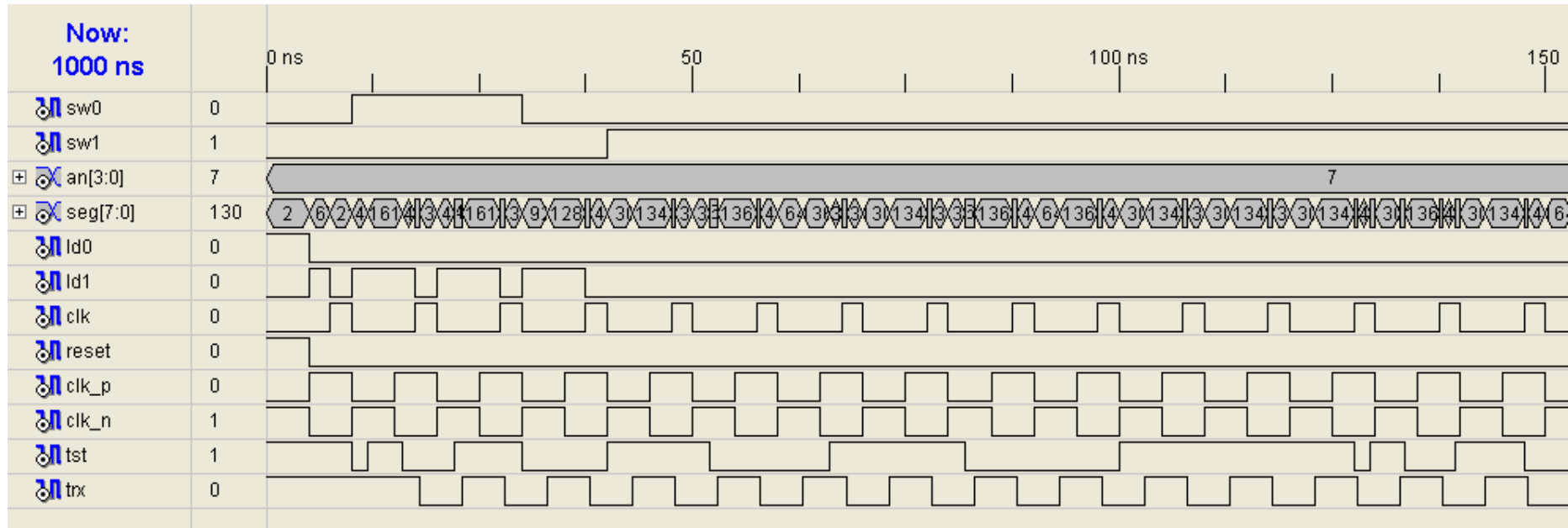
Instrukcja assert – pozwala na generowanie komunikatów i warunkowe przerwanie symulacji (występuje razem z report i severity).

Instrukcja wait until oczekuje na spełnienie warunku logicznego (w tym przypadku czeka na zbocze narastające sygnału clk).

Instrukcja wait on oczekuje na zmianę sygnału.

Instrukcja dokonuje rejestracji przyszłej zmiany sygnału trx. Jest to przykład opóźnienia transportowego.

# Symulacja projektu – testbench (5)



# Literały

-- Literały dziesiętne:

14

7755

156E7

188.993

88\_670\_551.453\_909

44.99E-22

-- Literały numeryczne:

16#FE#                   -- 254

2#1111\_1110#           -- 254

8#376#                   -- 254

16#D#E1                  -- 208

16#F.01#E+2             -- 3841.00

2#10.1111\_0001#E8      -- 1506.00

null       - literał pusty

-- Literały łańcuchowe:

"ERROR"

"Both S and Q equal to 1"

"X"

"BB\$CC"

""

"Quotation: ""REPORT..."""

-- Reprezentacja wektorów:

B"1111\_1111"

B"11111111"

X"FF"

O"377"

"11111111"

-- Literały typów wyliczeniowych

State0

Idle

TEST

\test\   -- literał rozszerzony pozwala na

\out\     -- nazwy zastrzeżone, ale rozróżnia

\OUT\     -- małe i duże litery

# Uzupełnienie informacji podstawowych

## -- to jest komentarz do końca linii

- Komentarze mogą zaczynać się w dowolnym miejscu linii, zawsze jednak zajmują resztę wiersza. Nie ma znaku komentarza który obejmowałby wiele linii.
- VHDL nie rozróżnia małych i dużych liter (mogą być wyjątki – literał rozszerzony).
- Średnik kończy wyrażenie (linię kodu) – występują nieliczne wyjątki.
- Identyfikatory muszą zaczynać się od litery i zawierać na następnych pozycjach znaki alfanumeryczne lub podkreślenie.
- Język jest silnie zorientowany na typ, automatyczna konwersja należy do rzadkości, możliwe jest przeciążanie operatorów, funkcji i procedur.
- Syntezowalny jest jedynie podzbiór języka.
- Żaden z systemów syntezy nie gwarantuje prawidłowej syntezy modelowania algorytmicznego.
- Wielkość układu po syntezie nie ma nic wspólnego z objętością kodu!

# Operatory, ich priorytet oraz synteżowalność

- |    |                               |                                          |         |                                             |
|----|-------------------------------|------------------------------------------|---------|---------------------------------------------|
| 1. | **                            | potęgowanie                              | [2 arg] | [ niesynteżowalny ]                         |
|    | abs                           | wartość bezwzględna                      | [1 arg] | [ synteżowalny ]                            |
|    | not                           | negacja logiczna                         | [1 arg] | [ synteżowalny ]                            |
| 2. | *                             | mnożenie                                 | [2 arg] | [ synteżowalny ]                            |
|    | /                             | dzielenie                                | [2 arg] | [ niesynteżowalny ]                         |
|    | mod                           | dzielenie modulo dla wart. całkowitych   | [2 arg] | [ niesynteżowalny ]                         |
|    | rem                           | reszta z dzielenia dla wart. całkowitych | [2 arg] | [ niesynteżowalny ]                         |
| 3. | +                             | operator zachowania znaku                | [1 arg] | [ synteżowalny ]                            |
|    | -                             | operator zmiany znaku                    | [1 arg] | [ synteżowalny ]                            |
| 4. | +                             | dodawanie                                | [2 arg] | [ synteżowalny ]                            |
|    | -                             | odejmowanie                              | [2 arg] | [ synteżowalny ]                            |
|    | &                             | łączenie tablic (wektorów, łańcuchów)    | [2 arg] | [ synteżowalny ]                            |
| 5. | sll                           | przesunięcie logiczne w lewo             | [2 arg] | [ synteżowalny dla bit_vector ]             |
|    | srl                           | przesunięcie logiczne w prawo            | [2 arg] | [ synteżowalny dla bit_vector ]             |
|    | sll                           | przesunięcie arytmetyczne w lewo         | [2 arg] | [ synteżowalny dla bit_vector ]             |
|    | sra                           | przesunięcie arytmetyczne w prawo        | [2 arg] | [ synteżowalny dla bit_vector ]             |
|    | rol                           | rotacja w lewo                           | [2 arg] | [ synteżowalny dla bit_vector ]             |
|    | ror                           | rotacja w prawo                          | [2 arg] | [ synteżowalny dla bit_vector ]             |
| 6. | =                             | równe                                    | /=      | różne                                       |
|    | <                             | mnijjsze                                 | <=      | mnijjsze lub równe                          |
|    | >                             | większe                                  | >=      | większe lub równe                           |
|    | } [ 2 arg] [ synteżowalny ]   |                                          |         |                                             |
| 7. | and, or, nand, nor, xor, xnor |                                          |         | operatory logiczne [2 arg] [ synteżowalne ] |

# Typy danych (1)

- Typ **Bit** – rzadko stosowany typ wyliczeniowy – może przyjmować tylko 2 wartości '0' lub '1'. Niekiedy występuje konieczność jego użycia (niektóre operatory i funkcje biblioteczne mogą wymagać argumentów i/lub wyników w tym typie), np. operatory przesunięcia i rotacji: sll, srl, sla, sra, rol, ror wymagają typu pochodnego **Bit\_vector**
- Typ **Integer** – obejmuje całkowite wartości numeryczne z zakresu -2147483648 to +2147483647. Wskazane jest **wykorzystywanie podtypów** typu Integer powstałych przez ograniczenie zakresu możliwych wartości. Dzięki temu możliwa jest **kontrola błędów** oraz **zmniejszenie rozmiarów układu** po syntezie (bez ograniczeń typ Integer ma realizację fizyczną szerokości 32-bitów). Na sygnałach i zmiennych typu Integer można wykonywać operacje: abs, \*\*, +, -, \*, /, mod, rem oraz zmiany znaku. Dzielenie, mod oraz rem są syntezywalne tylko wtedy gdy dzielnik jest stały i jest potęgą liczby 2. Potęgowanie jest syntezywalne tylko gdy podstawa jest stała i jest potęgą liczby 2. Można również wykonywać wszystkie operacje relacji. Nie można wykonywać operacji logicznych (not, and, or, nand, nor, xor, xnor).

# Typy danych (2)

- Typ **Natural** – typ Integer okrojony do zakresu od 0 do +2147483647.
- Typ **Positive** – typ Integer okrojony do zakr. od 1 do +2147483647.

Przykład definiowania własnego typu okrojonego:

```
type Int_64K is range -65536 to 65535;  
type WORD is range 0 to 31;  
type MUX_ADDRESS is range (2**(N+1))- 1 downto 0;  
subtype podtyp_ograniczony1 is integer range 4 to 5;  
subtype podtyp_ograniczony2 is WORD range 1 to 30;
```

- Typ **Boolean** – typ wyliczeniowy mogący przyjmować 2 wartości: **false** i **true**. Jest on stosowany do reprezentacji rezultatów działania operatorów relacyjnych. Do działań na tym typie można stosować operatory: not, and, or, nand, nor, xor, xnor oraz operatory relacyjne.

```
bool1 <= true;  
bool2 <= y > 7;  
bool3 <= bool2 = bool1;  
bool4 <= not (bool1 and bool2) xor bool3;  
...  
if bool4 then ...  
...  
abc <= a and b when bool2 else a or b;
```



# Typy danych (3)

- Typ **wyliczeniowy**:

```
type type_name is (type_element, type_element, ...);
```

Przykłady:

```
type Boolean is (False, True);
```

```
type FSM_States is (Init, Read, Decode, Execute, Write);
```

- Typ **wyliczeniowy** jest syntezywalny, często używa się go do opisu zmiennych stanu.
- Typ **Character** – jest to typ opisujący znaki kodu ASCII (ISO 8859-1), jest rzadko stosowany. Przykład deklaracji sygnału tego typu:  
  

```
signal znak : Character := 'Q';
```
- Na typie **Character** działają tylko operatory relacyjne i & (sklejania).
- Typ **Real** – stosuje się do opisu liczb zmiennoprzecinkowych. Nie jest syntezywalny.

# Typy danych (4)

- Typ tablicowy:

**type type\_name is array (range) of element\_type**

**type type\_name is array (type range <>) of element\_type**

Pierwszy przykład to tzw. constrained array, zaś drugi to unconstrained array (podczas syntezy rozmiar tablicy musi być jednak już zawsze określony).

Unconstrained array przydaje się np. do budowania funkcji operujących na tablicach różnej długości. Pakiet STANDARD definiuje dwa typy tablicowe: String i Bit\_vector:

**type String is array (Positive range <>) of Character;**

**type Bit\_Vector is array (Natural range <>) of Bit;**

- Pakiet IEEE.std\_logic\_1164 dodatkowo definiuje podstawowy typ syntezy układów logicznych: std\_logic oraz jego wektorową wersję, tablicę:

**type Std\_logic\_vector is array (NATURAL range <>) of Std\_logic;**

- Możliwe jest także definiowanie tablic dwuwymiarowych, zazwyczaj tylko 2-wymiarowe tablice typu „vectors of vectors” są syntezywalne:

**type Std\_logic\_2D is array (NATURAL range <>, NATURAL range <>) of Std\_logic;**

# Przypisanie sygnałów (współbieżne)

```
signal_1 <= signal_2;    signal_2 <= signal_3;  
signal_2 <= signal_3;    signal_1 <= signal_2;
```

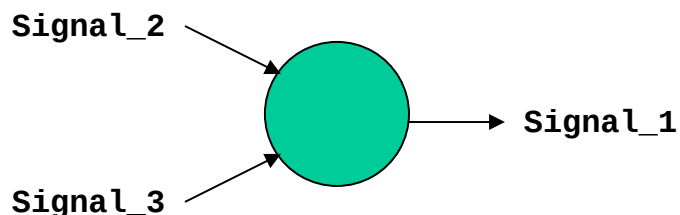
Opisy równoważne (w ciele architektury  
– jest to obszar współbieżny).

Czy wobec tego mamy problem ☹ ?

```
Signal_1 <= Signal_2;  
Signal_1 <= Signal_3;
```

**Uwaga !!!**

Można go rozwiązać definiując  
automatyczną funkcję rezolucji ☺ !



Funkcja rezolucji jest zdefiniowana  
tylko dla typu **std\_logic** oraz  
**std\_logic\_vector**. Pierwowzorem  
typu **std\_logic** bez zdefiniowanej  
funkcji rezolucji jest typ **std\_ulogic**.  
Zaleca się stosowanie typów  
**std\_logic** i **std\_logic\_vector**.

# Typy sygnałów – funkcja rezolucji

Funkcja rezolucji dla typu **std\_logic**:

| U | X | 0 | 1 | Z | W | L | H | - |   |
|---|---|---|---|---|---|---|---|---|---|
| U | U | U | U | U | U | U | U | U | U |
| U | X | X | X | X | X | X | X | X | X |
| U | X | 0 | X | 0 | 0 | 0 | 0 | X | 0 |
| U | X | X | 1 | 1 | 1 | 1 | 1 | X | 1 |
| U | X | 0 | 1 | Z | W | L | H | X | Z |
| U | X | 0 | 1 | W | W | W | W | X | W |
| U | X | 0 | 1 | L | W | L | W | X | L |
| U | X | 0 | 1 | H | W | W | H | X | H |
| U | X | X | X | X | X | X | X | X | - |

- **Std\_logic** oznacza typ sygnału. Jest to podstawowy typ opisujący sygnał logiczny. Może on przyjmować wartości **0** i **1** ale także:

**Z** stan wysokiej impedancji;

- don't care (wartość logiczna bez znaczenia, np. funkcja `std_match`);

Pozostałe wartości dotyczą tylko symulacji:

**U** niezainicjowany (domyślnie nadawany na początku symulacji);

**X** nieokreślony silny sygnał; **W** nieokreślony słaby sygnał;

**L, H** do opisu słabych sygnałów 0 i 1.

Funkcja rezolucji pozwala na syntezę logiki trójstanowej wewnątrz układu.

Syntezer nie zgodzi się na przypisanie do sygnału innej wartości niż 0, 1 lub Z.

Nie wszystkie układy programowalne posiadają wewnętrzne bramki

trójstanowe! Uwaga: sygnały ZUXLHW nie mogą być pisane małymi literami!

# Przypisanie sygnałów – funkcje

- Realizacja funkcji logicznych za pomocą instrukcji przypisania sygnałów: (uwaga na nawiasy – ze względu na jednakowy priorytet operatorów są konieczne!)

```
x <= (a and b) or c;           -- synteza funkcji logicznej
a <= a1 + a2 + a3 + a4;       -- synteza funkcji
                                -- arytmetycznej (sumatory)
b <= (b1 + b2) + (b3 + b4);    -- położenie nawiasów może
c <= ((c1 + c2) + c3) + c4;    -- mieć wpływ na optymalność
                                -- syntezy funkcji arytmetycznej

with sel select
  y <= "10" when "000" | "001", -- optymalizacja funkcji
      "11" when "101",         -- logicznej z wykorzystaniem
      "-1" when "010",         -- przypisania don't care
      "00" when others;
opt <= "000" when bool1 else "--1";
```

- Kolejność przypisań sygnałów we współbieżnej części kodu VHDL jest dowolna.
- Należy pamiętać o tym, że funkcje logiczne podlegają optymalizacji i nie muszą być syntezerowane dokładnie w taki sposób jak zostały zdefiniowane.
- Aby ułatwić optymalizację można stosować przypisanie wartości don't care zamiast konkretnej wartości 0 lub 1 – o ile oczywiście to możliwe.

# Operacje przypisań na wektorach (1)

- Wektory są jednowymiarowymi tablicami:

```
signal x: std_logic_vector(7 downto 0);
```

```
x <= "11001010";
```

Powyższa linia jest równoważna poniższym przypisaniom poszczególnych bitów:

```
x(7) <= '1'; x(6) <= '1'; x(5) <= '0'; x(4) <= '0';
```

```
x(3) <= '1'; x(2) <= '0'; x(1) <= '1'; x(0) <= '0';
```

- Można deklarować wektory z dowolnym zakresem indeksów rosnącym lub malejącym. Indeksy mogą mieć dowolne wartości z typu Natural (całkowite większe lub równe 0).

```
signal y: std_logic_vector(0 to 7);
```

```
y <= "11001010";
```

```
y <= x;
```

Powyższa linia jest równoważna poniższym przypisaniom:

```
y(0) <= x(7); y(1) <= x(6); y(2) <= x(5); y(3) <= x(4);
```

```
y(4) <= x(3); y(5) <= x(2); y(6) <= x(1); y(7) <= x(0);
```

- Przypisania są możliwe zawsze jeżeli zgadza się liczba i typ elementów wektorów po obu stronach przypisania, w trakcie przypisania nie zmienia się kolejność bitów w wektorach niezależnie od kierunku wzrostu indeksów.

# Operacje przypisań na wektorach (2)

- Należy zaznaczyć że wskazane jest używanie zmniejszającego indeksowania typu downto:

```
signal q: std_logic_vector(1 downto 0);
```

Jest to spowodowane tym, że większość funkcji bibliotecznych operujących na std\_logic\_vector zakłada taką właśnie kolejność bitów (MSB po lewej stronie).

- Możliwe jest przypisanie fragmentów magistrali (należy pamiętać tylko o tym, że liczba elementów subwektorów po obu stronach musi być taka sama):

```
x(7 downto 6) <= q;  
x(5 downto 3) <= y(2 to 4);
```

- Nie można zmieniać kierunku indeksowania właściwego dla danego wektora, zapis **x(5 downto 3) <= y(4 downto 2);** jest **błędny!!!**
- Tworzenie wektorów z poszczególnych elementów lub mniejszych wektorów za pomocą **operatora sklejania &**:

```
signal s: std_logic;
```

```
x <= "10" & q & '1' & q & '0';
```

```
vec_from_sig <= s & s & s & s & s & s & s & s;
```

# Operacje przypisań na wektorach (3)

- Tworzenie wektorów z poszczególnych elementów za pomocą konstrukcji tablicowej (agregatu):

```
x <= ( '1', '0', q(1), q(0), '1', q(1), q(0), others => '0' );  
x <= ( 6=>'0', 7=>'1', 4=>q(0), 5=>q(1), 0=>'0', 2=>q(1),  
1=>q(0), others => '1' );
```

```
zero_vector <= ( others => '0' );
```

```
vec_from_sig <= ( others => s );
```

```
vec_from_sig1 <= ( s1, s2, s3, s4, others => s );
```

```
vec_from_sig2 <= ( 0=>'1', 6 downto 2 => s2, 7|1 => s1 );
```

↑  
Słowo „others” może występować tylko na końcu agregatu!

## Operacje logiczne na wektorach:

```
signal temp: std_logic_vector(7 downto 0);
```

```
temp <= ( others => s );
```

```
y <= ( temp and x1 ) or ( not temp and x0 );
```

Powyższy przykład przedstawia implementację 2-wejściowego multipleksera 8-bitowego z wykorzystaniem równań logicznych.

- Na wektorach `std_logic_vector` i `bit_vector` można wykonywać wszystkie operacje logiczne: `not`, `and`, `or`, `nand`, `nor`, `xor`, `xnor`.
- Zawsze są one wykonywane na odpowiadających sobie pozycjami (a nie indeksami!) bitach poszczególnych wektorów.
- Wszystkie wektory biorące udział w operacji muszą mieć taką samą długość.



# Operacje relacji na wektorach (1)

```
signal x: std_logic_vector(7 downto 0);  
signal str: string(1 to 3);  
x <= "00100111";  
str <= "Bul";  
bool1 <= x > "00100010";    -- TRUE  
bool2 <= x <= "11001100";    -- TRUE  
bool3 <= str < "Bulba";      -- TRUE  
bool4 <= x = "100111";       -- FALSE!
```

- Na wszystkich wektorach (niezależnie od typu) można wykonać operacje relacyjne równości (równe i różne).
- Na wektorach jednowymiarowych (np: string, bit\_vector, std\_logic\_vector) możliwe są także operacje większości i mniejszości. Działają one według zasady „słownikowej”. Zasada ta pozwala na porównywanie wektorów o różnych długościach (tak jak w słowniku – krótsze słowa przed dłuższymi). Ma to sens w przypadku łańcuchów ale może być mylące dla wektorów binarnych.
- Jeżeli porównujemy dwa wektory binarne o jednakowych długościach – nie ma problemu – zasada słownikowa prowadzi do takiego samego rozwiązania co porównywanie słów binarnych bez znaku.
- **Jeżeli jednak przez pomyłkę użyjemy wektorów o różnych długościach – porównanie da niespodziewane wyniki.** Syntezer nie zgłosi błędu!

# Operacje relacji na wektorach (2)

- Z tego względu że wektory mogą reprezentować dane binarne w różny sposób trzeba zwrócić uwagę na tę interpretację podczas implementacji operacji relacyjnych. Np. inaczej operacja relacyjna większości będzie wykonywana dla wektorów reprezentujących dane binarne ze znakiem (najstarszy bit opisuje znak).
- Aby rozwiązać ten problem stworzono dwie biblioteki (zawsze używamy tylko jedną z nich):

```
use IEEE.std_logic_unsigned.all;
```

```
use IEEE.std_logic_signed.all;
```

- Biblioteki te przeciążają operatory relacji (i nie tylko) dla typu `std_logic_vector`, ustalając ich działanie według wybranej reprezentacji (liczby bez znaku – unsigned, liczby ze znakiem w kodzie U2 – signed). Przeciążone operatory pozwalają także na porównywanie wektorów z obiektami typu integer:

```
bool1 <= x > 56;
```

- Użycie tych bibliotek powoduje też normalizację działania operatorów relacji dla wektorów o różniących się długościami. W tym przypadku krótszy z wektorów jest po prostu rozszerzany tak aby nie zmienić wartości którą reprezentuje i następnie porównywany.

# Operacje relacji na wektorach (3)

- Do porównań wektorów można także wykorzystać funkcję

```
bool1 <= std_match("000-111-", x);
```

Funkcja ta jest częścią pakietu **IEEE.numeric\_std** i pozwala na porównywanie wektorów z wykorzystaniem znaków "-" oznaczających dowolną wartość (don't care).

- Jeżeli w jednej jednostce projektowej zachodzi konieczność użycia równocześnie wektorów ze znakiem i bez znaku można użyć typów zdefiniowanych w pakiecie **IEEE.numeric\_std**:

```
type SIGNED is array (NATURAL range <>) of STD_LOGIC;
```

```
type UNSIGNED is array (NATURAL range <>) of STD_LOGIC;
```

Typy te przeciążają operatory relacji i arytmetyczne. Dostępnych jest także wiele funkcji konwersji pomiędzy signed, unsigned, integer i std\_logic\_vector.

- Można także porównywać wektory bezpośrednio z agregatami (np. może to być wygodny sposób na porównywanie długich wektorów). Jeżeli agregat jest wykorzystywany przy porównywaniu (jako jeden z argumentów operatora relacji) kolejność indeksów agregatu jest ustalona domyślnie jako rosnąca od 0 w górę nawet, jeżeli w agregacie występują inne kierunki.

```
signal x8 : std_logic_vector(7 downto 0) := "11110000";
```

```
bool2 <= x8 = (7 downto 4 =>'0', 3 downto 0 =>'1'); -- TRUE!
```

# Operacje relacji na wektorach (4)

- Jeżeli chcemy użyć w agregacie słowa **others** musimy określić dokładny rozmiar wektora agregatu. Można to zrobić za pomocą operatora kontroli typu (type mark): '

```
subtype std_logic_8 is std_logic_vector(7 downto 0);  
bool2 <= x8 = std_logic_8'(7 downto 4 => '1', others => '0');  
-- TRUE!
```

Operator ten równocześnie ustala zakres i kolejność indeksów w agregacie na takie, jakie występują w narzuconym typie.

- Kolejność i zakres indeksów w agregacie używanym do przypisania. Jeżeli używamy agregatu w którym określamy indeksy poszczególnych bitów np.:  
**(7=>'1', 4 downto 2 => '1', others => '0')**

to o tym, jaka będzie kolejność tych indeksów (rosnąca czy malejąca) oraz ich zakres decyduje lewa strona przypisania. Np:

```
signal x: std_logic_vector(7 downto 0);  
signal y: std_logic_vector(0 to 7);  
x <= (7=>'1', 4 downto 2 => '1', others => '0');  
y <= (7=>'1', 4 downto 2 => '1', others => '0');  
bool1 <= x = "10011100";      -- TRUE  
bool2 <= y = "00111001";      -- TRUE
```

Zakres i kolejność indeksów można także określić operatorem kontroli typu.

# Operacje arytmetyczne na wektorach

- Charakterystycznym typem danych dla operacji arytmetycznych jest typ `integer`. Jednak również na typie `std_logic_vector` jest możliwe wykonywanie operacji arytmetycznych (np. sumatory, mnożniki). Do tego celu należy użyć jednej z bibliotek: **`std_logic_unsigned`** lub **`std_logic_signed`**. Zalecane jest używanie typów **`signed`** i **`unsigned`** zdefiniowanych w pakiecie **`IEEE.numeric_std`** zamiast typu **`std_logic_vector`**.
- Przy operacjach dodawania i odejmowania wektor wyniku ma długość równą najdłuższemu z wektorów wejściowych. W związku z tym, aby móc otrzymać wynik zawierający przeniesienie/pożyczkę należy zwiększyć rozmiar wektorów składowych (przynajmniej jednego). W przykładzie poniżej zastosowano rozszerzenie znakiem (najstarszym bitem) ponieważ wektor jest typu `signed`.

```
signal x, y : signed(7 downto 0);  
signal res : signed(8 downto 0);  
signal cin : std_logic;  
res <= (x(7) & x) + y + cin;
```

- Przy operacjach mnożenia wektor wyniku ma długość równą sumie długości wektorów wejściowych (czynniki). Rozszerzanie długości wektorów wejściowych nie jest więc w tym przypadku konieczne, bo wektor wyniku ma wystarczającą długość do przedstawienia wszystkich rezultatów.

# Konwersja typów (1)

Do konwersji `std_logic_vector`  $\leftrightarrow$  signed/unsigned wystarczy typecast:

**`std_logic_vector(ARG: UNSIGNED/SIGNED)`**

**`signed(ARG: STD_LOGIC_VECTOR)`**

**`unsigned(ARG: STD_LOGIC_VECTOR)`**

Poniżej wymieniono popularne funkcje konwersji typów:

- Biblioteka **`IEEE.numeric_std`** zawiera funkcje zamiany typu integer na unsigned/signed (należy określić rozmiar wektora w parametrze **`SIZE`**):

**`TO_UNSIGNED(ARG: INTEGER; SIZE: INTEGER)`**

**`TO_SIGNED(ARG: INTEGER; SIZE: INTEGER)`**

- zamiana typów unsigned/signed na typ integer:

**`TO_INTEGER(ARG: UNSIGNED)`**

**`TO_INTEGER(ARG: SIGNED)`**

- rozszerzenie wektorów unsigned/signed do długości **`size`**:

**`RESIZE(ARG: UNSIGNED/SIGNED; SIZE: INTEGER)`**

Dla signed jest to rozszerzenie najstarszym bitem (sign extend).

- Biblioteka **`IEEE.std_logic_1164`** zawiera funkcje konwersji typu bit\_vector:

**`TO_BITVECTOR(S: STD_LOGIC_VECTOR)`**

**`TO_STDLOGICVECTOR(B: BIT_VECTOR)`**

# Konwersja typów (2)

Poniżej wymieniono starsze funkcje konwersji typów, ich stosowanie jest obecnie niezalecane:

- Biblioteki **IEEE.std\_logic\_unsigned** oraz **IEEE.std\_logic\_signed** zawierają funkcję zamiany typu **std\_logic\_vector** na typ **integer**:

**CONV\_INTEGER(ARG: STD\_LOGIC\_VECTOR)**

Aby zamienić typ **std\_logic\_vector** na **integer** lepiej zastosować **typecast** na typ **unsigned/signed** i konwersję na **integer**:

```
outp <= to_integer(unsigned(inp));
```

```
outp <= to_integer(signed(inp));
```

- Biblioteka **IEEE.std\_logic\_arith** zawiera następujące funkcje konwersji:
  - zamiana typu **integer** na **std\_logic\_vector** (należy określić rozmiar wektora):

**CONV\_STD\_LOGIC\_VECTOR(ARG: INTEGER; SIZE: INTEGER)**

- rozszerzenie **std\_logic\_vector** zerami do długości **size**:

**EXT(ARG: STD\_LOGIC\_VECTOR; SIZE: INTEGER)**

- rozszerzenie **std\_logic\_vector** najstarszym bitem (sign extend) do dług. **SIZE**:

**SXT(ARG: STD\_LOGIC\_VECTOR; SIZE: INTEGER)**

Aby zamienić typ **integer** na **std\_logic\_vector** lepiej zastosować konwersję na typ **unsigned/signed** i **typecast** na **std\_logic\_vector**:

```
outp <= std_logic_vector(to_unsigned(inp, outp'length));
```

```
outp <= std_logic_vector(to_signed(inp, outp'length));
```

# Procesy

- Procesy są podstawowym budulcem złożonych opisów sprzętu ponieważ dysponują możliwością algorytmicznego (behawioralnego) jego opisu. Instrukcje umieszczone wewnątrz procesu są analizowane sekwencyjnie (a nie współbieżnie) aby określić jego reakcję na zmiany sygnałów wejściowych.

**architecture behavioral of multiplexer is**

**begin**

Opcjonalna  
etykieta →

**mux: process (x0, x1, s) is**

Lista czułości  
procesu

**begin**

**if s = '1' then**

**y <= x1;**

**else**

**y <= x0;**

**end if;**

**end process mux;**

Ciało procesu – instrukcje  
sekwencyjne!

**end architecture behavioral;**



# Procesy kombinacyjne (1)

- Ciało procesu rozpoczyna działanie tylko wtedy, gdy nastąpi zmiana któregośkolwiek z sygnałów znajdujących się na liście czułości procesu.
- Jeżeli proces ma realizować układ kombinacyjny (czyli bez przerzutników lub zatrząsków) **wszystkie sygnały wejściowe** takiej funkcji logicznej muszą znajdować się na liście czułości procesu. Np. multiplexer z przykładu musi mieć na liście czułości oba wejścia danych i wejście sterujące. Dodatkowo **wszystkie sygnały wyjściowe** muszą być **we wszystkich możliwych przypadkach** użyte przynajmniej raz w instrukcji przypisania.

- Przykład prawidłowego procesu kombinacyjnego:

```
mux: process (x0, x1, s) is
Begin
    y <= x0;
    if s = '1' then
        y <= x1;
    end if;
end process mux;
```

- Przykład nieprawidłowego procesu kombinacyjnego (brak x1 na liście czułości, brak przypisania do y gdy s='0'):

```
mux: process (s) is
begin
    if s = '1' then
        y <= x1;
    end if;
end process mux;
```

# Procesy kombinacyjne (2)

- Sytuacja w której dla pewnej kombinacji sygnałów wejściowych nie następuje przypisanie sygnału wyjściowego powoduje powstanie zatrasku – elementu pamięciowego – czyli proces przestaje opisywać układ kombinacyjny!
- Aby uniknąć tego problemu należy zadbać o to aby zawsze jakąś wartość do sygnału wyjściowego przypisać (np. należy dodać odpowiednie klauzule else do instrukcji if, itp.)
- Drugą metodą jest inicjacja wszystkich sygnałów wyjściowych wartością domyślną:

```
mux: process (x0, x1, s) is  
begin  
    y <= x0;  
    if s = '1' then  
        y <= x1;  
    end if;  
end process mux;
```

Przypisanie do sygnałów działa w procesach nieco inaczej niż poza procesami: jeżeli to tego samego sygnału jest w procesie więcej przypisań – wykonane zostanie ostatnie. Proces wykonuje po prostu swój kod sekwencyjnie i nowe przypisanie anuluje poprzednie. Faktyczna realizacja przypisania następuje dopiero po zakończeniu kodu procesu (jest to kod sekwencyjny)!

# Procesy kombinacyjne (3)

- Możliwa jest sytuacja taka, że sygnał wyjściowy procesu kombinacyjnego jest równocześnie jego sygnałem wejściowym. Należy wtedy pamiętać w wypadku tego sygnału o równoczesnym spełnieniu warunków dla wejść (obecność na liście czułości) oraz wyjść (obowiązkowe przypisanie do sygnału):

```
process(a, b, c, x, y) is  
begin  
    x <= a and b;  
    y <= x or c;  
    z <= y xor a;  
end process;
```

Poniżej przedstawiono działanie procesu:

|             |                                |
|-------------|--------------------------------|
| t=0 ns      | (a=0, b=1, c=0, x=0, y=0, z=0) |
| t=1 ns      | (a=1, b=1, c=0, x=0, y=0, z=0) |
| t=1 ns + 1d | (a=1, b=1, c=0, x=1, y=0, z=1) |
| t=1 ns + 2d | (a=1, b=1, c=0, x=1, y=1, z=1) |
| t=1 ns + 3d | (a=1, b=1, c=0, x=1, y=1, z=0) |

- Przypisanie do sygnału w obrębie procesu można wykonać wiele razy – ostatecznie wykonane zostanie ostatnie, chyba że kolejne cykle delta spowodują ponowne wykonanie procesu – wtedy ważne będzie ostatnie przypisanie wśród wszystkich cykli delta.
- Można zaobserwować ciekawą właściwość: kolejność przypisań do różnych sygnałów w obrębie procesu jest dowolna.
- Należy pamiętać, że przypisanie do tego samego sygnału w różnych procesach wymaga zastosowania funkcji rezolucji (tak jak dla przypisań współbieżnych) i jest za wyjątkiem np. trójstanowych magistral – zabronione.

# Procesy i zmienne (1)

- Wewnątrz procesów można stosować zmienne (variables). Są one widoczne tylko w obrębie procesu dla którego są zdefiniowane i mają dwie ciekawe właściwości:
  - **zachowują swoją wartość w czasie kiedy proces nie jest używany (kiedy sygnały z listy czułości nie zmieniają swojego stanu),**
  - **ich zmiana następuje natychmiast po napotkaniu przypisania zmiennej := w trakcie sekwencyjnego działania procesu.**

```
process(a, b, c) is
variable x,y:std_logic_vector(7 downto 0);
begin
    x := a and b;
    y := x or c;
    z <= y xor a;
end process;
```

W przypadku zmiennych kolejność przypisań ma oczywiście znaczenie. Zamiana pierwszej i drugiej linii da rezultat:  
(x=1, y=0, z=1)

Poniżej przedstawiono działanie procesu:

```
t=0 ns      (a=0, b=1, c=0, x=0, y=0, z=0)
t=1 ns      (a=1, b=1, c=0, x=0, y=0, z=0)
t=1 ns + 1d (a=1, b=1, c=0, x=1, y=1, z=0)
```

# Procesy i zmienne (2)

- Do zmiennej można przypisać wartość początkową:

```
variable x: std_logic_vector(7 downto 0) := "00100010";  
variable i: integer range 0 to 7 := 4;
```

- W zmiennych deklarowanych w procesach wartość początkowa jest przypisywana do zmiennej tylko na samym początku symulacji (syntezy), później wartość zmiennej jest zachowywana pomiędzy aktywacjami procesu.
- Zmienne można także deklarować w procedurach i funkcjach. W tym przypadku zmienne którym przypisano wartość początkową są inicjowane tą wartością przy każdym uruchamianiu funkcji lub procedury.
- Zmienne są zalecane dla symulacji ponieważ nie muszą zawierać informacji o przyszłości tak jak sygnały. Dzięki temu są znacznie mniejszym obciążeniem dla symulatora niż sygnały.
- Zmienne mają ograniczone zastosowanie dla syntezy – konstrukcje z nimi związane mogą nie zawsze być syntezywalne.
- Zakres dostępności zmiennej jest ograniczony do procesu, funkcji lub procedury w której została zdefiniowana.

# Wartości początkowe (1)

- Do sygnału również można przypisać wartość początkową:

```
signal x: std_logic_vector(7 downto 0) := "00100010";  
signal i: integer range 0 to 7 := 4;
```

- Dla sygnałów i zmiennych z wartościami początkowymi mogą wystąpić problemy z syntezą!
- Jedyna interpretacja wartości początkowej to początkowy stan rejestru po konfiguracji układu FPGA. Nie wszystkie układy FPGA mają możliwość ustawiania określonych stanów w rejestrach podczas konfiguracji. **Układy firmy Xilinx mają taką możliwość.**
- W przypadku gdy nie przypisano wartości początkowej nie oznacza że takowej nie będzie – jest nią domyślnie lewa granica zakresu zdefiniowanego dla typu. Jeżeli typ jest agregatem – to każdy element agregatu ma taką samą wartość (taką jak typ z którego jest złożony agregat).
- Jeżeli zachowujemy wartość zmiennej pomiędzy aktywacjami procesu – tworzymy rejestr – a więc proces sekwencyjny! **Aby utworzyć proces kombinacyjny musimy inicjować lub zmieniać wartości wszystkich zmiennych zanim będą użyte (najlepiej na początku procesu).**

# Wartości początkowe (2)

- Przykłady wartości początkowych domyślnych:

```
signal i: integer range 1 to 7;      -- wart. pocz.: 1
signal i: integer range 7 downto 0; -- wart. pocz.: 7
signal x: std_logic_vector(7 downto 0);
-- wart. pocz. dla symul: "UUUUUUUU" dla syntezy: "00000000"
signal x: bit_vector(7 downto 0); -- wart. pocz.: "00000000"
```

- Przykład wartości początkowej i domyślnej dla typu wyliczeniowego:

```
type state_type is (clear, initiate, run, stop);
signal state_1: state_type := run;  -- wart. pocz.: run
signal state_2: state_type;         -- wart. pocz.: clear
```

# Stałe i aliasy

- Wewnątrz jednostek projektowych (entity), architektur, procesów i pakietów można również definiować **stałe**:

```
constant x: std_logic_vector(7 downto 0) := "00100010";  
constant y: std_logic_vector(7 downto 0) := ( others => '1');  
constant i: integer := 4;  
type state_type is (clear, initiate, run, stop);  
constant state_stop: state_type := stop;
```

- Stałe są przydatne do parametryzowania projektów (jeżeli np. chcemy aby projekt był realizowalny dla różnej liczby bitów magistrali – liczba bitów może być zdefiniowana w stałej do której odwołujemy się kiedy ta informacja jest nam potrzebna).

- **Alias** służą do nadania nowej przyjaźniejszej nazwy innemu obiektowi lub kawałkowi obiektu (np. wycinkowi z wektora).

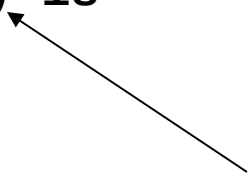
```
signal iw: std_logic_vector(28 downto 0);  
alias opcode: std_logic_vector(4 downto 0) is iw(28 downto 24);  
alias src: std_logic_vector(7 downto 0) is iw(23 downto 16);  
alias dst: std_logic_vector(7 downto 0) is iw(15 downto 8);  
alias arg: std_logic_vector(3 downto 0) is iw(7 downto 4);  
alias option: std_logic_vector(2 downto 0) is iw(3 downto 1);  
alias reserved: std_logic_vector(3 downto 0) is iw(3 downto 0);
```



# Instrukcja warunkowa (1)

- W procesach można używać konstrukcji warunkowej **if**, **then**, **elsif**, **else**, **end if**. Jeżeli warunek po **if** jest spełniony wykonywany jest kod po **then** w przeciwnym wypadku wykonywany jest kod po **else** lub sprawdzany następny warunek po **elsif**. Każdą konstrukcję **if** powinno kończyć słowo **end if**.

```
-- signal s:std_logic_vector(2 downto 0);
-- signal y:std_logic_vector(1 downto 0);
priority_encoder: process (s) is
begin
    if s(2) = '1' then
        y <= "11";
    elsif s(1) = '1' then
        y <= "10";
    elsif s(0) = '1' then
        y <= "01";
    else
        y <= "00";
    end if;
end process priority_encoder;
```



Pamiętajmy – wszystkie bity sygnału **s** są na liście czułości procesu, sygnał **y** jest tylko wyjściem więc nie musi być na tej liście.

## Instrukcja warunkowa (2)

- Dekoder priorytetowy możemy również wykonać korzystając z tego że ostatnie przypisanie do sygnału ma priorytet.

```
-- signal s:std_logic_vector(2 downto 0);
-- signal y:std_logic_vector(1 downto 0);
priority_encoder: process (s) is
begin
    y <= "00";           -- teraz najniższy priorytet
    if s(0) = '1' then   -- jest na początku!
        y <= "01";
    end if;
    if s(1) = '1' then
        y <= "10";
    end if;
    if s(2) = '1' then
        y <= "11";
    end if;
end process priority_encoder;
```

# Instrukcja warunkowa (3)

```
air_conditioning: process (temp) is
begin
  if temp > 22 then
    if temp > 26 then
      if temp > 28 then
        fan <= '1'; cool <= 2; heat <= 0; window <= '0';  -- temp > 28
      else
        fan <= '0'; cool <= 2; heat <= 0; window <= '0';  -- 26 < temp <= 28
      end if;
    else
      if (temp > 24) then
        fan <= '0'; cool <= 1; heat <= 0; window <= '0';  -- 24 < temp <= 26
      else
        fan <= '1'; cool <= 0; heat <= 0; window <= '1';  -- 22 < temp <= 24
      end if;
    end if;
  else
    if (temp > 18) then
      if (temp > 20) then
        fan <= '0'; cool <= 0; heat <= 0; window <= '1';  -- 20 < temp <= 22
      else
        fan <= '0'; cool <= 0; heat <= 1; window <= '0';  -- 18 < temp <= 20
      end if;
    else
      fan <= '0'; cool <= 0; heat <= 2; window <= '0';    -- temp <= 18
    end if;
  end if;
end process air_conditioning;
```

Jeżeli wszystkie warunki w złożonej instrukcji if są wzajemnie wykluczające się – stosowanie tej instrukcji może prowadzić do nieoptymalnej syntezy. Niepotrzebne staje się kolejne sprawdzanie warunków. W niektórych sytuacjach syntezer sam to wykryje i zoptymalizuje, jednak zaleca się stosowanie w takim przypadku o ile to możliwe instrukcji wyboru – case.

# Instrukcja wyboru (1)

- Najpierw obliczana jest wartość wyrażenia po słowie case, a następnie wykonywany jest kod przypisany do wyliczonej wartości. Zawsze musi być podana specyfikacja wszystkich możliwych wartości wyrażenia – jeżeli to jest niemożliwe lub trudne należy stosować słowo others (będzie konieczne np. dla typu std\_logic\_vector).

```
-- signal s:std_logic_vector(2 downto 0);
-- signal y:std_logic_vector(1 downto 0);
priority_encoder: process (s) is
begin
    case s is
        when "111" | "110" | "101" | "100" => y <= "11";
        when "011" | "010" => y <= "10";
        when "001" => y <= "01";
        when others => y <= "00";
    end case;
end process priority_encoder;
```

# Instrukcja wyboru (2)

- W instrukcji wyboru możemy (dla typów skalarnych i wyliczeniowych) stosować zakresy (np: downto ). Wewnątrz instrukcji wyboru możemy wykonywać dowolny kod składający się z wielu instrukcji, może też to być kod pusty. Po słowie when można stosować stałe lub wyrażenia ze stałymi.

```
-- signal s:std_logic_vector(2 downto 0);
-- signal y:std_logic_vector(1 downto 0);
-- constant jeden:integer := 1;
priority_encoder: process (s) is
begin
    y <= "10";
    case CONV_INTEGER(s) is
        when 7 downto 4 => y <= "11";
                        if s = 5 then
                            z <= 123;
                        end if;
        when 3 downto jeden + 1 => null;
        when jeden => y <= "01";
        when others => y <= "00";
    end case;
end process priority_encoder;
```

← Instrukcja pusta

# Instrukcja wyboru (3)

- Instrukcja wyboru najlepiej nadaje się w sytuacji gdy występuje konieczność wyboru z wielu warunków bez wyróżniania priorytetów.
- Instrukcja nadaje się też do maszyn stanu zbudowanych w oparciu o wyliczeniowe zmienne stanu (słowo `others` nie jest potrzebne w tym przypadku).

```
-- type state_type is (clear, initiate, run, stop);  
-- signal current_state, next_state: state_type;  
state_machine: process (current_state) is  
begin  
    case current_state is  
        when clear      => next_state <= initiate;  
        when initiate   => next_state <= run;  
        when run        => next_state <= stop;  
        when stop       => next_state <= clear;  
    end case;  
end process state_machine;
```

# Instrukcja pętli (1)

- Instrukcja pętli pozwala na łatwą syntezę powtarzalnych elementów systemu cyfrowego.
- **Synteżowalna pętla musi mieć ograniczoną liczbę iteracji !**
- Pętla typu while wykonuje się dopóki warunek po słowie while jest prawdą.
- Pętle while i for można przerwać całkowicie słowem exit.

```
priority_encoder: process (s) is
variable i:integer;
begin
    i := 2;
    y <= "00";
L1:   while i >= 0 loop
        if s(i) = '1' then
            y <= CONV_STD_LOGIC_VECTOR(i+1, 2);
            exit L1;
        end if;
        i := i - 1;
    end loop L1;
end process priority_encoder;
```

# Instrukcja pętli (2)

- Pętla for pozwala na domyślną deklarację zmiennej iteracyjnej (w przykładzie zmienna i). Zmiennej tej przypisywane są kolejne wartości z podanego zakresu.
- **Syntezywalna pętla musi mieć ograniczoną liczbę iteracji !**
- W określeniu zakresów iteracji pomocny może być atrybut 'range'.

```
bit_counter: process (s) is -- licznik jedynek w słowie s
variable num_bits:integer range 0 to s'length; -- atrybut
begin -- LENGTH
    num_bits := 0;
L1:    for i in s'range loop -- atrybut RANGE
        if s(i) = '1' then
            num_bits := num_bits + 1;
        end if;
    end loop L1;
    q <= num_bits;
end process bit_counter;
```



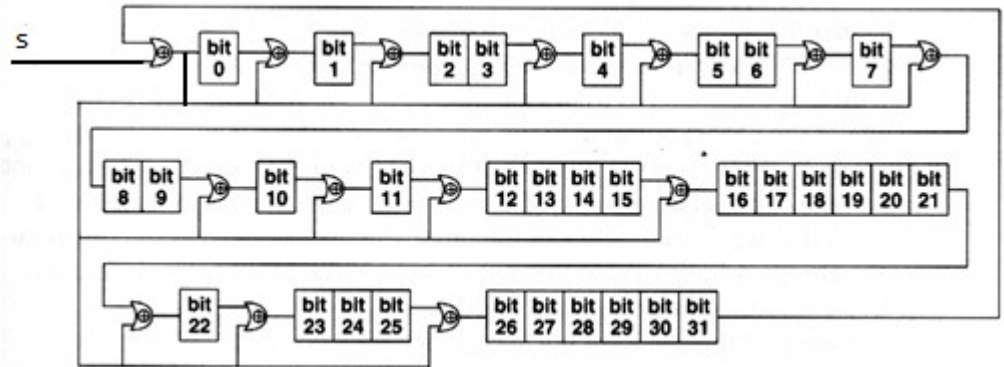
# Instrukcja pętli (3)

- Każdą iterację pętli while i for można także przerwać przechodząc do następnej iteracji – instrukcją next.
- **Syntezywalna pętla musi mieć ograniczoną liczbę iteracji !**

```
-- signal s:std_logic;
-- signal crc_in, crc_out:std_logic_vector(31 downto 0);
crc32_logic: process (s, crc_in) is
constant coef:std_logic_vector(31 downto 0) :=
"0000_0100_1100_0001_0001_1101_1011_0111";
variable tmp:std_logic;
begin
  for i in 0 to 31 loop
    if i = 0 then
      tmp := '0';
    else
      tmp := crc_in(i-1);
    end if;
    crc_out(i) <= tmp;
    if coef(i) = '0' then
      next;
    end if;
    crc_out(i) <= tmp xor crc_in(31) xor s;
  end loop;
end process crc32_logic;
```

$$c(x) = 1+x+x^2+x^4+x^5+x^7+x^8+x^{10}+x^{11}+x^{12}+x^{16}+x^{22}+x^{23}+x^{26}+x^{32}$$

-- crc\_in <= crc\_out when rising\_edge(clk);



# Instrukcja pętli (4)

- Pętle mogą być również wielokrotnie zagłębione – ale uwaga na synteżowalność!
- **Synteżowalna pętla musi mieć ograniczoną liczbę iteracji !**

```
-- type mem_table is array (0 to 7) of std_logic_vector(7 downto 0);
-- signal mem : mem_table;
mem_control: process (clk, rst) is
begin
    if rst = '1' then
L1:      for i in 0 to 7 loop                -- inicjacja pamięci
L2:      for j in 7 downto 0 loop          -- przesuwającymi się jedynekami
            if i = j then
                mem(i)(j) <= '1';
            else
                mem(i)(j) <= '0';
            end if;
        end loop L2;
    end loop L1;
    elsif rising_edge(clk) then
        ...
    end if;
end process mem_control;
```

# Atrybuty

- Atrybuty można przypisać do jednostek projektowych, architektur, typów i sygnałów.

**type cnt is integer range 0 to 127;**

**type state is (idle, start, stop, reset);**

**type word is array(15 downto 0) of std\_logic;**

- Typowe atrybuty dotyczące typów i sygnałów to: 'left 'right 'high 'low i 'length:

**cnt'left = 0; state'left = idle; word'left = 15;**

**cnt'right = 127; state'right = reset; word'right = 0;**

**cnt'high = 127; state'high = reset; word'high = 15;**

**cnt'low = 0; state'low = idle; word'low = 0;**

**cnt'length = 128; state'length = 4; word'length = 16;**

- Często także stosuje się atrybut 'range który określa zakres i kierunek zmian indeksów w wektorach: **word'range = 15 downto 0;**

- Innym często używanym atrybutem jest 'event który pozwala wykryć zmianę sygnału z którym jest skojarzony. Używamy go do realizacji procesów synchronicznych.

- Atrybut 'pos pozwala na określenie pozycji jaką konkretna wartość zajmuje na liście typu, np: **state'pos(start) = 1; character'pos('A') = 65;**

- Atrybut 'val pozwala na wykonanie operacji odwrotnej, np:

**state'val(2) = stop; character'val(66) = 'B';**

- Atrybuty 'pos i 'val pozwalają na łatwą konwersję pomiędzy znakiem a jego kodem ASCII i odwrotnie.

- W języku VHDL oraz rozszerzeniach syntezerów i symulatorów występuje więcej atrybutów, niektóre zostaną omówione w dalszej części wykładu.

# Implementacja buforów trójstanowych (1)

- Bufory trójstanowe powinny być implementowane jako układy kombinacyjne. Wykorzystujemy tu wartość metalogiczną 'Z'.

```
-- signal sig,o,ena:std_logic;  
process (sig,ena) is  
begin  
    if ena = '1' then  
        o <= sig;  
    else  
        o <= 'Z';  
    end if;  
end process;
```

- Można także prościej – z wykorzystaniem współbieżnego przypisania warunkowego.

```
-- signal sig,o:std_logic_vector(127 downto 0);  
-- signal ena: std_logic;  
o <= sig when ena = '1' else 'Z';
```

# Implementacja buforów trójstanowych (2)

- Bufory trójstanowe pozwalają na implementację magistral wewnątrz układów programowalnych. W niektórych układach programowalnych może także nie być wewnętrznych buforów trójstanowych – w takiej sytuacji syntezer zazwyczaj implementuje multipleksery (które mogą być bardzo duże).
- Należy pamiętać że syntezer ma ograniczone możliwości sprawdzania prawidłowości sterowania bufora sygnałem enable. Jeżeli stanie się tak, że kilka nadajników na wewnętrznej magistrali rozpocznie pracę równocześnie nastąpi zwarcie, przepływ dużego prądu i być może nawet uszkodzenie układu.
- Jeżeli sygnał zdefiniowany w sekcji port jednostki projektowej ma być trójstanowy i równocześnie pełnić funkcje wejścia i wyjścia (dwukierunkowy) należy zdefiniować ten sygnał jako **inout**.
- Bufory trójstanowe są przeważnie zawsze zaimplementowane w końcówkach I/O układów programowalnych. Umożliwia to implementację zewnętrznych magistral trójstanowych. W tym przypadku należy także wyznaczyć odpowiednie sygnały sterujące taką magistralą aby zabezpieczyć się przed równoczesnym włączeniem nadajników linii w więcej niż jednym układzie.

# Implementacja buforów trójstanowych (3)

- Przykład (bufor trójstanowy dwukierunkowy 8-bitowy):

```
library IEEE;
use IEEE.std_logic_1164.all;

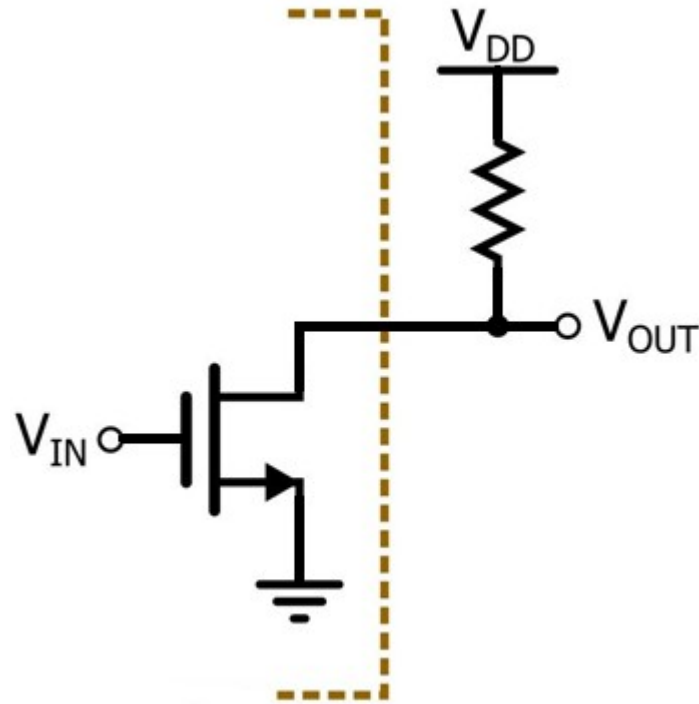
entity bidir_buffer8 is
port (
    ena : in std_logic;
    data : inout std_logic_vector(7 downto 0);
    in_data : out std_logic_vector(7 downto 0);
    out_data : in std_logic_vector(7 downto 0)
);
end entity bidir_buffer8;

architecture data_flow of bidir_buffer8 is
begin
    data <= out_data when ena = '1' else (others => 'Z');
    in_data <= data;
end architecture data_flow;
```

# Implementacja wyjścia typu otwarty dren

- Przykład:

```
-- signal sig,o:std_logic;  
process (sig) is  
begin  
    if sig = '0' then  
        o <= '0';  
    else  
        o <= 'Z';  
    end if;  
end process;
```



# Rodzaje sygnałów w deklaracji „entity”

- W deklaracji „entity” możemy wyróżnić trzy rodzaje sygnałów:
- **in** – sygnał wejściowy (podawany z zewnątrz, odczytywany w jednostce projektowej) – wewnątrz jednostki nie może występować po lewej stronie  $\leq$
- **out** - sygnał wyjściowy (generowany w jednostce projektowej, odczytywany na zewnątrz) – wewnątrz jednostki nie może występować po prawej stronie  $\leq$
- **inout** - sygnał dwukierunkowy - wewnątrz jednostki projektowej podłączony do dwukierunkowego bufora trójstanowego
- **buffer** - sygnał wyjściowy z możliwością odczytu wewnątrz jednostki projektowej, z różnych względów zaleca się jednak korzystać z sygnału out i dodać sygnał pośredniczący:

```
-- bez sygnału pośredniczącego
-- (wersja niezalecana)
entity buff is
port (
    x : buffer std_logic;
    y : out std_logic;
    a : in std_logic
);
end entity buff;
architecture data_flow of buff is
Begin
    x <= not a;
    y <= x and a;
end architecture data_flow;
```

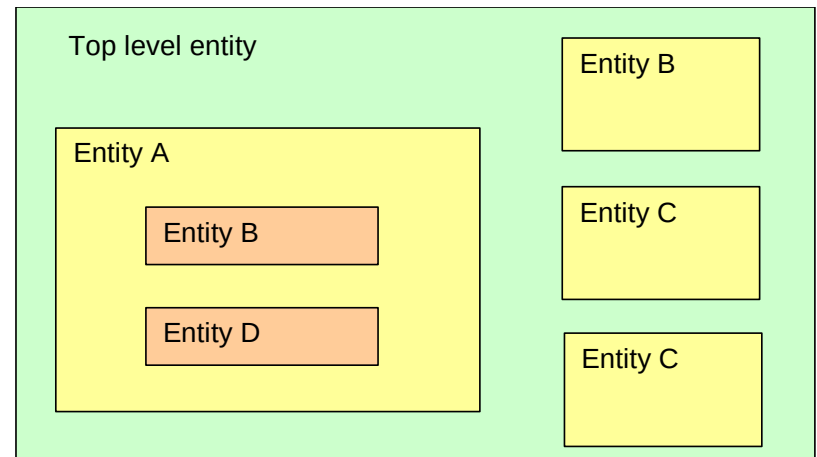
```
-- z sygnałem pośredniczącym
entity buff_ok is
port (
    x : out std_logic;
    y : out std_logic;
    a : in std_logic
);
end entity buff_ok;
architecture data_flow of buff_ok is
signal x_in : std_logic;
begin
    x_in <= not a;
    y <= x_in and a;
    x <= x_in;
end architecture data_flow;
```



# Hierarchia jednostek projektowych (1)

- Jednostki projektowe mogą być używane wielokrotnie wewnątrz tego samego projektu na różnych poziomach hierarchii.
- Można budować projekt w oparciu o strukturalną dekompozycję na mniejsze fragmenty które mogą być realizowane w oparciu o różne poziomy abstrakcji.
- Jednostki projektowe mogą być umieszczane w osobnych plikach, pakietach, bibliotekach i włączane do naszego projektu w miarę potrzeb.
- Interfejsem dla jednostki projektowej jest deklaracja „entity”.
- Deklaracją użycia jednostki projektowej jest słowo: component.
- Realizacją użycia wcześniej zadeklarowanego komponentu jest konstrukcja: „port map”.

```
component bufor port(  
    d, enable: in std_logic;  
    q: out std_logic);  
end component bufor;  
.....  
begin  
.....  
buf1: bufor port map(res,ena,outp(0));  
buf2: bufor port map(ack,ena,outp(1));
```



# Hierarchia jednostek projektowych (2)

- Przykład - sumator jednobitowy:

```
entity fulladder is port(  
    a,b,cin:  in std_logic;  
    sum,cout: out std_logic);  
end entity fulladder;  
architecture dataflow of fulladder is  
begin  
    sum <= a xor b xor cin;  
    cout <= (a and b) or (a and cin) or (b and cin);  
end architecture dataflow;
```

- Wykorzystanie sumatora do budowy sumatora wielobitowego:

```
entity adder_4 is  
port(a,b: in std_logic_vector(3 downto 0);  
    cin: in std_logic;  
    sum: out std_logic_vector(3 downto 0);  
    cout: out std_logic);  
end entity adder_4;  
architecture structural of adder_4 is  
component fulladder port(a,b,cin:in std_logic;  
    sum,cout:out std_logic);  
end component fulladder;  
signal c0,c1,c2: std_logic;  
begin  
fa0: fulladder port map(a(0),b(0),cin,sum(0),c0);  
fa1: fulladder port map(a(1),b(1),c0,sum(1),c1);  
fa2: fulladder port map(a(2),b(2),c1,sum(2),c2);  
fa3: fulladder port map(a(3),b(3),c2,sum(3),cout);  
end architecture structural;
```

# Hierarchia jednostek projektowych (3)

- Podłączanie sygnałów do końcówek komponentu można dokonywać w dowolnej kolejności, ale tylko z przywołaniem nazwy portu.
- Słowo open oznacza sygnał komponentu który nie jest używany – czyli pozostaje nie podłączony. Jeżeli taki sygnał jest wejściem komponentu – musi mieć w sposób jawny określoną wartość początkową w deklaracji komponentu!):  
`fa2: fulladder port map(a(2), b(2), c1, sum(2), c2);`  
`fa2: fulladder port map(b=>b(2), a=>a(2), sum=>sum(2), cout=>c2, cin=>c1);`  
`fa2: fulladder port map(a(2), b(2), c1, sum(2), open);`  
`fa2: fulladder port map(b=>b(2), a=>a(2), sum=>sum(2), cout=>open, cin=>c1);`
- W opisie układu przywołujemy wcześniej zdefiniowane jednostki projektowe, dołączając sygnały do odpowiednich portów tych jednostek (tworzenie powiązań).
- Typy i zakresy sygnałów aktualnych i lokalnych (w przywoływanych jednostkach) muszą być zgodne (w przypadku wektorów musi zgadzać się długość natomiast kierunek zmian indeksów oraz ich wartości graniczne mogą być dowolne – przypisanie odbywa się według położenia: „lewy do lewego, prawy do prawego”):

```
entity borg is port(a: in std_logic_vector(3 to 7));  
end entity borg;
```

```
-----  
component borg  
    port(a:in std_logic_vector(10 downto 6));  
end component borg;
```

# Hierarchia jednostek projektowych (4)

- W przypadku, gdy w komponencie użyto wektora „unconstrained” – można do niego „podłączyć” wektor tego samego typu o dowolnej długości. Kod opisujący taką jednostkę projektową musi być odpowiedni – tzn. musi działać prawidłowo niezależnie od długości wektora i zakresu indeksów, często przydatny jest do tego atrybut ‘range.

```
entity borg is port(a: in std_logic_vector);  
end entity borg;
```

-----

```
component borg  
  port(a:in std_logic_vector(567 downto 123));  
end component borg;
```

- Przy tworzeniu powiązań między portami obowiązuje zgodność kierunków przesyłania sygnałów (in, out, inout, buffer). Kierunki zadeklarowane w deklaracji komponentu dotyczą jego „punktu widzenia”. Czyli do sygnałów które są dla komponentu „in” podłączamy wyjścia a do sygnałów które są dla komponentu „out” podłączamy wejścia.
- W jednostkach projektowych można także tworzyć parametry ogólne (generic). Parametry te w odróżnieniu od stałych mogą się zmieniać. O ich wartości decydujemy w momencie użycia jednostki projektowej jako komponentu na wyższym poziomie hierarchii.

# Parametryzacja jednostek projektowych

```
entity adder_n is
  generic (size : integer := 4);
  port (a,b      :in std_logic_vector(size-1 downto 0);
        sum      :out std_logic_vector(size-1 downto 0);
        cout     :out std_logic);
end adder_n;
```

Wartość domyślna parametru (opcjonalnie).

Na parametry możemy powoływać się już przy deklaracji sygnałów interfejsu.

- Domyślną wartość parametru można zmienić w deklaracji komponentu (pominięcie parametru oznacza wybór wartości domyślnej z jednostki projektowej):

```
component adder_n is
  generic (size : integer := 8);
  port (a,b      :in std_logic_vector(size-1 downto 0);
        sum      :out std_logic_vector(size-1 downto 0);
        cout     :out std_logic);
end adder_n;
```

- lub dopiero w momencie użycia komponentu, ale tylko jeżeli w deklaracji wystąpił wpis „generic” (obowiązkowo jeżeli ten wpis nie podał wartości domyślnej):

```
fa2: adder_n generic map(size=>12)
      port map(a,b,sum,carry);
```

```
fa3: adder_n generic map(14)
      port map(sum=>s, a=>d1, b=>d2, carry=>p);
```

# Instrukcja generate (1)

- Podłączanie komponentów odbywa się w części współbieżnej opisu architektury. Aby zrealizować automatyczne podłączanie wielu komponentów (i nie tylko) należy użyć instrukcji „generate”. Jest to współbieżny odpowiednik pętli „for” znanej z procesów.

```
entity fulladder_n is
  generic (size : integer := 4);
  port(a,b : in std_logic_vector(size-1 downto 0);
       sum:out std_logic_vector(size-1 downto 0);
       cin:in std_logic;
       cout:out std_logic);
end entity fulladder_n;

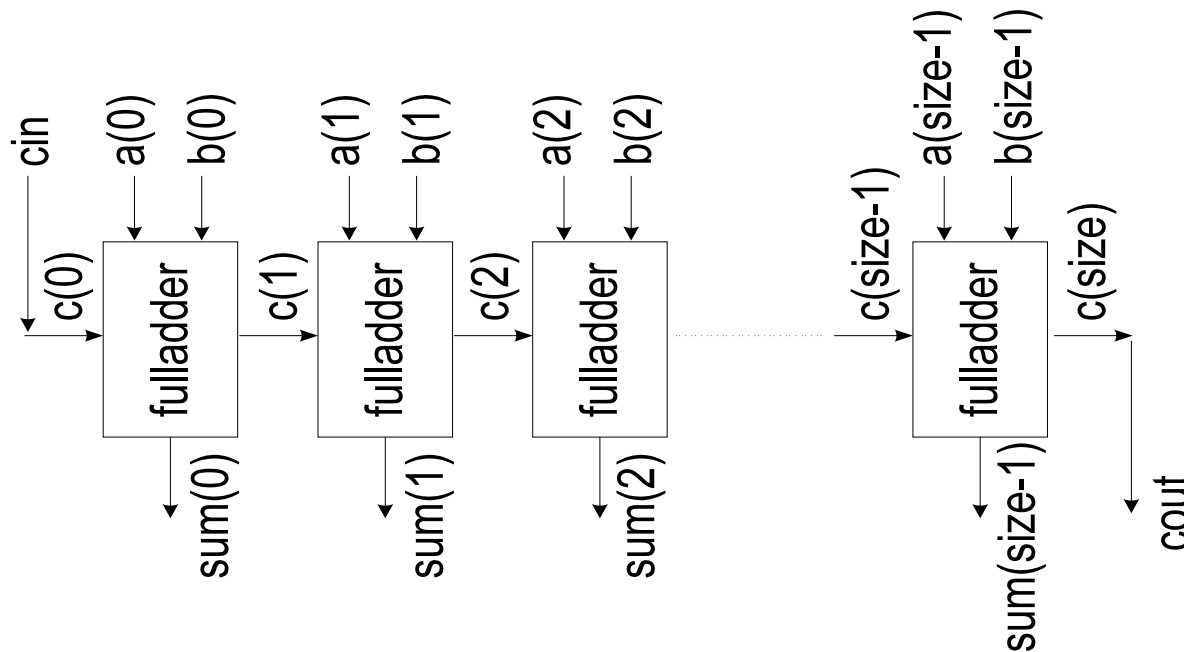
architecture dataflow of fulladder_n is
  signal c:std_logic_vector(size downto 0);
  begin
    c(0) <= cin;
    G: for i in 0 to size-1 generate
      sum(i) <= a(i) xor b(i) xor c(i);
      c(i+1) <= (a(i) and b(i)) or (a(i) and c(i)) or (b(i) and c(i));
    end generate;
    cout <= c(size);
  end architecture dataflow;
```

Etykieta jest wymagana!

Deklaracja zmiennej i jest domyślna

# Instrukcja generate (2)

- Instrukcja „generate” służy do automatycznej generacji struktur regularnych, tworzonych na bazie struktury wzorcowej (fizyczny efekt to powielenie podukładów wzorcowych).
- Wewnątrz struktury generate może być powielana dowolna instrukcja współbieżna łącznie z samą instrukcją generate.
- Istnieją dwa schematy generacji: generacja „for” dla układów w pełni regularnych oraz generacja „if” gdy istnieją nieregularności w układzie.



# Instrukcja generate (3)

- Przykład generacji struktur nieregularnych (półsumator dla najmłodszego bitu):

```
entity adder_n is
    generic (size : integer := 4 );
    port (a,b: in std_logic_vector(size-1 downto 0);
          sum: out std_logic_vector(size-1 downto 0);
          cout: out std_logic);
end adder_n;

architecture dataflow of adder_n is
    signal c:std_logic_vector(size downto 1);
begin
    G1: for i in 0 to size-1 generate
    G2:   if i=0 generate
            sum(i)<=a(i) xor b(i);
            c(i+1)<=a(i) and b(i);
        end generate;           --gałęzie else i elsif nie są dozwolone!!!
    G3:   if i>0 generate
            sum(i)<=a(i) xor b(i) xor c(i);
            c(i+1)<=(a(i) and b(i)) or (a(i) and c(i)) or (b(i) and c(i));
        end generate;
    end generate;
    cout <= c(size);
end architecture dataflow;
```



# Procesy sekwencyjne (1)

- Procesy mogą modelować układy sekwencyjne (przerzutnikowe) jeżeli wykorzystują pamięć. Mogą wykorzystywać pamięć sygnałów i zmiennych.
- Pamięć sygnałów - jeżeli wartość sygnału wyjściowego nie zależy stale od stanów sygnałów wejściowych (np. jeżeli zapomniano o przypisaniu wartości do sygnału w jakimś przebiegu procesu kombinacyjnego).
- Pamięć zmiennych – jeżeli nie inicjujemy zmiennej na początku kodu procesu – zostaje zachowana jej wartość z poprzedniego uruchomienia procesu i powstaje pamięć.
- Aby proces sekwencyjny był syntezywalny konieczne jest ściśle stosowanie się do reguł budowy procesów które poniżej zostaną przedstawione:

Przykład prawidłowego procesu sekwencyjnego:

```
latch: process (ena, d) is  
begin  
    if ena = '1' then  
        q <= d;  
    end if;  
end process latch;
```

Zatrząsk typu D (inaczej przerzutnik D wyzwalany poziomem). Dopóki sygnał ena jest aktywny dopóty wyjście q kopiuje stan wejścia d, kiedy ena staje się nieaktywny stan wyjścia q zostaje zamrożony.

# Procesy sekwencyjne (2)

- Zatrzaski są rzadko stosowane. Występują jednak w układach programowalnych, zazwyczaj są implementowane poprzez wyłączenie części przerzutnika czułego na zbocze.
- Przeważnie jednak zatrzaski są rezultatem błędu (np. braku przypisania wartości do sygnału). Niektóre syntezy informują o wykryciu zatrzasków w komunikatach typu: „Warning: latch inferred ... ”

Przykład prawidłowego procesu sekwencyjnego wrażliwego na zbocze sygnału zegarowego:

```
dff: process (clk) is  
begin  
    if clk = '1' and clk'event then  
        q <= d;  
    end if;  
end process dff;
```

↑  
'0' daje czułość na  
zbocze opadające

Przerzutnik typu D (wyzwalany zboczem narastającym). Jeżeli zmieni się stan sygnału clk wartość logiczna clk'event stanie się na chwilę prawdą. Jeżeli ta zmiana clk nastąpiła na „1” zostanie wykonane przepisanie sygnału wejściowego d na wyjście q i stan wyjścia q zostanie zamrożony.

# Procesy sekwencyjne (3)

- Atrybut 'event' ma jednak pewną wadę: wykrywa każdą zmianę sygnału, nie tylko z '0' na '1' i z '1' na '0' ale także np: z 'U' na '1' i '1' na 'U', etc.
- Zalecane jest stosowanie konstrukcji rising\_edge() lub falling\_edge() które są czułe wyłącznie na zmiany pomiędzy zerem a jedynką.

```
dff: process (clk) is
begin
    if rising_edge(clk) then
        q <= d;
    end if;
end process dff;
```

```
dff: process (clk) is
begin
    if falling_edge(clk) then
        q <= d;
    end if;
end process dff;
```

**Uniwersalny przepis na układ synchroniczny taktowany zboczem narastającym/opadającym:**

```
nazwa_układu: process (zegar) is
begin
    if rising/falling_edge(zegar) then
        ... -- Sekwencyjny opis
        ... -- układu synchronicznego,
        ... -- wszystkie przypisania do sygn.
        ... -- wykonywane są w takt zbocza
        ... -- sygnału zegarowego.
        ...
    end if;
end process nazwa_układu;
```

# Procesy sekwencyjne (4)

- W praktyce często stosuje się w przerzutnikach asynchroniczne wejścia zerujące lub ustawiające (reset, preset).

```
drff: process (clk, rst) is
begin
    if rst = '1' then
        q <= '0';
    elsif rising_edge(clk) then
        q <= d;
    end if;
end process drff;
```

```
dsff: process (clk, set) is
begin
    if set = '1' then
        q <= '1';
    if falling_edge(clk) then
        q <= d;
    end if;
end process dsff;
```

**Uniwersalny przepis na układ synchroniczny taktowany zboczem narastającym/opadającym z asynchronicznym wejściem reset:**

```
nazwa_układu: process (zegar, reset) is
begin
    if reset = '1' then    -- lub '0'
        sig <= "00110";
        sig <= const;
        ...
    elsif rising/falling_edge(zegar) then
        ... -- Sekwencyjny opis
        ... -- układu synchronicznego,
        ... -- wszystkie przypisania do sygn.
        ... -- wykonywane są w takt zbocza
        ... -- sygnału zegarowego.
    end if;
end process nazwa_układu;
```

# Procesy sekwencyjne (5)

- Niektóre układy programowalne posiadają przerzutniki wyposażone równocześnie w wejścia asynchroniczne kasujące i ustawiające.

```
drsff: process (clk, rst, set) is
begin
    if rst = '1' then
        q <= '0';
    elsif set = '1' then
        q <= '1';
    elsif rising_edge(clk) then
        q <= d;
    end if;
end process drsff;
```

```
drsff: process (clk, rst, set) is
begin
    if rst = '1' then
        q <= "001001";
    elsif set = '1' then
        q <= "100111";
    elsif rising_edge(clk) then
        q <= d;
    end if;
end process drsff;
```

# Procesy sekwencyjne (6)

- Niektóre układy programowalne posiadają przerzutniki wyposażone w możliwość asynchronicznego ładowania. Przykład – przerzutnik typu D z asynchronicznym resetem i ładowaniem:

```
drlff: process (clk, rst, load, ld) is  
begin  
    if rst = '1' then  
        q <= '0';  
    elsif load = '1' then  
        q <= ld;  
    elsif rising_edge(clk) then  
        q <= d;  
    end if;  
end process drlff;
```

# Procesy sekwencyjne (7)

## Ważne uwagi!

- Po warunku testującym zbocze zegara nie może wystąpić żadna gałąź else lub elsif !
- Sygnał zegarowy oraz sygnały z obsługą asynchroniczną muszą być umieszczone na liście czułości procesu !
- Część procesu opisująca właściwości asynchroniczne przerzutnika powinna odpowiadać możliwościom technicznym przerzutników w wykorzystywanym układzie FPGA.
- Proces może opisywać układ reagujący na zbocze tylko jednego sygnału (brak fizycznych przerzutników sterowanych zboczami wielu sygnałów)

```
bad_process: process (rst) is      -- brak zegara na liście czułości!
begin
    if rst = '1' then
        q <= ad;
    elsif rising_edge(clk) or falling_edge(clk) then    -- wykrywanie kilku
        q <= d;                                           -- zboczy!
    else
        q <= d2;                                           -- dodatkowa gałąź else!
    end if;
end process bad_process;
```

# Procesy sekwencyjne (8)

- Poniższy kod może, ale nie musi prowadzić to syntezy układu bramkującego zegar (clock gating). Clock gating jest nie zalecany ze względu na problemy wynikające z opóźnienia sygnału zegarowego (hold time violation), a także ze względu na możliwą obecność hazardów w układach logicznych, co może powodować powstawanie szpilek na wejściach bramkujących (i błędną pracę przerzutników).

```
dceff: process (clk, rst) is
begin
  if rst = '1' then
    q <= '0';
  elsif clk_ena = '1' then
    if rising_edge(clk) then
      q <= d;
    end if;
  end if;
end process dceff;
```

- Jeżeli sygnał zegarowy jawnie przepuścimy przez bramkę poza procesem (jak w poniższym przykładzie) to syntezer prawie zawsze wykona clock gating:

```
clk <= clk_in and clk_ena;
```



# Procesy sekwencyjne (9)

## **UWAGI DOTYCZĄCE RESETOWANIA PRZERZUTNIKÓW:**

**Najbezpieczniej jest stosować synchroniczne (czyli realizowane w takt zbocza zegara) wersje funkcji reset, preset, load, clock enable, itp. Dzięki temu unika się problemów związanych z hazardami i wyścigami.**

**Globalny reset całego systemu najlepiej wykonywać korzystając z dedykowanego wejścia do globalnego resetu (dodatkowa funkcja zapewniana przez producentów układów FPGA). Stosując zwykłe asynchroniczne lub nawet synchroniczne wejścia resetu przerzutników do celu resetu globalnego nie mamy pewności, że wszystkie przerzutniki zaczną pracę na tym samym zboczu zegara – czas propagacji sygnału resetu przez układ scalony może to uniemożliwić (ewentualnie można spróbować wyłączyć zegar podczas trwania globalnego resetu).**

**Można także zrezygnować z klasycznego globalnego resetu i wykorzystać wartości początkowe przerzutników po konfiguracji FPGA. U większości producentów FPGA wartość początkową przerzutnika w momencie uruchomienia systemu można ustalić w kodzie VHDL, za pomocą wartości początkowej sygnału lub zmiennej. Zamiast resetu globalnego można wtedy wykorzystać wejście twardego resetu układu FPGA (wejście PROG w układach Xilinx), które wymusza ponowną konfigurację (zazwyczaj z pamięci FLASH).**

# Procesy sekwencyjne (10)

- Przykład procesu z synchronicznym clock enable:

```
dffce: process (clk, rst) is
begin
    if rst = '1' then
        q <= '0';
    elsif rising_edge(clk) then
        if clk_ena = '1' then
            q <= d;
        end if;
    end if;
end process dffce;
```

- Sygnały synchroniczne nie powinny znajdować się na liście czułości.

# Procesy sekwencyjne (11)

- Przykład procesu z synchronicznym resetem i presetem:

```
dffrs: process (clk) is
begin
    if rising_edge(clk) then
        if rst = '1' then
            q <= '0';
        elsif set = '1' then
            q <= '1';
        else
            q <= d;
        end if;
    end if;
end process dffrs;
```

- W kodzie współbieżnym (przypisanie warunkowe) synteza prostych przerzutników jest również możliwa, ale nie jest zalecana.  
Przykład - przerzutnik typu D reagujący na zbocze:

```
q <= d when rising_edge(clk) else q;
```

# Procesy sekwencyjne (12)

- Przykłady procesów realizujących inne rodzaje przerzutników z wejściami synchronicznymi:

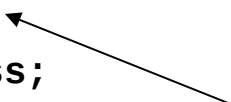
```
-- przerzutnik T:
tff: process (clk) is
begin
    if rising_edge(clk) then
        if t = '1' then
            q <= not q;
        end if;
    end if;
end process tff;

-- przerzutnik JK:
jkff: process (clk) is
begin
    if rising_edge(clk) then
        if j = '1' and k = '1' then
            q <= not q;
        elsif j = '0' and k = '1' then
            q <= '0';
        elsif j = '1' and k = '0' then
            q <= '1';
        end if;
    end if;
end process jkff;
```

# Procesy sekwencyjne (13)

- Zasady obowiązujące dla pojedynczych przerzutników można przenieść na szerszą klasę układów rejestrowych – o szerokości wielu bitów:

```
process (clk, rst) is
variable x:std_logic_vector(5 downto 0);
begin
    if rst = '1' then
        x := "000000";
    elsif rising_edge(clk) then
        x := x + 1;
        x := x * "10";
    end if;
    q <= x;
end process;
```



- W procesie synchronicznym nie dopuszcza się przypisań poza główną instrukcją if, z wyjątkiem przypisania sygnałowi wartości zmiennej obliczonej wewnątrz instrukcji warunkowej if.
- Poza główną instrukcją if rozpoczyna się obszar w którym można opisywać układy kombinacyjne – należy jednak pamiętać o zasadach tworzenia procesów kombinacyjnych. Łączenie procesów sekwencyjnych z kombinacyjnymi nie jest zalecane.

# Liczniki synchroniczne (1)

- Przykładowy proces licznika linii w generatorze sygnału video:

```
process (clk, rst) is
begin
    if rst = '1' then
        line <= 0;
        h_sync_old <= '1';
    elsif rising_edge(clk) then
        h_sync_old <= h_sync;
        if h_sync = '0' and h_sync_old = '1' then
            line <= line + 1;
            if line = 518 then
                v_sync <= '1';
            end if;
            if line = 520 then
                v_sync <= '0';
                line <= 0;
            end if;
            if line = 67 then
                display_enable <= '1';
            end if;
            if line = 434 then
                display_enable <= '0';
            end if;
        end if;
    end if;
end process;
```

# Liczniki synchroniczne (2)

- Przykładowy licznik rewersyjny z synchronicznym ładowaniem, wartością maksymalną, synchronicznym wejściem enable i asynchronicznym resetem:

```
library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.std_logic_unsigned.all;
use IEEE.std_logic_arith.all;

entity univ_counter is
generic ( cnt_size : integer := 8 );
port ( clk, rst : in std_logic;
      load_in : in std_logic_vector(cnt_size-1 downto 0);
      cnt_down : in std_logic;
      cnt_load : in std_logic;
      cnt_ena : in std_logic;
      cnt_max : in std_logic_vector(cnt_size-1 downto 0);
      cnt : out std_logic_vector(cnt_size-1 downto 0));
end entity univ_counter;

architecture behav of univ_counter is
signal int_cnt : std_logic_vector(cnt_size-1 downto 0);
begin
    cnt <= int_cnt;
```

# Liczniki synchroniczne (3)

```
process (clk, rst) is
begin
    if rst = '1' then
        int_cnt <= (others => '0');
    elsif rising_edge(clk) then
        if cnt_load = '1' then
            int_cnt <= load_in;
        elsif cnt_ena = '1' then
            if cnt_down = '1' then
                if int_cnt = CONV_STD_LOGIC_VECTOR(0, cnt_size) then
                    int_cnt <= cnt_max;
                else
                    int_cnt <= int_cnt - 1;
                end if;
            else
                if int_cnt = cnt_max then
                    int_cnt <= ( others => '0' );
                else
                    int_cnt <= int_cnt + 1;
                end if;
            end if;
        end if;
    end if;
end process;
end architecture behav;
```



# Dzielniki częstotliwości (1)

- Przykładowy dzielnik częstotliwości przez 2,3,4 i 64:

```
dzielnik: process (clk, rst) is
begin
    if rst = '1' then
        counter <= ( others => '0' );
        div3 <= '0';
        div3_cntr <= 0;
    elsif rising_edge(clk) then
        counter <= counter + 1;
        div3_cntr <= div3_cntr + 1;
        if div3_cntr = 1 then
            div3 <= '1';
        end if;
        if div3_cntr = 2 then
            div3 <= '0';
            div3_cntr <= 0;
        end if;
    end if;
end process dzielnik;
div2 <= counter(0);
div4 <= counter(1);
div64 <= counter(5);
```

# Dzielniki częstotliwości (2)

Ze stosowaniem w układzie dużej liczby sygnałów zegarowych wiąże się kilka problemów:

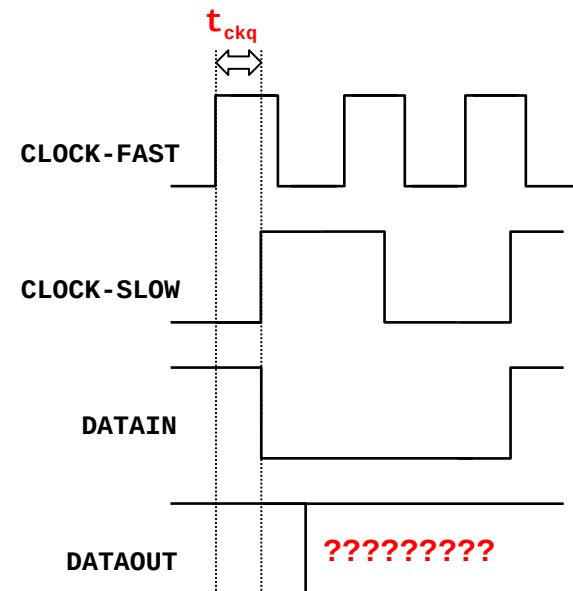
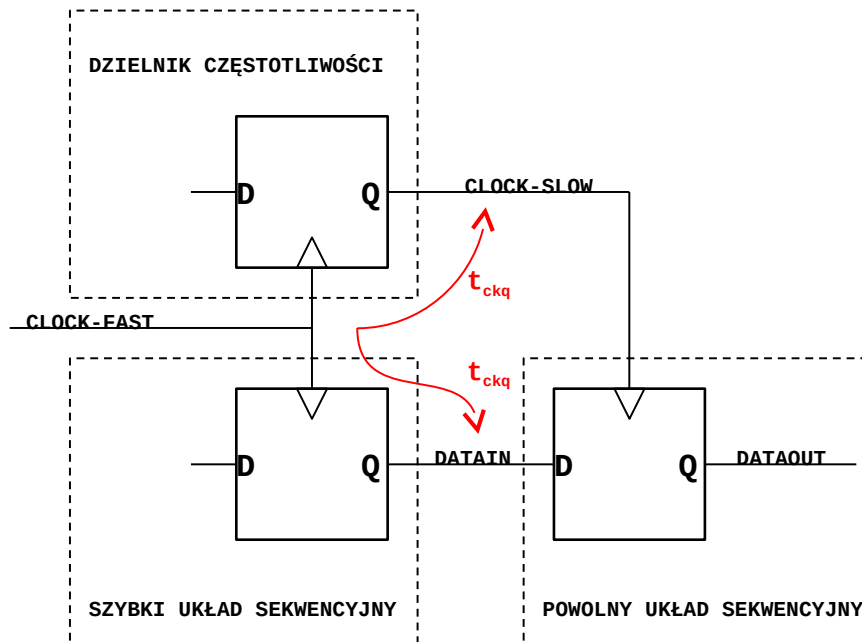
- ograniczona liczba zasobów do trasowania sygnałów zegarowych w układach programowalnych,
- przesunięcie fazowe sygnałów zegarowych względem siebie (skew) i zegarów zewnętrznych które utrudnia lub uniemożliwia przekazywanie danych pomiędzy domenami zegarowymi,
- do wytwarzania zegarów wskazane jest stosowanie specjalizowanych modułów kontrolerów sygnałów zegarowych (znajdujących się układach programowalnych).

## **Rady dla początkujących:**

- 1. Stosować w całym projekcie tylko jeden zegar.**
- 2. Zegar ten można otrzymać w dzielniku – ale nie należy wytwarzać już więcej zegarów ani korzystać z zegara oryginalnego ani też wymieniać jakichkolwiek dodatkowych sygnałów pomiędzy dzielnikiem a taktowanym przez niego układem.**

# Dzielniki częstotliwości (3)

Poniżej przedstawiona została demonstracja problemu z przesyłaniem danych z układu taktowanego zegarem szybkim do układu taktowanego zegarem powolnym, wygenerowanym w dzielniku. Ponieważ opóźnienie sygnału wyjściowego Q przerzutników w stosunku do aktywnego zbocza zegara ( $t_{ckq}$ ) jest (prawie) takie samo dla przerzutników w dzielniku i przerzutników w „szybkim układzie sekwencyjnym”, aktywne zbocze zegara podzielonego przychodzi w (prawie) tym samym momencie co nowa dana na linii DATAIN. Powoduje to błędną pracę przerzutnika w „powolnym układzie sekwencyjnym”.



# Dzielniki częstotliwości (4)

- Poniższy przykład opisuje nieprawidłową implementację dzielnika – wyjście podzielonego sygnału zegarowego jest zrealizowane z układu kombinacyjnego – mogą wystąpić hazardy.

```
process(clk_i, rst_i, q)
begin
    if (rst_i = '1') then
        q <= ( others => '0' );
    elsif rising_edge(clk_i) then
        q <= q + 1;
        if (q = 99999) then
            q <= ( others => '0' );
        end if;
    end if;
    -- UWAGA! poniższy kod opisuje układ kombinacyjny (q jest wejściem więc
    -- musi być dopisane do listy czułości)
    if (q >= 50000) then
        clk_div <= '1';
    else
        clk_div <= '0';
    end if;
end process;
```

# Dzielniki częstotliwości (5)

- Prawidłowa implementacja dzielnika z poprzedniego przykładu – wyjście podzielonego sygnału zegarowego jest zrealizowane za pomocą wyjścia Q przerzutnika.

```
process(clk_i, rst_i)
begin
    if (rst_i = '1') then
        q <= ( others => '0' );
        clk_div <= '0';
    elsif rising_edge(clk_i) then
        q <= q + 1;
        if (q = 99999) then
            q <= ( others => '0' );
        end if;
        if (q >= 50000) then
            clk_div <= '1';
        else
            clk_div <= '0';
        end if;
    end if;
end process;
```

-- wersja zoptymalizowana:

```
process(clk_i, rst_i)
begin
    if (rst_i = '1') then
        q <= ( others => '0' );
        clk_div <= '0';
    elsif rising_edge(clk_i) then
        q <= q + 1;
        if (q = 99999) then
            q <= ( others => '0' );
            clk_div <= '0';
        end if;
        if (q = 49999) then
            clk_div <= '1';
        end if;
    end if;
end process;
```

# Rejestry przesuwne (1)

- Do implementacji rejestrów przesuwnych nie jest wskazane używanie operatorów: sll, srl, sla, sra, rol, ror (działają one na bit\_vector).
- Przykład: 2 rejestry przesuwne (jeden w prawo, drugi w lewo) 16-bitowe z wejściem/wyjściem szeregowym i ładowaniem synchronicznym równoległym.

```
shift_regs: process(clk, rst)
begin
    if rst = '1' then
        reg_l <= (others => '0');
        reg_r <= (others => '0');
    elsif rising_edge(clk) then
        if load = '1' then
            reg_l <= data_l;
            reg_r <= data_r;
        else
            serial_data_out_l <= reg_l(15);
            reg_l(15 downto 1) <= reg_l(14 downto 0);
            reg_l(0) <= serial_data_in_l;
            serial_data_out_r <= reg_r(0);
            reg_r(14 downto 0) <= reg_r(15 downto 1);
            reg_r(15) <= serial_data_in_r;
        end if;
    end if;
end process;
```

# Rejestry przesuwne (2)

- Przykładowy rejestr przesuwny ze sprzężeniem zwrotnym LFSR (Linear Feedback Shift Register):

```
LFSR: process(clk, rst)
```

```
begin
```

```
  if rst = '1' then
```

```
    reg <= (others => '1');
```

```
  elsif rising_edge(clk) then
```

```
    if clk_enable = '1' then
```

```
      reg(15 downto 1) <= reg(14 downto 0);
```

```
      reg(0) <= reg(15) xor reg(14) xor reg(13) xor reg(4);
```

```
    end if;
```

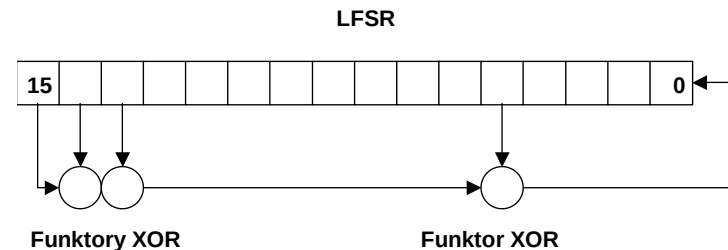
```
    if init_load = '1' then
```

```
      reg <= data_in;
```

```
    end if;
```

```
  end if;
```

```
end process;
```



# Rejestry przesuwne (3)

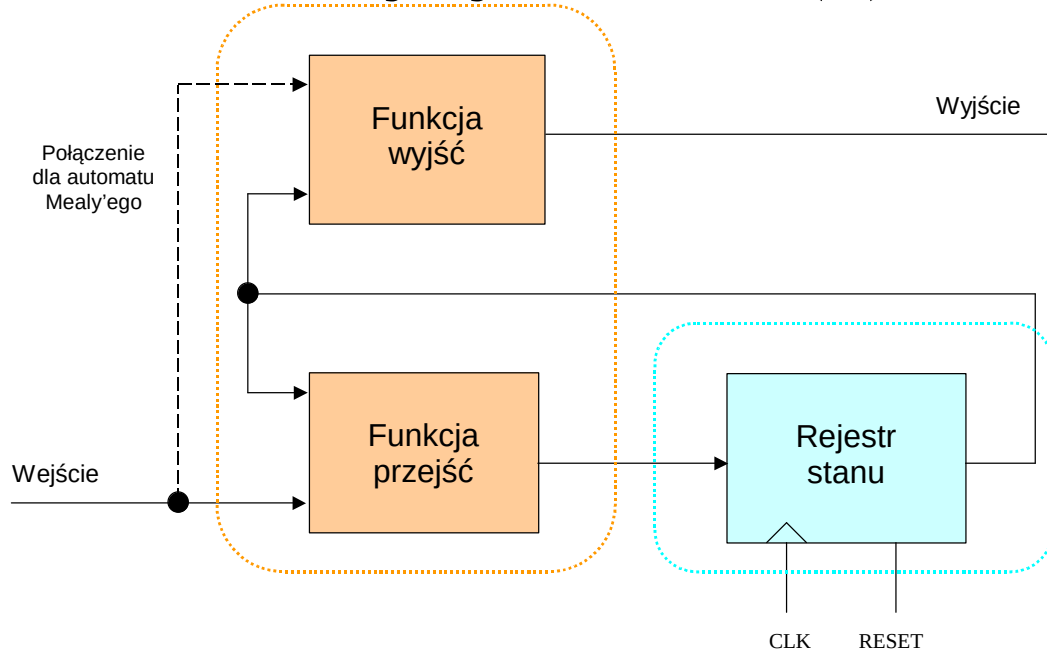
- Przykładowy rejestr przesuwany typu barrel shifter (16-bitowy, rotacja w prawo):

```
signal SEL_A: STD_LOGIC_VECTOR(1 downto 0);
signal SEL_B: STD_LOGIC_VECTOR(1 downto 0);
signal C: STD_LOGIC_VECTOR(15 downto 0);
-----
SEL_A <= SEL(1 downto 0);
SEL_B <= SEL(3 downto 2);
process(SEL_A,B_INPUT)
begin
  case SEL_A is
    when "00" =>    -- rotacja o 0 bitów
      C <= B_INPUT;
    when "01" =>    -- rotacja o 1 bit
      C(15) <= B_INPUT(0);
      C(14 downto 0) <= B_INPUT(15 downto 1);
    when "10" =>    -- rotacja o 2 bity
      C(15 downto 14) <= B_INPUT(1 downto 0);
      C(13 downto 0) <= B_INPUT(15 downto 2);
    when "11" =>    -- rotacja o 3 bity
      C(15 downto 13) <= B_INPUT(2 downto 0);
      C(12 downto 0) <= B_INPUT(15 downto 3);
    when others =>
      C <= B_INPUT;
  end case;
end process;
```

```
process(SEL_B,C)
begin
  case SEL_B is
    when "00" => -- dodatkowo o 0 bitów
      B_OUTPUT <= C;
    when "01" => -- dodatkowo o 4 bity
      B_OUTPUT(15 downto 12) <= C(3 downto 0);
      B_OUTPUT(11 downto 0) <= C(15 downto 4);
    when "10" => -- dodatkowo o 8 bitów
      B_OUTPUT(15 downto 8) <= C(7 downto 0);
      B_OUTPUT(7 downto 0) <= C(15 downto 8);
    when "11" => -- dodatkowo o 12 bitów
      B_OUTPUT(15 downto 4) <= C(11 downto 0);
      B_OUTPUT(3 downto 0) <= C(15 downto 12);
    when others => B_OUTPUT <= C;
  end case;
end process;
process(clk)
begin
  if rising_edge(clk) then
    if (data_load = '1') then
      B_INPUT <= data_in;
    else
      B_INPUT <= B_OUTPUT;
    end if;
  end if;
end process;
```



# Maszyny stanów (1)



- Maszyny stanów Moora i Mealy'ego mogą być w łatwy sposób implementowane w języku VHDL.
- Kolorem niebieskim zaznaczono bloki implementowane za pomocą procesów sekwencyjnych synchronicznych.
- Kolorem pomarańczowym zaznaczono bloki implementowane za pomocą procesów kombinacyjnych.
- Oba bloki kombinacyjne mogą być zaimplementowane w jednym procesie.

# Maszyny stanów (2)

- Do reprezentacji zmiennych stanu używa się zazwyczaj typu wyliczeniowego. Można także użyć innego typu (np. `std_logic_vector` lub `integer`), ale typ wyliczeniowy pozwala na swobodny dobór przez syntezer odpowiedniej reprezentacji dla każdego ze stanów:

```
type StateType is (idle, operate, finish);  
signal present_state, next_state : StateType;
```

- Można wpływać na sposób reprezentacji stanów:
  - globalnie za pomocą opcji syntezy,
  - lokalnie za pomocą atrybutów dla sygnału reprezentującego aktualny stan maszyny – czyli wyjścia rejestru stanu (przykład dla Vivado):

```
attribute fsm_encoding: string;  
attribute fsm_encoding of present_state: signal is "gray";  
-- "{auto|one_hot|gray|sequential|johnson|none}"
```

# Maszyny stanów (3)

```
-- przykład maszyny stanów  
-- typu Moore'a
```

```
type StateType is (idle, operate,  
finish);  
signal present_state : StateType;  
signal next_state : StateType;
```

```
seq: process (reset, clk) is  
begin  
    if (reset = '1') then  
        present_state <= idle;  
    elsif rising_edge(clk) then  
        present_state <= next_state;  
    end if;  
end process seq;
```

```
comb: process (present_state, go, brk, cln,  
rdy) is  
begin  
    case present_state is  
        when idle =>  
            if (go = '1') then  
                next_state <= operate;  
            else  
                next_state <= idle;  
            end if;  
            power <= '0';  
            direction <= '0';  
        when operate =>  
            if (brk = '1') then  
                next_state <= idle;  
            elsif (cln = '1') then  
                next_state <= finish;  
            else  
                next_state <= operate;  
            end if;  
            power <= '1';  
            direction <= '0';  
        when finish =>  
            if (rdy = '1') then  
                next_state <= idle;  
            else  
                next_state <= finish;  
            end if;  
            power <= '1';  
            direction <= '1';  
    end case;  
end process comb;
```

# Maszyny stanów (4)

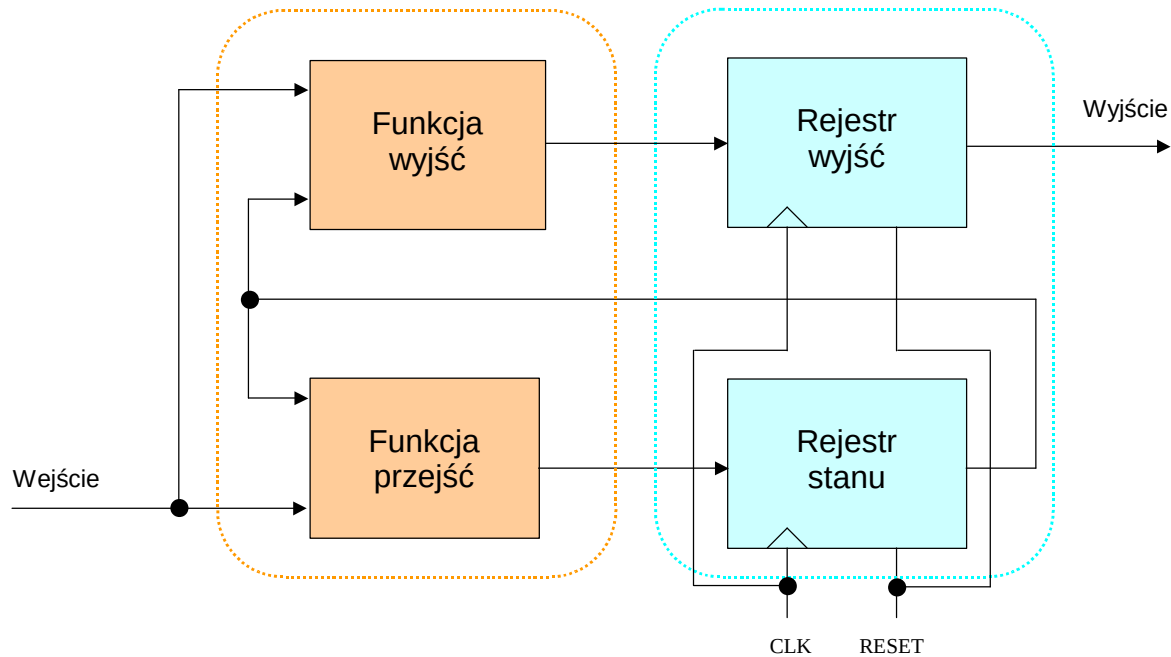
```
-- przykład maszyny stanów  
-- typu Mealy'ego
```

```
type StateType is (idle, operate,  
finish);  
signal present_state : StateType;  
signal next_state : StateType;
```

```
seq: process (reset, clk) is  
begin  
    if (reset = '1') then  
        present_state <= idle;  
    elsif rising_edge(clk) then  
        present_state <= next_state;  
    end if;  
end process seq;
```

```
comb: process (present_state, go, brk, cln,  
rdy, emergency_off) is  
begin  
    -- unikamy zatrząsków: wartość domyślna:  
    next_state <= present_state;  
    case present_state is  
        when idle =>  
            if (go = '1') then  
                next_state <= operate;  
            end if;  
            power <= '0';  
            direction <= '0';  
        when operate =>  
            if (brk = '1') then  
                next_state <= idle;  
            elsif (cln = '1') then  
                next_state <= finish;  
            end if;  
            power <= not (emergency_off or brk);  
            direction <= '0';  
        when finish =>  
            if (rdy = '1') then  
                next_state <= idle;  
            end if;  
            power <= not emergency_off;  
            direction <= '1';  
    end case;  
end process comb;
```

# Maszyny stanów (5)



- Powyżej przedstawiono schemat maszyny stanów Mealy'ego z wyjściem synchronicznym.
- Kolorem niebieskim zaznaczono bloki implementowane za pomocą procesów sekwencyjnych synchronicznych.
- Kolorem pomarańczowym zaznaczono bloki implementowane za pomocą procesów kombinacyjnych.
- Maszyna Mealy'ego z wyjściem synchronicznym może być także zaimplementowana w jednym procesie sekwencyjnym, ponieważ wyjście maszyny generowane jest wyłącznie z wyjść przerzutników.

# Maszyny stanów (6)

```
-- przykład maszyny stanów  
-- Mealy'ego z wyjściem  
-- synchronicznym  
-- w jednym procesie
```

```
type StateType is (idle, operate,  
finish);
```

```
signal fsm_state : StateType;
```

```
sync_mealy: process (reset, clk) is  
begin
```

```
    if reset = '1' then  
        fsm_state <= idle;  
        power <= '0';  
        direction <= '0';  
    elsif rising_edge(clk) then
```

```
        case fsm_state is  
            when idle =>  
                if (go = '1') then  
                    fsm_state <= operate;  
                    power <= '1';  
                end if;  
            when operate =>  
                if (brk = '1') then  
                    fsm_state <= idle;  
                    power <= '0';  
                elsif (cln = '1') then  
                    fsm_state <= finish;  
                    direction <= '1';  
                end if;  
            when finish =>  
                if (rdy = '1') then  
                    fsm_state <= idle;  
                    power <= '0';  
                    direction <= '0';  
                end if;  
        end case;  
    end process sync_mealy;
```

# Maszyny stanów (7)

## Problem stanów zabronionych

- Zazwyczaj nie stosuje się specjalnych zabezpieczeń. W systemach „mission-critical” można włączyć odpowiednią opcję syntezy (jeżeli jest dostępna) generującą maszyny stanów zabezpieczone przed zatrzaśnięciem się w stanie niedozwolonym. Jednak maszyny te zajmują znacznie więcej elementów logicznych i są powolniejsze.
- Jeżeli nie ma odpowiedniej opcji w syntezy można:
  1. Użyć maszyny stanów z bezpośrednim kodowaniem stanów i określić właściwe zachowanie się maszyny dla wszystkich, także nieużywanych stanów.
  2. Dodać układ logiczny wykrywający stan niedozwolony.

# Maszyny stanów (8)

-- Przykład maszyny stanów z bezpośrednim kodowaniem stanów:

```
signal present_state, next_state : std_logic_vector(2 downto 0);  
constant idle      : std_logic_vector(2 downto 0) := "000";  
constant prepare   : std_logic_vector(2 downto 0) := "001";  
constant operate    : std_logic_vector(2 downto 0) := "010";  
constant finish     : std_logic_vector(2 downto 0) := "100";  
constant failure    : std_logic_vector(2 downto 0) := "111";
```

-- maszyna z ochroną przed stanami zabronionymi:  
when others => next\_state <= idle;

-- maszyna bez ochrony przed stanami zabronionymi:  
when others => next\_state <= "---";



# Maszyny stanów (9)

-- Wykrywanie stanów zabronionych  
-- dla maszyn z kodowaniem one-hot:

```
idl <= state = idle;
prepar <= state = prepare;
operat <= state = operate;
finis <= state = finish;
failur <= state = failure;

inv <= (idl and (prepar or operat or finis or failur)) or
      (prepar and (operat or finis or failur)) or
      (operat and (finis or failur)) or
      (finis or failur) or
      (not (idl or prepar or operat or finis or failur));

....
if (inv = '1') then
    state <= idle;
else
    ....
....
```

# Maszyny stanów (10)

W syntezerze Vivado zamiast opcji globalnej można wykorzystać atrybut:

```
attribute fsm_safe_state : string;  
attribute fsm_safe_state of present_state : signal is "power_on_state";  
-- "{reset_state|power_on_state|default_state|auto_safe_state}"
```

Ustawienie tego atrybutu powoduje syntezę maszyny stanów zdolnej do wykrycia stanu zabronionego i automatycznej reakcji (w zależności od wartości atrybutu):

- **reset\_state**: powrót maszyny stanów do stanu ustawianego przez wejście resetu przerzutników pamięci stanu (stan taki jak po sygnale reset),
- **power\_on\_state**: powrót maszyny stanów do stanu ustawianego przez wartość początkową sygnału opisującego przerzutniki pamięci stanu (stan taki jak po uruchomieniu konfiguracji FPGA),
- **default\_state**: powrót maszyny stanów do stanu określonego przez **when others** w opisie funkcji przejść (nawet jeżeli stan opisywany przez **when others** jest nieosiągalny).

Powyższe metody wykorzystują w kodowaniu stanów kod Hamming-2 (wykrywający przekłamanie pojedynczego bitu w przerzutnikach stanu).

- **auto\_safe\_state**: powrót maszyny do stanu jaki występował przed przekłamaniem.

Powyższa metoda wykorzystuje w kodowaniu stanów kod Hamming-3 (wykrywający i potrafiący skorygować przekłamanie pojedynczego bitu w przerzutnikach stanu).

# Maszyny stanów (11)

## Unikanie stanów zabronionych:

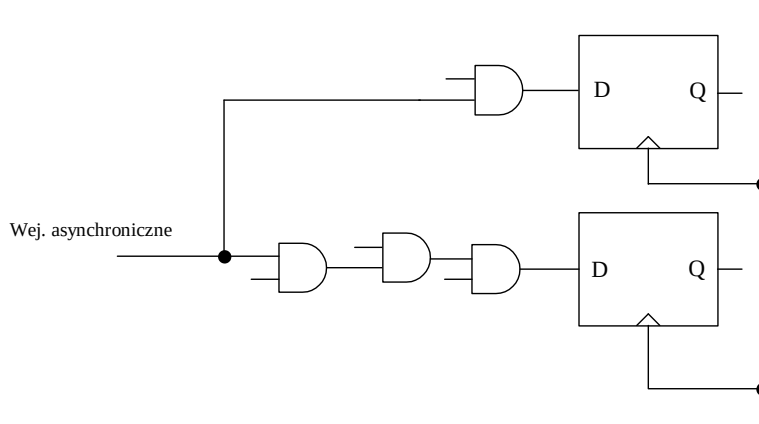
Najczęstsze przyczyny przechodzenia maszyn do stanów zabronionych (lub do innych stanów niewłaściwych):

- Zbyt duża częstotliwość zegarowa systemu (opóźnienie ścieżki krytycznej jest zbyt duże i nie są spełnione parametry  $t_{\text{setup}}$  przerzutników).

- **Sygnały sterujące maszyną stanów nie spełniają parametru  $t_{\text{setup}}$  przerzutnika lub są **asynchroniczne**.**

**Sygnały wytwarzane przez elementy zewnętrzne takie jak: przyciski, czujniki, linie przerwań itp. zazwyczaj są asynchroniczne.**

Różne czasy propagacji wewnątrz logiki kombinacyjnej maszyny stanów mogą powodować niewłaściwe jej działanie jeżeli stan wejścia zmienia się tuż przed aktywnym zboczem sygnału zegarowego:

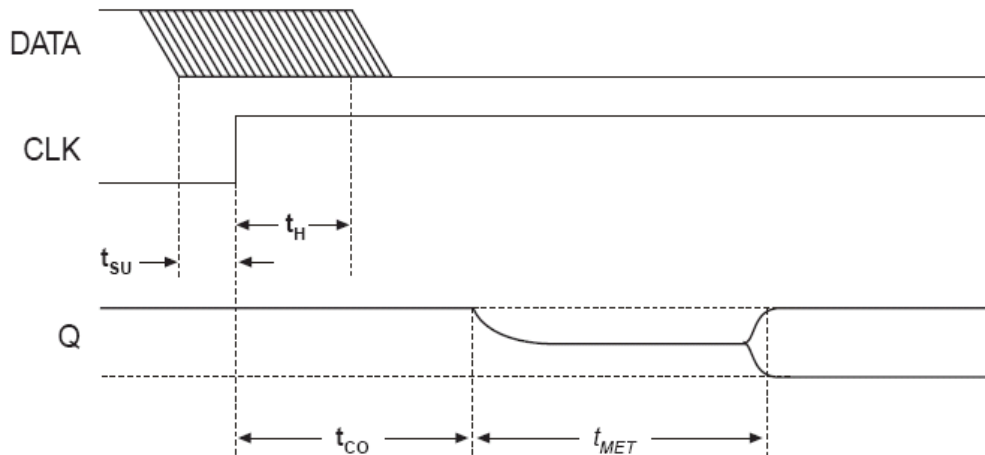




# Metastabilność (1)

## Unikanie stanów zabronionych, c.d.:

Użycie potoku złożonego z dwóch lub więcej przerzutników synchronizujących może być konieczne jeżeli występuje duże prawdopodobieństwo pojawienia się zjawiska metastabilności w pierwszym przerzutniku synchronizującym. Zjawisko to polega na pozostawaniu przerzutnika przez pewien czas w równowadze chwiejnej pomiędzy dwoma stanami stabilnymi. Na rysunku czas ten zaznaczono jako  $t_{MET}$ .



- przykład procesu realizującego
- dwa przerzutniki synchronizujące

```
signal asynch_in : std_logic;  
signal q : std_logic;  
signal synch_out : std_logic;
```

```
synch: process (clk) is  
begin  
    if rising_edge(clk) then  
        q <= asynch_in;  
        synch_out <= q;  
    end if;  
end process;
```

# Metastabilność (2)

$$MTBF = \frac{e^{(C_2 \times t_{MET})}}{C_1 \times f_{CLOCK} \times f_{DATA}}$$

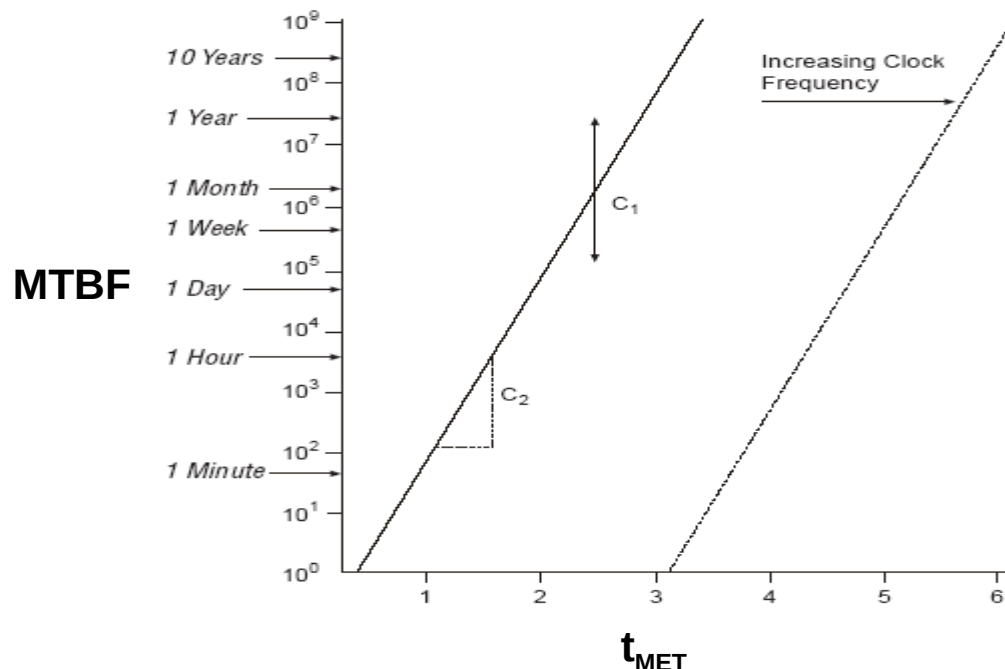
MTBF – Mean Time Between Failures

$f_{data}$  – częstotliwość zmian w sygnale danych

$f_{clock}$  – częstotliwość zegara

$t_{MET}$  – dodatkowy czas zarezerwowany dla ustalenia się stanu wyjściowego

Stałe  $C_1$  i  $C_2$  zależą od właściwości przerzutników



# Metastabilność (3)

-- przykład procesu realizującego dwa przerzutniki synchronizujące  
-- oraz układ likwidacji drgań zestyków

```
signal key_in : std_logic;          -- sygnał bezpośrednio z
przycisku
signal q : std_logic;
signal key_synch : std_logic;
signal key_stable_out : std_logic;  -- stabilny sygnał przycisku
(out)
constant delay_const : integer := 10000; -- opóźnienie syg.
stabilnego
```

```
synch: process (clk) is
variable delay_cntr : integer := 0;
begin
    if rising_edge(clk) then
        q <= key_in;
        key_synch <= q;
        if (key_synch = key_stable_out) then
            delay_cntr := 0;
        else
            delay_cntr := delay_cntr + 1;
        end if;
        if (delay_cntr = delay_const) then
            key_stable_out <= key_synch;
            delay_cntr := 0;
        end if;
    end if;
```

# Funkcje i procedury

- Funkcje pozwalają na realizację operacji sekwencyjnych zakończoną zwróceniem pojedynczej wartości.
- Funkcje mogą pełnić rolę komponentów o charakterze kombinacyjnym.
- Funkcje mogą służyć do przeciążania operatorów (nadając lub zmieniając ich standardowe znaczenie w odniesieniu do konkretnych typów danych).
- Procedury pozwalają na realizację operacji sekwencyjnych zakończoną modyfikacją kilku wartości.
- Procedury pozwalają na wykonywanie instrukcji zegarowanych (mogą opisywać układy synchroniczne).
- Funkcje i procedury mogą być przeciążane.
- Funkcje i procedury mogą być deklarowane i definiowane w architekturach oraz w pakietach.
- Wewnątrz procedur i funkcji można deklarować tylko stałe i zmienne (jeżeli są one inicjowane – to inicjacja dotyczy każdej aktywacji procedury lub funkcji – inaczej niż w przypadku procesów).
- Funkcje które mogą generować różne rezultaty przy tych samych argumentach nazywają się „impure” – zazwyczaj nie są syntezywalne!
- Procedury i funkcje mogą być zagnieżdżane. Można również stosować rekurencję.



# Funkcje (1)

-- Przykład funkcji widocznej tylko w architekturze:

```
...
architecture systemx of systemx is
    function majority (a, b, c: std_logic) return std_logic is
    begin
        return ((a and b) or (a and c) or (b and c));
    end function majority;
begin
    sum <= a xor b xor carry_in;
    carry_out <= majority(a, b, carry_in);
end;
```

-- Można także wykorzystać pakiety (w celu dodania funkcji do  
-- biblioteki tak aby można było z niej korzystać globalnie):

```
package moje_funkcje is
    function majority (a, b, c: std_logic) return std_logic;
end package moje_funkcje;
package body moje_funkcje is
    function majority (a, b, c: std_logic) return std_logic is
    begin
        return ((a and b) or (a and c) or (b and c));
    end function majority;
end package body moje_funkcje;
```

-- Wykorzystanie pakietu moje\_funkcje:  
use work.moje\_funkcje.majority;

## Funkcje (2)

-- Przykład funkcji zamieniającej miejscami pozycje bitów  
-- w wektorze:

```
function mirror(in_vec: std_logic_vector) return std_logic_vector is
variable tmp: std_logic_vector(in_vec'range);
begin
    for i in 1 to in_vec'length loop
        if (in_vec'left > in_vec'right) then
            tmp(in_vec'left-i+1) := in_vec(in_vec'right+i-1);
        else
            tmp(in_vec'left+i-1) := in_vec(in_vec'right-i+1);
        end if;
    end loop;
    return (tmp);
end function mirror;
```

# Funkcje (3)

-- Przykład funkcji realizującej dekodery 7-segmentowy LED:

```
function seven_seg(data_in: std_logic_vector(3 downto 0)) return std_logic_vector is
variable tmp : std_logic_vector(6 downto 0);
begin
    --      0
    --      ---
    --  5 |    | 1
    --      ---    <- 6
    --  4 |    | 2
    --      ---
    --      3
    case data_in is
        when "0001" => tmp := "1111001";    --1
        when "0010" => tmp := "0100100";    --2
        when "0011" => tmp := "0110000";    --3
        when "0100" => tmp := "0011001";    --4
        when "0101" => tmp := "0010010";    --5
        when "0110" => tmp := "0000010";    --6
        when "0111" => tmp := "1111000";    --7
        when "1000" => tmp := "0000000";    --8
        when "1001" => tmp := "0010000";    --9
        when "1010" => tmp := "0001000";    --A
        when "1011" => tmp := "0000011";    --b
        when "1100" => tmp := "1000110";    --C
        when "1101" => tmp := "0100001";    --d
        when "1110" => tmp := "0000110";    --E
        when "1111" => tmp := "0001110";    --F
        when others => tmp := "1000000";    --0
    end case;
    return (tmp);
end function seven_seg;
```

# Funkcje (4)

Za pomocą funkcji jest możliwe przeciążanie operatorów:

```
library IEEE;
use IEEE.STD_LOGIC_1164.all;
use IEEE.STD_LOGIC_UNSIGNED.all;

package moje_funkcje is
    function "/" (x, y: std_logic_vector) return std_logic_vector;
end package moje_funkcje;

package body moje_funkcje is
    function "/" (x, y: std_logic_vector) return std_logic_vector is
        variable result : std_logic_vector(x'length-1 downto 0) := ( others => '0' );
        variable remainder : std_logic_vector(y'length downto 0) := ( others => '0' );
        begin
            for i in x'range loop
                remainder(0) := x(i);
                if (remainder >= y) then
                    result(i) := '1';
                    remainder := remainder - y;
                end if;
                remainder := remainder(y'length-1 downto 0) & '0';
            end loop;
            return (result);
        end function "/";
    end package body moje_funkcje;
```

# Funkcje (5)

Użycie pakietu „moje\_funkcje” przeciąża operator „/” dodając obsługę tego operatora w odniesieniu do argumentów typu std\_logic\_vector:

```
library IEEE;
use IEEE.STD_LOGIC_1164.all;
use work.moje_funkcje.all;
...
signal dzielna, dzielnik, iloraz : std_logic_vector(7 downto 0);
...
iloraz <= dzielna / dzielnik;
...
```

Możliwe jest także przeciążenie samych funkcji (poniżej przeciążona funkcja mirror z poprzedniego przykładu – dodaje obsługę łańcuchów):

```
function mirror(in_str: string) return string is
variable tmp: string(in_str'range);
begin
  for i in 1 to in_str'length loop
    if (in_str'left > in_str'right) then
      tmp(in_str'left-i+1) := in_str(in_str'right+i-1);
    else
      tmp(in_str'left+i-1) := in_str(in_str'right-i+1);
    end if;
  end loop;
  return (tmp);
end mirror;
```

# Procedury (1)

Procedury pozwalają na zwracanie więcej niż jednego rezultatu.

Parametry mogą być typu: constant (domyślnie dla parametrów wejściowych), variable (domyślnie dla parametrów wyjściowych) lub signal. Możliwe kierunki przekazywania sygnałów: in (domyślnie), out, inout.

```
procedure dziel (constant x, y: std_logic_vector;  
                signal result, remainder: out std_logic_vector) is  
variable dv : std_logic_vector(x'length-1 downto 0) := ( others => '0' );  
variable rm : std_logic_vector(y'length downto 0) := ( others => '0' );  
begin  
  for i in x'range loop  
    rm(0) := x(i);  
    if (rm >= y) then  
      dv(i) := '1';  
      rm := rm - y;  
    end if;  
    rm := rm(y'length-1 downto 0) & '0';  
  end loop;  
  remainder <= rm(y'length downto 1);  
  result <= dv;  
end dziel;  
  
...  
dziel(dzielna, dzielnik, iloraz, reszta);  
...
```

# Procedury (2)

Procedury pozwalają na realizację instrukcji zależnych od zegara.

```
package moje_funkcje is
    procedure dff (signal clk, rst: std_logic;
                   signal d: std_logic_vector;
                   signal q: out std_logic_vector);
end package moje_funkcje;

package body moje_funkcje is
    procedure dff (signal clk, rst: std_logic;
                   signal d: std_logic_vector;
                   signal q: out std_logic_vector) is
    begin
        if rst = '1' then
            q <= ( d'range => '0' );
        elsif (clk'event and clk = '1') then
            q <= d;
        end if;
    end procedure dff;
end package body moje_funkcje;
```

# Procedury (3)

Procedury pozwalają na łatwą implementację komponentów synchronicznych (o ile nie zawierają słowa kluczowego „wait”).

Poniższy przykład pokazuje implementację trzech rejestrów 8-bitowych za pomocą wcześniej zdefiniowanej procedury dff.

```
library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.std_logic_unsigned.all;
use work.moje_funkcje.all;

entity delay_line is
    port ( clk, rst : in std_logic;
          di : in std_logic_vector(7 downto 0);
          do : out std_logic_vector(7 downto 0));
end entity delay_line;

architecture arch1 of delay_line is
    signal tmp, tmp1: std_logic_vector(7 downto 0);
begin
    dff(clk, rst, di, tmp);
    dff(clk, rst, tmp, tmp1);
    dff(clk, rst, tmp1, do);
end architecture arch1;
```



# Procedury (4)

Przykład procedury ze argumentem typu inout (zmienna).

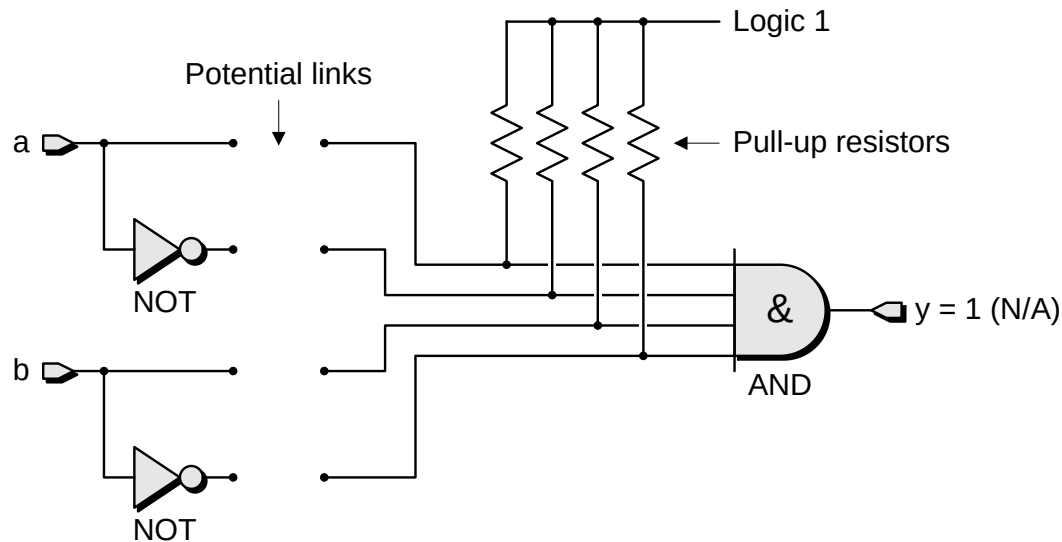
Instrukcja „assert” umożliwia wykrycie błędnej długości wektora wejściowego (jeżeli warunek po „assert” jest „false” następuje przerwanie syntezy z komunikatem o błędzie podanym po słowie „report”).

```
procedure transcoder_bcd_2_binary (variable value: inout std_logic_vector) is
variable res: integer range 0 to (2**value'length)-1;
begin
    assert ((value'length mod 4) = 0)
        report "BCD value must be multiple of 4 bits"
        severity Error;
    res := 0;
    for i in 0 to (value'length-1)/4 loop
        res := res + CONV_INTEGER(value((4*i)+3 downto 4*i))*(10**i);
    end loop;
    value := CONV_STD_LOGIC_VECTOR(res, value'length);
end procedure transcoder_bcd_2_binary;

....
process (xcoder_in) is
variable x: std_logic_vector(xcoder_in'range);
begin
    x := xcoder_in;
    transcoder_bcd_2_binary(x);
    xcoder_out <= x;
end process;
```

# Budowa logiki programowalnej

# Budowa logiki programowalnej - wstęp

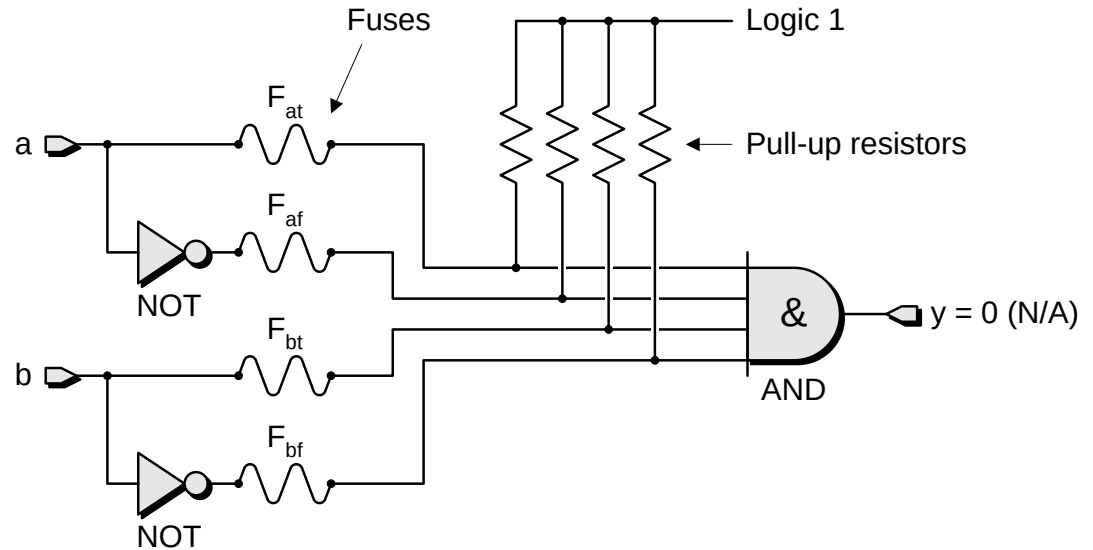


Podstawowym elementem logicznych układów programowalnych są programowalne połączenia.

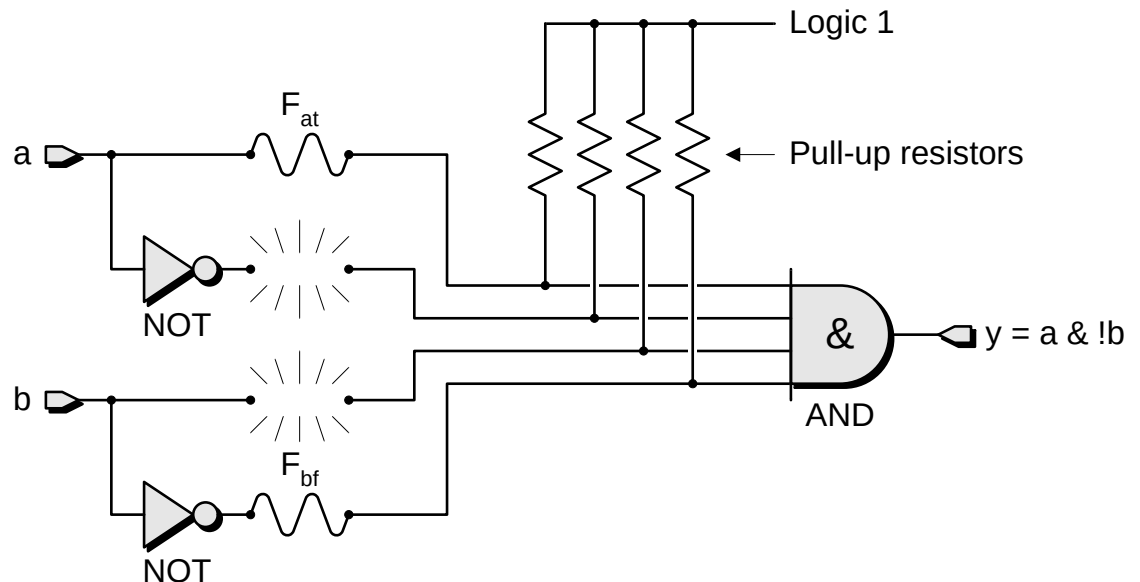
# Połączenia przepalane - fuse

Przykładowy  
programowalny funkcyj  
logiczny z przepalanymi  
połączeniami -

przed programowaniem:



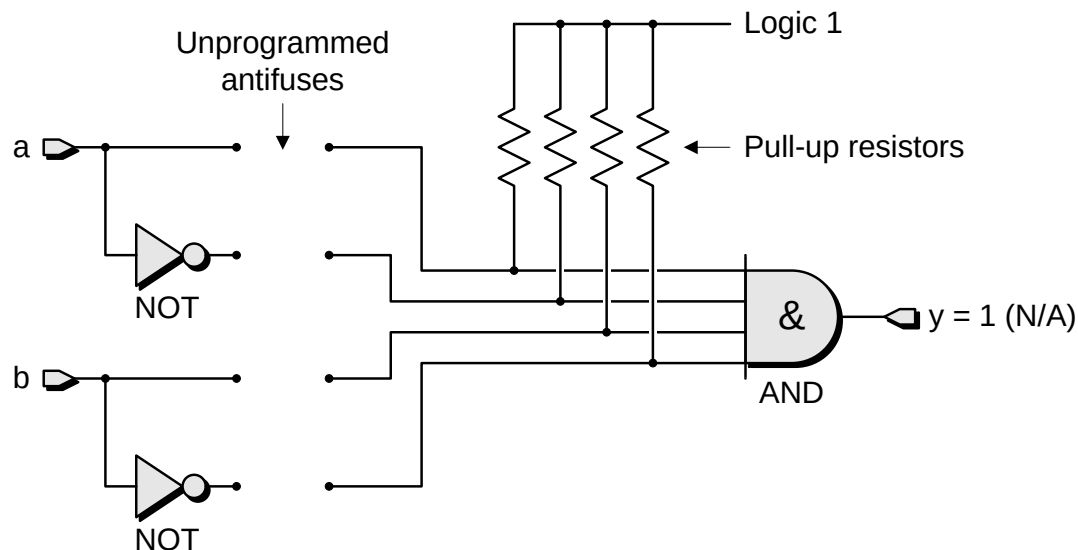
i po programowaniu:



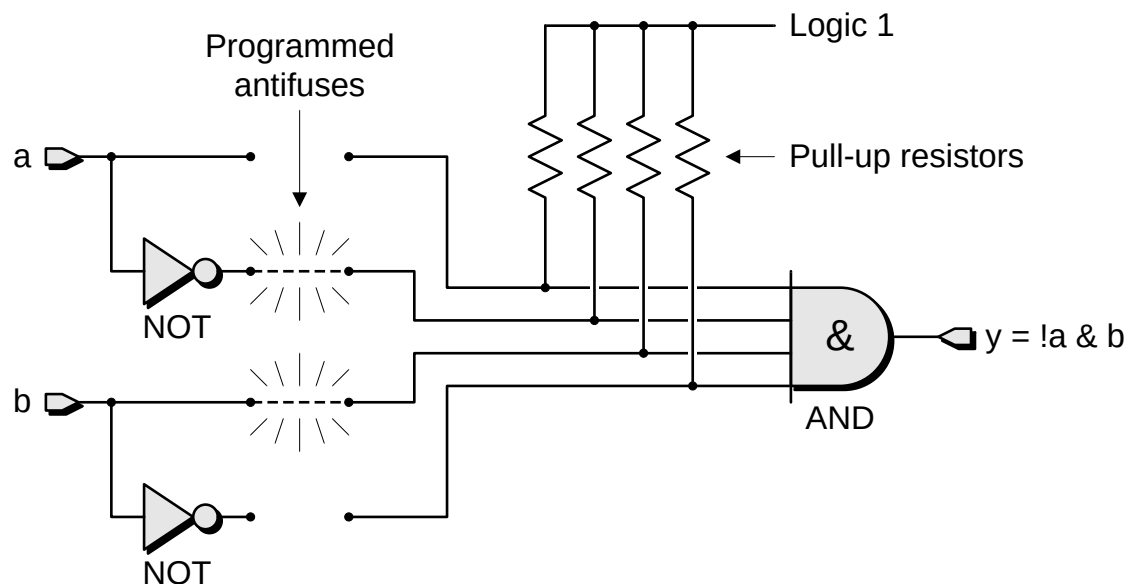
# Połączenia generowane - antifuse (1)

Przykładowy  
programowalny funkcyj  
logiczny z generowanymi  
połączeniami –

przed programowaniem:

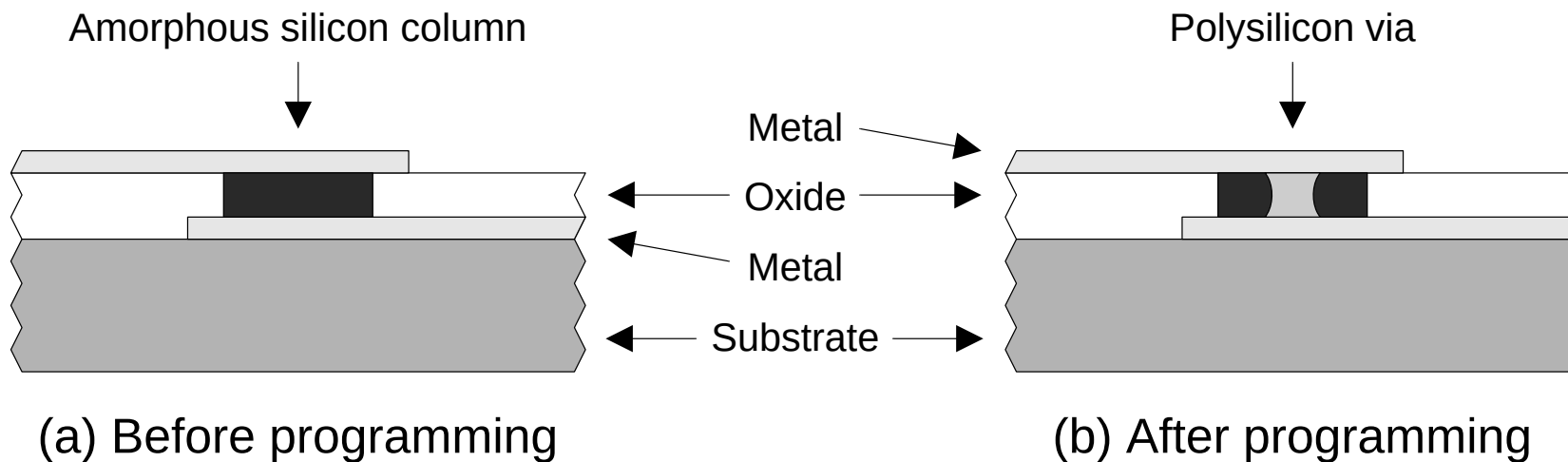


i po programowaniu:



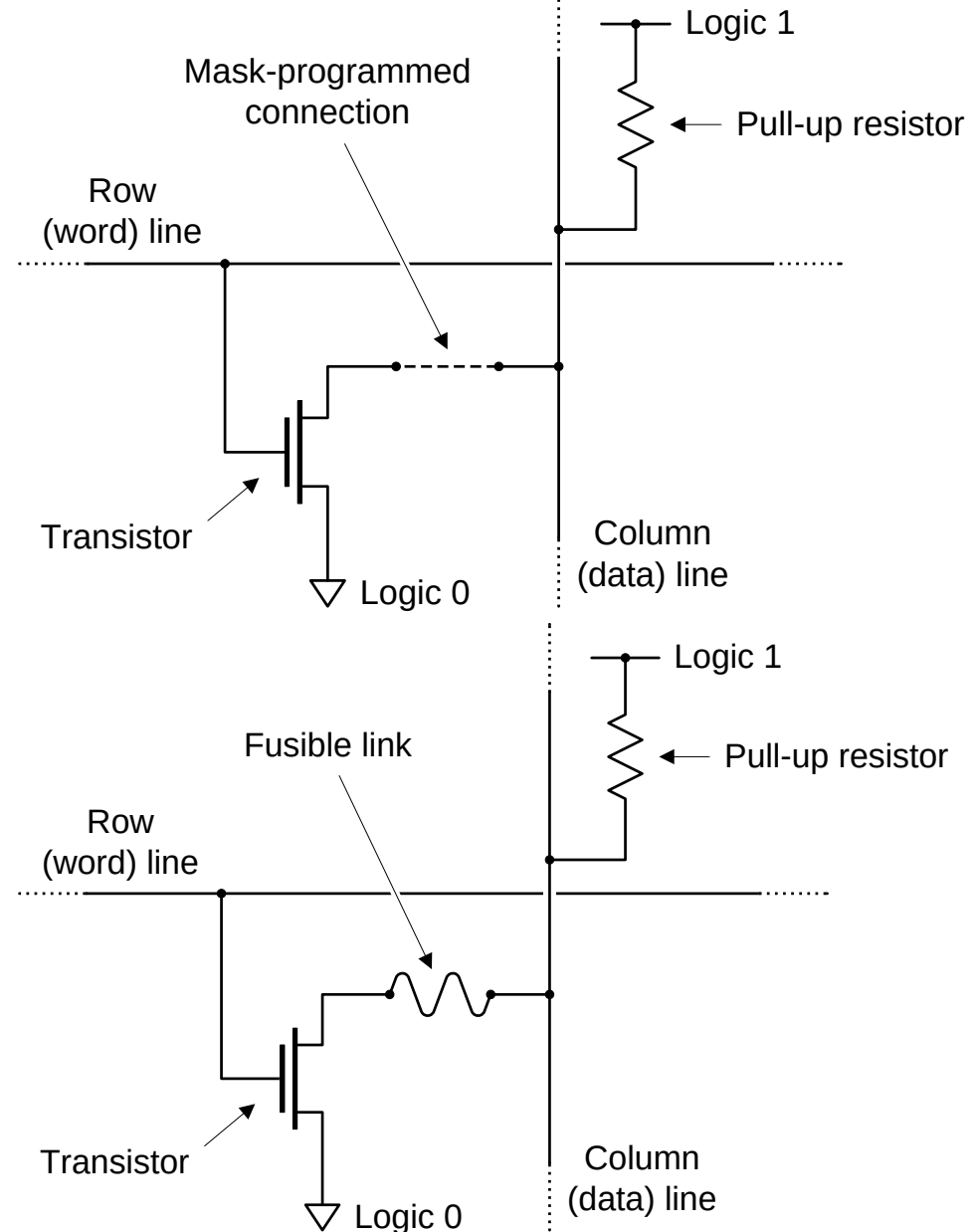
# Połączenia generowane - antifuse (2)

- Budowa połączeń generowanych (antifuse).
- Połączenia tego typu stosowane są np. w układach programowalnych FPGA firm Actel i QuickLogic.



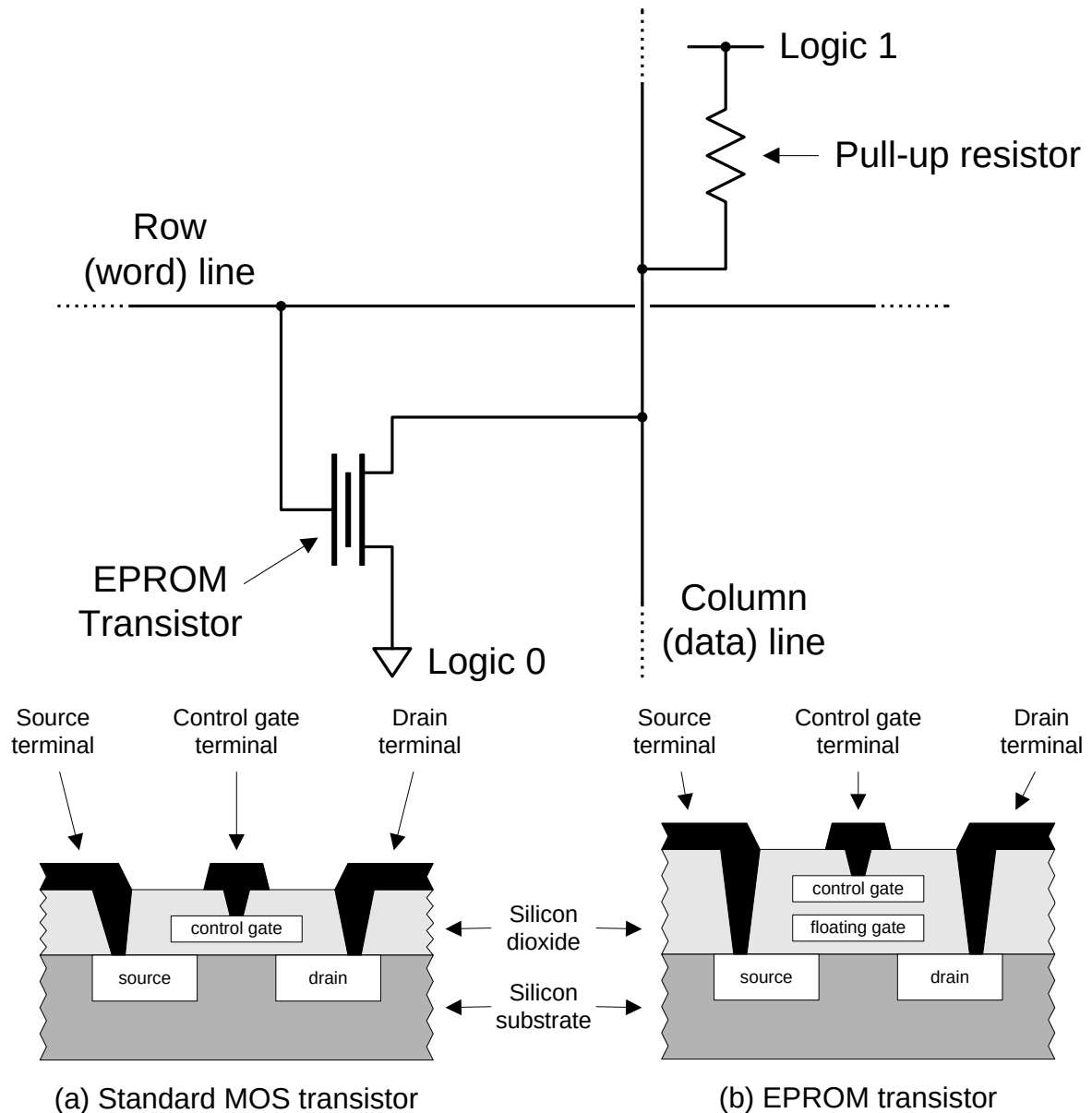
# Budowa pamięci programowalnej (1)

Pamięci programowalne typu PROM:



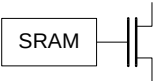
# Budowa pamięci programowalnej (2)

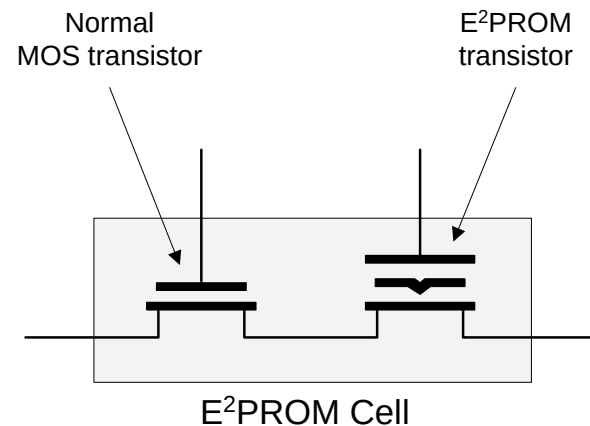
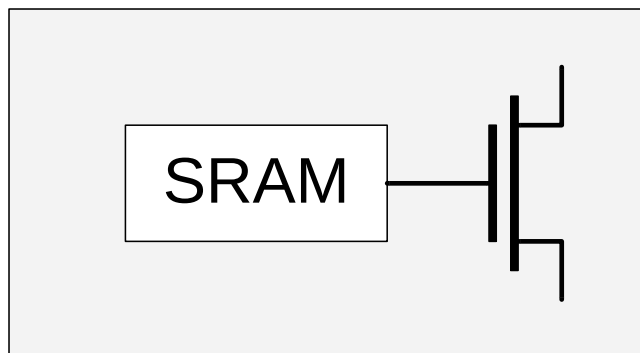
Pamięci programowalne typu EPROM i FLASH:





# Rodzaje połączeń i układów programowalnych

| Technology                | Symbol                                                                            | Predominantly associated with ... |
|---------------------------|-----------------------------------------------------------------------------------|-----------------------------------|
| Fusible-link              |  | SPLDs                             |
| Antifuse                  |  | FPGAs                             |
| EPROM                     |  | SPLDs and CPLDs                   |
| E <sup>2</sup> PROM/FLASH |  | SPLDs and CPLDs (some FPGAs)      |
| SRAM                      |  | FPGAs (some CPLDs)                |



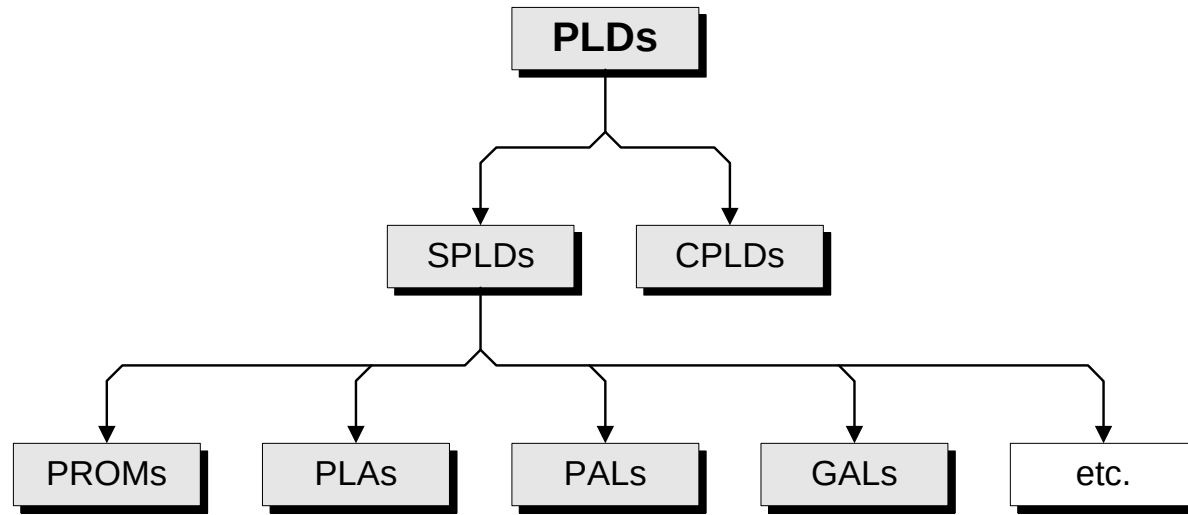
# Rodzaje połączeń i ich właściwości (1)

| Połączenie            | Reprogramowalność                             | Powierzchnia | Rezystancja           | Pojemność               |
|-----------------------|-----------------------------------------------|--------------|-----------------------|-------------------------|
| EPROM                 | poza układem (okienko)<br>lub brak (OTP)      | mała         | $\approx 1000 \Omega$ | $\approx 10 \text{ fF}$ |
| EEPROM                | w układzie                                    | średnia      | $\approx 1000 \Omega$ | $\approx 10 \text{ fF}$ |
| Antifuse<br>(PLICE)   | brak                                          | mała         | $\approx 300 \Omega$  | $\approx 3 \text{ fF}$  |
| Antifuse<br>(ViaLink) | brak                                          | mała         | $\approx 30 \Omega$   | $< 1 \text{ fF}$        |
| SRAM                  | w układzie – zawsze po<br>włączeniu zasilania | duża         | $\approx 1000 \Omega$ | $\approx 10 \text{ fF}$ |

# Rodzaje połączeń i ich właściwości (2)

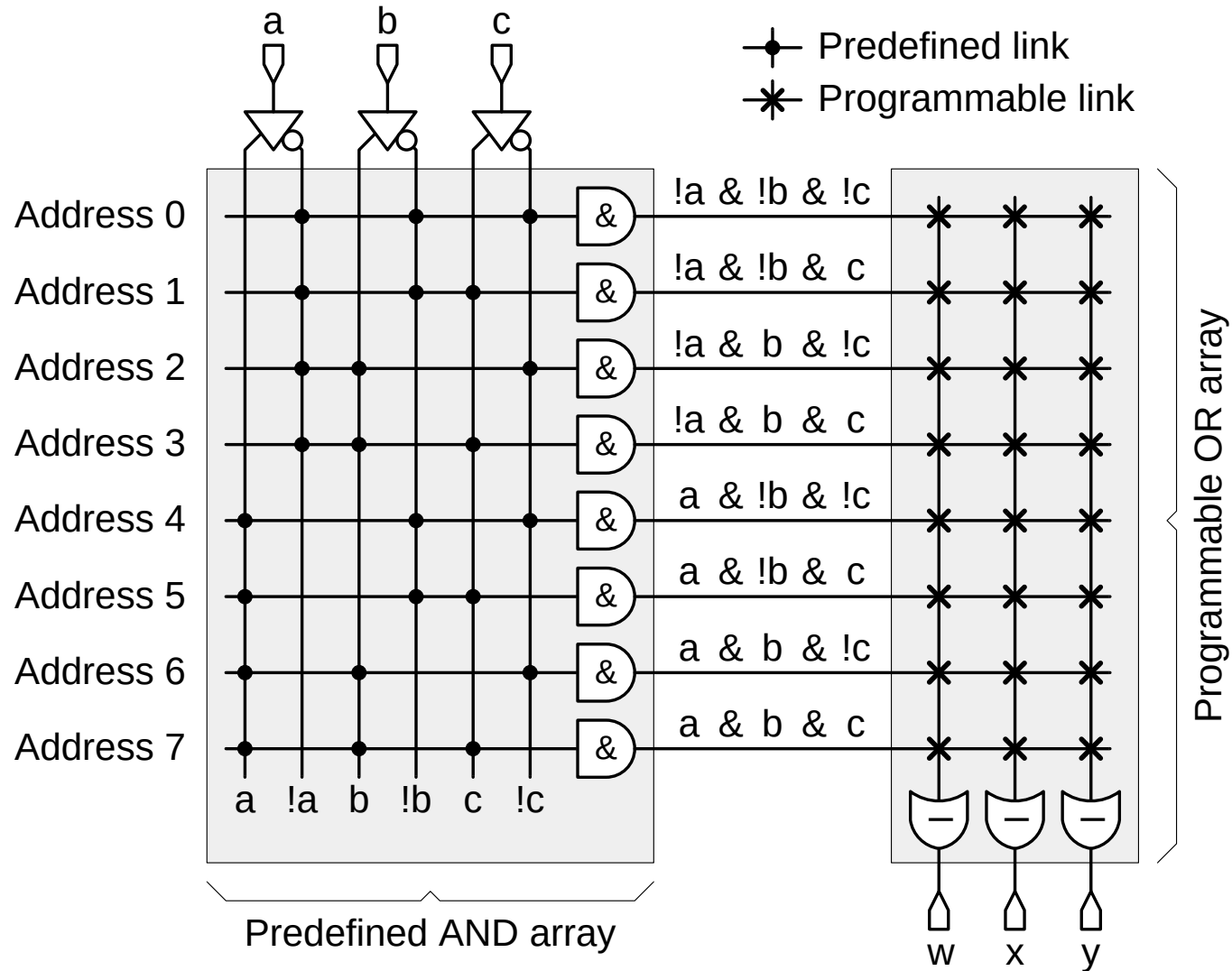
| Feature                                   | SRAM                                                    | Antifuse                       | E2PROM / FLASH                 |
|-------------------------------------------|---------------------------------------------------------|--------------------------------|--------------------------------|
| Technology node                           | State-of-the-art                                        | One or more generations behind | One or more generations behind |
| Reprogrammable                            | Yes (in system)                                         | No                             | Yes (in-system or offline)     |
| Reprogramming speed (inc. erasing)        | Fast                                                    | ----                           | 3x slower than SRAM            |
| Volatile (must be programmed on power-up) | Yes                                                     | No                             | No (but can be if required)    |
| Requires external configuration file      | Yes                                                     | No                             | No                             |
| Good for prototyping                      | Yes (very good)                                         | No                             | Yes (reasonable)               |
| Instant-on                                | No                                                      | Yes                            | Yes                            |
| IP Security                               | Acceptable (especially when using bitstream encryption) | Very Good                      | Very Good                      |
| Size of configuration cell                | Large (six transistors)                                 | Very small                     | Medium-small (two transistors) |
| Power consumption                         | Medium                                                  | Low                            | Medium                         |
| Rad Hard                                  | No                                                      | Yes                            | Not really                     |

# Podział układów programowalnych (bez FPGA)

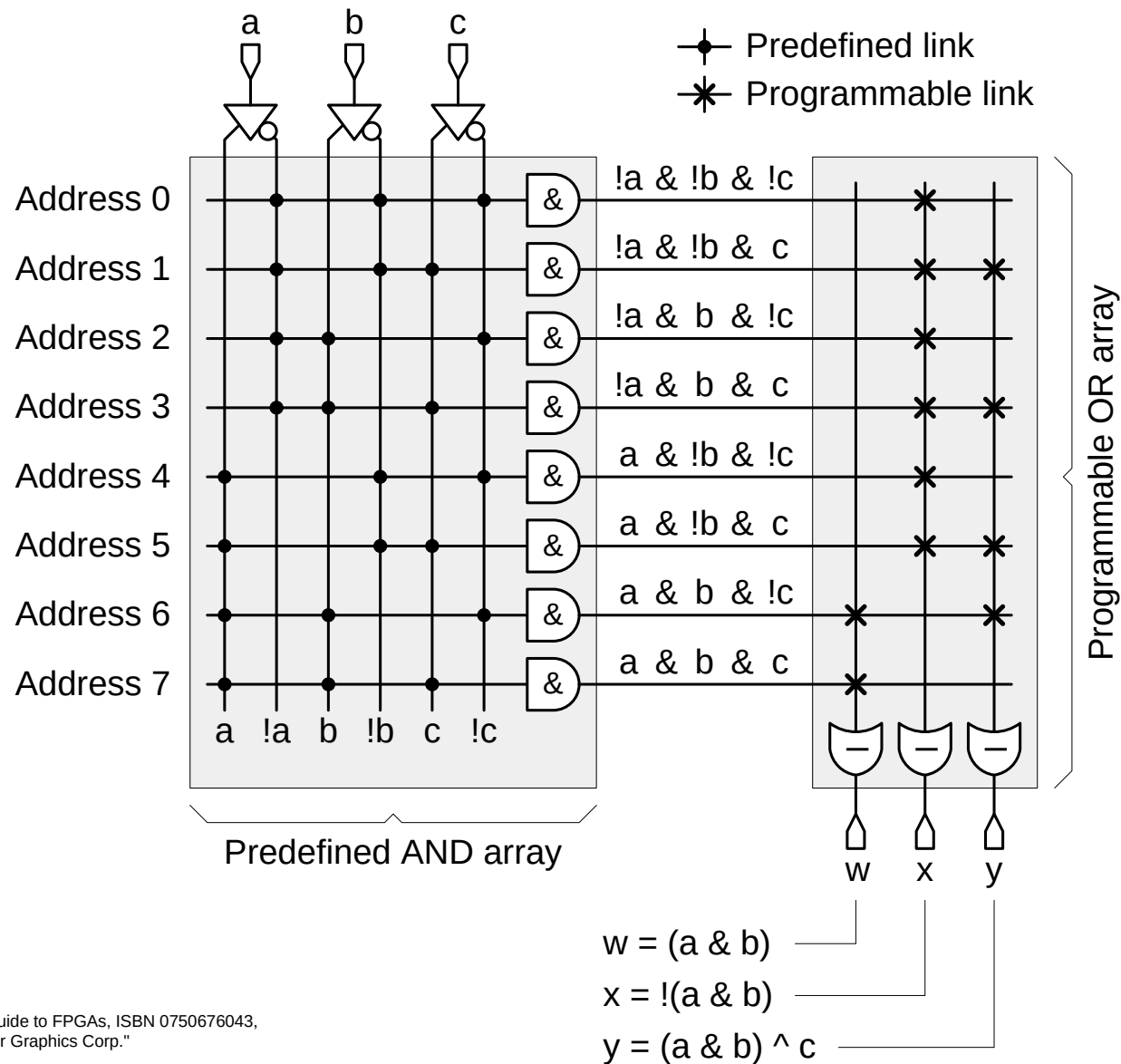


- SPLD – Simple Programmable Logic Devices  
(do 500 bramek, max. 24 makrokomórki, max. 40 linii I/O)
- CPLD – Complex Programmable Logic Devices
- PROM – Programmable Read Only Memory
- PLA – Programmable Logic Array  
1975 Signetics/Philips, układ PLS100. Duża cena, mała szybkość działania.
- PAL – Programmable Array Logic  
1978 Monolithic Memories/AMD
- GAL – Generic Array Logic  
1985 Lattice

# Zasada działania pamięci PROM



# Pamięć PROM jako układ logiczny



# Logika w układach programowalnych (1)

- Każdą funkcję boolowską  $F(A,B,C,D, \dots)$  można przedstawić w jednej z dwóch postaci (twierdzenie Shannona):

$$F(A,B,C,D,\dots) = A * F(1,B,C,D,\dots) + \sim A * F(0,B,C,D,\dots)$$

$$F(A,B,C,D,\dots) = (A + F(0,B,C,D,\dots)) * (\sim A + F(1,B,C,D,\dots))$$

- Z powyższego wynika że każdą funkcję logiczną możemy przedstawić w postaci sumy tzw. mintermów:

$$F = m_0 + m_3 + m_7$$

lub iloczynu makstermów:

$$G = M_1 * M_2 * M_4 * M_5 * M_6$$

gdzie mintermy to iloczyny zmiennych wejściowych lub ich negacji,  
zaś makstermy to sumy zmiennych wejściowych lub ich negacji.

$$\text{Np.: } m_0 = \sim a * \sim b * \sim c \quad m_3 = \sim a * b * c \quad m_7 = a * b * c$$

$$\text{Np.: } M_1 = a + b + \sim c \quad M_2 = a + \sim b + c \quad M_4 = \sim a + b + c \quad M_5 = \sim a + b + \sim c \quad M_6 = \sim a + \sim b + c$$

- Optymalizacja funkcji pozwala na redukcję zbędnych termów:

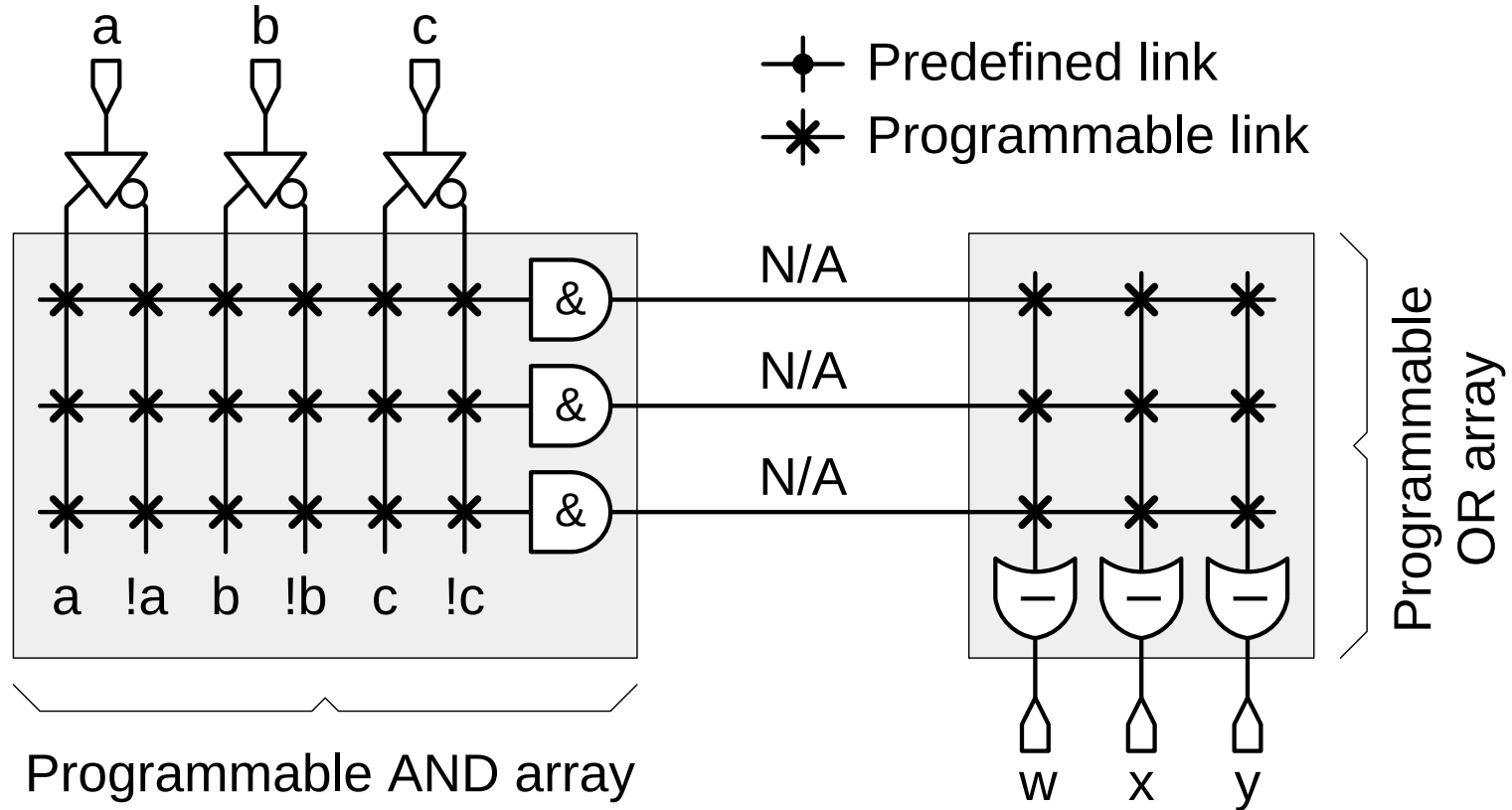
$$\text{Np.: } F = \sim a * \sim b * \sim c + b * c$$

# Logika w układach programowalnych (2)

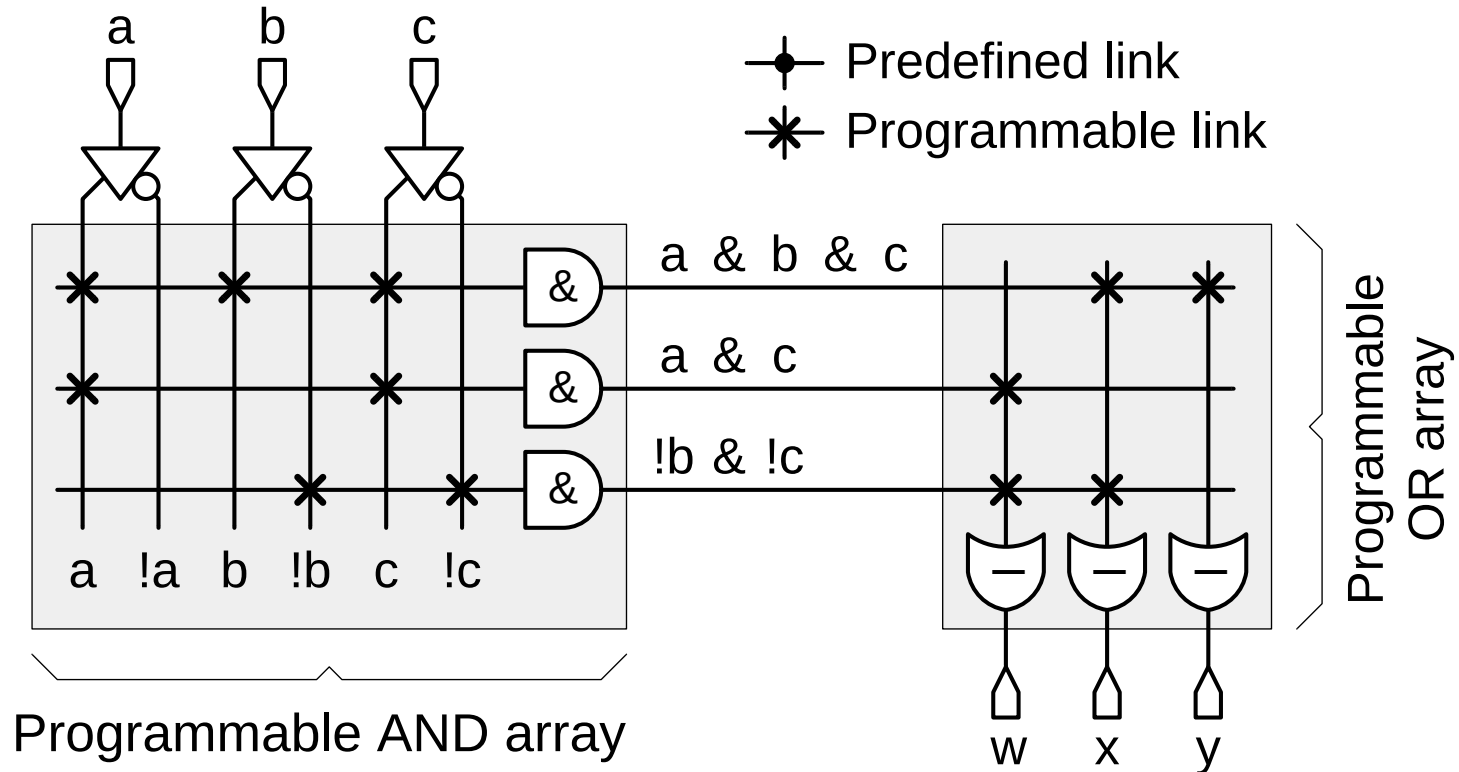
- Funkcje realizowane w praktyce dają się łatwo przedstawić w postaci sumy mintermów lub iloczynu makstermów.
- W praktyce częściej stosuje się rozkład na mintermy.
- Dzięki takiemu rozkładowi i optymalizacji można zrealizować typowe funkcje logiczne w układach programowalnych znacznie mniejszym kosztem niż za pomocą pamięci PROM.
- Istnieje pewna klasa funkcji których reprezentacja w postaci iloczynu mintermów lub sumy makstermów nie daje możliwości optymalizacji (powstaje konieczność wykorzystania bardzo dużej liczby termów): są to funkcje typu XOR i XNOR. W tym przypadku stosuje się specjalne układy programowalne wyposażone w funktry XOR lub XNOR.
- W typowych zastosowaniach układów programowalnych funkcje XOR i XNOR występują rzadko.
- Typowe zastosowania układów SPLD to: dekodery adresowe, bramy I/O, logika sklejaąca (glue logic), konwertery poziomów logicznych.



# Zasada działania układów PLA



# Układ PLA zaprogramowany do funkcji logicznej

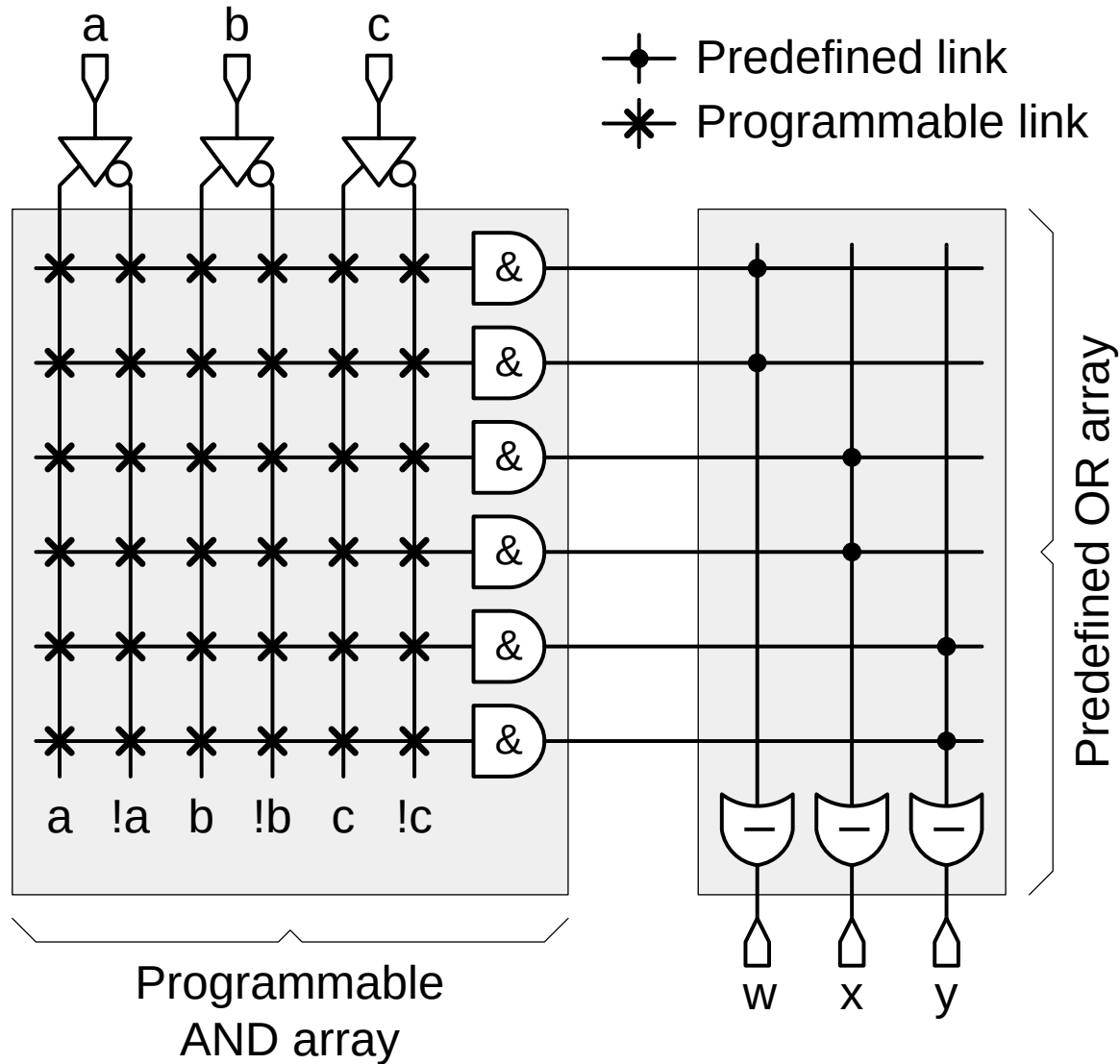


$$w = (a \& c) \mid (\neg b \& \neg c)$$

$$x = (a \& b \& c) \mid (\neg b \& \neg c)$$

$$y = (a \& b \& c)$$

# Zasada działania układu PAL



# Układy programowalne SPLD

# Układy programowalne SPLD

- Układy pamięci (E)PROM można interpretować jako układy programowalne z programowalną matrycą OR. Za pomocą układów (E)PROM można zrealizować każdą funkcję logiczną o liczbie wejść równej lub mniejszej od liczby linii adresowych pamięci. Pamięci (E)PROM są jednak powolne i nie nadają się do realizacji funkcji logicznych z dużą liczbą wejść.
- Układy typu PLA posiadają programowalne matryce AND i OR. Daje to możliwość wielokrotnego wykorzystania termów do syntezy kilku funkcji logicznych.
- Układy typu PAL (GAL) posiadają tylko programowalną matrycę AND, ale są szybsze od układów PLA.

Układy PLA okazały się zbyt powolne i drogie. Ze względu na właściwości i cenę najczęściej stosowano układy PAL które następnie zostały zastąpione przez układy GAL (wykorzystujące pamięć EEPROM – mogą być więc reprogramowalne). Wada układów PAL polegająca na stałej liczbie termów przypisanych do końcówki wyjściowej została częściowo usunięta w zaawansowanych układach GAL gdzie możliwe jest „pożyczanie” termów z sąsiednich komórek.

# Układy programowalne SPLD typu GAL (1)

Przykład: GAL16V8

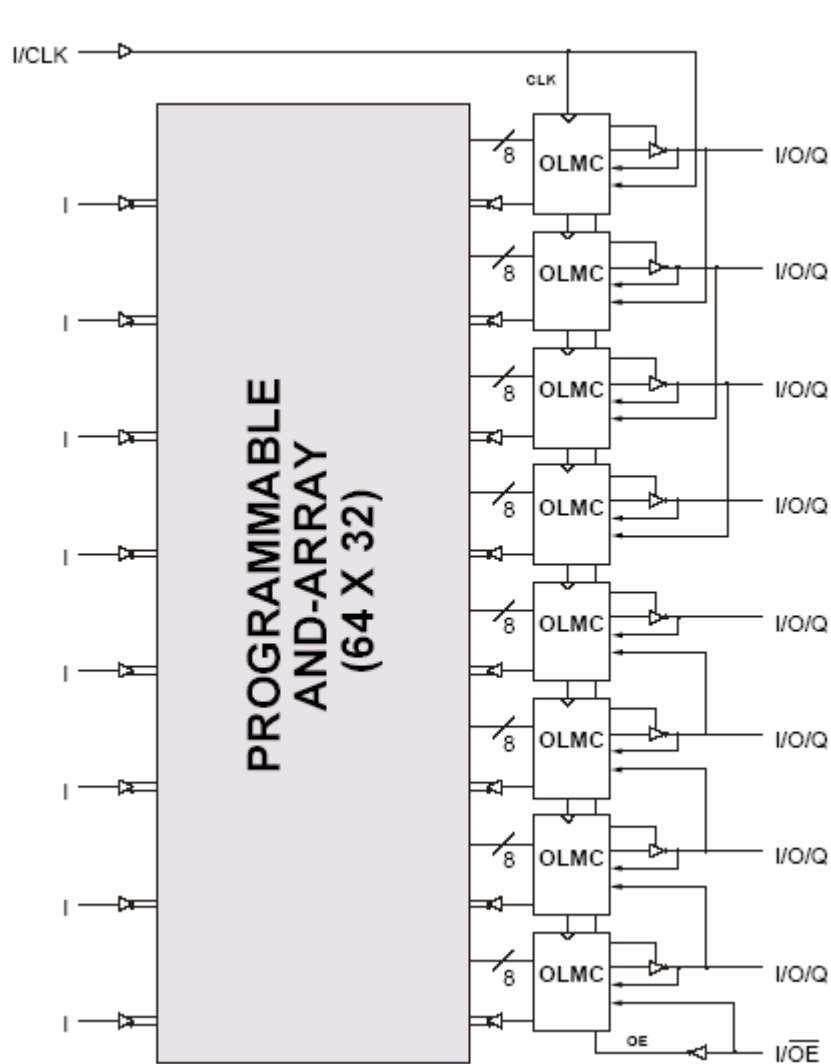
- czas propagacji: 3.5 ns,
- high performance CMOS EEPROM technology,
- szybkie kasowanie (<100ms),
- architektura uniwersalna (generic) może emulować każdą z rodzin układów PAL,
- 100 cykli kasowania i zapisu z gwarancją 20 letniego czasu zachowania pamięci.

Wybór trybu pracy (syntezer ABEL):

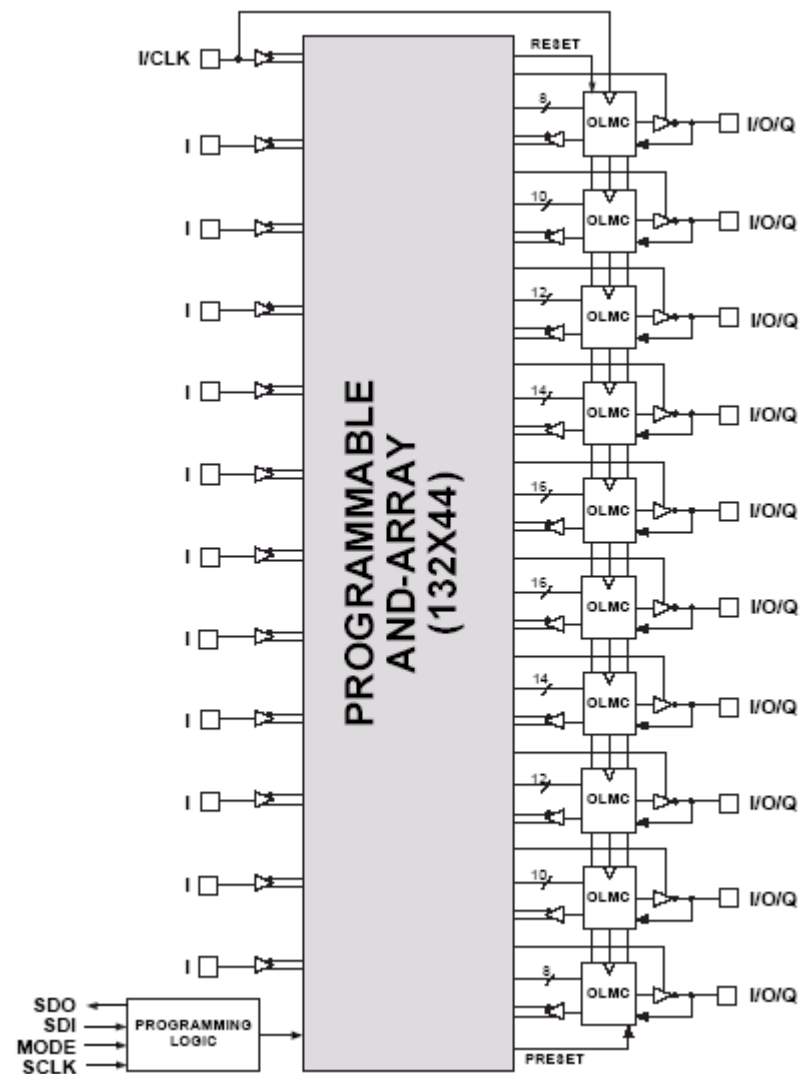
- P16V8R (tryb registered),
- P16V8C (tryb complex),
- P16V8AS (tryb simple),
- P16V8 (tryb auto).

| PAL Architectures<br>Emulated by GAL16V8 | GAL16V8<br>Global OLMC Mode |
|------------------------------------------|-----------------------------|
| 16R8                                     | Registered                  |
| 16R6                                     | Registered                  |
| 16R4                                     | Registered                  |
| 16RP8                                    | Registered                  |
| 16RP6                                    | Registered                  |
| 16RP4                                    | Registered                  |
| 16L8                                     | Complex                     |
| 16H8                                     | Complex                     |
| 16P8                                     | Complex                     |
| 10L8                                     | Simple                      |
| 12L6                                     | Simple                      |
| 14L4                                     | Simple                      |
| 16L2                                     | Simple                      |
| 10H8                                     | Simple                      |
| 12H6                                     | Simple                      |
| 14H4                                     | Simple                      |
| 16H2                                     | Simple                      |
| 10P8                                     | Simple                      |
| 12P6                                     | Simple                      |
| 14P4                                     | Simple                      |
| 16P2                                     | Simple                      |

# Układy programowalne SPLD typu GAL (2)

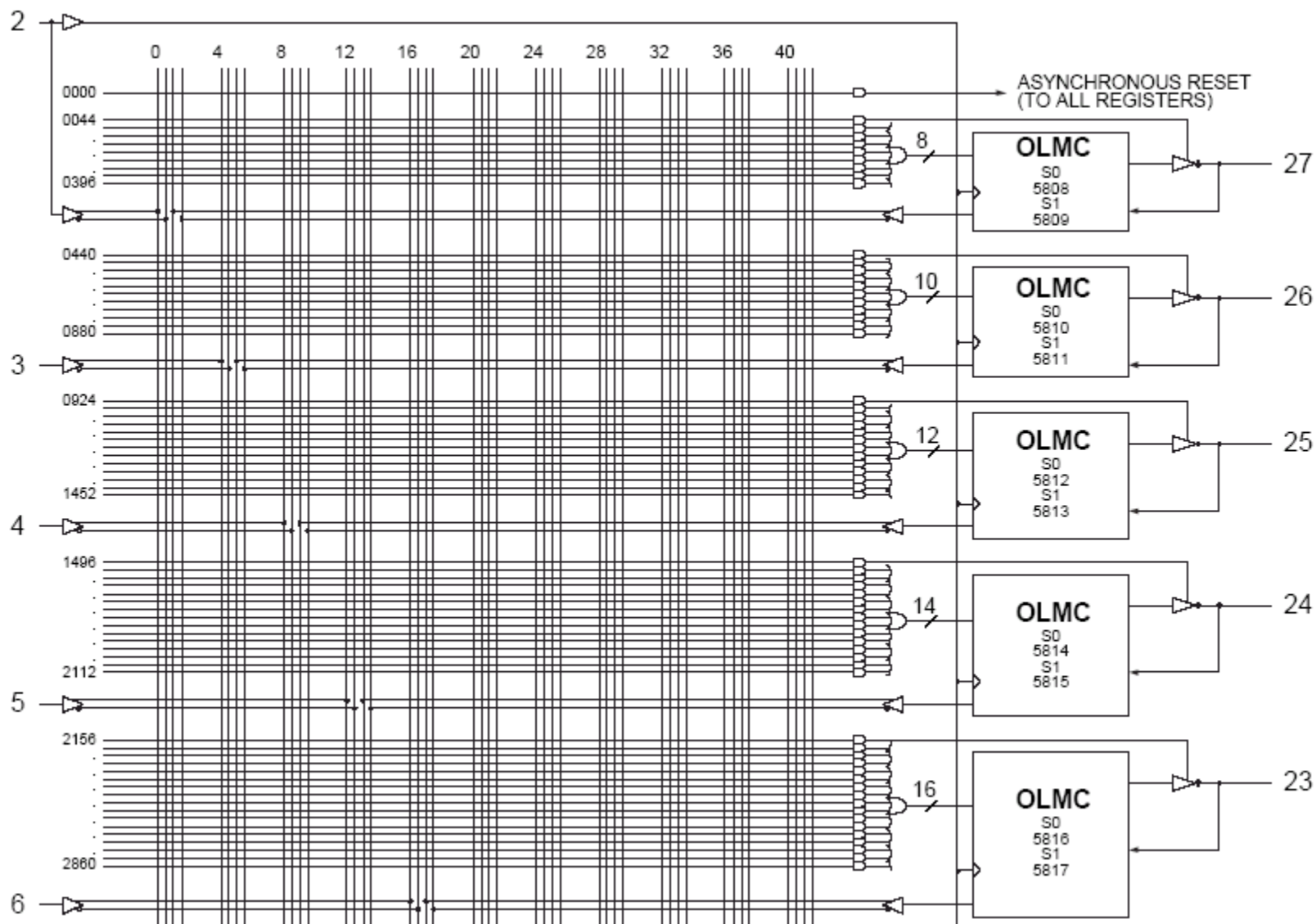


GAL 16V8



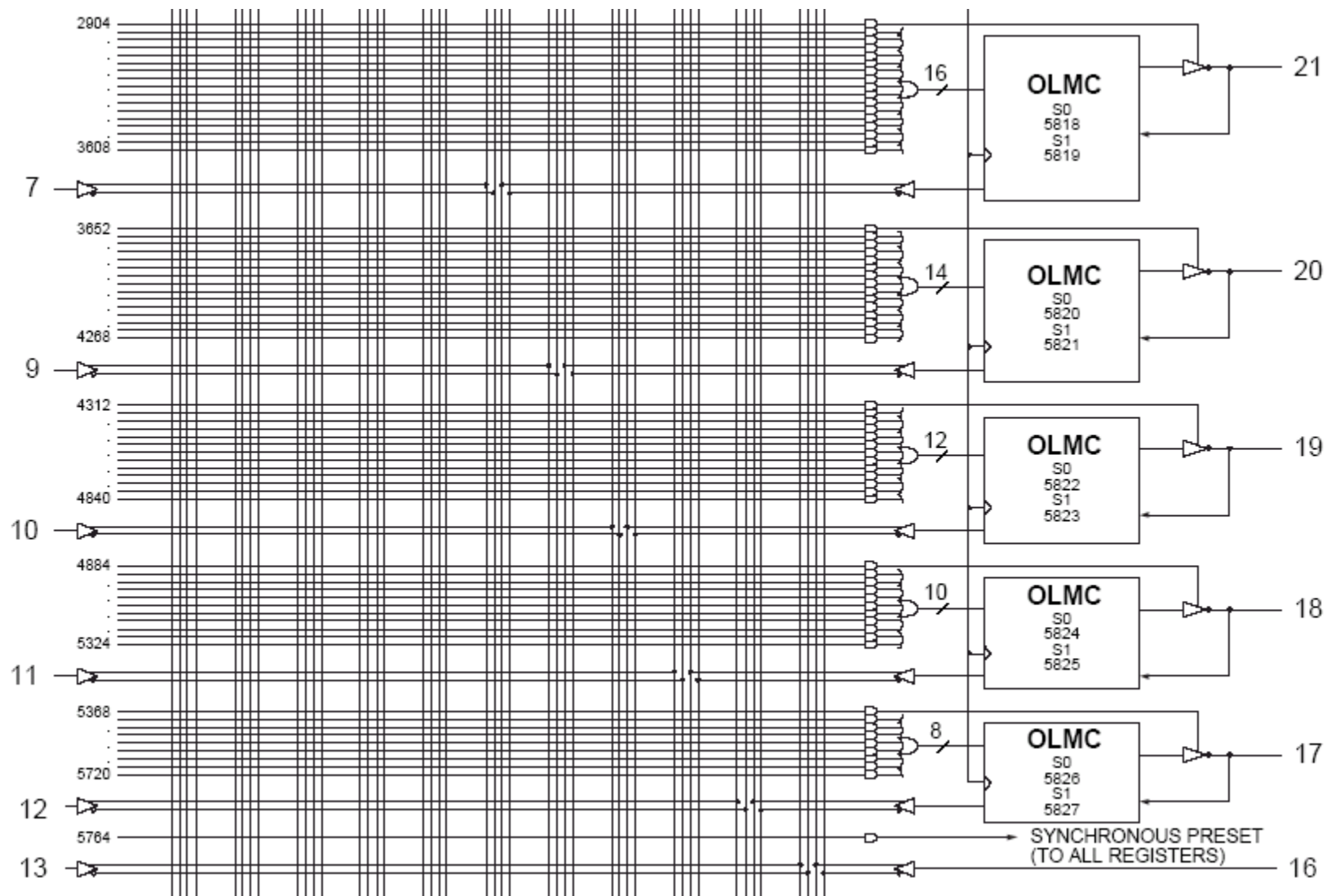
ispGAL 22V10

# Budowa układów SPLD GAL 22V10 (1)

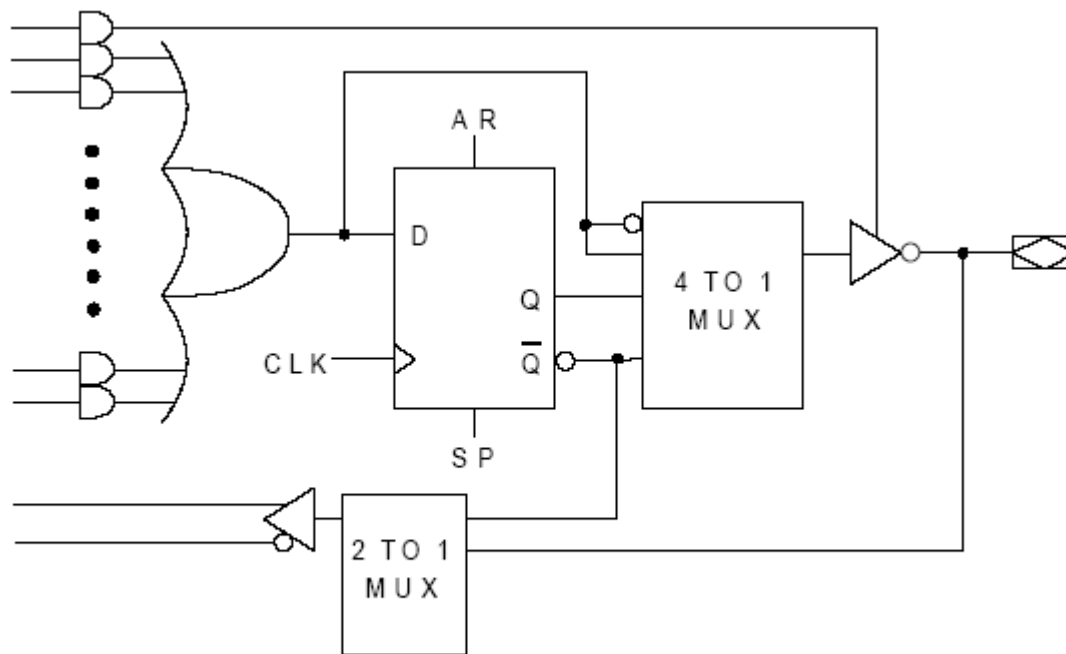




# Budowa układów SPLD GAL 22V10 (2)



# Budowa układów SPLD GAL 22V10 (3)



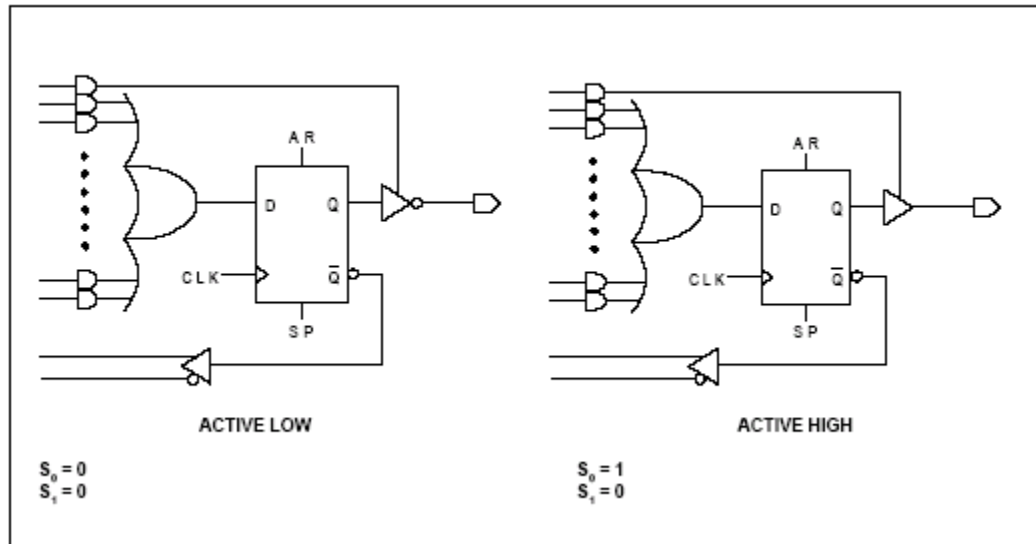
ispGAL22V10 OUTPUT LOGIC MACROCELL (OLMC)

Budowa pojedynczej wyjściowej komórki logicznej.

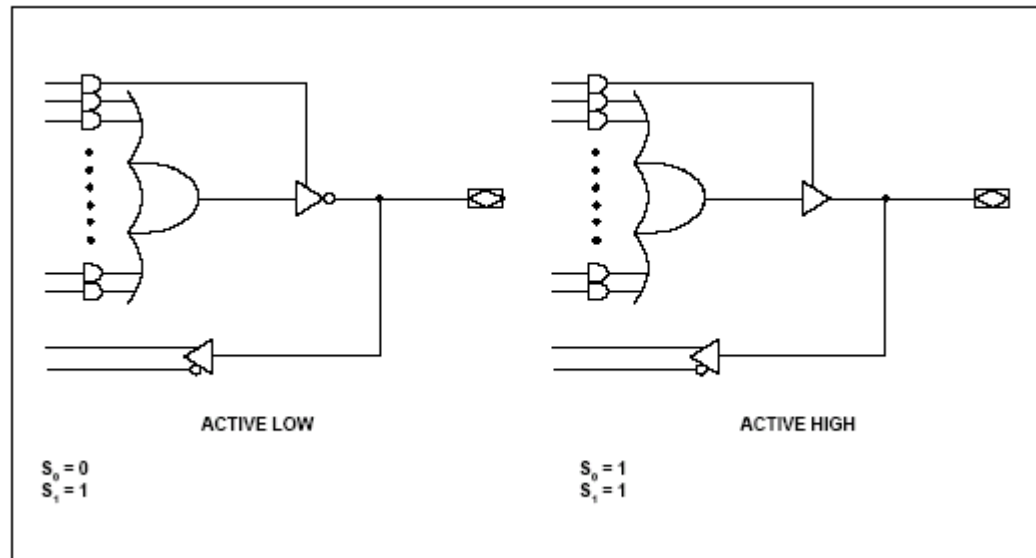
# Budowa układów SPLD GAL 22V10 (4)

Możliwe tryby pracy wyjściowej komórki logicznej (OLMC).

## Registered Mode



## Combinatorial Mode



# Właściwości układów SPLD GAL 22V10 (1)

- Podpis elektroniczny: 64 bity. Dostępne dzięki dodatkowym bezpiecznikom. Możliwy odczyt także po zaprogramowaniu security-cell. Zastosowanie: numer wersji, firmware'u, produktu, etc.

|                |        |        |        |                      |        |        |        |                |  |
|----------------|--------|--------|--------|----------------------|--------|--------|--------|----------------|--|
| 5828, 5829 ... |        |        |        | Electronic Signature |        |        |        | ... 5890, 5891 |  |
| Byte 7         | Byte 6 | Byte 5 | Byte 4 | Byte 3               | Byte 2 | Byte 1 | Byte 0 |                |  |
| M              | L      |        |        |                      |        |        |        |                |  |
| S              | S      |        |        |                      |        |        |        |                |  |
| B              | B      |        |        |                      |        |        |        |                |  |

- Komórka bezpieczeństwa (security-cell) zabezpiecza przed kopiowaniem konfiguracji układu programowalnego. Po zaprogramowaniu dostęp do odczytu stanu zaprogramowanych komórek jest zabroniony. Komórkę bezpieczeństwa można skasować tylko kasując cały układ.
- ispGAL22V10 jest wyposażony w układ zabezpieczający przed efektem latch-up (ujemna polaryzacja podłoża, n-channel pullup na wyjściu).
- ispGAL22V10 posiada funkcję „In-System Programmable”. Dzięki zintegrowaniu układów wytwarzających podwyższone napięcie programujące możliwe jest programowanie (i reprogramowanie) bez wyjmowania układu z płytki (przy zasilaniu 5V). Programowanie odbywa się szeregowo poprzez 4-ro przewodowy interface (SDI, SDO, SCLK, MODE).
- Funkcja „Output Register Preload” – programowanie początkowego stanu rejestrów w celu testowania maszyn stanu.

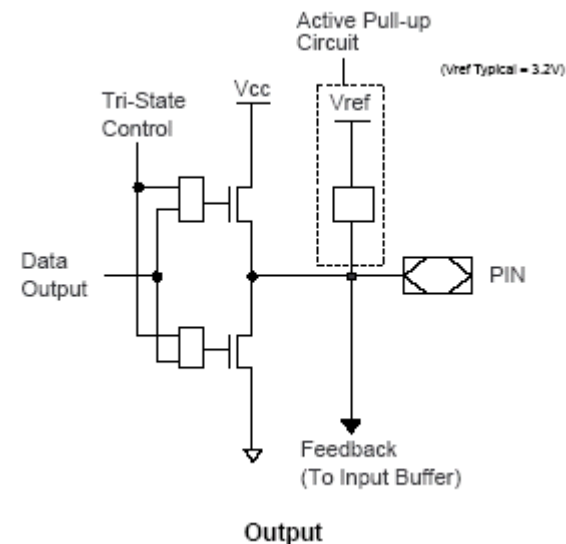
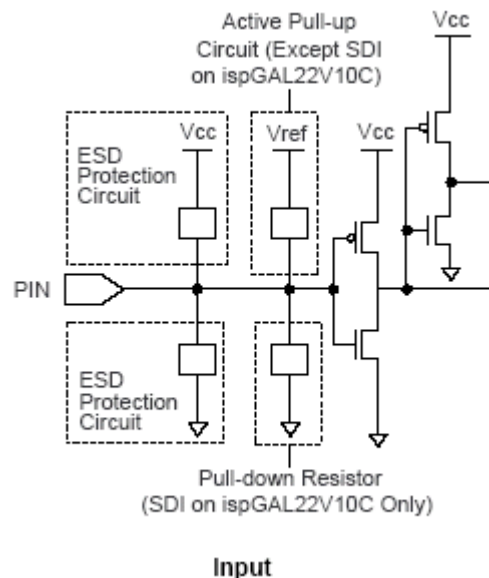
# Właściwości układów SPLD GAL 22V10 (2)

Właściwości komórek wejściowych:

- wysoka impedancja,
- active pullup (zalecenie: stosować dodatkowy zewnętrzny pullup/pulldown).

Właściwości komórek wyjściowych:

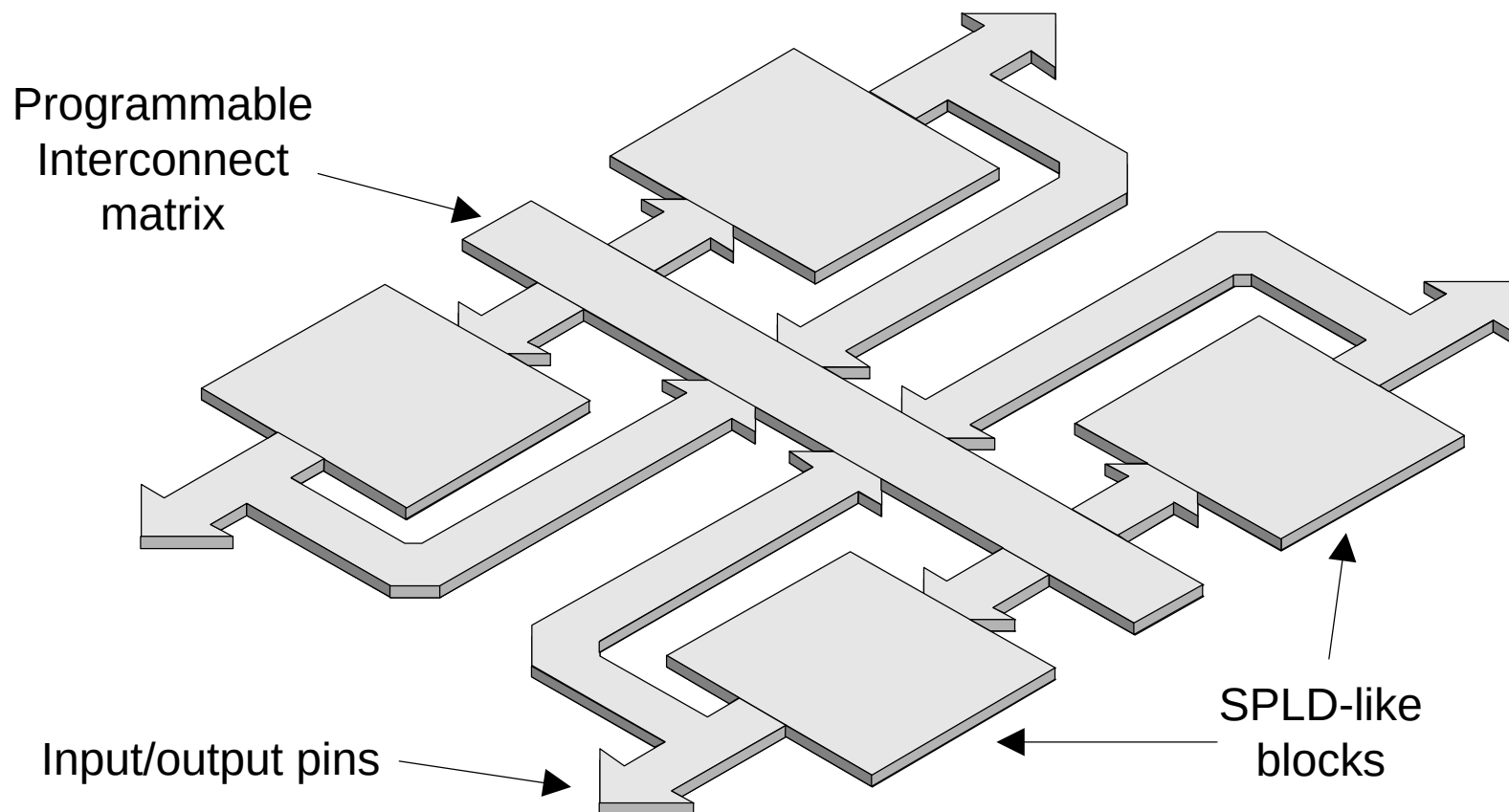
- power-on reset: rejestry są zerowane automatycznie podczas włączania zasilania, wyjścia rejestrowe są zerowane albo ustawiane w zależności od włączenia opcjonalnego inwertera wyjściowego.
- warunki prawidłowego zerowania power-on: monotoniczna zmiana napięcia zasilania, nie pracujący podczas zerowania zegar.



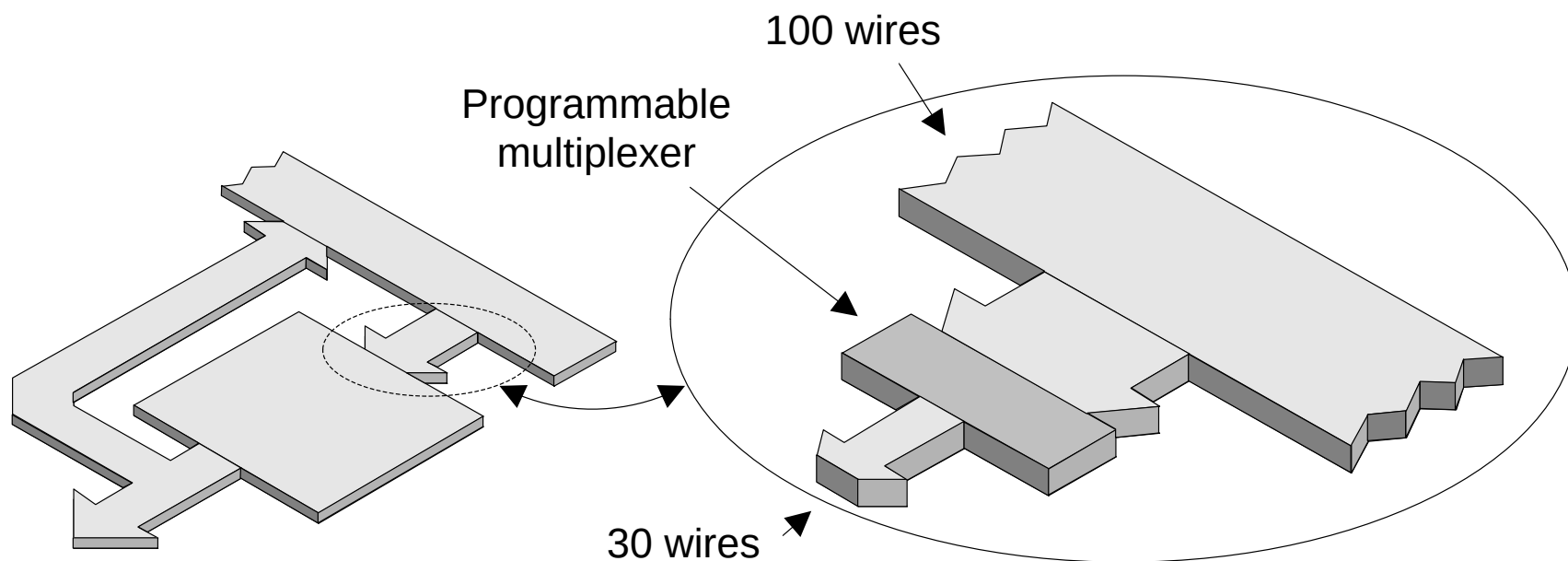
# Układy programowalne CPLD

# Układy CPLD (1)

Układy typu Complex Programmable Logic Device składają się z bloków typu SPLD połączonych dodatkową wewnętrzną programowalną magistralą połączeniową:



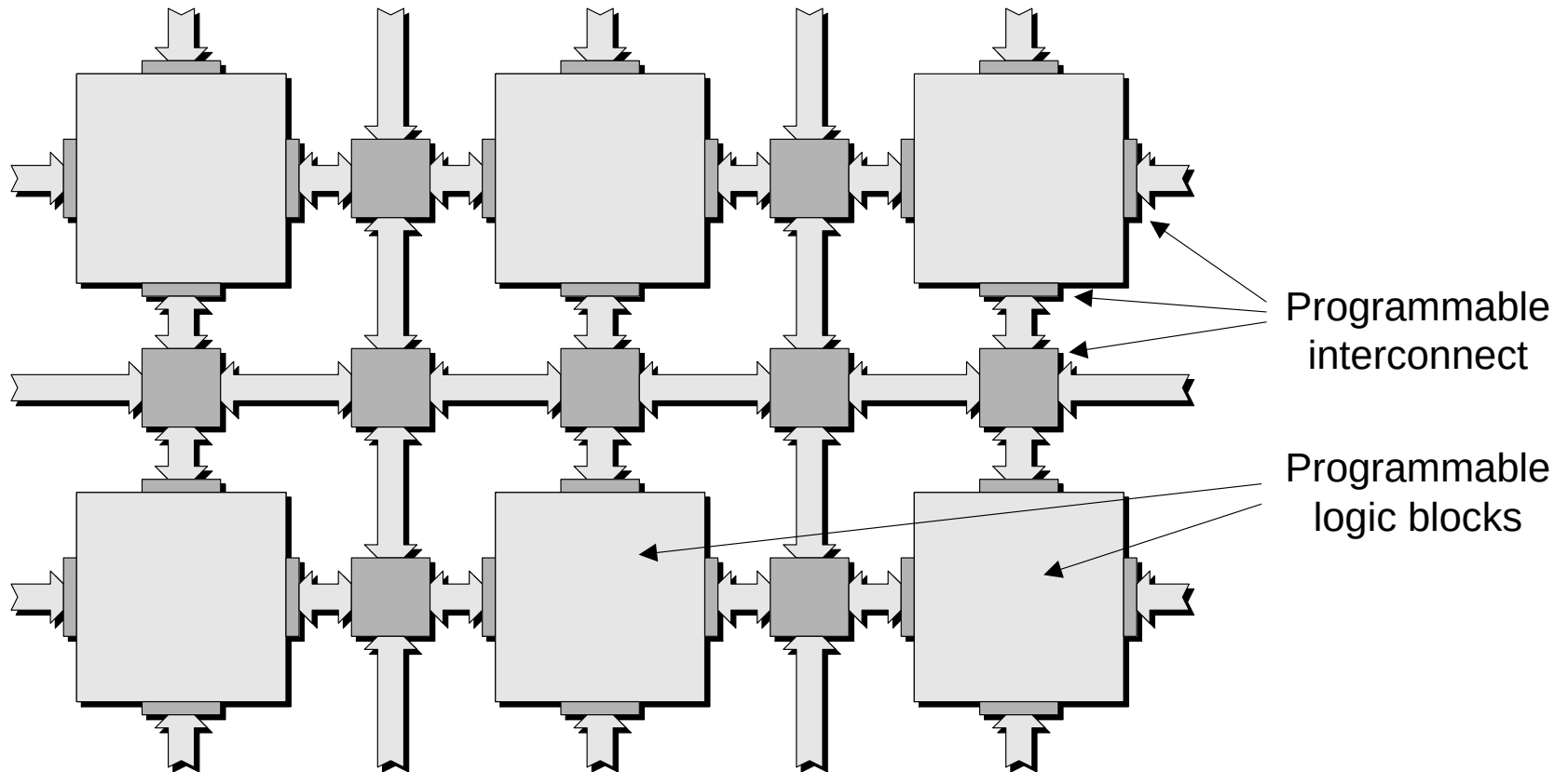
# Układy CPLD (2)





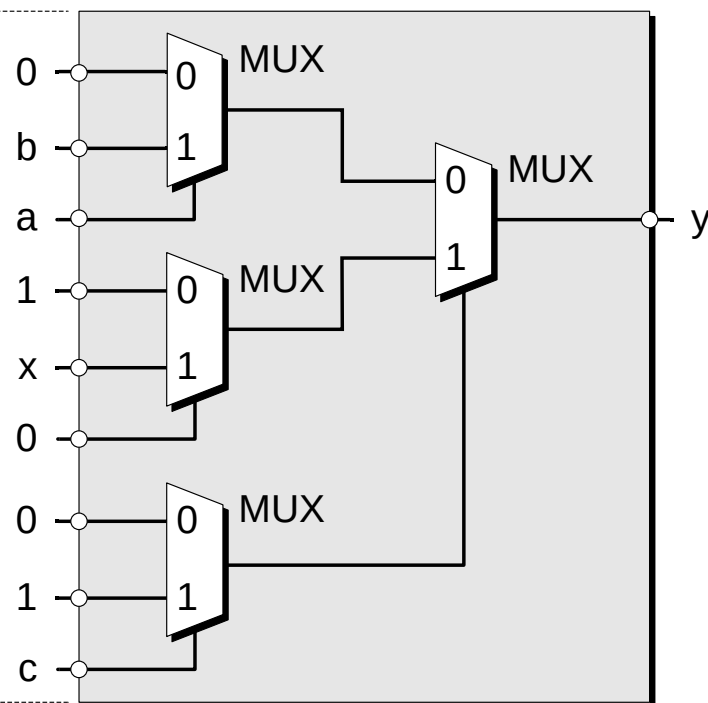
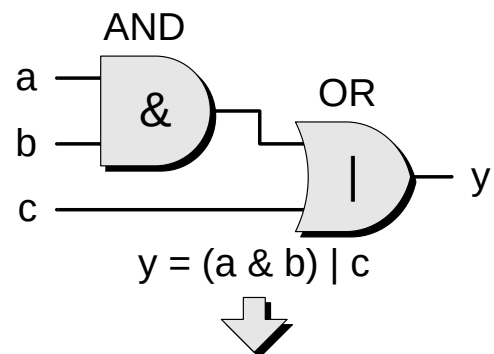
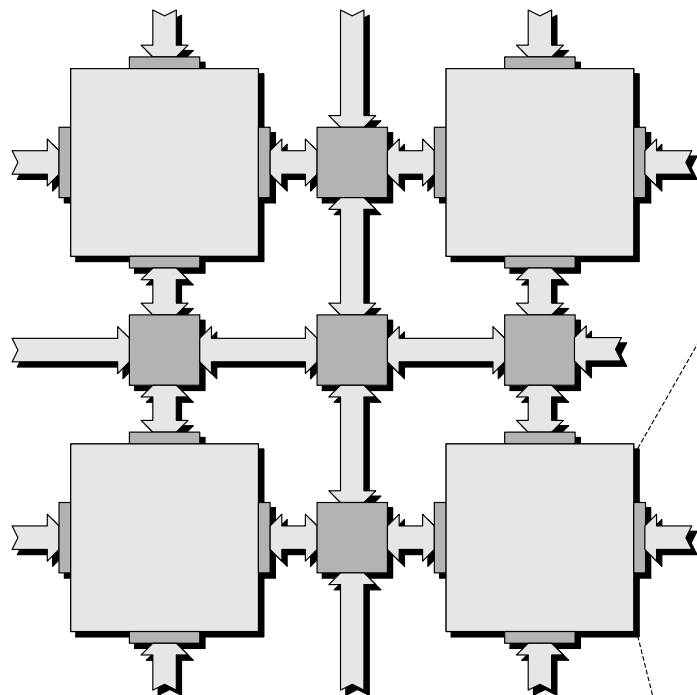
# Układy programowalne FPGA

# Budowa układów FPGA (1)



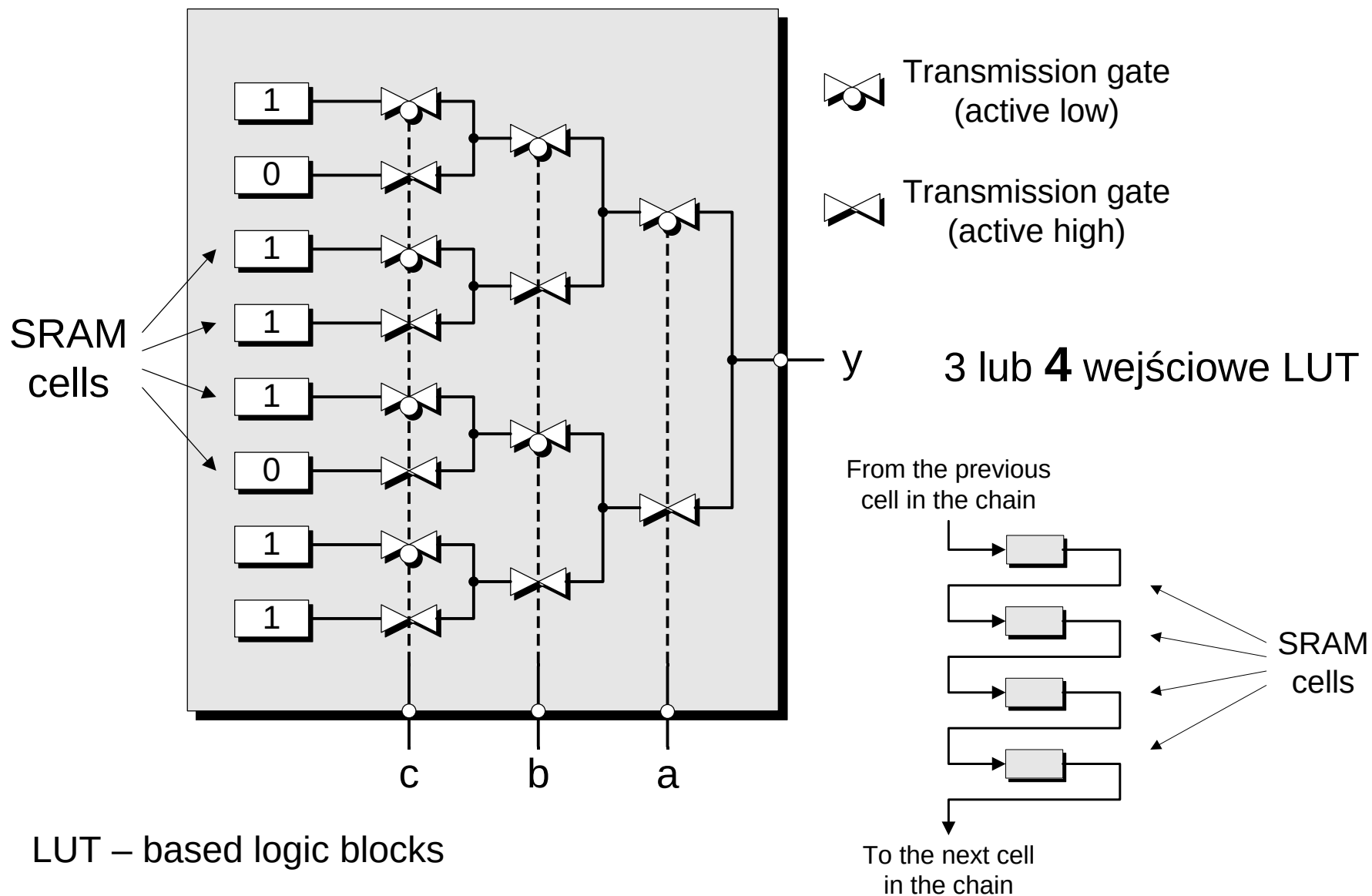
- Coarse grained architectures
- Medium grained architectures
- Fine grained architectures
- Local routing
- General purpose routing
- Global routing
- Dedicated routing

# Budowa układów FPGA (2)

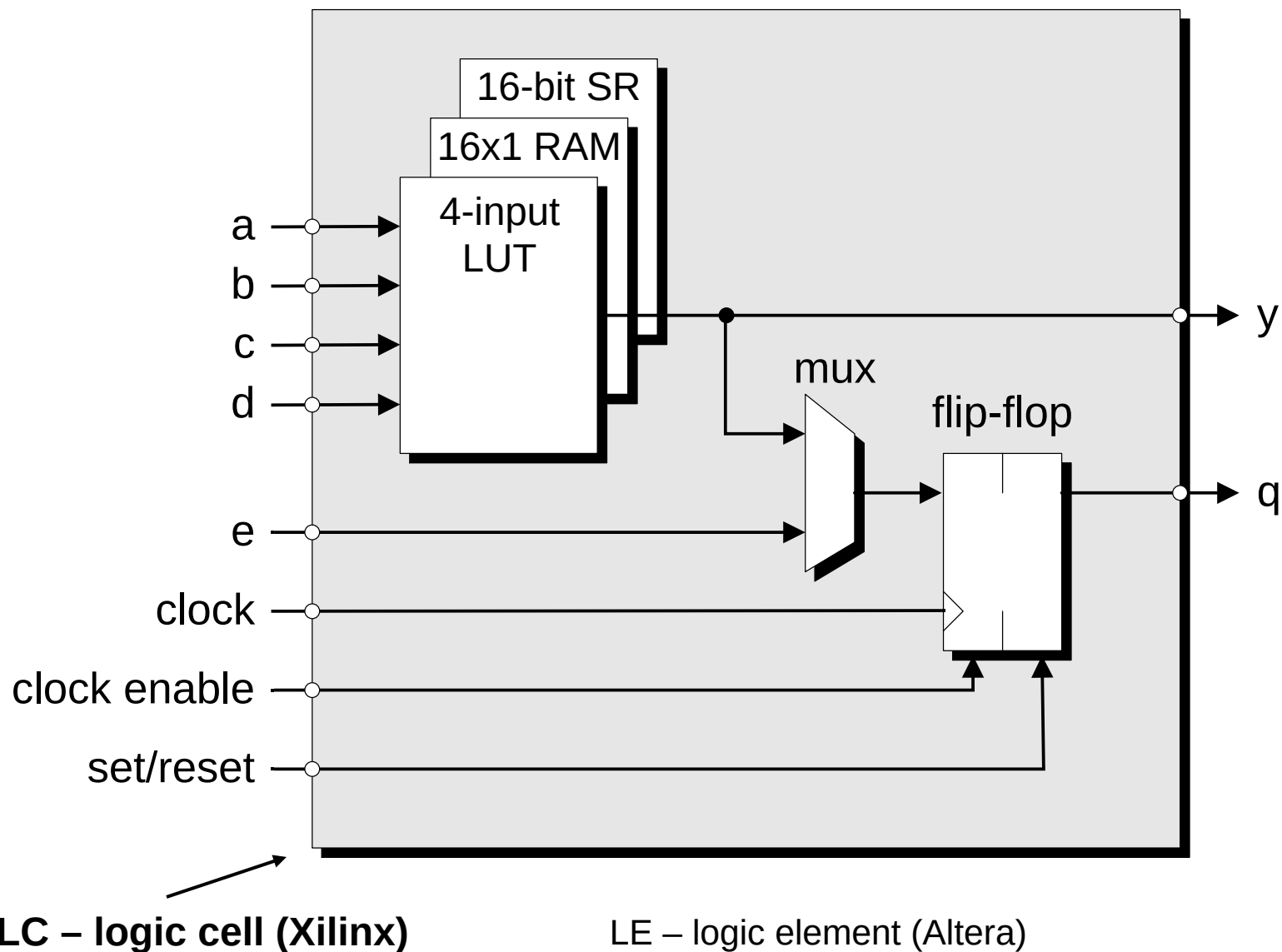


MUX – based logic blocks

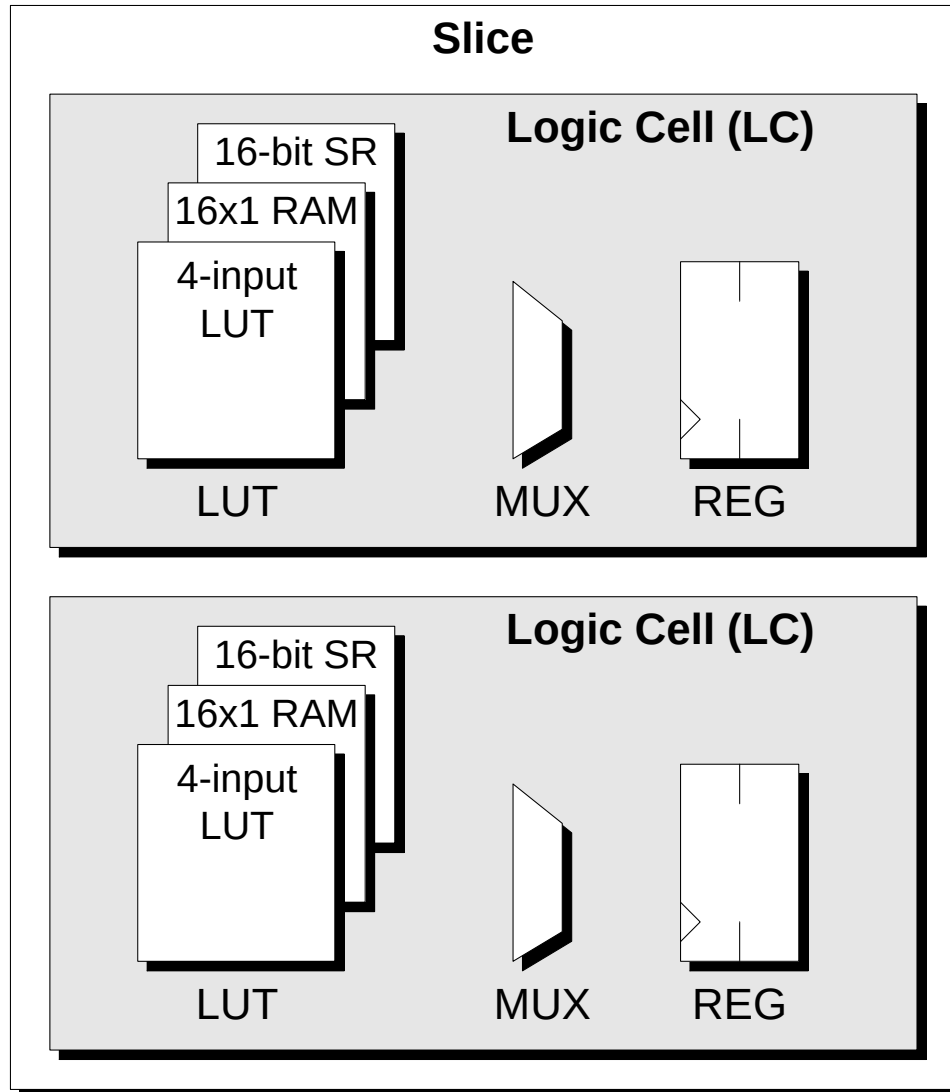
# Budowa układów FPGA (3)



# Budowa układów FPGA (4)

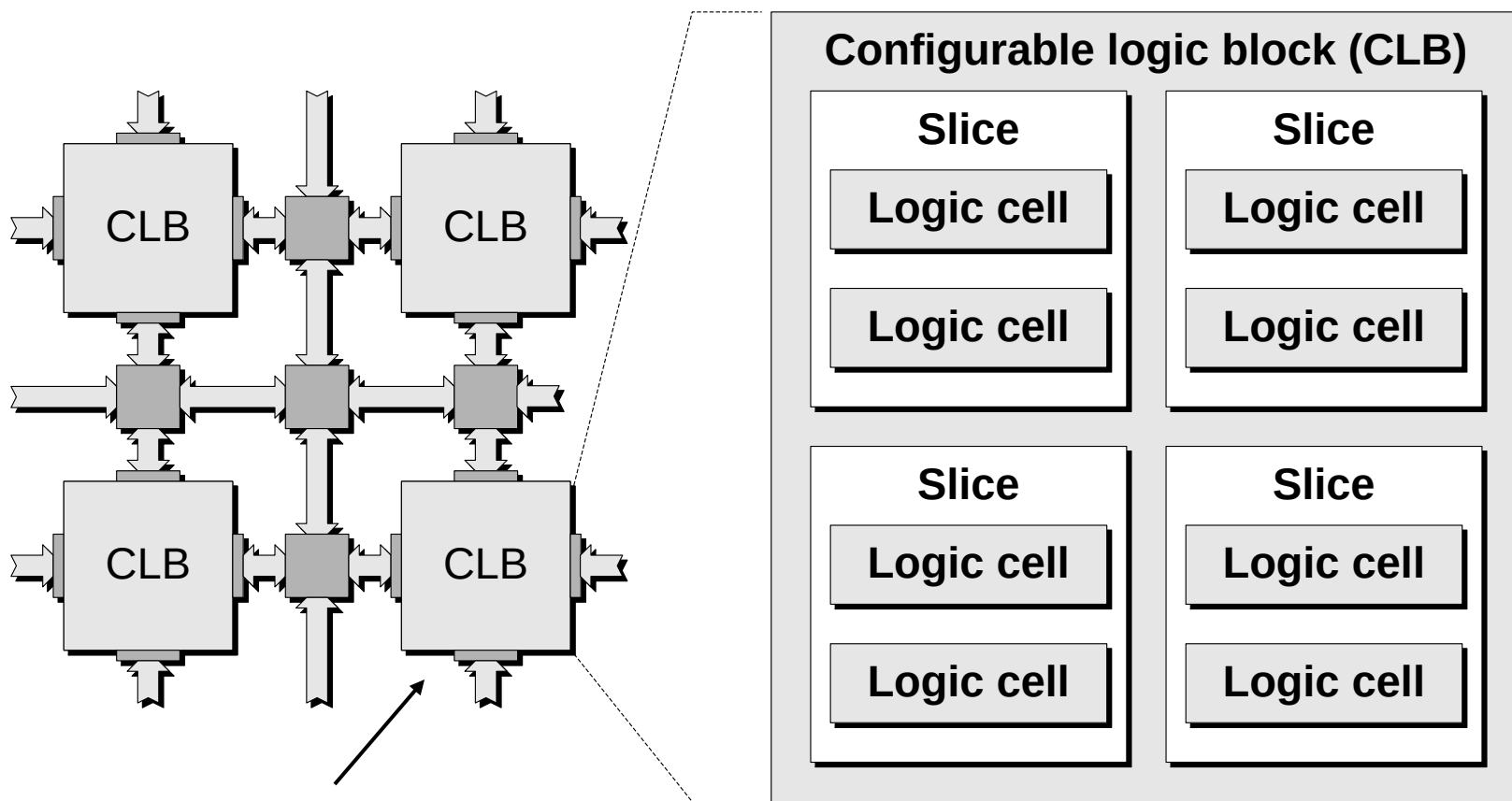


# Budowa układów FPGA (5)



Slice (Xilinx) – łączy dwie LC w większą strukturę (wspólny zegar i inne sygnały).

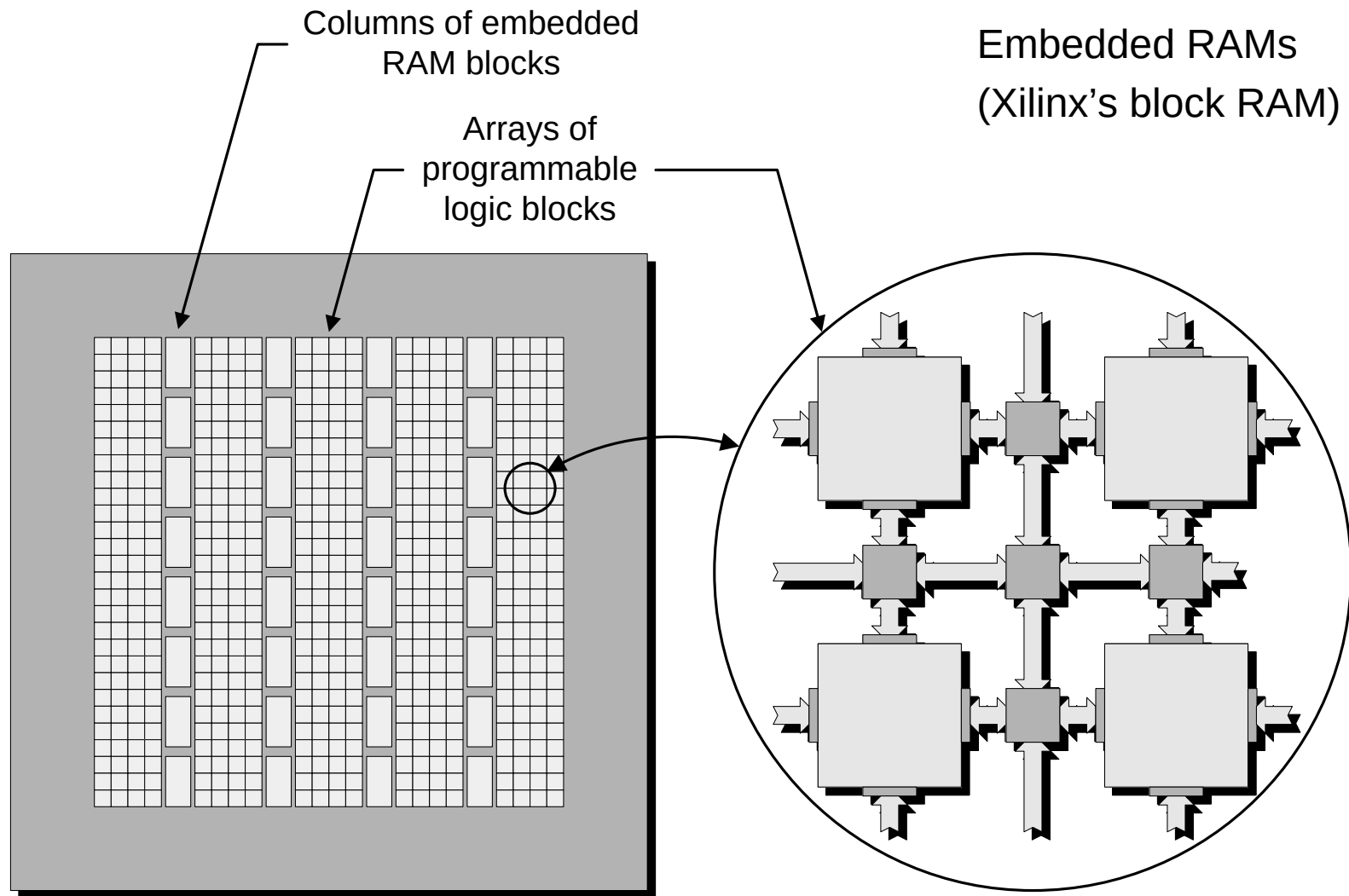
# Budowa układów FPGA (6)



**CLB (Xilinx) – Configurable Logic Block** - podstawowy blok logiczny w macierzy FPGA.

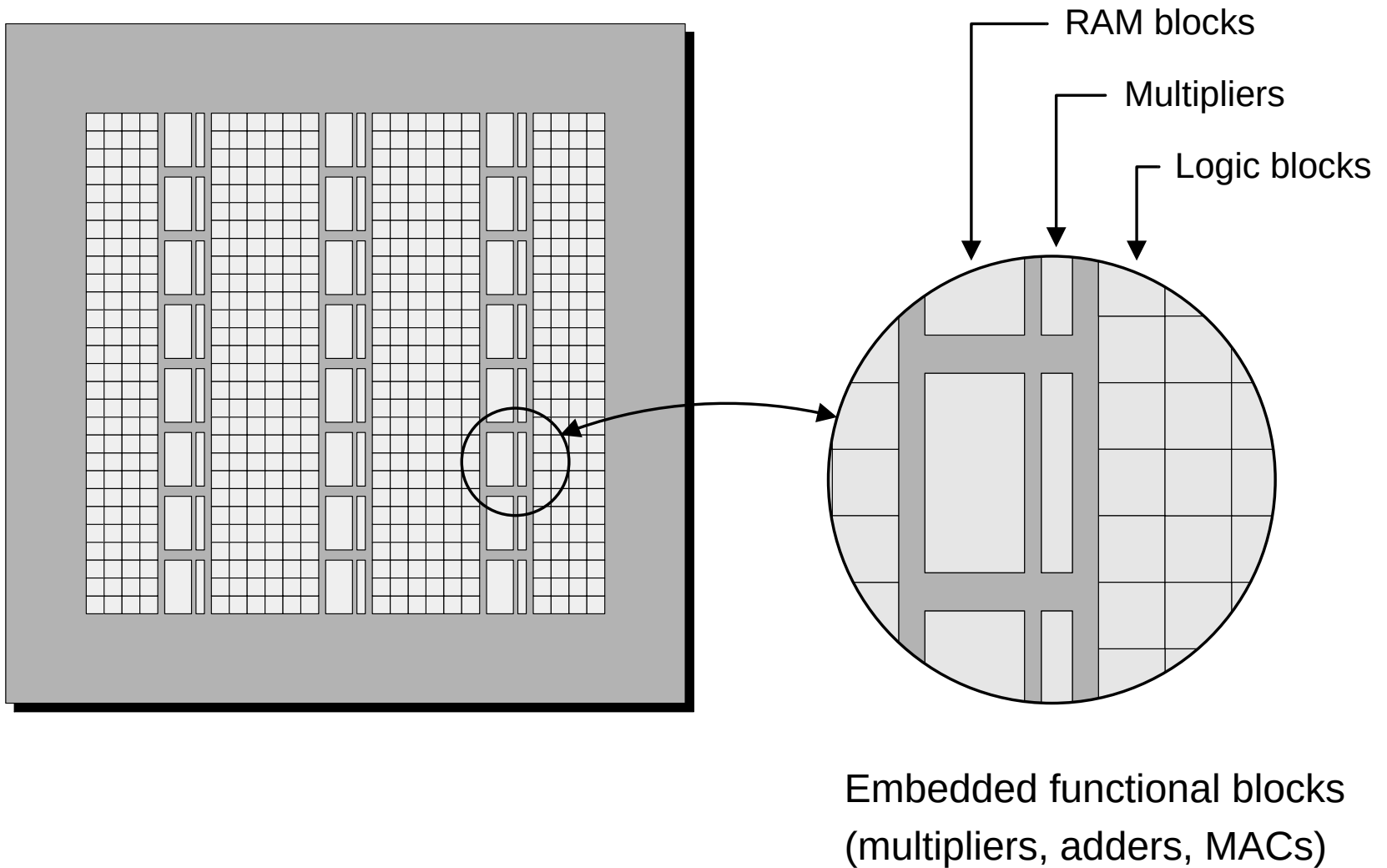
LAB (Altera) – Logic Array Block

# Budowa układów FPGA (7)

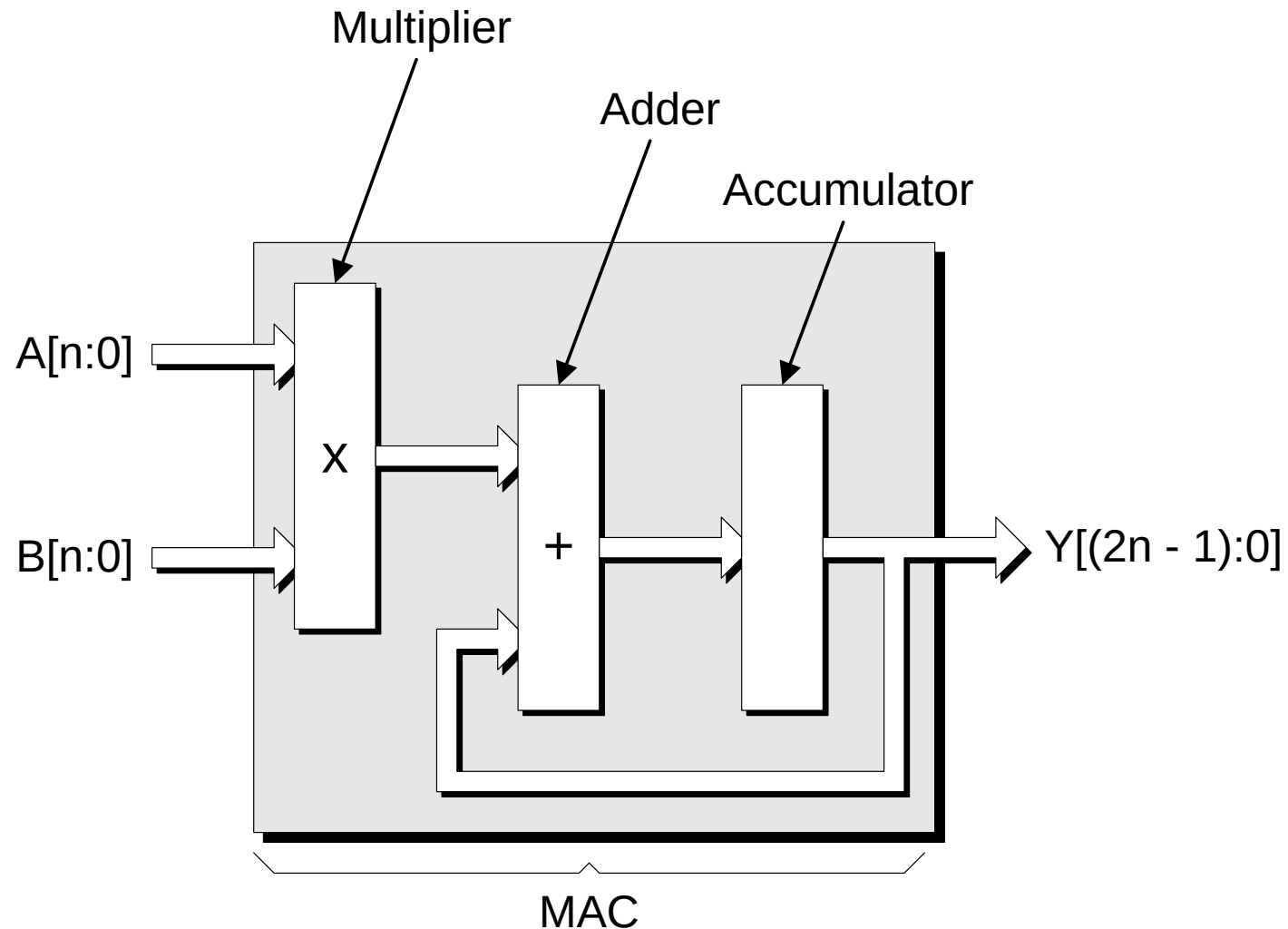




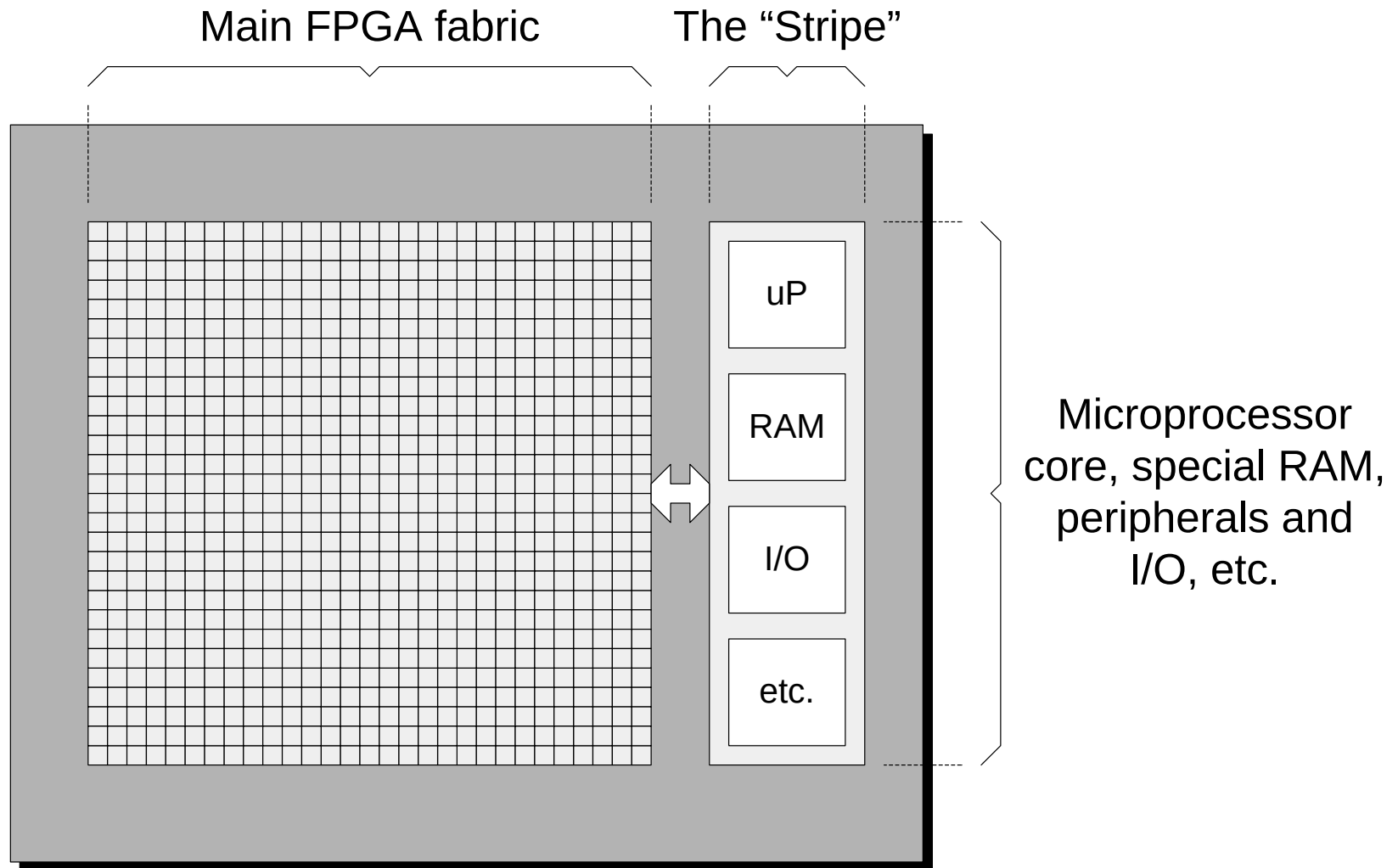
# Budowa układów FPGA (8)



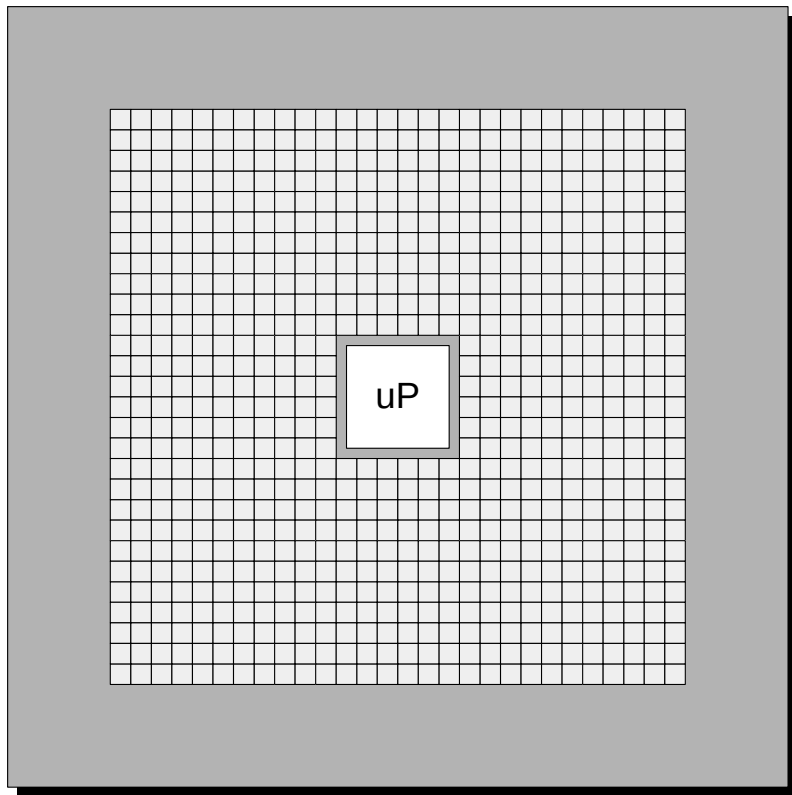
# Budowa układów FPGA (9)



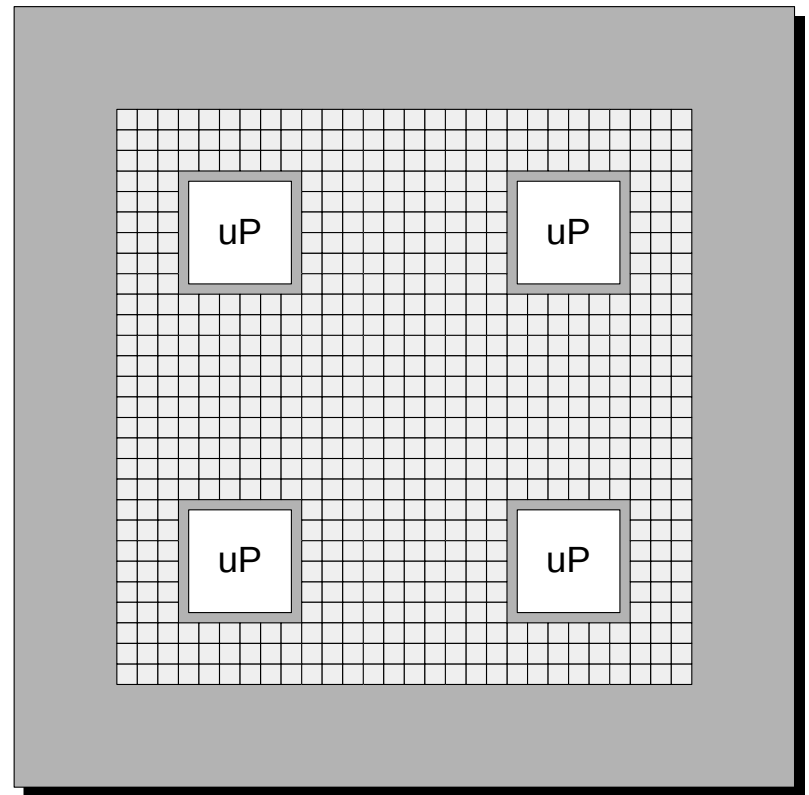
# Budowa układów FPGA (10)



# Budowa układów FPGA (11)



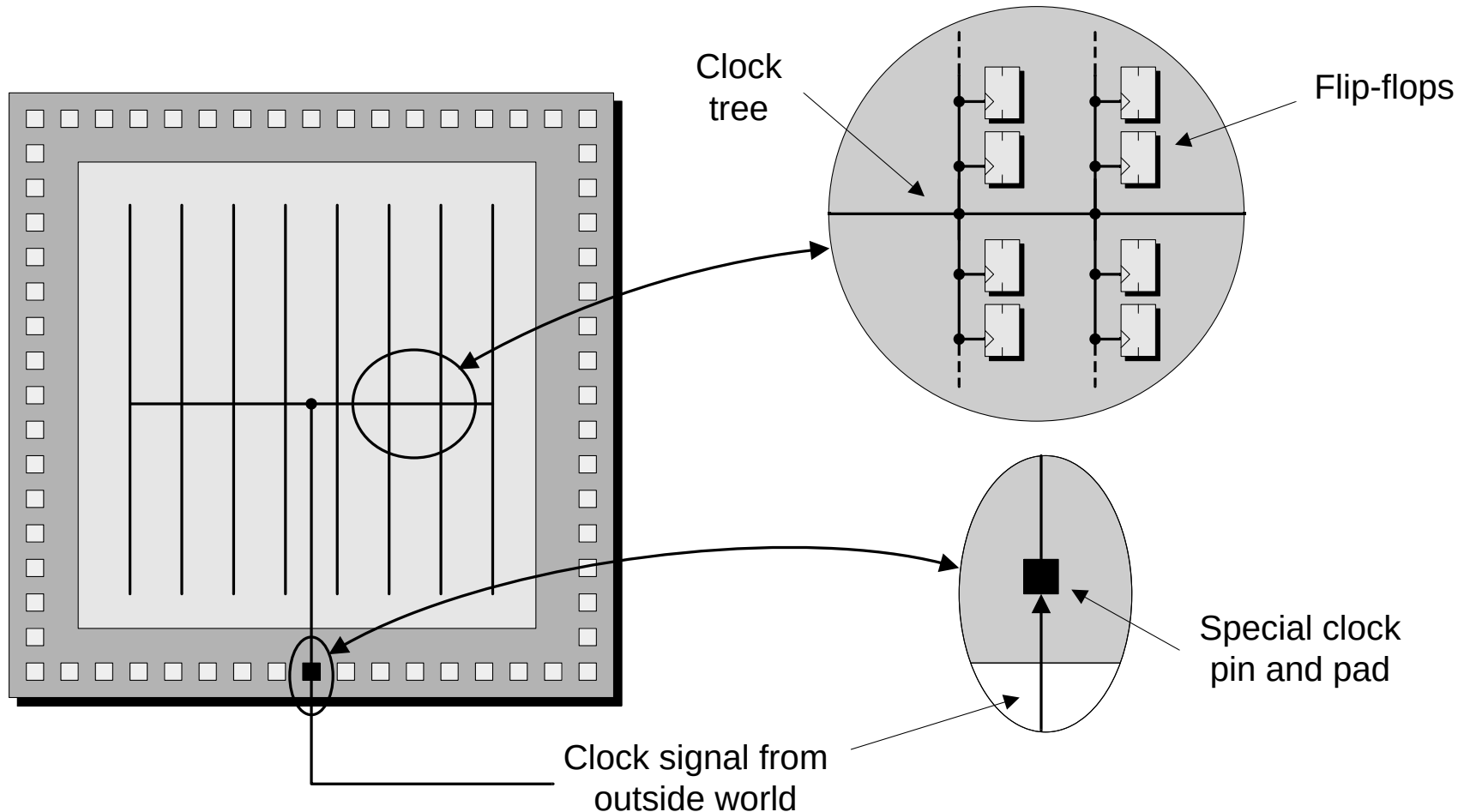
(a) One embedded core



(b) Four embedded cores

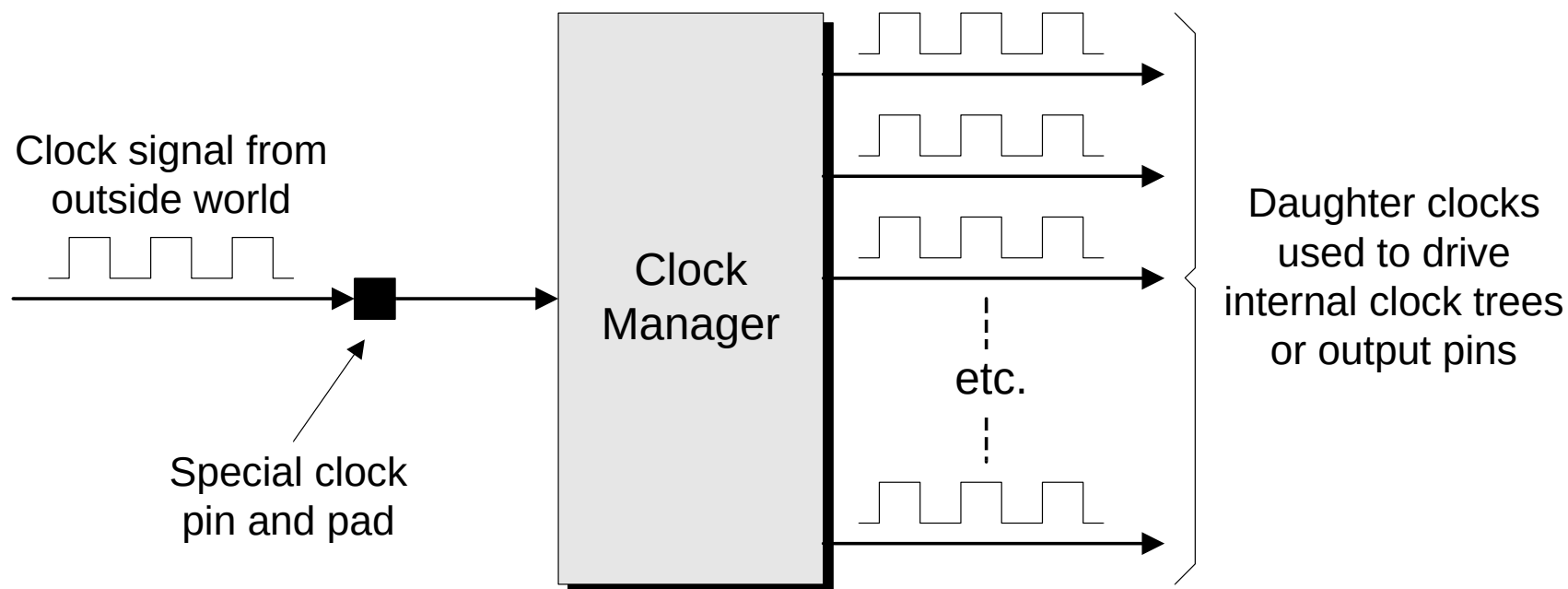
- HARD core
- SOFT core

# Budowa układów FPGA (12)



Proste drzewo dystrybucji zegara –  
minimalizacja *clock skew*

# Budowa układów FPGA (13)

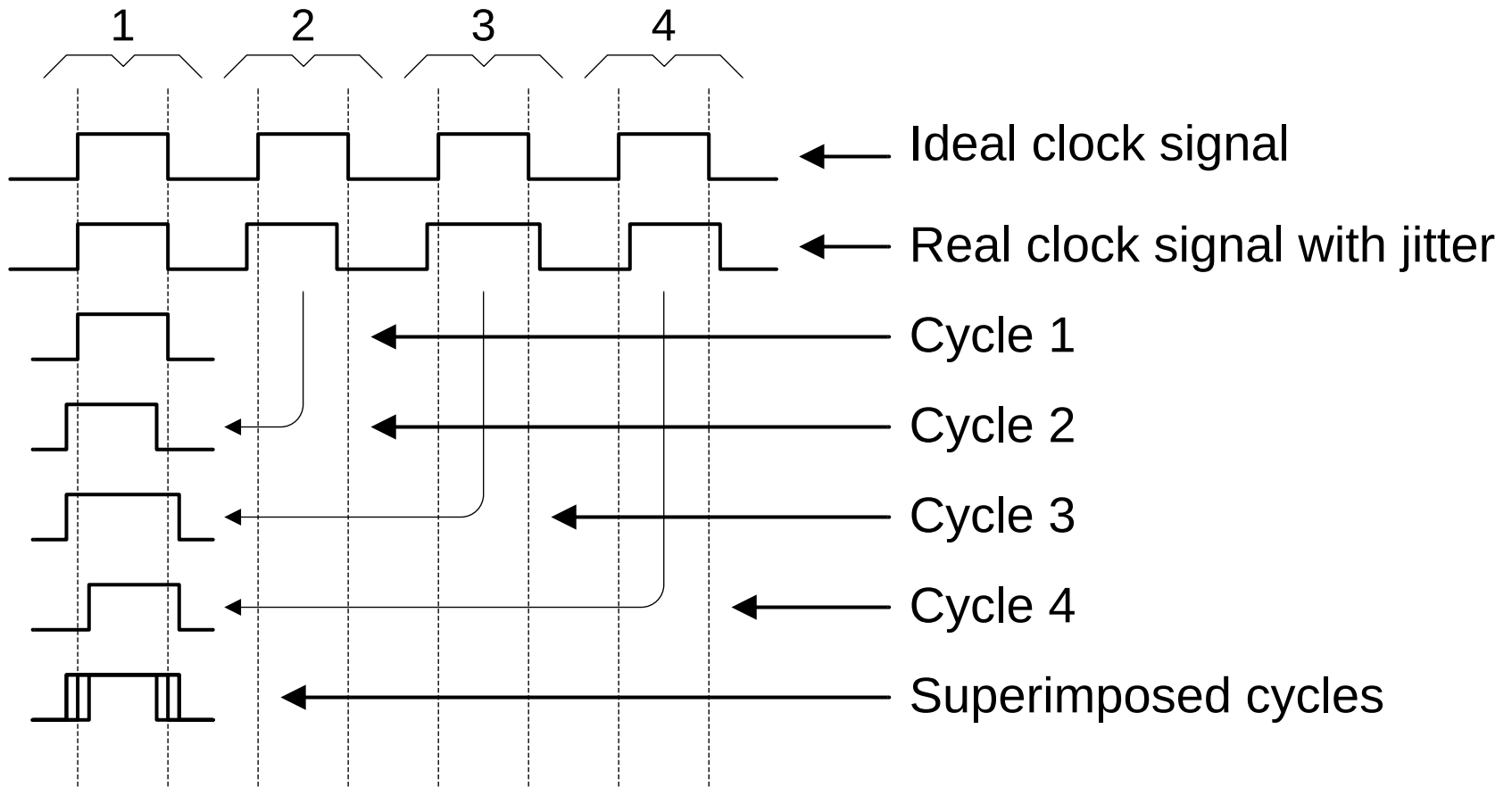


Układ zarządzający sygnałami zegarowymi.

Funkcje:

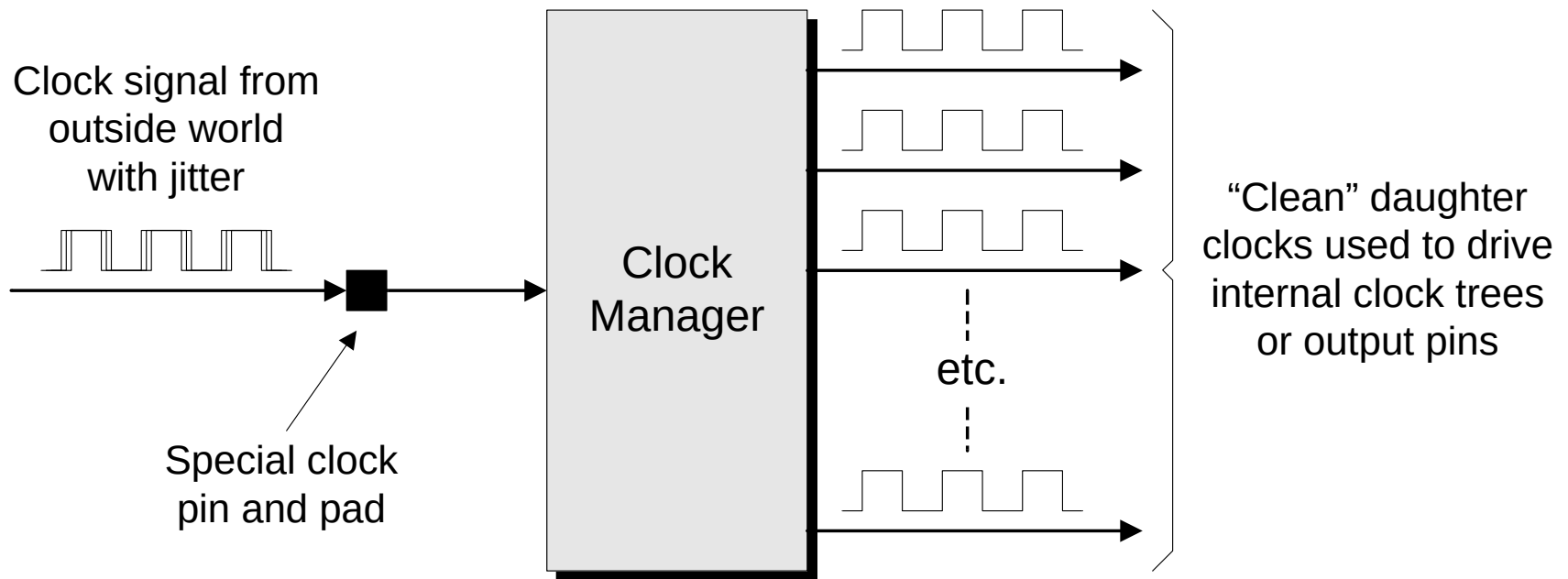
- Minimalizacja jitteru.
- Minimalizacja *clock skew*.
- Synteza częstotliwości.
- Przesunięcie fazy zegara.

# Budowa układów FPGA (14)



Wpływ jitteru na jakość sygnału zegarowego.

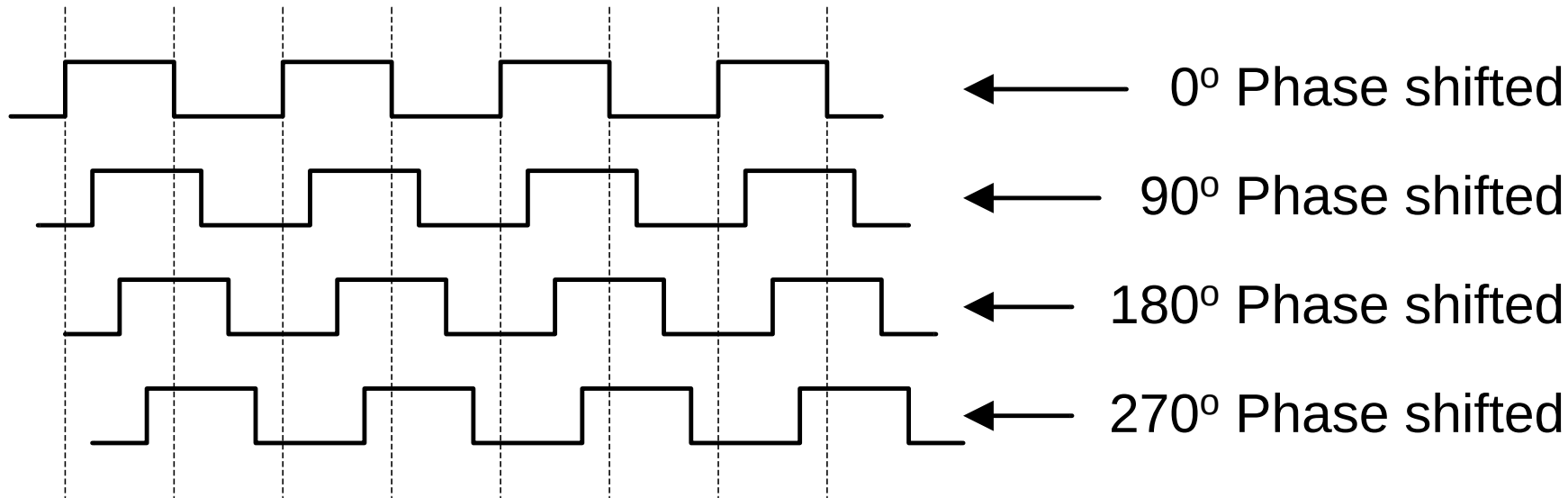
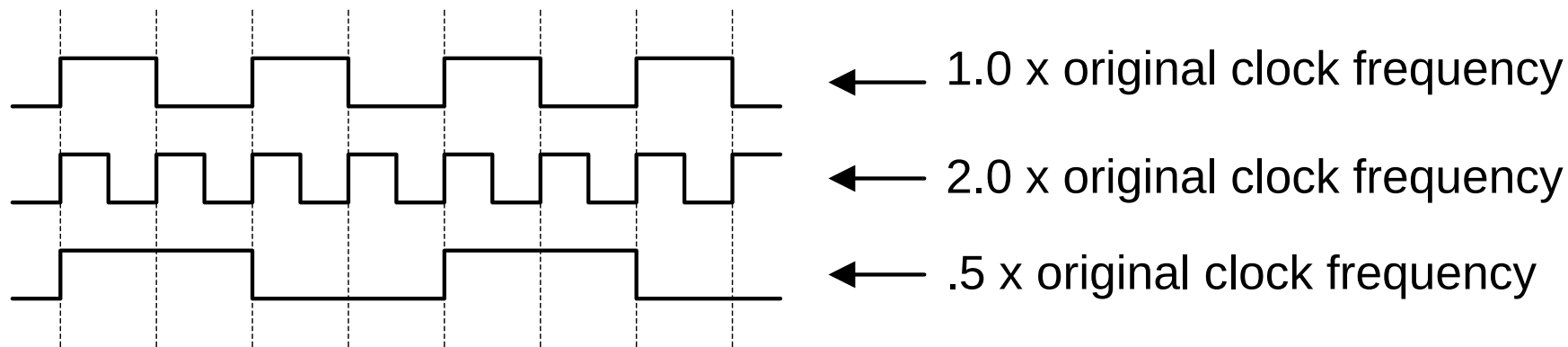
# Budowa układów FPGA (15)



Minimalizacja jitteru.

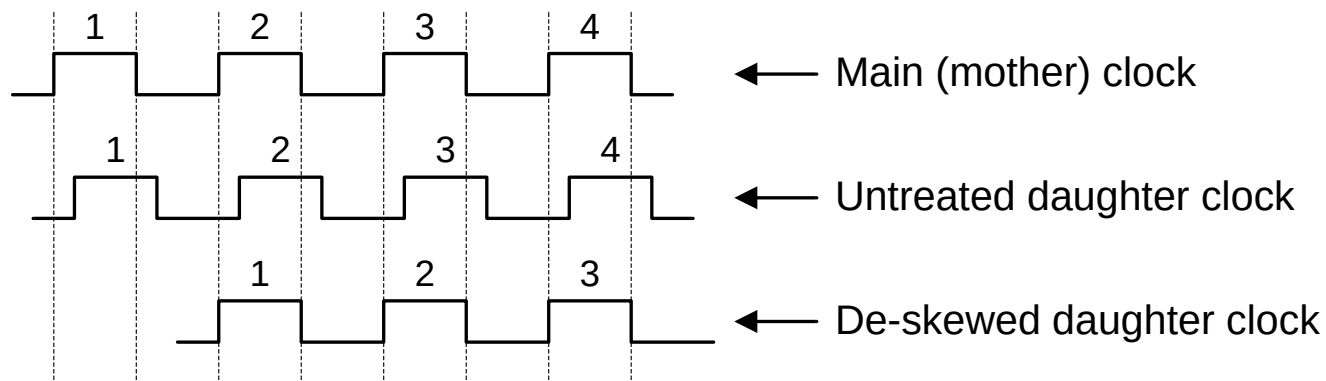
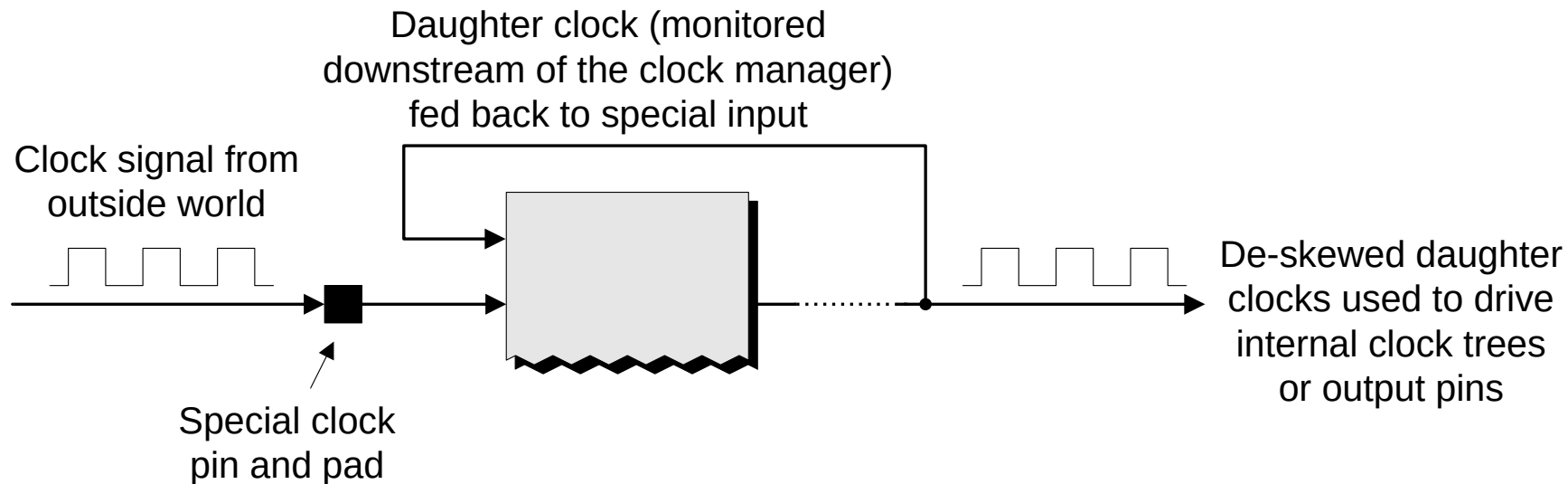


# Budowa układów FPGA (16)



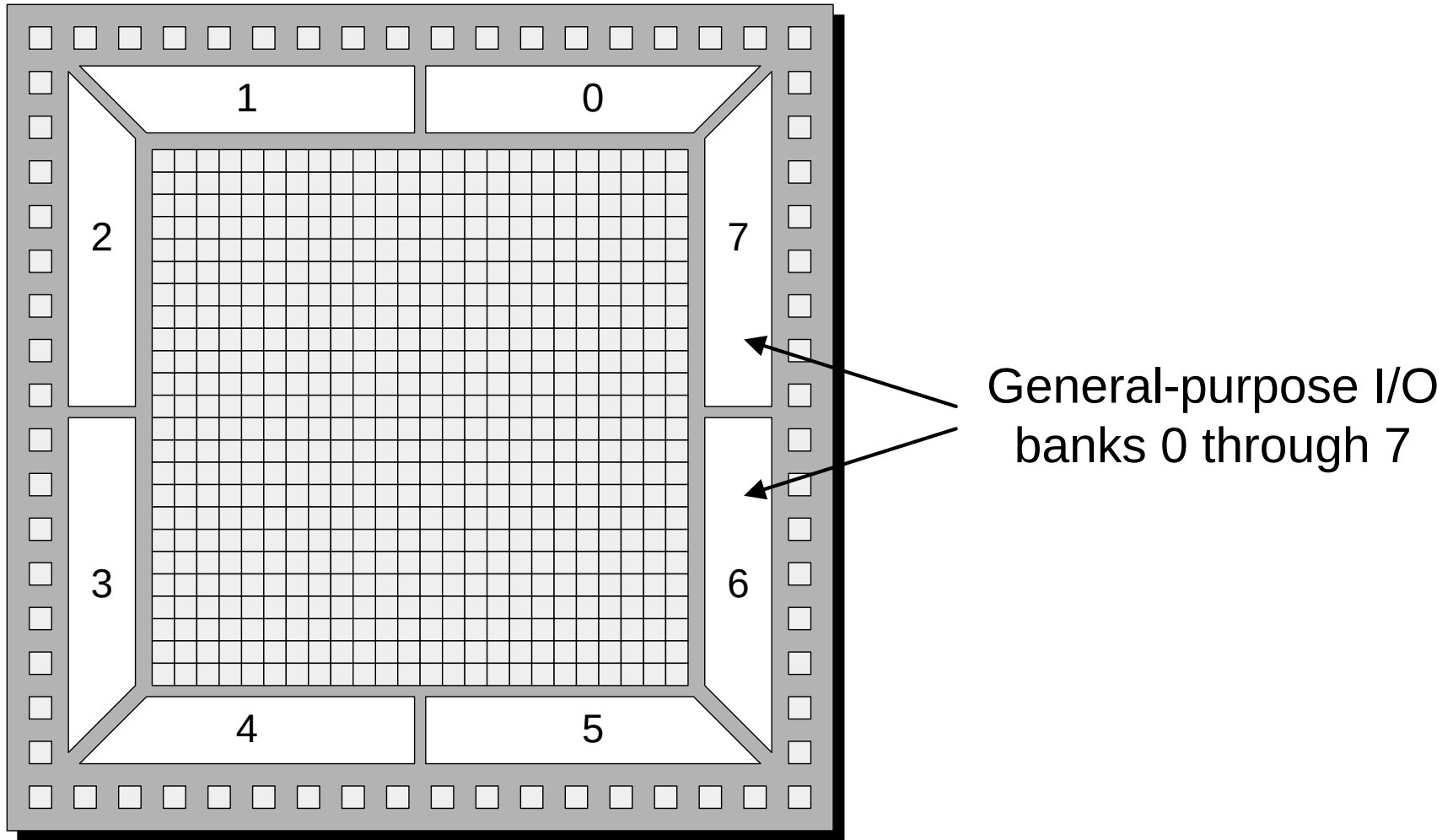
Synteza częstotliwości i przesunięcie fazy.

# Budowa układów FPGA (17)

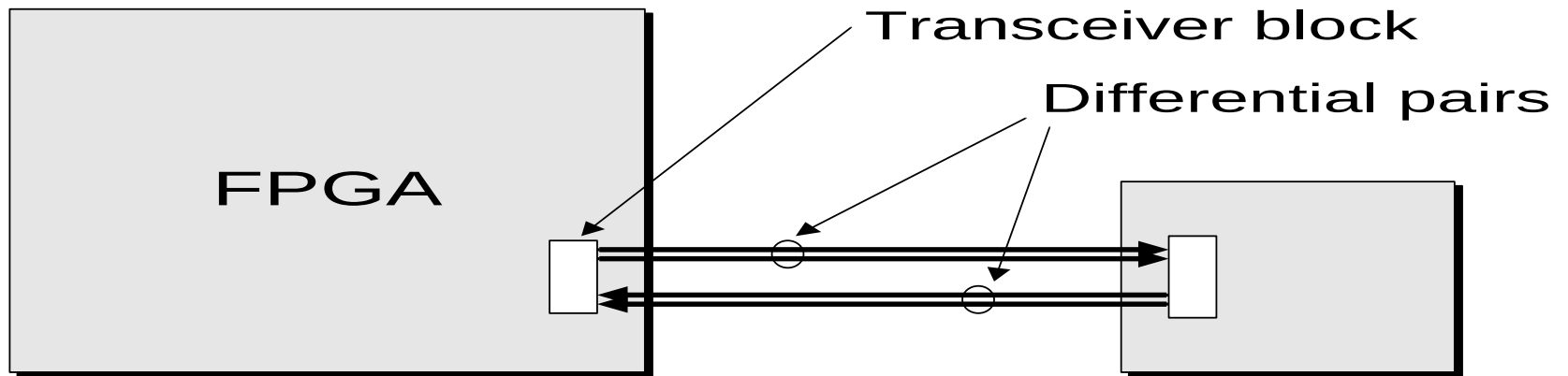
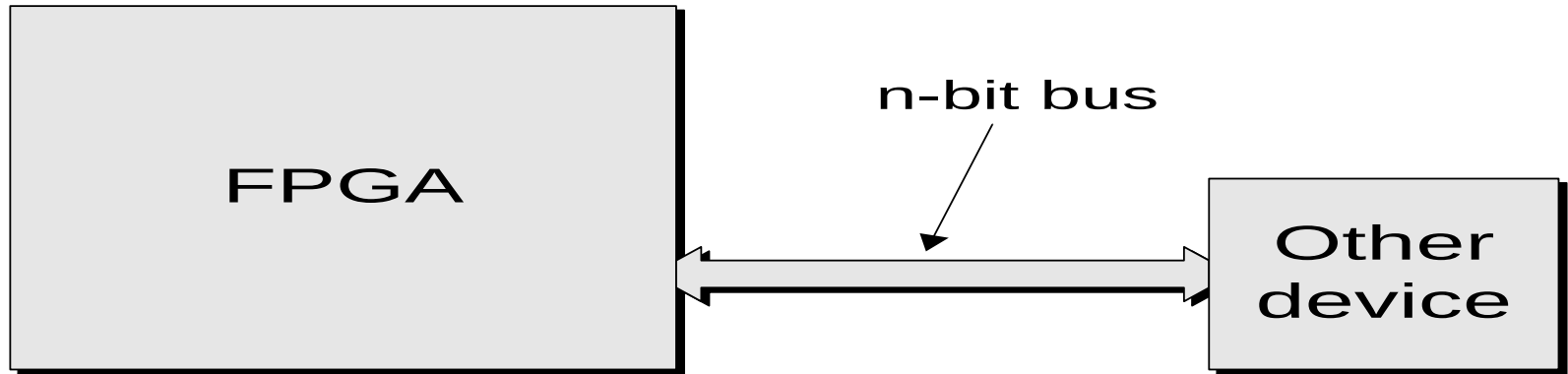


Minimalizacja *clock skew*.

# Budowa układów FPGA (18)



# Budowa układów FPGA (19)



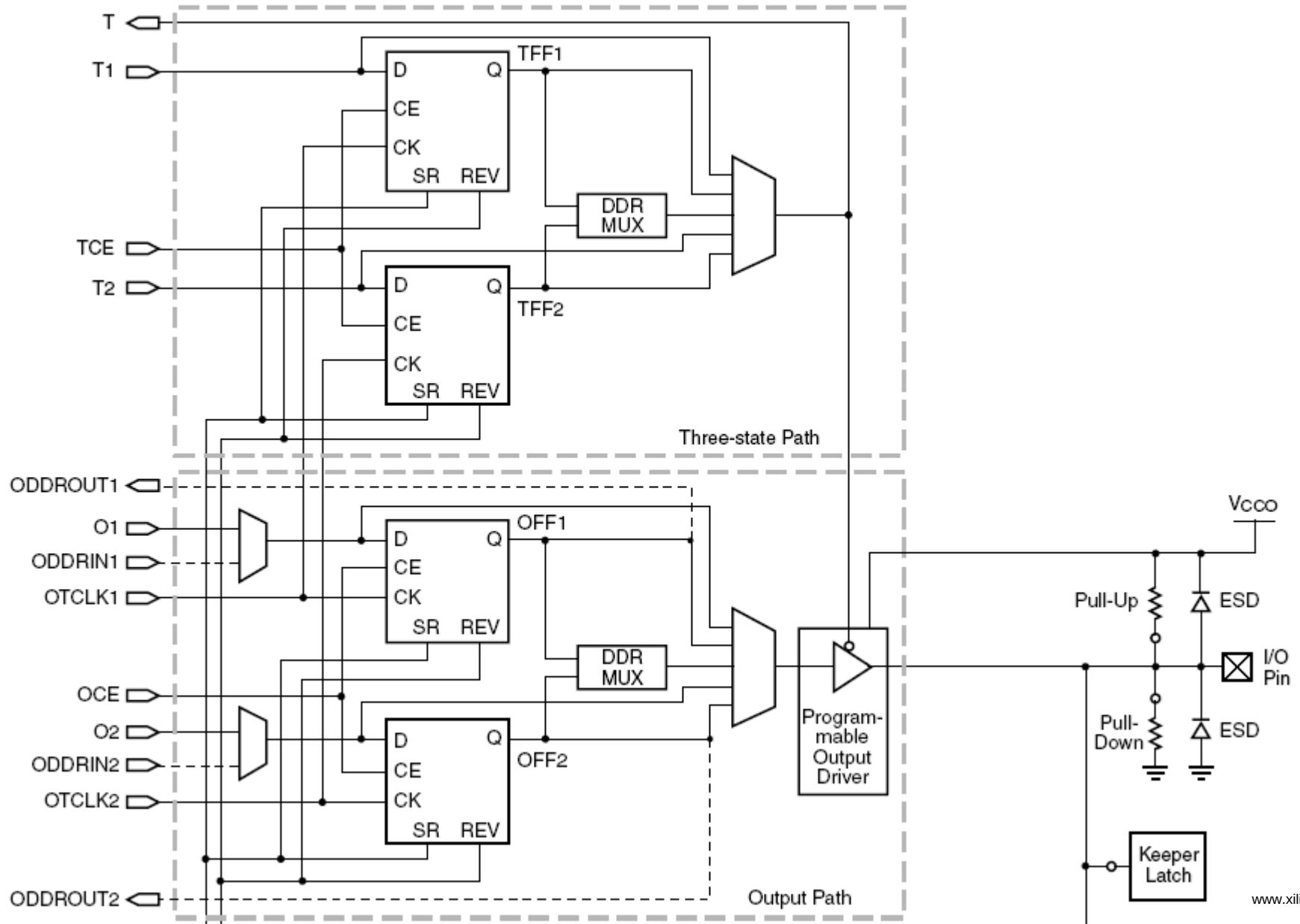
# Układy FPGA Spartan-3E (1)

| Device    | System Gates | Equivalent Logic Cells | CLB Array<br>(One CLB = Four Slices) |         |            |              | Distributed RAM bits <sup>(1)</sup> | Block RAM bits <sup>(1)</sup> | Dedicated Multipliers | DCMs | Maximum User I/O | Maximum Differential I/O Pairs |
|-----------|--------------|------------------------|--------------------------------------|---------|------------|--------------|-------------------------------------|-------------------------------|-----------------------|------|------------------|--------------------------------|
|           |              |                        | Rows                                 | Columns | Total CLBs | Total Slices |                                     |                               |                       |      |                  |                                |
| XC3S100E  | 100K         | 2,160                  | 22                                   | 16      | 240        | 960          | 15K                                 | 72K                           | 4                     | 2    | 108              | 40                             |
| XC3S250E  | 250K         | 5,508                  | 34                                   | 26      | 612        | 2,448        | 38K                                 | 216K                          | 12                    | 4    | 172              | 68                             |
| XC3S500E  | 500K         | 10,476                 | 46                                   | 34      | 1,164      | 4,656        | 73K                                 | 360K                          | 20                    | 4    | 232              | 92                             |
| XC3S1200E | 1200K        | 19,512                 | 60                                   | 46      | 2,168      | 8,672        | 136K                                | 504K                          | 28                    | 8    | 304              | 124                            |
| XC3S1600E | 1600K        | 33,192                 | 76                                   | 58      | 3,688      | 14,752       | 231K                                | 648K                          | 36                    | 8    | 376              | 156                            |

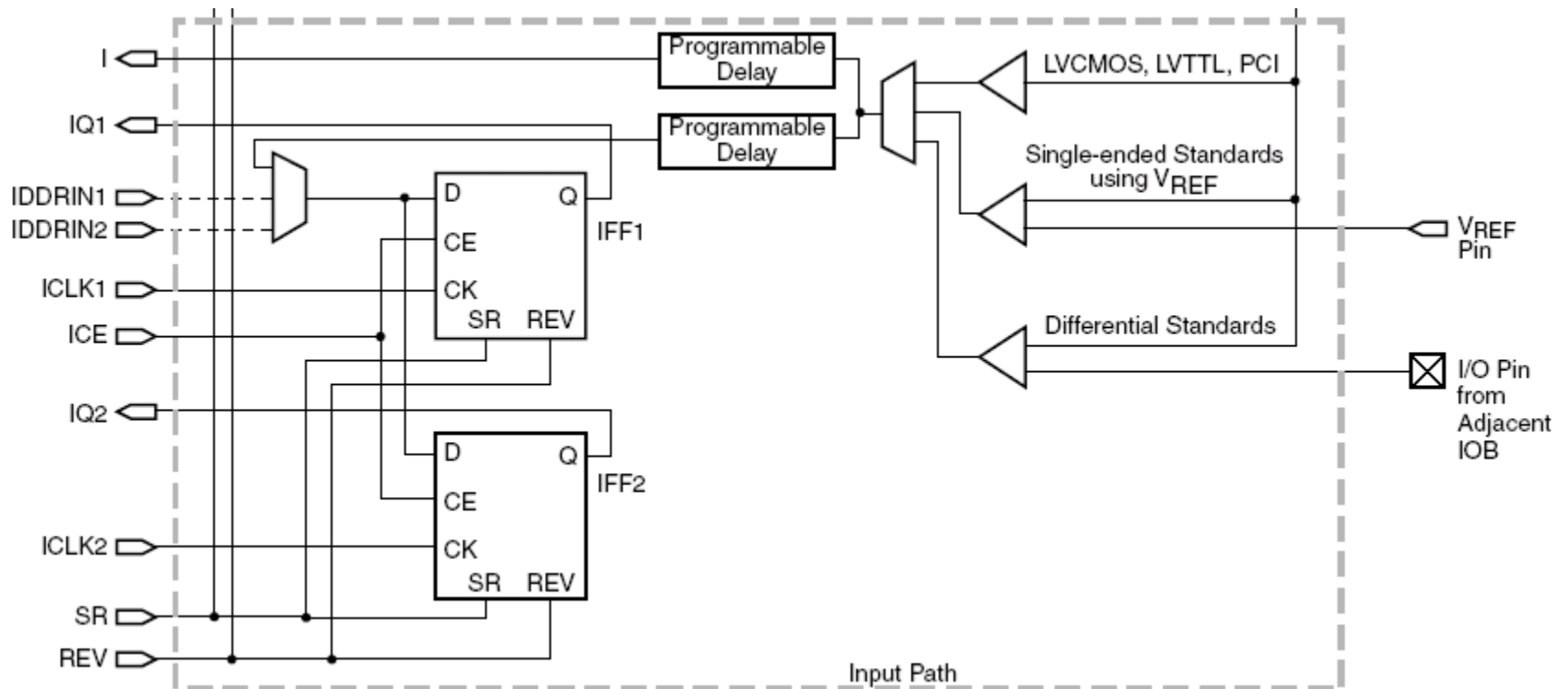
[www.xilinx.com](http://www.xilinx.com)

- IOB (Input output blocks)
- Dedicated inputs
- Special function pins

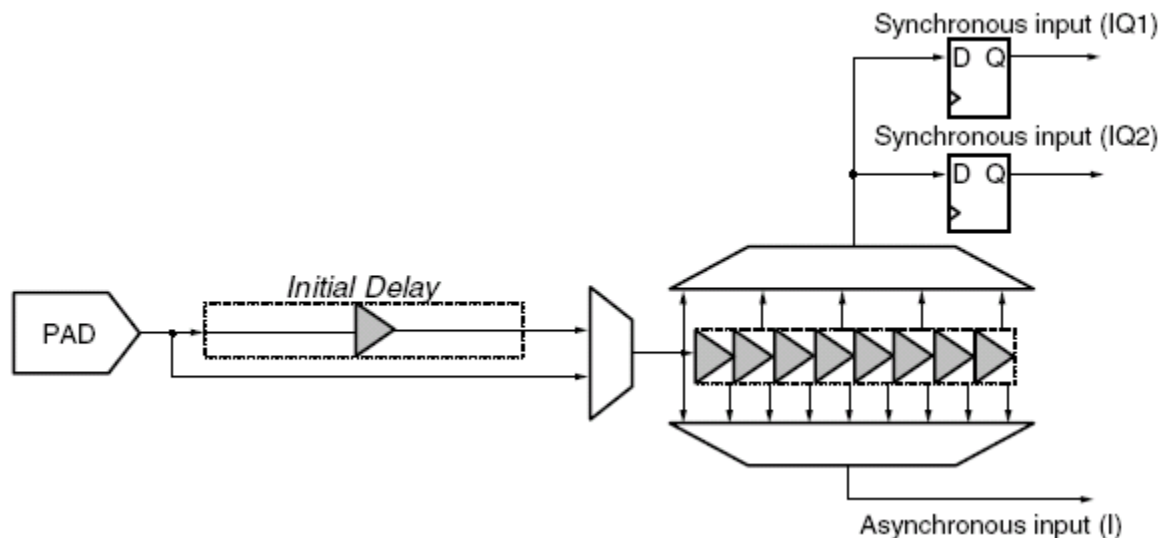
# Spartan-3E (2) IOB - output



# Spartan-3E (3) IOB - input



# Spartan-3E (4) IOB - delay



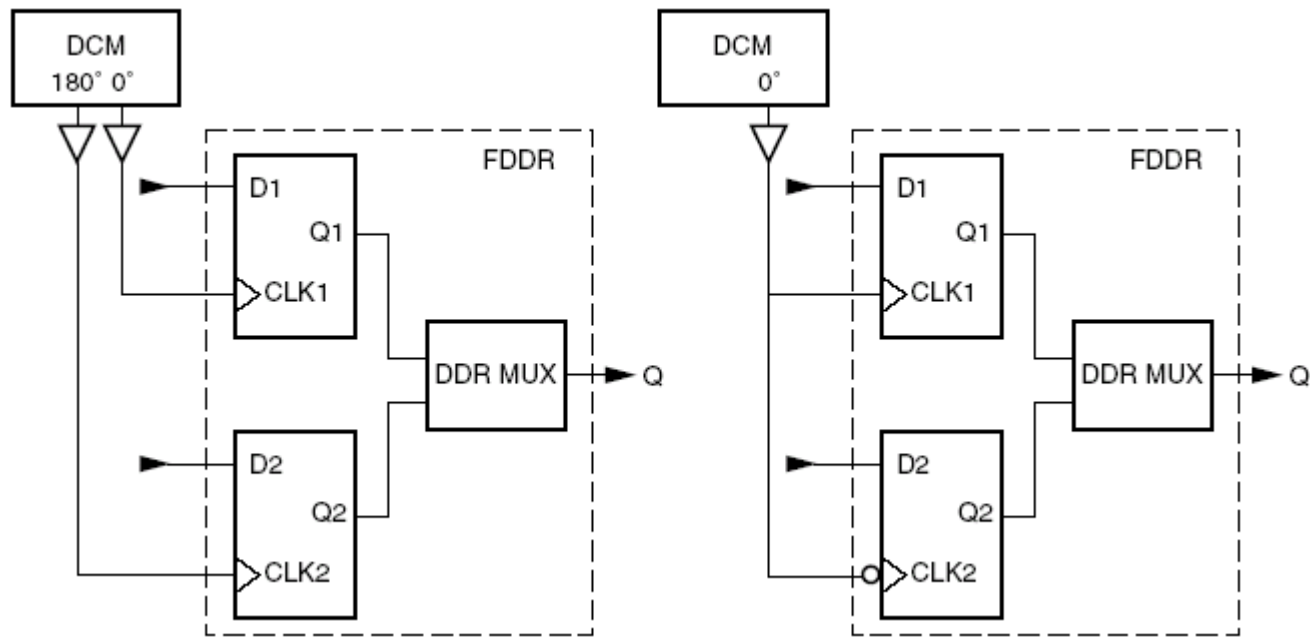
- Regulowana linia opóźniająca: dla wejścia asynchronicznego 8 odczepów co 250 ps (max. 4000 ps).
- Przykładowe zastosowania: kompensacja *clock skew* w systemie, dopasowanie do parametrów czasowych układów I/O np. pamięci.



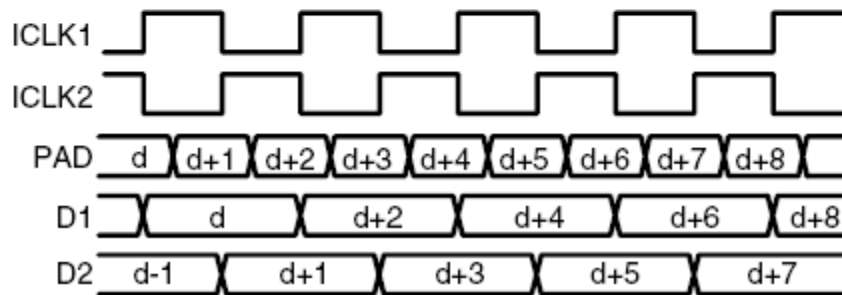
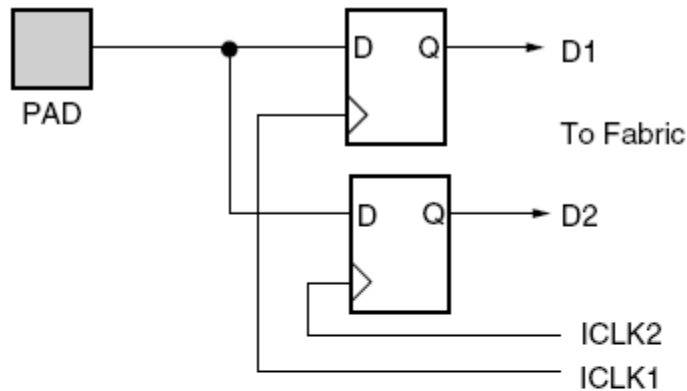
# Spartan-3E (5) IOB - DDR

Konfigurowalne właściwości przerzutników IOB i CLB:

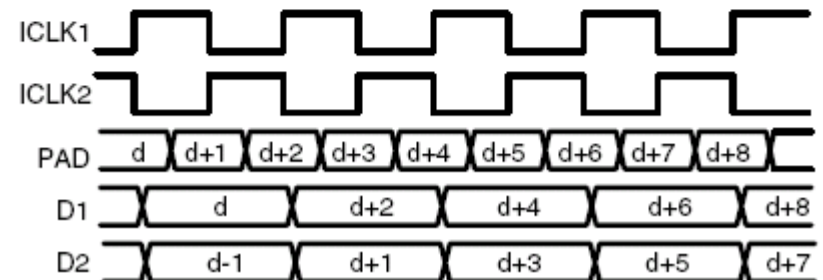
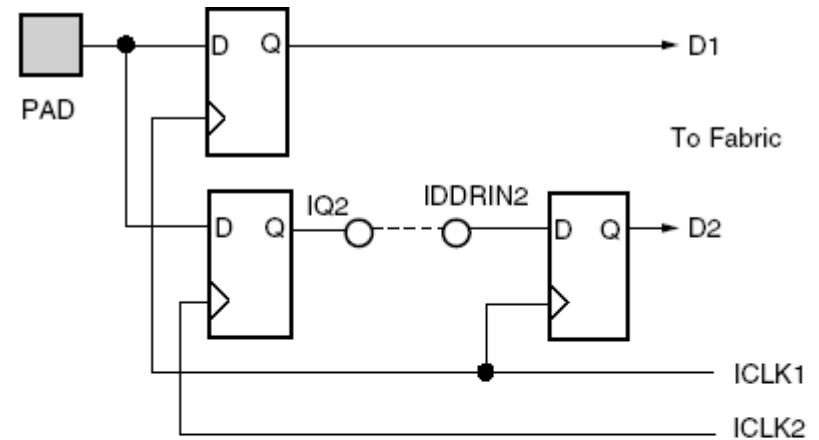
- FF / Latch
- SYNC / ASYNC (dla set-reset)
- SRHIGH / SRLOW
- INIT1 / INIT0 (po konfiguracji lub GSR)



# Spartan-3E (6) IOB – DDR - cascade



Cascade feature off



Cascade feature on

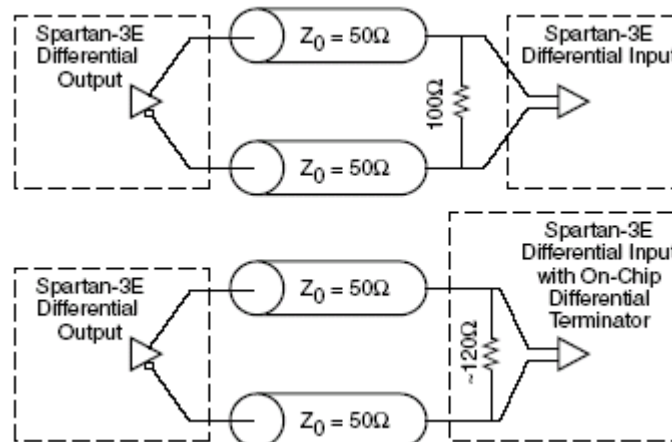
# Spartan-3E (7) IOB – standards

| Single-Ended<br>IOSTANDARD | V <sub>CC0</sub> Supply/Compatibility |                  |                  |                  |                                 |                      | Input Requirements |                                              |
|----------------------------|---------------------------------------|------------------|------------------|------------------|---------------------------------|----------------------|--------------------|----------------------------------------------|
|                            | 1.2V                                  | 1.5V             | 1.8V             | 2.5V             | 3.0V                            | 3.3V                 | V <sub>REF</sub>   | Board Termination Voltage (V <sub>TT</sub> ) |
| LVTTTL                     | -                                     | -                | -                | -                | -                               | Input/<br>Output     | N/R <sup>(1)</sup> | N/R                                          |
| LVCN0533                   | -                                     | -                | -                | -                | -                               | Input/<br>Output     | N/R                | N/R                                          |
| LVCN0525                   | -                                     | -                | -                | Input/<br>Output | Input                           | Input                | N/R                | N/R                                          |
| LVCN0518                   | -                                     | -                | Input/<br>Output | Input            | Input                           | Input                | N/R                | N/R                                          |
| LVCN0515                   | -                                     | Input/<br>Output | Input            | Input            | Input                           | Input                | N/R                | N/R                                          |
| LVCN0512                   | Input/<br>Output                      | Input            | Input            | Input            | Input                           | Input                | N/R <sup>(1)</sup> | N/R                                          |
| PCI33_3                    | -                                     | -                | -                | -                | Input/<br>Output <sup>(2)</sup> | Input <sup>(3)</sup> | N/R                | N/R                                          |
| PCI66_3                    | -                                     | -                | -                | -                | Input/<br>Output <sup>(2)</sup> | Input <sup>(3)</sup> | N/R                | N/R                                          |
| PCIX                       |                                       |                  |                  |                  | Input/<br>Output <sup>(2)</sup> | Input <sup>(3)</sup> | N/R                | N/R                                          |
| HSTL_L_18                  | -                                     | -                | Input/<br>Output | Input            | Input                           | Input                | 0.9                | 0.9                                          |
| HSTL_IIL_18                | -                                     | -                | Input/<br>Output | Input            | Input                           | Input                | 1.1                | 1.8                                          |
| SSTL18_I                   | -                                     | -                | Input/<br>Output | Input            | Input                           | Input                | 0.9                | 0.9                                          |
| SSTL2_I                    | -                                     | -                | -                | Input/<br>Output | Input                           | Input                | 1.25               | 1.25                                         |

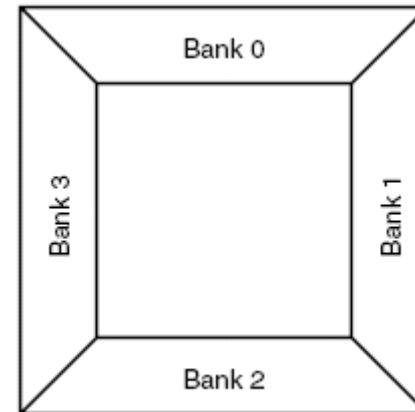
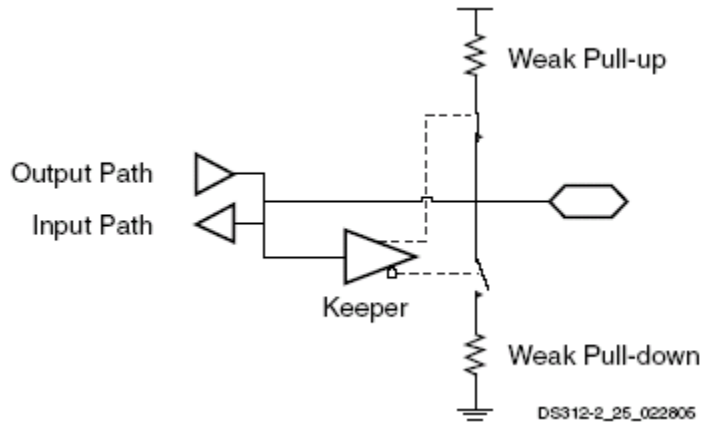
IOB standards – ustawiane za pomocą atrybutu IOSTANDARD

# Spartan-3E (8) IOB – standards

| Differential IOSTANDARD | Vcco Supply      |                                                       |       | Input Requirements:<br>$V_{REF}$                    | Differential Bank Restriction <sup>(1)</sup>                                           |
|-------------------------|------------------|-------------------------------------------------------|-------|-----------------------------------------------------|----------------------------------------------------------------------------------------|
|                         | 1.8V             | 2.5V                                                  | 3.3V  |                                                     |                                                                                        |
| LVDS_25                 | Input            | Input,<br>On-chip Differential Termination,<br>Output | Input | $V_{REF}$ is not used<br>for these I/O<br>standards | Applies to<br>Outputs Only                                                             |
| RSDS_25                 | Input            | Input,<br>On-chip Differential Termination,<br>Output | Input |                                                     | Applies to<br>Outputs Only                                                             |
| MINI_LVDS_25            | Input            | Input,<br>On-chip Differential Termination,<br>Output | Input |                                                     | Applies to<br>Outputs Only                                                             |
| LVPECL_25               | Input            | Input                                                 | Input |                                                     | No Differential<br>Bank Restriction<br>(other I/O bank<br>restrictions might<br>apply) |
| BLVDS_25                | Input            | Input,<br>Output                                      | Input |                                                     |                                                                                        |
| DIFF_HSTL_I_18          | Input,<br>Output | Input                                                 | Input |                                                     |                                                                                        |
| DIFF_HSTL_III_18        | Input,<br>Output | Input                                                 | Input |                                                     |                                                                                        |
| DIFF_SSTL18_I           | Input,<br>Output | Input                                                 | Input |                                                     |                                                                                        |
| DIFF_SSTL2_I            | Input            | Input,<br>Output                                      | Input |                                                     |                                                                                        |



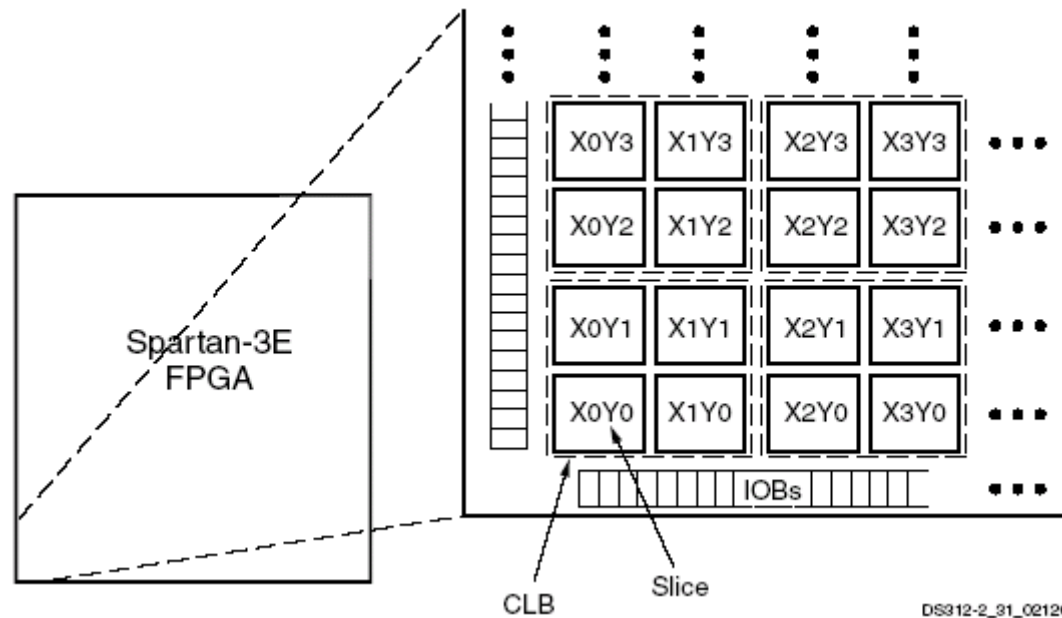
# Spartan-3E (9) IOB - właściwości



- Szybkość zmian napięcia wyjściowego: SLOW (default) lub FAST.

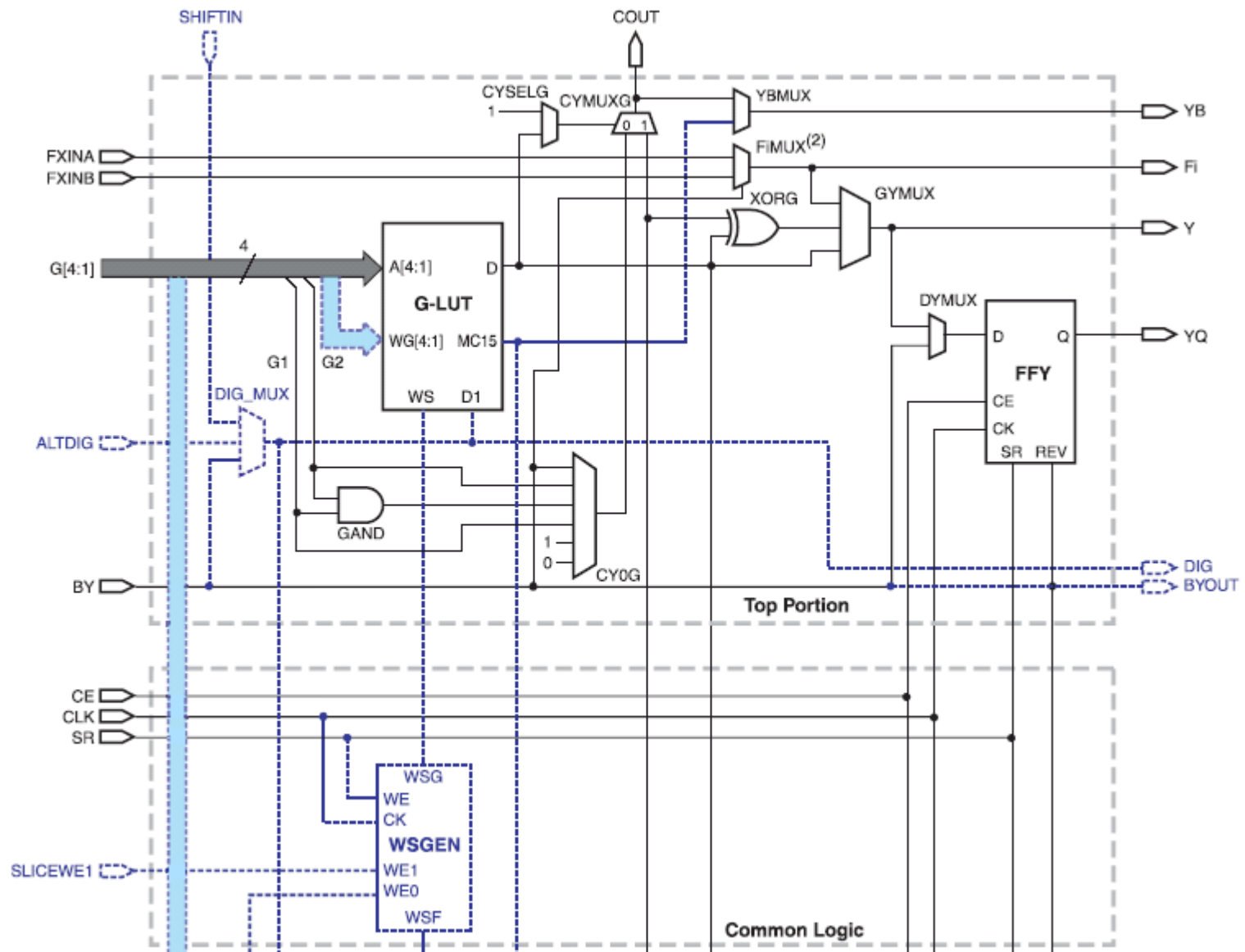
| IOSTANDARD | Output Drive Current (mA) |   |   |   |    |    |
|------------|---------------------------|---|---|---|----|----|
|            | 2                         | 4 | 6 | 8 | 12 | 16 |
| LVTTL      | ✓                         | ✓ | ✓ | ✓ | ✓  | ✓  |
| LVC MOS33  | ✓                         | ✓ | ✓ | ✓ | ✓  | ✓  |
| LVC MOS25  | ✓                         | ✓ | ✓ | ✓ | ✓  | -  |
| LVC MOS18  | ✓                         | ✓ | ✓ | ✓ | -  | -  |
| LVC MOS15  | ✓                         | ✓ | ✓ | - | -  | -  |
| LVC MOS12  | ✓                         | - | - | - | -  | -  |

# Spartan-3E (10) Logika programowalna

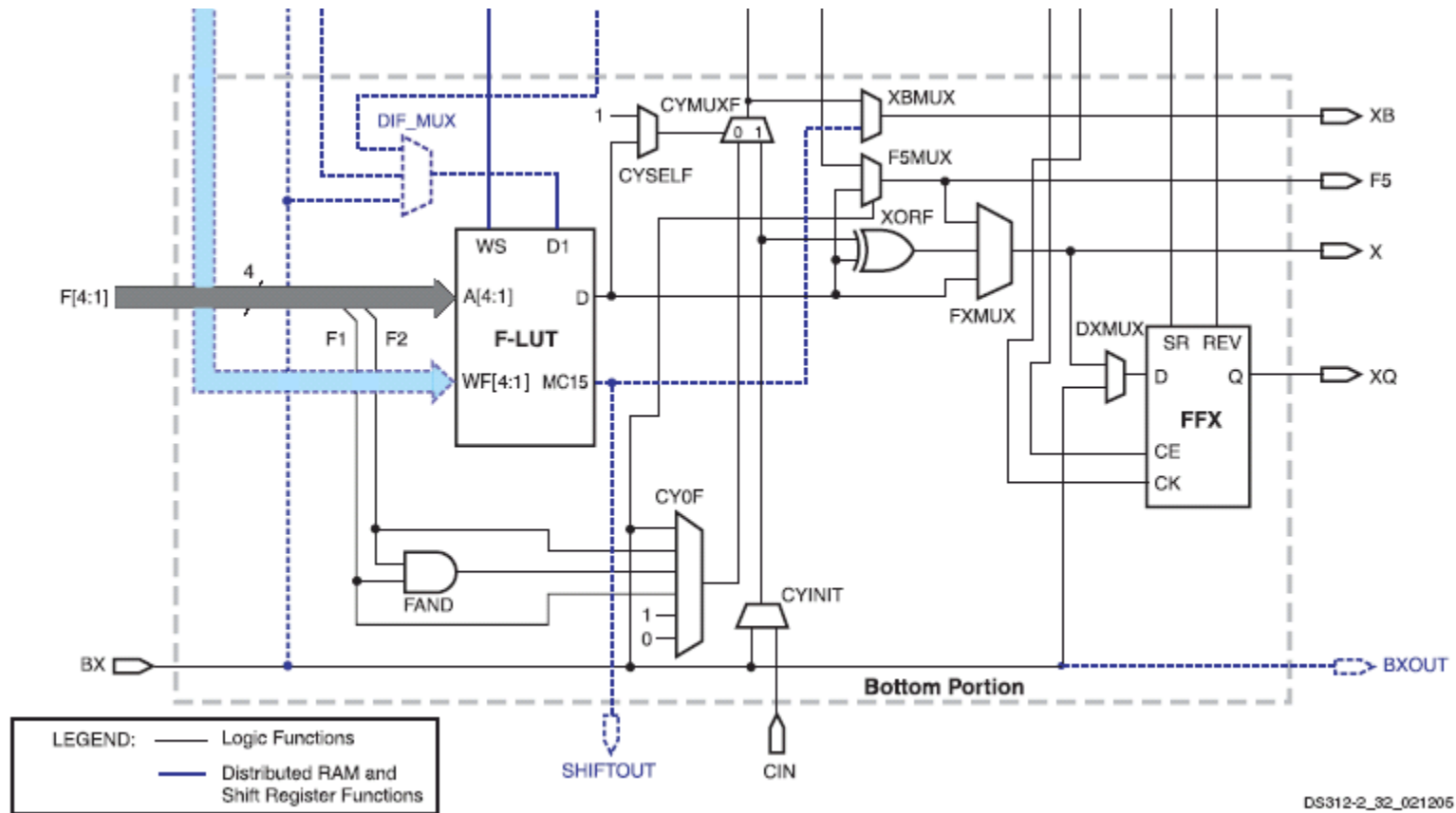


DS912-2\_31\_021205

# Spartan-3E (11) Slice

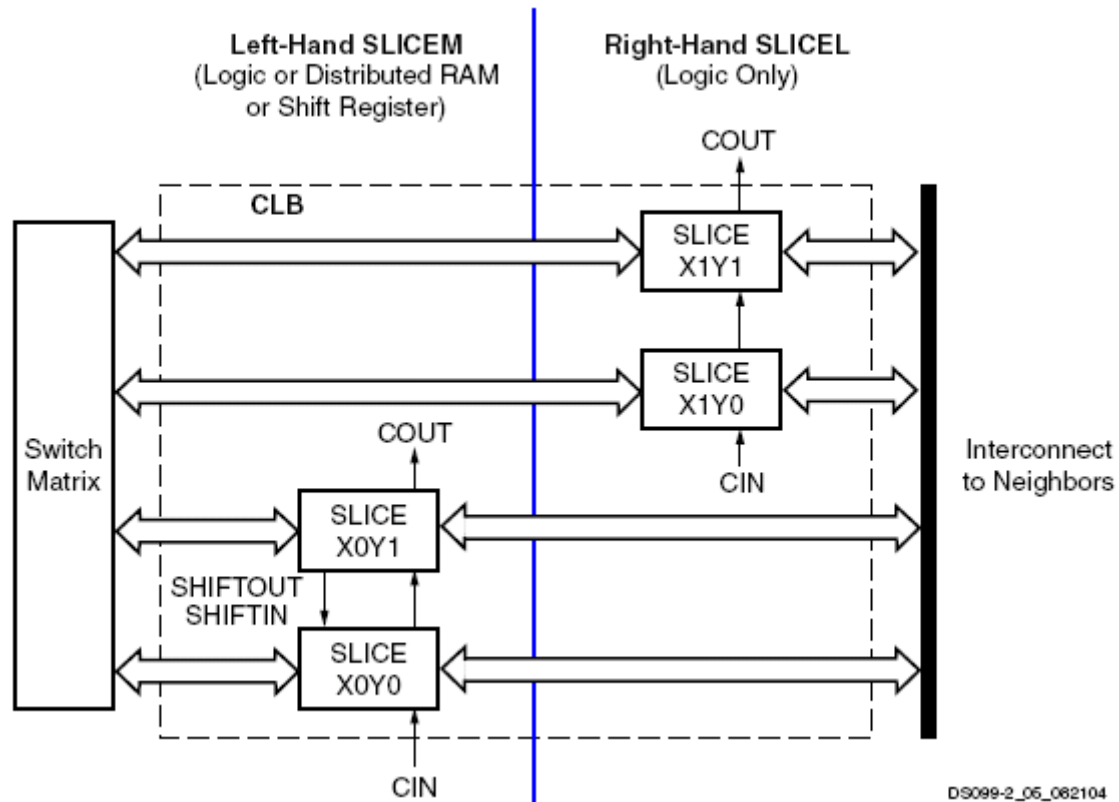


# Spartan-3E (12) Slice

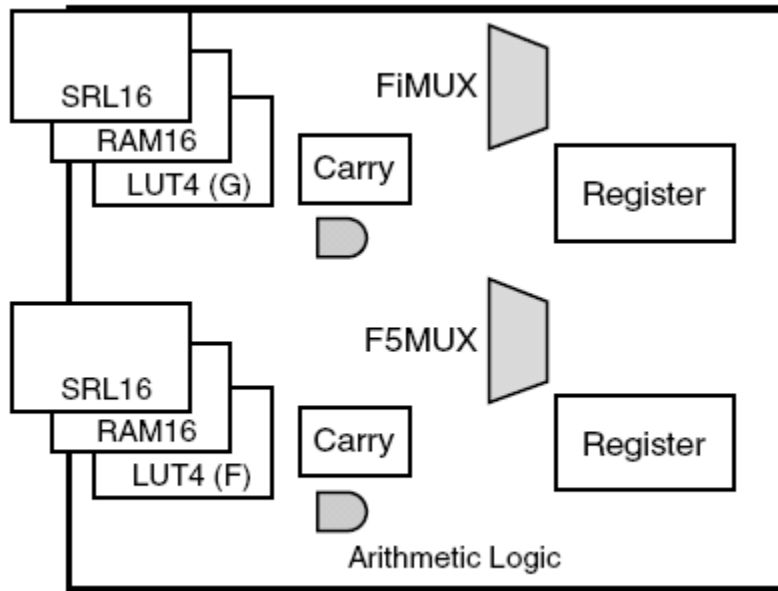




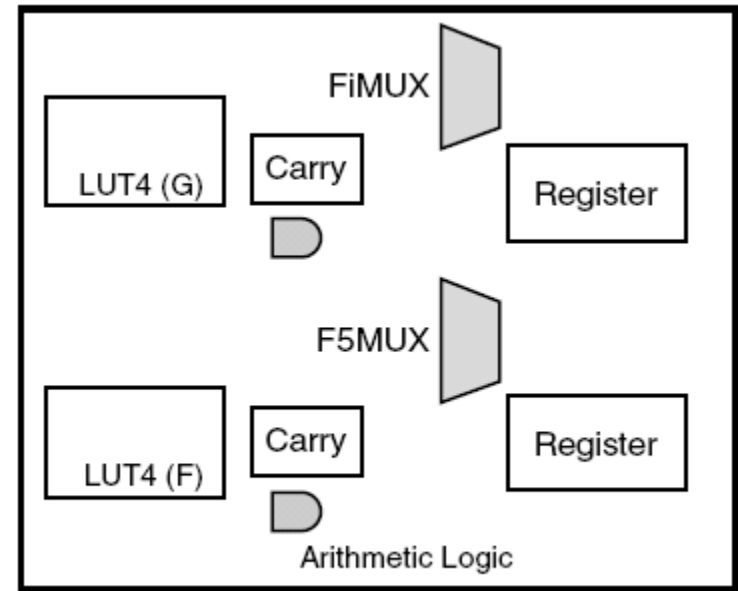
# Spartan-3E (13) CLB



# Spartan-3E (14) Slice - rodzaje



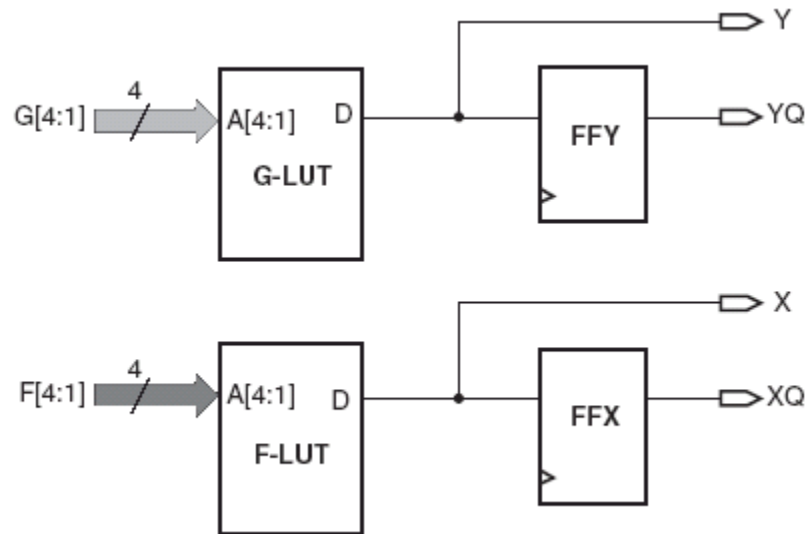
**SLICEM**



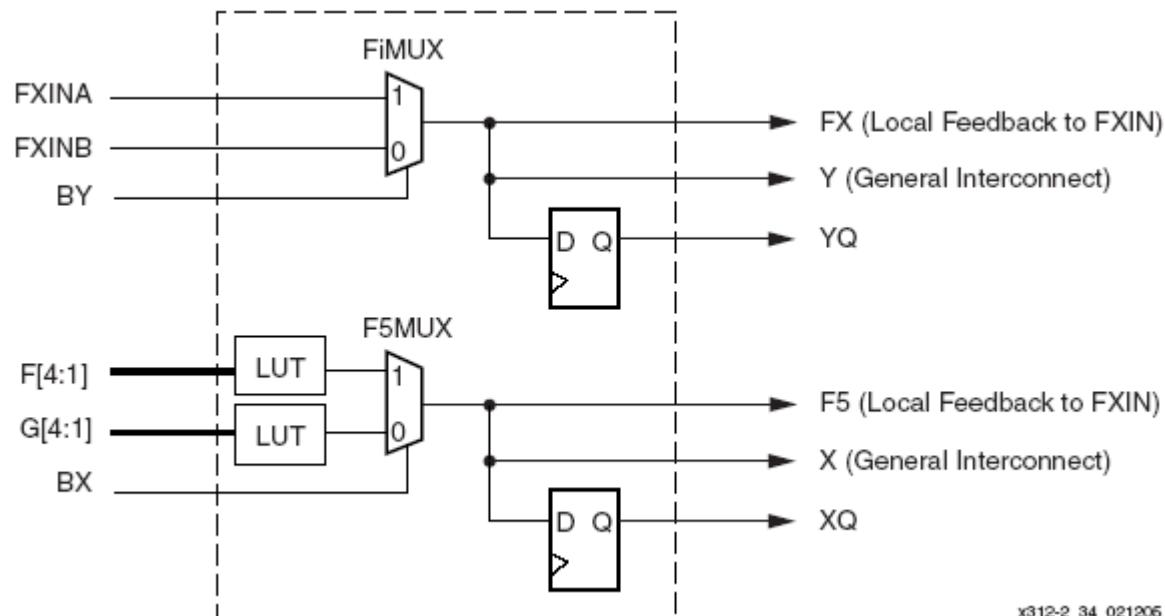
**SLICEL**

DS312-2\_13\_020905

# Spartan-3E (15) LUT

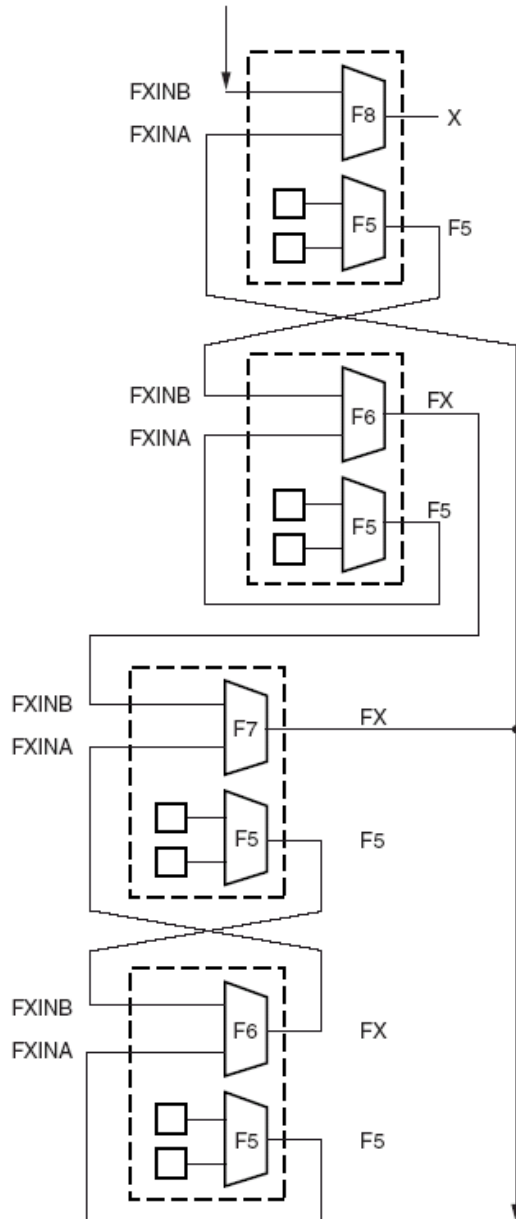


DS312-2\_33\_111105

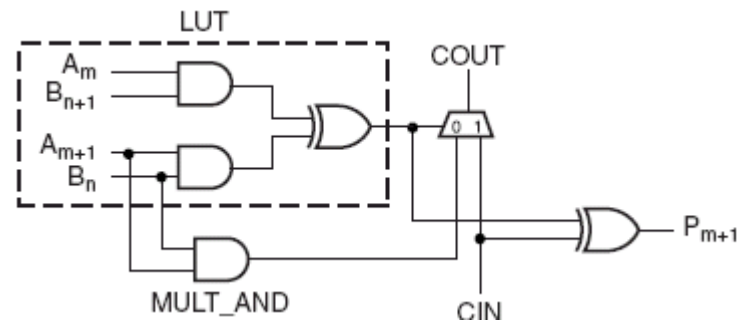
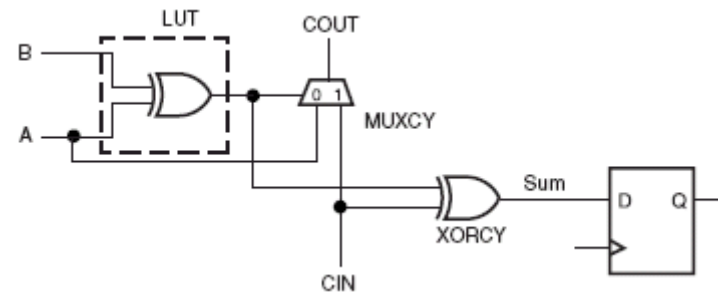


x312-2\_34\_021205

# Spartan-3E (16) CLB - logika dedykowana

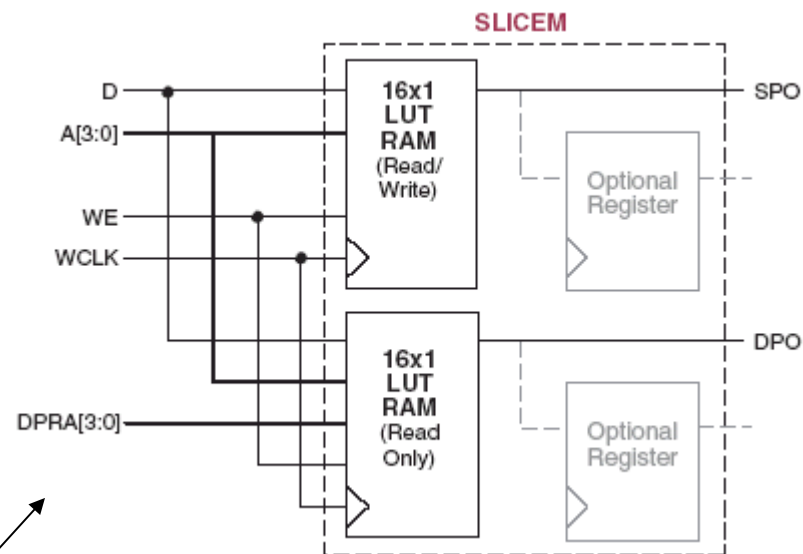
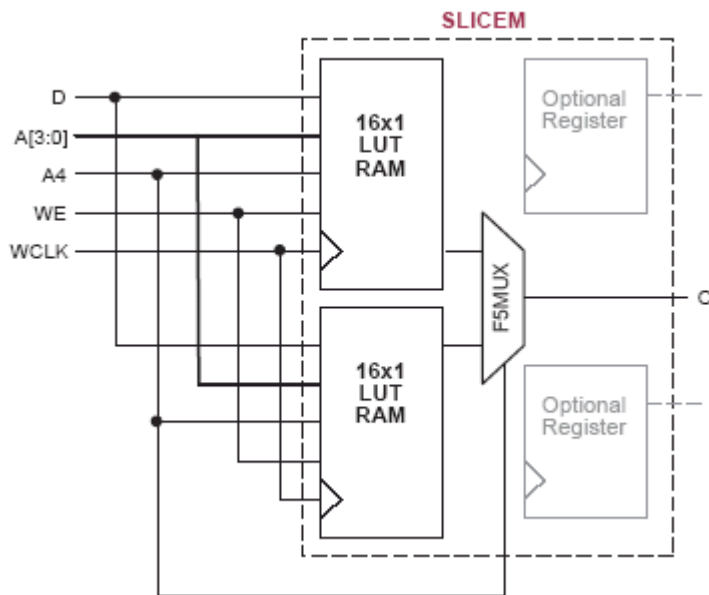


| Mux   | Usage | Input Source | Total Number of Inputs per Function |               |                       |
|-------|-------|--------------|-------------------------------------|---------------|-----------------------|
|       |       |              | For Any Function                    | For Mux       | For Limited Functions |
| F5MUX | F5MUX | LUTs         | 5                                   | 6 (4:1 mux)   | 9                     |
| FiMUX | F6MUX | F5MUX        | 6                                   | 11 (8:1 mux)  | 19                    |
|       | F7MUX | F6MUX        | 7                                   | 20 (16:1 mux) | 39                    |
|       | F8MUX | F7MUX        | 8                                   | 37 (32:1 mux) | 79                    |



# Spartan-3E (17)

# CLB – distributed RAM



- Single port SRAM
- Dual port SRAM

| Inputs    |      |   | Outputs |        |
|-----------|------|---|---------|--------|
| WE (mode) | WCLK | D | SPO     | DPO    |
| 0 (read)  | X    | X | data_a  | data_d |
| 1 (read)  | 0    | X | data_a  | data_d |
| 1 (read)  | 1    | X | data_a  | data_d |
| 1 (write) | ↑    | D | D       | data_d |
| 1 (read)  | ↓    | X | data_a  | data_d |

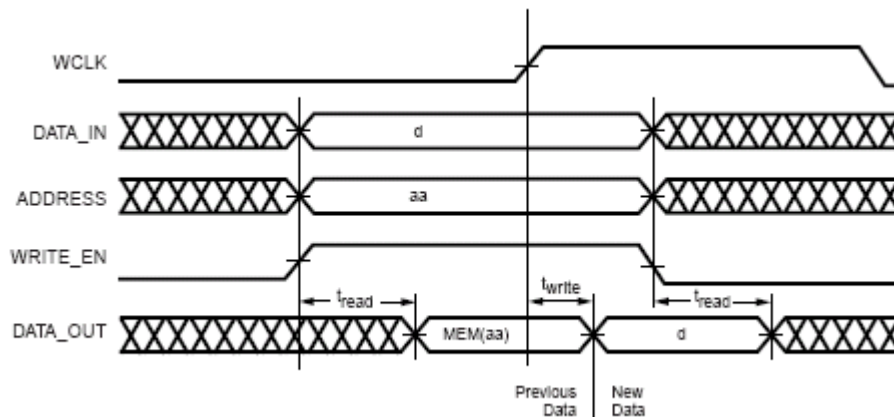
## Notes:

1. data\_a = word addressed by bits A3-A0.
2. data\_d = word addressed by bits DPRA3-DPRA0.

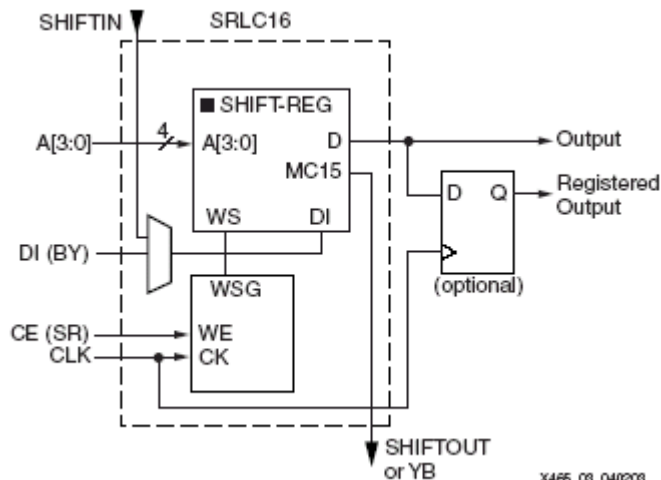
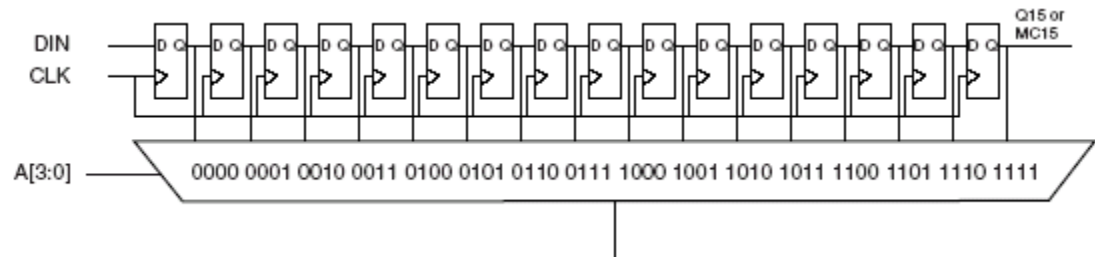
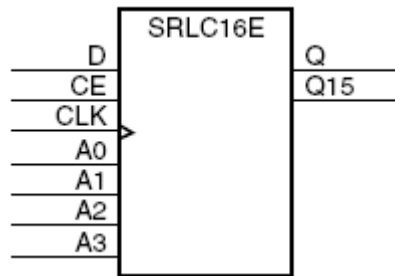
# Spartan-3E (18) CLB – distributed RAM

| Feature                      | Spartan-3/<br>Spartan-3L/<br>Spartan-3E<br>Families | Virtex/Virtex-E,<br>Spartan-II/Spartan-II E<br>Families | Virtex-II,<br>Virtex-II Pro<br>Families |
|------------------------------|-----------------------------------------------------|---------------------------------------------------------|-----------------------------------------|
| LUTs per CLB                 | 8                                                   | 4                                                       | 8                                       |
| ROM bits per CLB             | 128                                                 | 64                                                      | 128                                     |
| Single-port RAM bits per CLB | 64                                                  | 64                                                      | 128                                     |
| Dual-port RAM bits per CLB   | 32                                                  | 32                                                      | 64                                      |

| Family                                     | Single-Port RAM |      |      |       | Dual-Port RAM |      |      |
|--------------------------------------------|-----------------|------|------|-------|---------------|------|------|
|                                            | 16x1            | 32x1 | 64x1 | 128x1 | 16x1          | 32x1 | 64x1 |
| Spartan-3                                  | 4               | 2    | 1    |       | 2             |      |      |
| Spartan-II/Spartan-II E<br>Virtex/Virtex-E | 4               | 2    | 1    |       | 2             |      |      |
| Virtex-II/Virtex-II Pro                    | 8               | 4    | 2    | 1     | 4             | 2    | 1    |

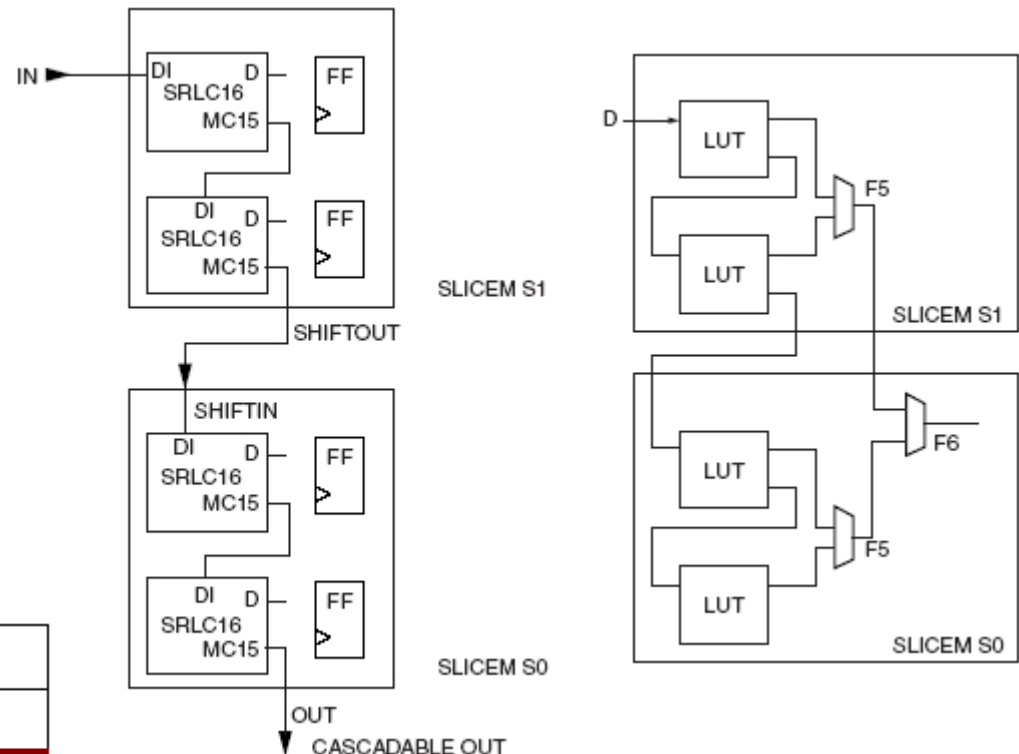


# Spartan-3E (19) CLB – shift reg. SRL16



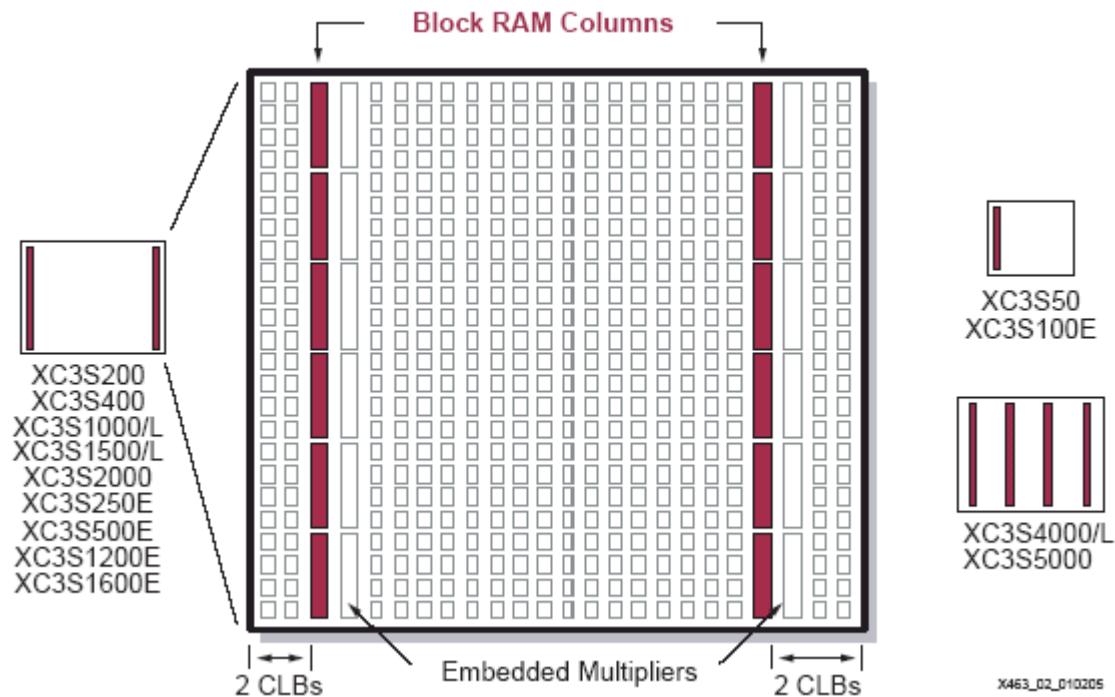
X465\_09\_040209

| Inputs |     |    |   | Outputs |       |
|--------|-----|----|---|---------|-------|
| Am     | CLK | CE | D | Q       | Q15   |
| Am     | X   | 0  | X | Q[Am]   | Q[15] |
| Am     | ↑   | 1  | D | Q[Am-1] | Q[15] |



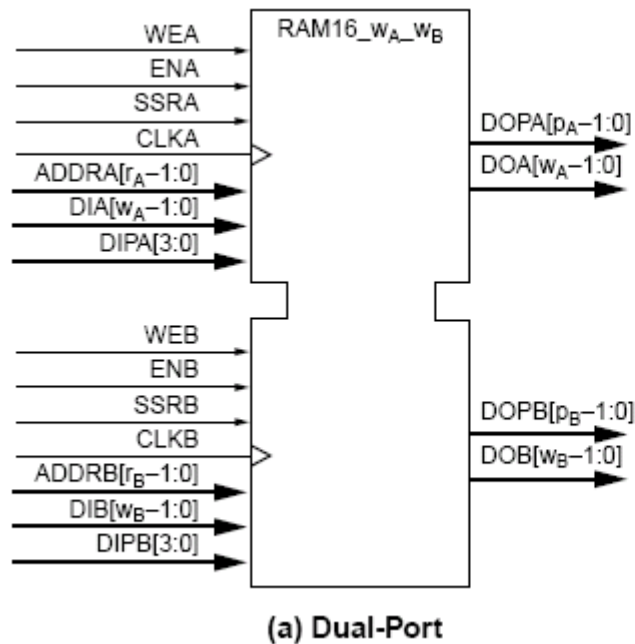
# Spartan-3E (20) Block RAM

| Device    | RAM Columns | RAM Blocks per Column | Total RAM Blocks | Total RAM Bits | Total RAM Kbits |
|-----------|-------------|-----------------------|------------------|----------------|-----------------|
| XC3S100E  | 1           | 4                     | 4                | 73728          | 72              |
| XC3S250E  | 2           | 6                     | 12               | 221184         | 216             |
| XC3S500E  | 2           | 10                    | 20               | 368640         | 360             |
| XC3S1200E | 2           | 14                    | 28               | 516096         | 504             |
| XC3S1600E | 2           | 18                    | 36               | 663552         | 648             |

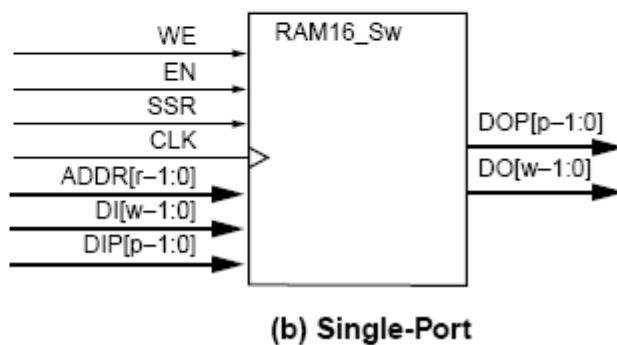




# Spartan-3E (21) Block RAM

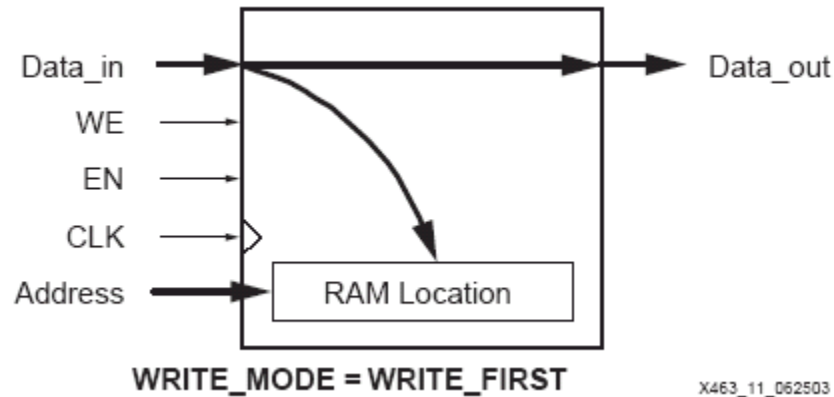


|        |        | Port A  |         |         |         |          |          |
|--------|--------|---------|---------|---------|---------|----------|----------|
|        |        | 16Kx1   | 8Kx2    | 4Kx4    | 2Kx9    | 1Kx18    | 512x36   |
| Port B | 16Kx1  | _S1_S1  |         |         |         |          |          |
|        | 8Kx2   | _S1_S2  | _S2_S2  |         |         |          |          |
|        | 4Kx4   | _S1_S4  | _S2_S4  | _S4_S4  |         |          |          |
|        | 2Kx9   | _S1_S9  | _S2_S9  | _S4_S9  | _S9_S9  |          |          |
|        | 1Kx18  | _S1_S18 | _S2_S18 | _S4_S18 | _S9_S18 | _S18_S18 |          |
|        | 512x36 | _S1_S36 | _S2_S36 | _S4_S36 | _S9_S36 | _S18_S36 | _S36_S36 |

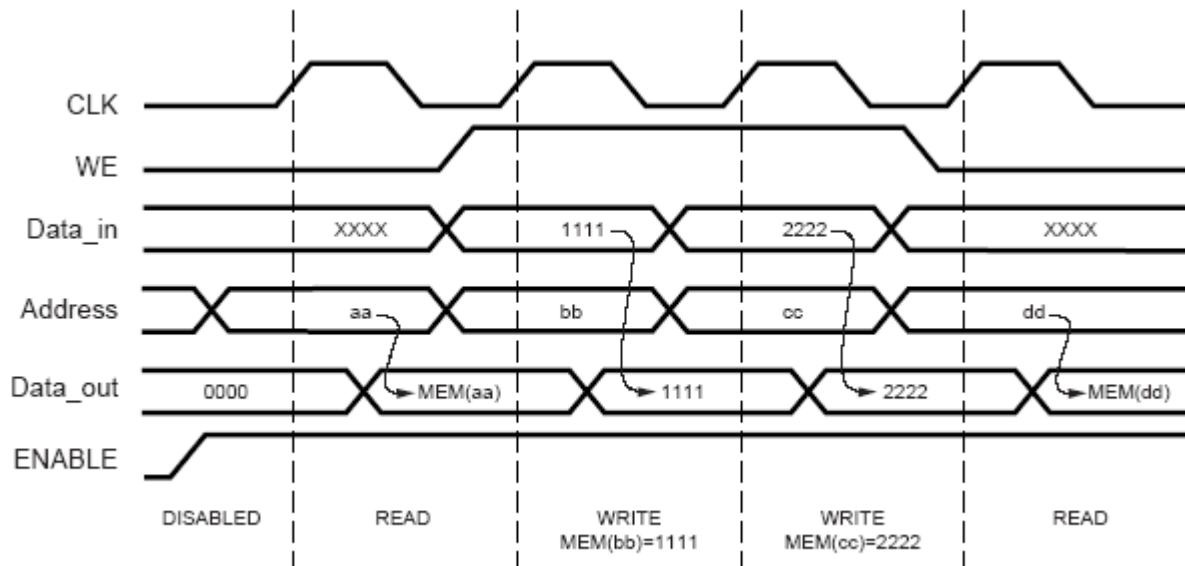


| Organization | Memory Depth | Data Width | Parity Width | DI/DO  | DIP/DOP | ADDR   | Single-Port Primitive | Total RAM Kbits |
|--------------|--------------|------------|--------------|--------|---------|--------|-----------------------|-----------------|
| 512x36       | 512          | 32         | 4            | (31:0) | (3:0)   | (8:0)  | RAMB16_S36            | 18K             |
| 1Kx18        | 1024         | 16         | 2            | (15:0) | (1:0)   | (9:0)  | RAMB16_S18            | 18K             |
| 2Kx9         | 2048         | 8          | 1            | (7:0)  | (0:0)   | (10:0) | RAMB16_S9             | 18K             |
| 4Kx4         | 4096         | 4          | -            | (3:0)  | -       | (11:0) | RAMB16_S4             | 16K             |
| 8Kx2         | 8192         | 2          | -            | (1:0)  | -       | (12:0) | RAMB16_S2             | 16K             |
| 16Kx1        | 16384        | 1          | -            | (0:0)  | -       | (13:0) | RAMB16_S1             | 16K             |

# Spartan-3E (22) Block RAM – modes



X463\_11\_062503



X463\_12\_020503

# Spartan-3E (23) Block RAM – modes

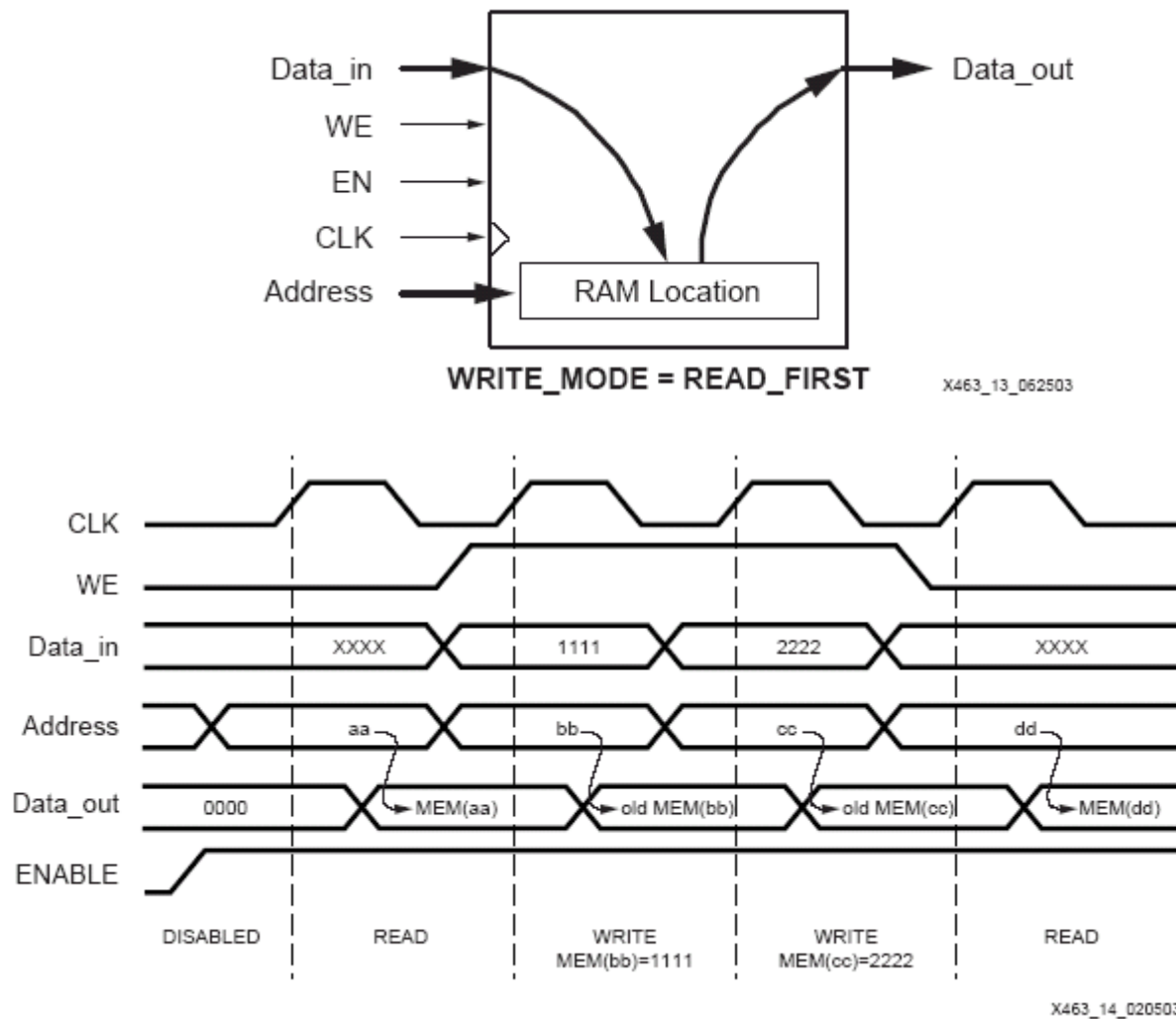
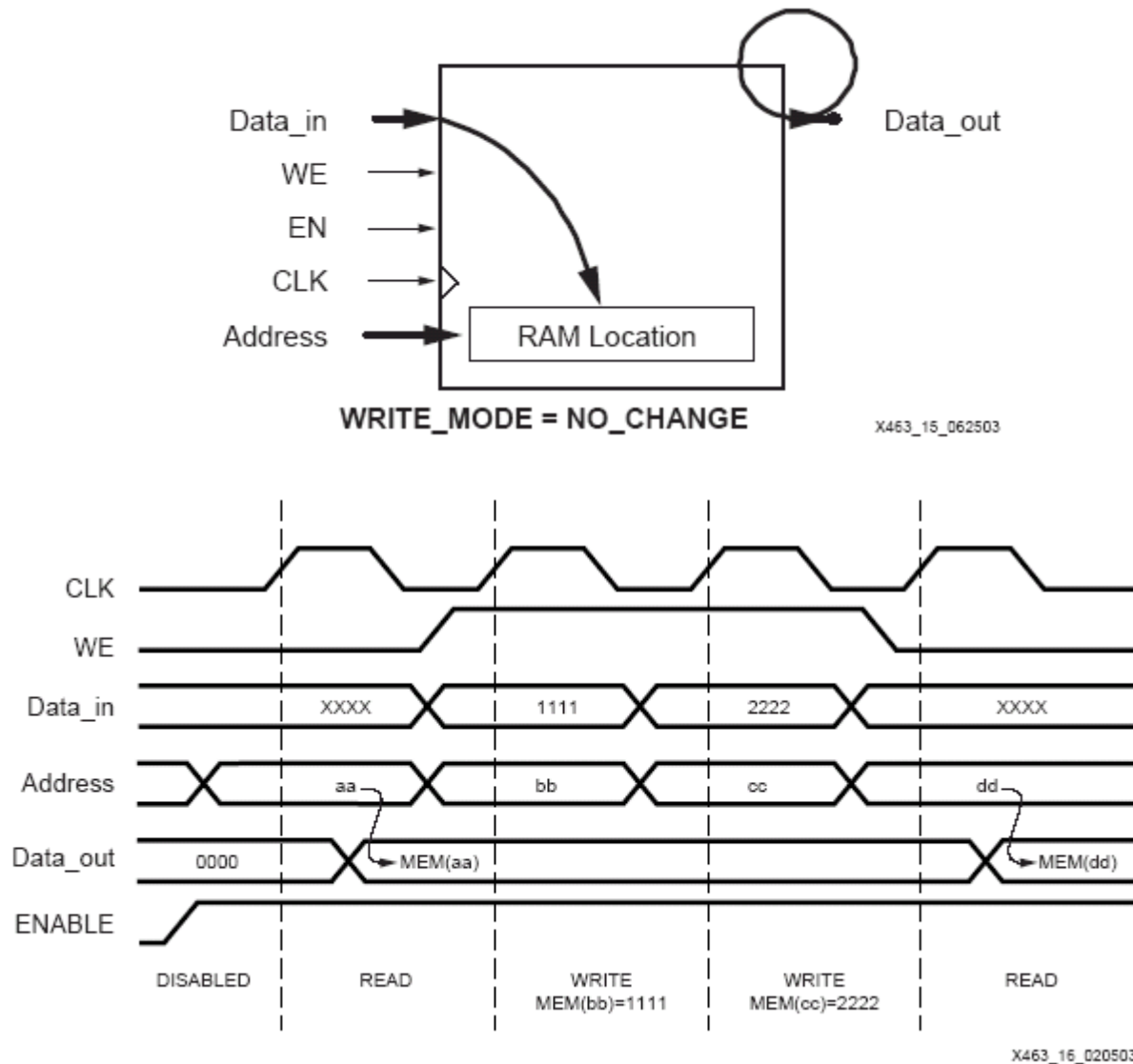
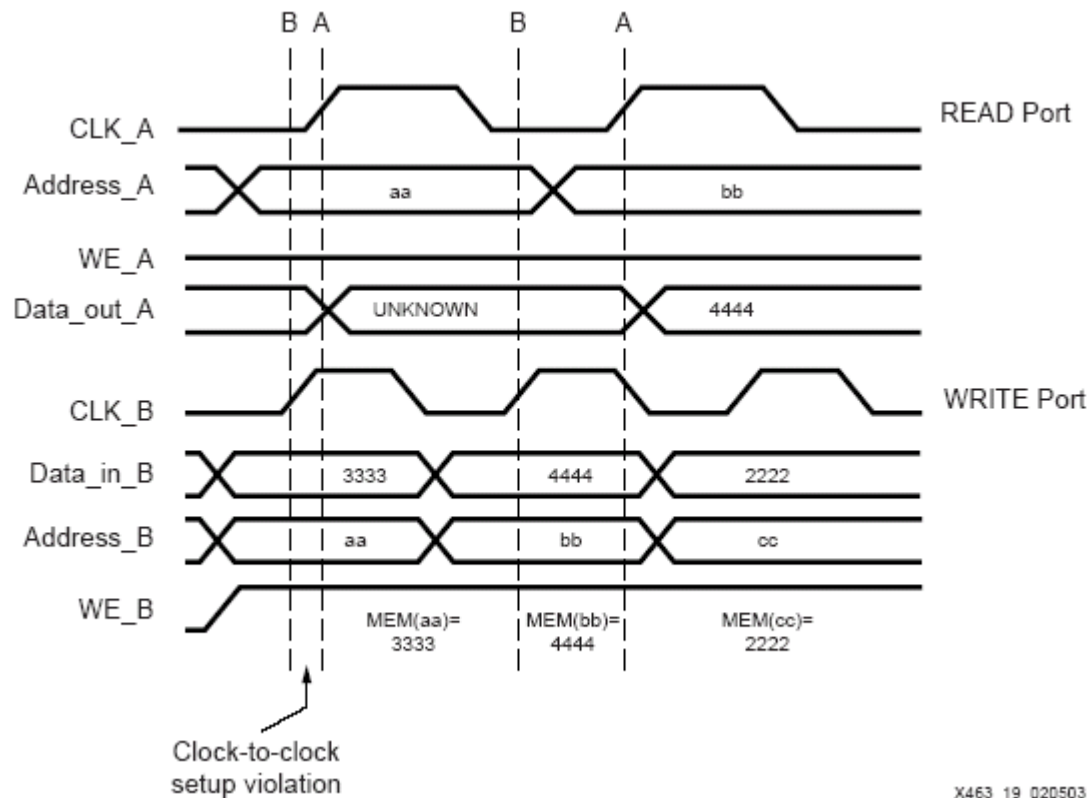


Figure 14: READ\_FIRST Mode Waveforms

# Spartan-3E (24) Block RAM – modes



# Spartan-3E (25) Block RAM – kolizje



X463\_19\_020503

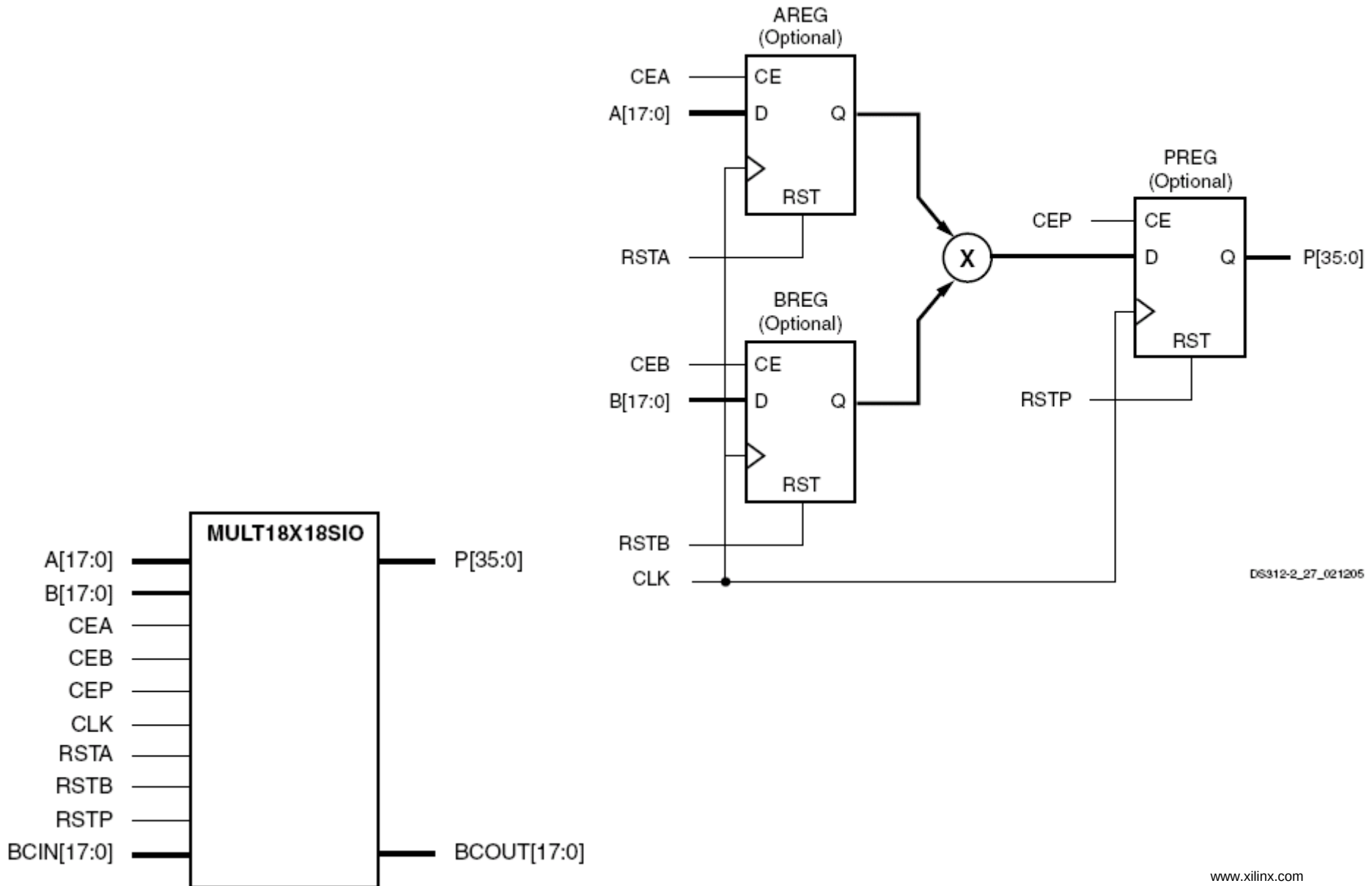
Problemy przy równoczesnym dostępie do tej samej komórki pamięci mogą wystąpić jeżeli:

1. aktywne zbocza obu zegarów (asynchronicznych) następują zbyt szybko po sobie (nie jest zapewniony clock-to-clock setup time);
2. oba porty zapisują inne dane (do tej samej komórki);
3. jeżeli port używa trybu zapisu: NO\_CHANGE lub WRITE\_FIRST, zapis do portu unieważnia dane odczytywane z drugiego portu.

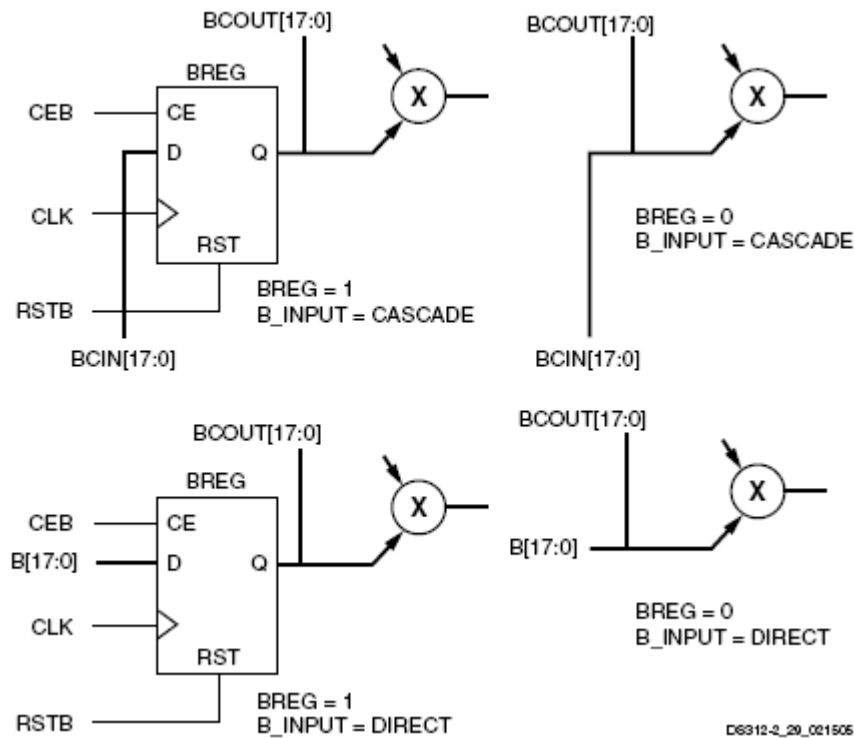
# Spartan-3E (26) Block RAM – zastosowania

- Duże pamięci synchroniczne SRAM jedno i dwuportowe z możliwością inicjacji zawartości podczas konfiguracji.
- Duże pamięci ROM.
- Synchroniczne i asynchroniczne FIFO.
- Bufory cyrkulacyjne.
- Linie opóźniające (np. do FIR).
- Rejestry przesuwne.
- Pamięć typu LIFO (stos) do procesorów.
- Złożone maszyny stanów.
- Szybkie liczniki.
- Szybka implementacja złożonych funkcji logicznych.
- Pamięci wieloportowe (>2).
- Pamięci CAM.
- Cyfrowa bezpośrednia synteza sygnałów.

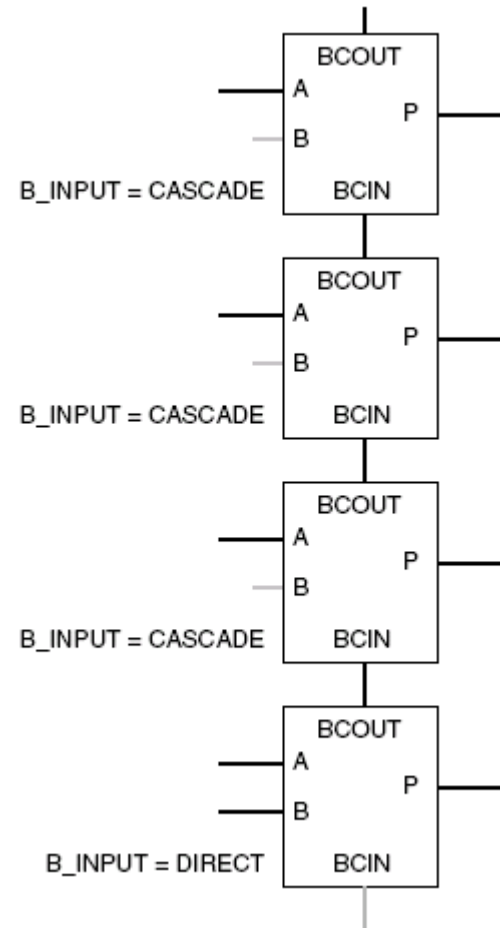
# Spartan-3E (27) Dedicated multiplier



# Spartan-3E (28) multiplier - cascade

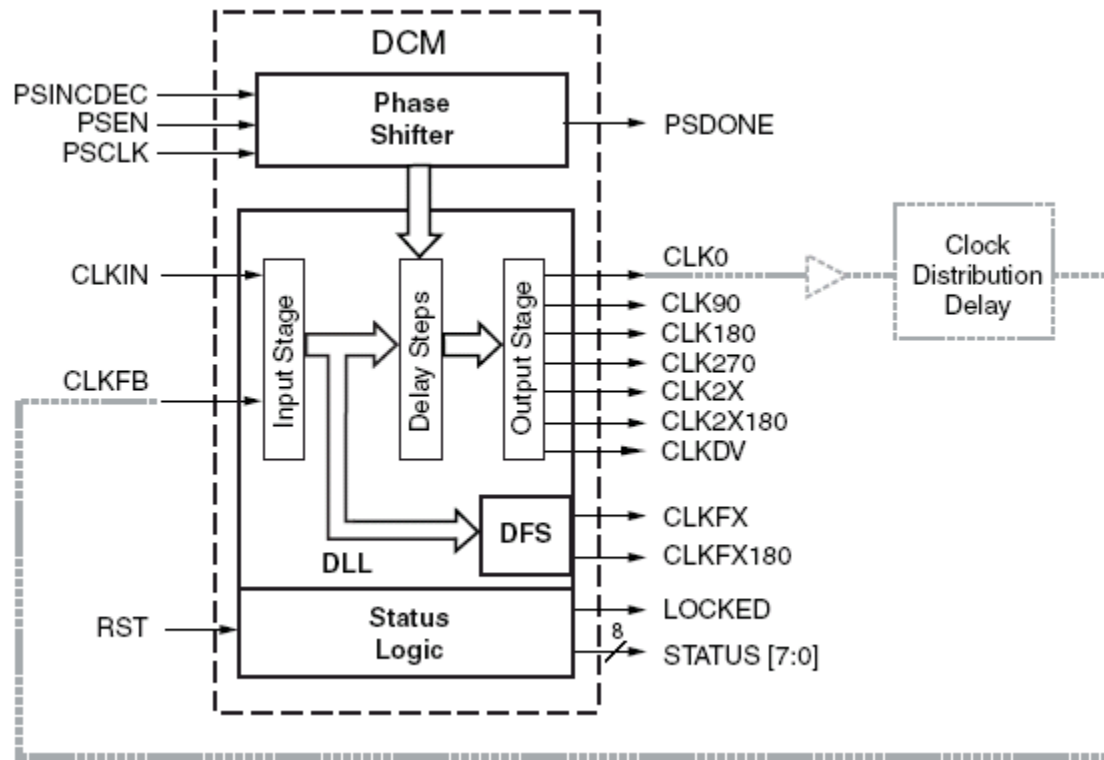


D6312-2\_20\_021505

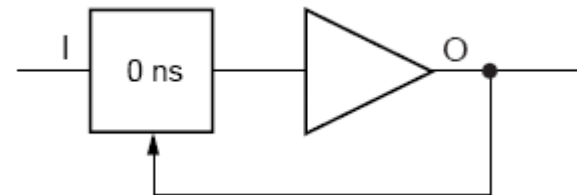




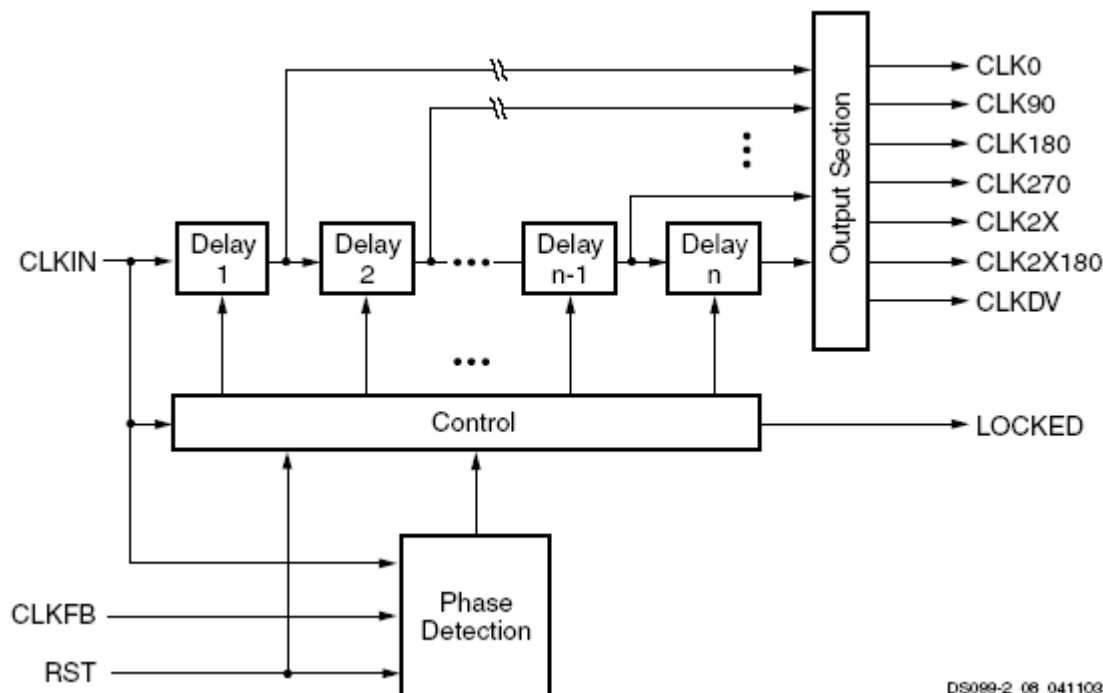
# Spartan-3E (29) DCM



- Clock de-skew
- Frequency synthesis
- Phase shift
- EMI reduction (VIRTEX2)



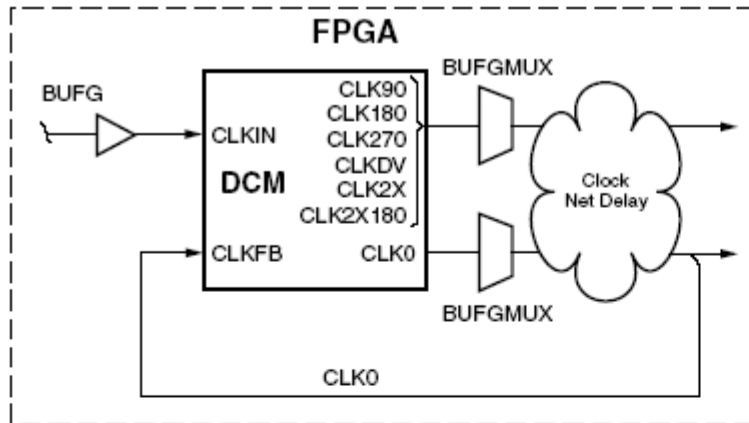
# Spartan-3E (30) DCM - DLL



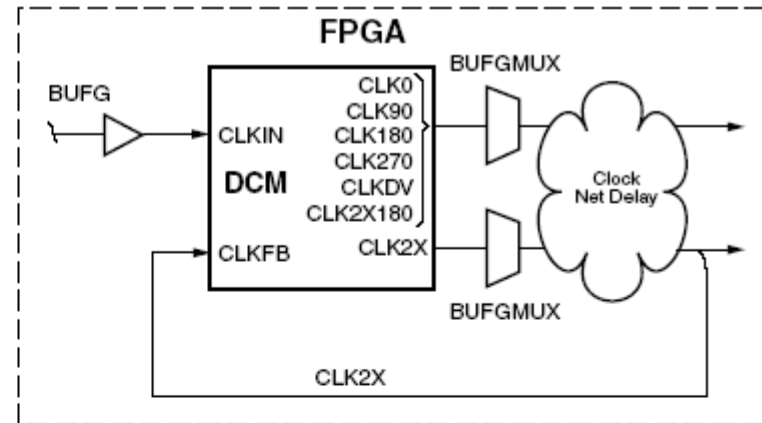
DS099-2\_08\_041103

| Attribute         | Description                                                                                                           | Values                                                                                        |
|-------------------|-----------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| CLK_FEEDBACK      | Chooses either the CLK0 or CLK2X output to drive the CLKFB input                                                      | <u>NONE</u> , 1X, 2X                                                                          |
| CLKIN_DIVIDE_BY_2 | Halves the frequency of the CLKIN signal just as it enters the DCM                                                    | <u>FALSE</u> , TRUE                                                                           |
| CLKDV_DIVIDE      | Selects the constant used to divide the CLKIN input frequency to generate the CLKDV output frequency                  | 1.5, 2, 2.5, 3, 3.5, 4, 4.5, 5, 5.5, 6.0, 6.5, 7.0, 7.5, 8, 9, 10, 11, 12, 13, 14, 15, and 16 |
| CLKIN_PERIOD      | Additional information that allows the DLL to operate with the most efficient lock time and the best jitter tolerance | Floating-point value representing the CLKIN period in nanoseconds                             |

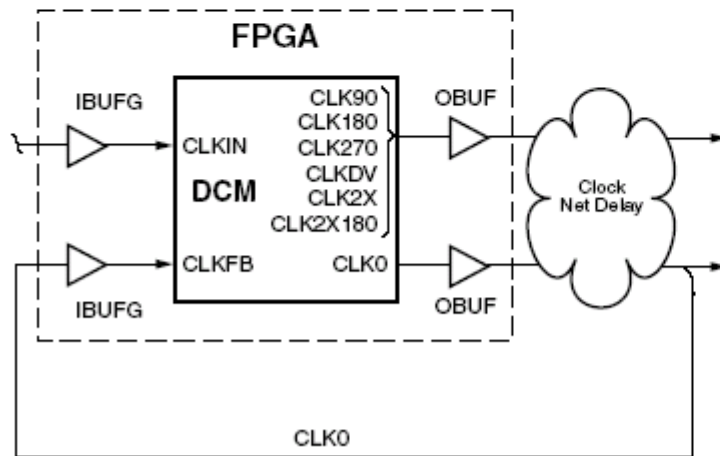
# Spartan-3E (31) DLL – feedback



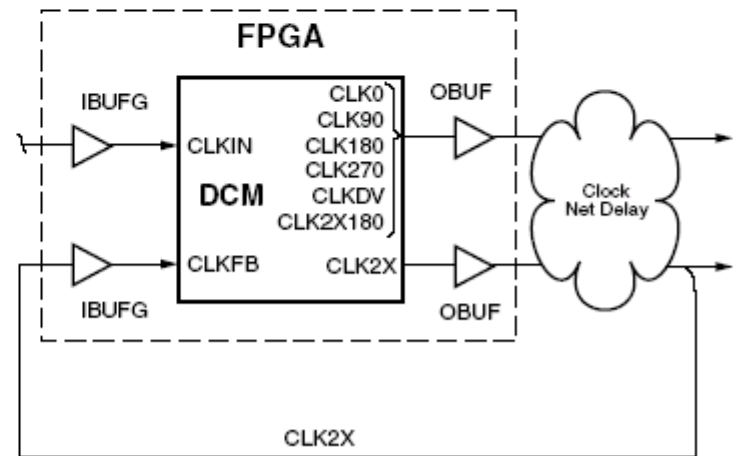
(a) On-Chip with CLK0 Feedback



(b) On-Chip with CLK2X Feedback



(c) Off-Chip with CLK0 Feedback



(d) Off-Chip with CLK2X Feedback

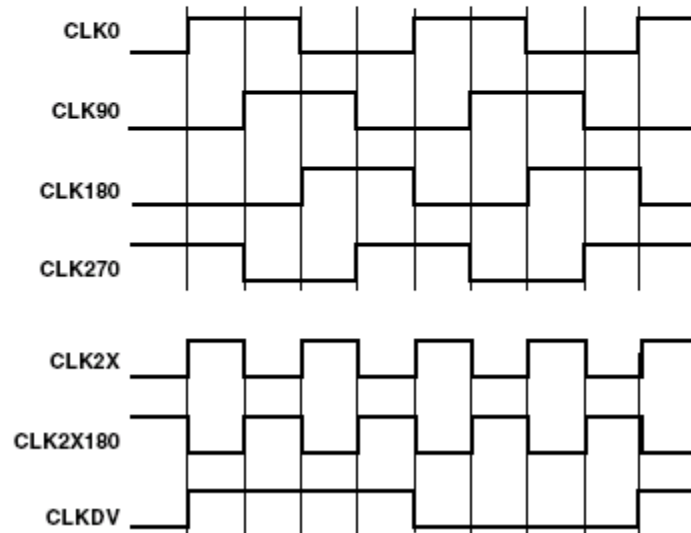
# Spartan-3E (32) DCM - output

Phase:      0°   90° 180° 270° 0°   90° 180° 270° 0°

Input Signal (40%/60% Duty Cycle)



Output Signal - Duty Cycle Corrected



DS000-2\_10\_101105

| Signal   | Direction | Description                                                                                                                                       |
|----------|-----------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| CLKFX    | Output    | Multiplies the CLKIN frequency by the attribute-value ratio (CLKFX_MULTIPLY/CLKFX_DIVIDE) to generate a clock signal with a new target frequency. |
| CLKFX180 | Output    | Generates a clock signal with the same frequency as CLKFX, but shifted 180° out-of-phase.                                                         |

$$f_{\text{CLKFX}} = f_{\text{CLKIN}} \cdot \left( \frac{\text{CLKFX\_MULTIPLY}}{\text{CLKFX\_DIVIDE}} \right)$$

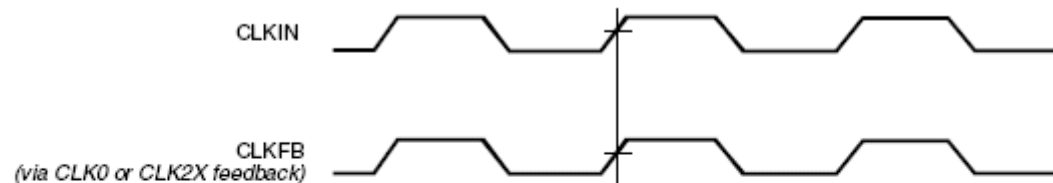
| Attribute      | Description                   | Values                          |
|----------------|-------------------------------|---------------------------------|
| CLKFX_MULTIPLY | Frequency multiplier constant | Integer from 2 to 32, inclusive |
| CLKFX_DIVIDE   | Frequency divisor constant    | Integer from 1 to 32, inclusive |

# Spartan-3E (33) DCM – phase shift

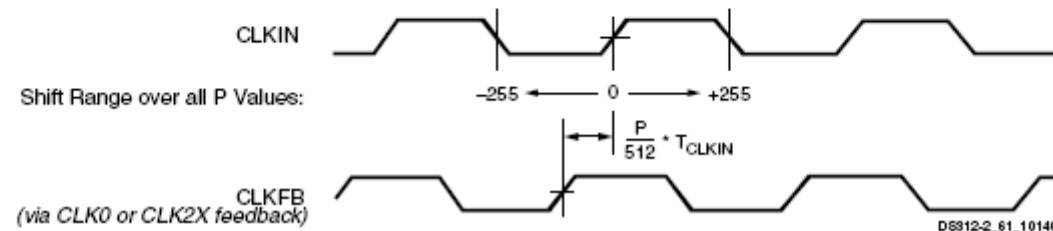
| Attribute          | Description                                                                        | Values                     |
|--------------------|------------------------------------------------------------------------------------|----------------------------|
| CLKOUT_PHASE_SHIFT | Disables the PS component or chooses between Fixed Phase and Variable Phase modes. | NONE, FIXED, VARIABLE      |
| PHASE_SHIFT        | Determines size and direction of initial fine phase shift.                         | Integers from -255 to +255 |

$$T_{PS} = (PHASE\_SHIFT/512) \cdot T_{CLKIN}$$

## a. CLKOUT\_PHASE\_SHIFT = NONE



## b. CLKOUT\_PHASE\_SHIFT = FIXED



# Spartan-3E (34) DCM – phase shift

| Signal                  | Direction | Description                                                                                                                                                                  |
|-------------------------|-----------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| PSEN <sup>(1)</sup>     | Input     | Enables the Phase Shift unit for variable phase adjustment.                                                                                                                  |
| PSCLK <sup>(1)</sup>    | Input     | Clock to synchronize phase shift adjustment.                                                                                                                                 |
| PSINCDEC <sup>(1)</sup> | Input     | When High, increments the current phase shift value. When Low, decrements the current phase shift value. This signal is synchronized to the PSCLK signal.                    |
| PSDONE                  | Output    | Goes High to indicate that the present phase adjustment is complete and PS unit is ready for next phase adjustment request. This signal is synchronized to the PSCLK signal. |

$$T_{PS}(\text{Max}) = \text{VALUE} \bullet \text{DCM\_DELAY\_STEP\_MAX}$$

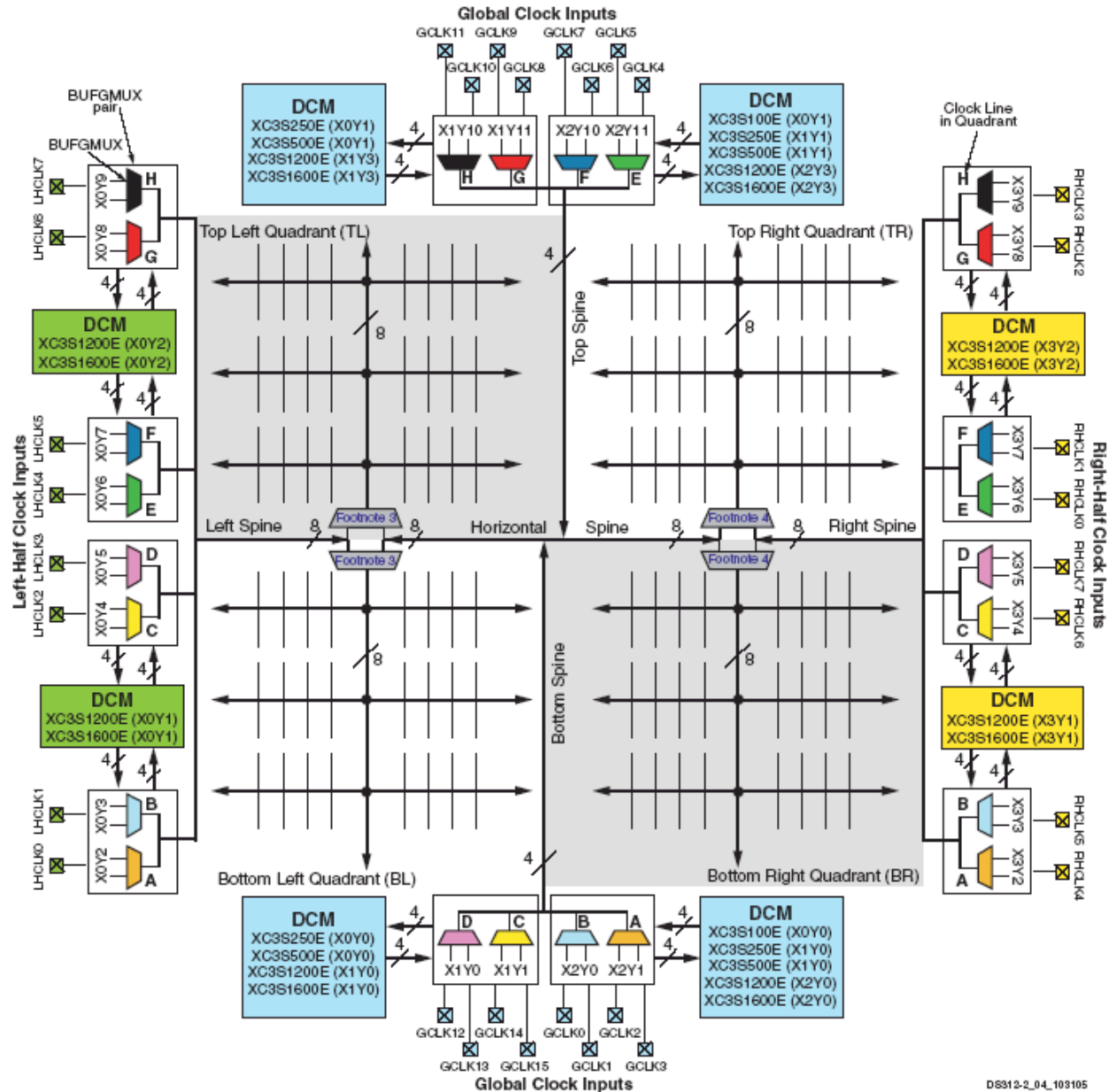
$$T_{PS}(\text{Min}) = \text{VALUE} \bullet \text{DCM\_DELAY\_STEP\_MIN}$$

$$\text{MAX\_STEPS} = \pm[\text{INTEGER}(20 \bullet (T_{\text{CLKIN}} - 3 \text{ ns}))]$$

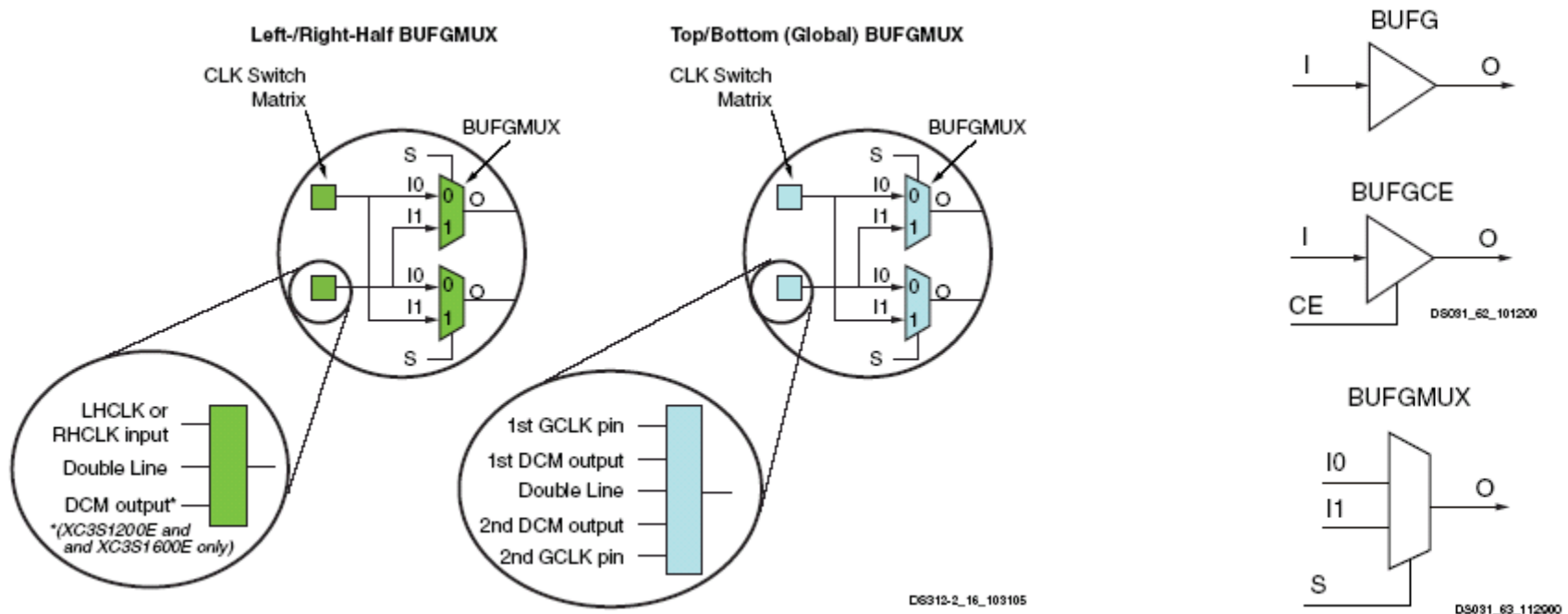
| Signal      | Direction | Description                                                                                                                                                       |
|-------------|-----------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| RST         | Input     | A High resets the entire DCM to its initial power-on state. Initializes the DLL taps for a delay of zero. Sets the LOCKED output Low. This input is asynchronous. |
| STATUS[7:0] | Output    | The bit values on the STATUS bus provide information regarding the state of DLL and PS operation                                                                  |
| LOCKED      | Output    | Indicates that the CLKIN and CLKFB signals are in phase by going High. The two signals are out-of-phase when Low.                                                 |

| Bit | Name          | Description                                                                                                                                                                            |
|-----|---------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 0   | Reserved      | -                                                                                                                                                                                      |
| 1   | CLKIN Stopped | When High, indicates that the CLKIN input signal is not toggling. When Low, indicates CLKIN is toggling. This bit functions only when the CLKFB input is connected. <sup>(1)</sup>     |
| 2   | CLKFX Stopped | When High, indicates that the CLKFX output is not toggling. When Low, indicates the CLKFX output is toggling. This bit functions only when the CLKFX or CLKFX180 output are connected. |

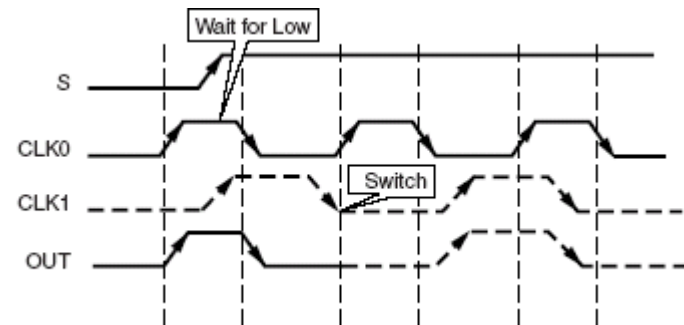
# Spartan-3E (35) dystrybucja zegarów



# Spartan-3E (36) dystrybucja zegarów

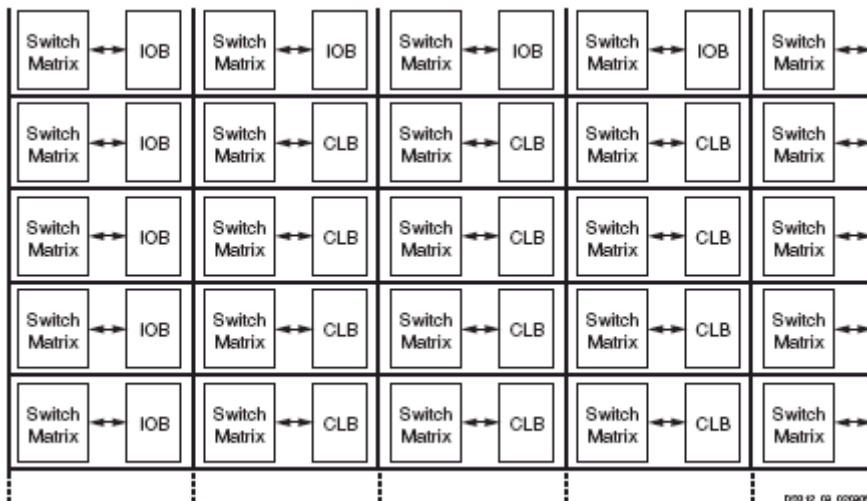
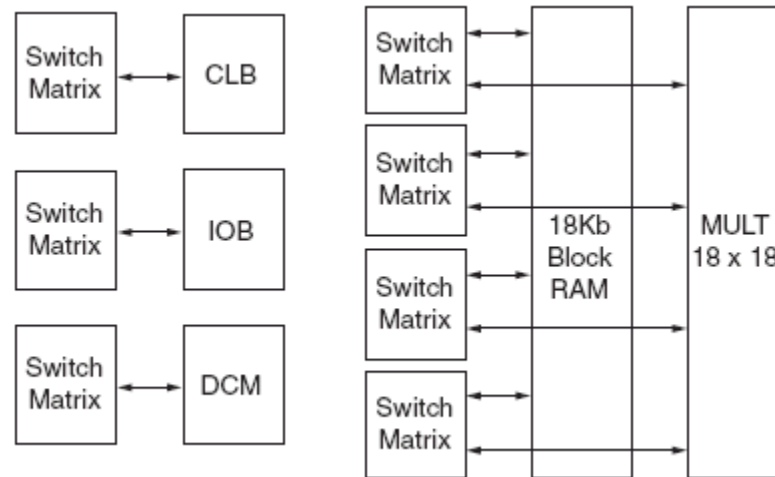


- 16 Global Clock inputs (GCLK0 through GCLK15) located along the top and bottom edges of the FPGA.
- 8 Right-Half Clock inputs (RHCLK0 through RHCLK7) located along the right edge.
- 8 Left-Half Clock inputs (LHCLK0 through LHCLK7) located along the left edge.



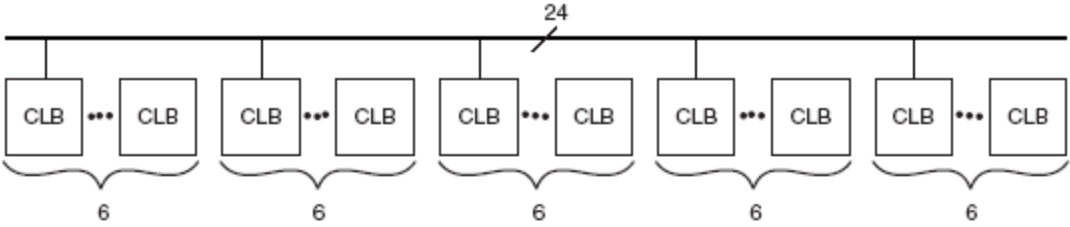
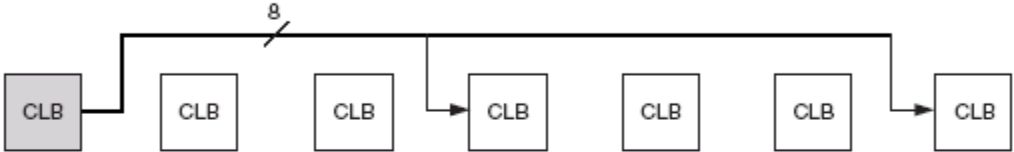
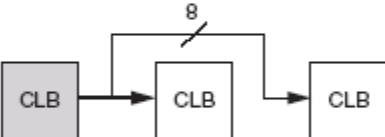
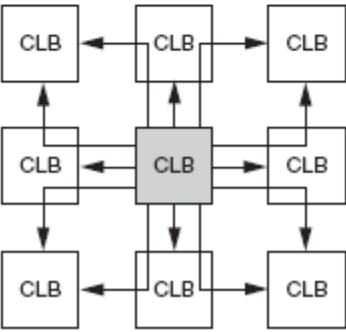


# Spartan-3E (37) połączenia programowalne



| Global Control Input | Description                                                                                                                                                                                                                                                           |
|----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| GSR                  | <b>Global Set/Reset:</b> When High, asynchronously places all registers and flip-flops in their initial state (see <a href="#">Initialization, page 32</a> ). Asserted automatically during the FPGA configuration process (see <a href="#">Start-Up, page 103</a> ). |
| GTS                  | <b>Global Three-State:</b> When High, asynchronously forces all I/O pins to a high-impedance state (Hi-Z, three-state).                                                                                                                                               |

# Spartan-3E (38) połączenia programowalne

|                                                                                         |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
|-----------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Horizontal and Vertical Long Lines</b><br>(horizontal channel shown as an example)   |  <p>The diagram shows a horizontal channel with a long line labeled '24' at the top. Below the line, there are five groups of Configurable Logic Blocks (CLBs). Each group consists of two CLBs connected by three dots, and each group is bracketed with the number '6' below it. The diagram illustrates a long horizontal connection spanning multiple CLB groups.</p> <p>DS312-2_10_022905</p> |
| <b>Horizontal and Vertical Hex Lines</b><br>(horizontal channel shown as an example)    |  <p>The diagram shows a horizontal channel with a line labeled '8' at the top. Below the line, there are six Configurable Logic Blocks (CLBs) arranged in a row. The first CLB is shaded gray. The diagram illustrates a horizontal connection spanning multiple CLBs.</p> <p>DS312-2_11_020005</p>                                                                                                |
| <b>Horizontal and Vertical Double Lines</b><br>(horizontal channel shown as an example) |  <p>The diagram shows a horizontal channel with a line labeled '8' at the top. Below the line, there are three Configurable Logic Blocks (CLBs) arranged in a row. The first CLB is shaded gray. The diagram illustrates a horizontal connection spanning multiple CLBs.</p> <p>DS312-2_15_022305</p>                                                                                              |
| <b>Direct Connections</b>                                                               |  <p>The diagram shows a 3x3 grid of Configurable Logic Blocks (CLBs). The central CLB is shaded gray. Arrows indicate direct connections between the central CLB and its eight neighbors (up, down, left, right, and diagonally). The diagram illustrates direct connections between CLBs.</p> <p>DS312-2_12_020005</p>                                                                           |

# Synteza bloków funkcjonalnych FPGA

# Primitives and macros – synteza (1)

```
Synthesizing Unit <timecore>.
Related source file is timecore.vhd.
Found finite state machine <FSM_0> for signal <state>.
...
Found 7-bit subtractor for signal <fsm_sig1>.
Found 7-bit subtractor for signal <fsm_sig2>.
Found 7-bit register for signal <min>.
Found 4-bit register for signal <points_tmp>.
...
Summary:
inferred 1 Finite State Machine(s).
inferred 18 D-type flip-flop(s).
inferred 10 Adder/Subtractor(s).
Unit <timecore> synthesized.
...
Synthesizing Unit <divider>.
Related source file is divider.vhd.
Found 18-bit up counter for signal <counter>.
Found 1 1-bit 2-to-1 multiplexers.
Summary:
inferred 1 Counter(s).
inferred 1 Multiplexer(s).
Unit <divider> synthesized. ...
```

# Primitives and macros – synteza (2)

```
=====
* Advanced HDL Synthesis *
=====
Implementing FSM <FSM_0> on signal <current_state> on BRAM.
INFO:Xst - Data output of ROM <Mrom_tmp_one_hot> in block <decode> is
tied to register <one_hot> in block <decode>.
INFO:Xst - The register is removed and the ROM is implemented as
readonly
block RAM.
...
...
=====
HDL Synthesis Report
Macro Statistics
# FSMs : 1
# ROMs : 4
16x7-bit ROM : 4
# Registers : 3
7-bit register : 2
4-bit register : 1
# Counters : 1
18-bit up counter : 1
# Multiplexers : 1
2-to-1 multiplexer : 1
# Adders/Subtractors : 10
7-bit adder : 4
7-bit subtractor : 6
=====
...
```

# Primitives and macros – synteza (3)

...

=====

## Final Results

...

## Macro Statistics

# FSMs : 1

# ROMs : 4

16x7-bit ROM : 4

# Registers : 7

7-bit register : 2

1-bit register : 4

18-bit register : 1

# Adders/Subtractors : 11

7-bit adder : 4

7-bit subtractor : 6

18-bit adder : 1

...

=====

...

# Inferred macros in VHDL (1)

```
-- synteza SRL16
library ieee;
use ieee.std_logic_1164.all;
entity shift is
port(
    C, SI : in std_logic;
    SO : out std_logic
);
end shift;

architecture archi of shift is
    signal tmp: std_logic_vector(7 downto 0);
begin
    process (C)
    begin
        if (C'event and C='1') then
            for i in 0 to 6 loop
                tmp(i+1) <= tmp(i);
            end loop;
            tmp(0) <= SI;
        end if;
    end process;
    SO <= tmp(7);
end archi;
```

```
...
Synthesizing Unit <shift>.
Related source file is
shift_registers_1.vhd.
Found 8-bit shift register for
signal <tmp<7>>.
Summary:
inferred 1 Shift register(s).
Unit <shift> synthesized.
=====
HDL Synthesis Report
Macro Statistics
# Shift Registers : 1
8-bit shift register : 1
=====
...
```

# Inferred macros in VHDL (2)

```
-- synteza SRL16
-- dynamiczny rejestr przesuwny
library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.std_logic_unsigned.all;

entity shiftregluts is
port(
  CLK : in std_logic;
  DATA : in std_logic;
  CE : in std_logic;
  A : in std_logic_vector(3 downto 0);
  Q : out std_logic
);
end shiftregluts;
```

```
architecture rtl of shiftregluts is
constant DEPTH_WIDTH : integer := 16;
type SRL_ARRAY is array (0 to DEPTH_WIDTH-1) of std_logic;
signal SRL_SIG : SRL_ARRAY;
begin
PROC_SRL16 : process (CLK)
begin
  if (CLK'event and CLK = '1') then
    if (CE = '1') then
      SRL_SIG <= DATA & SRL_SIG(0 to DEPTH_WIDTH-2);
    end if;
  end if;
end process;
Q <= SRL_SIG(conv_integer(A));
end rtl;
```

```
...
Synthesizing Unit <dynamic_srl>.
Related source file is dynamic_srl.vhd.
Found 1-bit 16-to-1 multiplexer for
signal <Q>.
Found 16-bit register for signal <data>.
Summary:
inferred 16 D-type flip-flop(s).
inferred 1 Multiplexer(s).
Unit <dynamic_srl> synthesized.
...
Macro Statistics
# Shift Registers : 1
# 16-bit dynamic shift register : 1
...
```



# Inferred macros in VHDL (3)

```
-- synteza enkodera z wykorzystaniem MUXCY
library ieee;
use ieee.std_logic_1164.all;
entity priority is
port (
    sel : in std_logic_vector (7 downto 0);
    code :out std_logic_vector (2 downto 0)
);
end priority;

architecture archi of priority is
begin
code <=
    "000" when sel(0) = '1' else
    "001" when sel(1) = '1' else
    "010" when sel(2) = '1' else
    "011" when sel(3) = '1' else
    "100" when sel(4) = '1' else
    "101" when sel(5) = '1' else
    "110" when sel(6) = '1' else
    "111" when sel(7) = '1' else
    "---";
end archi;
```

```
...
Synthesizing Unit <priority>.
Related source file is
priority_encoders_1.vhd.
Found 3-bit 1-of-9 priority encoder
for signal <code>.
Summary:
inferred 3 Priority encoder(s).
Unit <priority> synthesized.
=====
HDL Synthesis Report
Macro Statistics
# Priority Encoders : 1
3-bit 1-of-9 priority encoder : 1
=====
...
```

# Inferred macros in VHDL (4)

Sterowanie procesem syntezy za pomocą atrybutów:

```
attribute priority_extract: string;  
attribute priority_extract of {signal_name|entity_name}:  
{signal|entity} is "{yes|no|force}";
```

Przykład:

```
attribute priority_extract: string;  
attribute priority_extract of code: signal is "force";
```

# Inferred macros in VHDL (5)

```
-- synteza sumatora z wejściem i wyjściem carry
```

```
library ieee;
```

```
use ieee.std_logic_1164.all;
```

```
use ieee.std_logic_arith.all;
```

```
use ieee.std_logic_unsigned.all;
```

```
entity adder is
```

```
port(
```

```
    A, B : in std_logic_vector(7 downto 0);
```

```
    CI : in std_logic;
```

```
    SUM : out std_logic_vector(7 downto 0);
```

```
    CO : out std_logic
```

```
);
```

```
end adder;
```

```
architecture archi of adder is
```

```
    signal tmp: std_logic_vector(8 downto 0);
```

```
begin
```

```
    tmp <= conv_std_logic_vector((conv_integer(A) +  
                                   conv_integer(B) +  
                                   conv_integer(CI)),9);
```

```
    SUM <= tmp(7 downto 0);
```

```
    CO <= tmp(8);
```

```
end archi;
```

```
...
```

```
Synthesizing Unit <adder>.
```

```
Related source file is
```

```
arithmetic_operations_1.vhd.
```

```
Found 8-bit adder for signal <sum>.
```

```
Summary:
```

```
inferred 1 Adder/Subtractor(s).
```

```
Unit <adder> synthesized.
```

```
=====
```

```
HDL Synthesis Report
```

```
Macro Statistics
```

```
# Adders/Subtractors : 1
```

```
8-bit adder : 1
```

```
=====
```

```
...
```

# Inferred macros in VHDL (6)

```
library ieee; -- Synteza mnożnika typu potokowego
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;
entity mult is
generic(
  A_port_size: integer := 18;
  B_port_size: integer := 18
);
port(
  clk : in std_logic;
  A : in unsigned (A_port_size-1 downto 0);
  B : in unsigned (B_port_size-1 downto 0);
  MULT : out unsigned ((A_port_size+B_port_size-1) downto 0)
);
end mult;
architecture beh of mult is
signal a_in, b_in : unsigned (A_port_size-1 downto 0);
signal mult_res : unsigned ((A_port_size+B_port_size-1) downto 0);
signal pipe_2, pipe_3 : unsigned ((A_port_size+B_port_size-1) downto 0);
begin
  process (clk)
  begin
    if (clk'event and clk='1') then
      a_in <= A; b_in <= B;
      mult_res <= a_in * b_in;
      pipe_2 <= mult_res;
      pipe_3 <= pipe_2;
      MULT <= pipe_3;
    end if;
  end process;
end beh;
```

O syntezie mnożnika decyduje atrybut **mult\_style**:

```
attribute mult_style: string;
attribute mult_style of
{signal_name|entity_name}: {signal|entity} is
"{auto|block|lut|pipe_lut|pipe_block|KCM}";
```

Dla układów Virtex, Virtex-E, Spartan-II i SpartanIIE domyślnie jest auto. Dla układów Virtex-II, Virtex-II Pro, Virtex-II Pro X i Spartan-3 domyślnie jest lut.

# Inferred macros in VHDL (7)

Przykład współdzielenia zasobów (resource sharing)  
(kontrolowane przez atrybut: RESOURCE\_SHARING):

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_unsigned.all;

entity addsub is
port(
    A, B, C : in std_logic_vector(7 downto 0);
    OPER : in std_logic;
    RES : out std_logic_vector(7 downto 0)
);
end addsub;

architecture archi of addsub is
begin
    RES <= A + B when OPER='0' else A - C;
end archi;
```

INFO:Xst - HDL ADVISOR - Resource sharing  
has identified that some arithmetic  
operations in this design can share the  
same physical resources for reduced device  
utilization. For improved clock frequency  
you may try to disable resource sharing.  
...

# Inferred macros in VHDL (8)

```
-- Pamięć distributed RAM (jednoportowa) z odczytem asynchronicznym
```

```
library ieee;
```

```
use ieee.std_logic_1164.all;
```

```
use ieee.std_logic_unsigned.all;
```

```
entity raminfr is
```

```
port (
```

```
    clk : in std_logic;
```

```
    we : in std_logic;
```

```
    a : in std_logic_vector(4 downto 0);
```

```
    di : in std_logic_vector(3 downto 0);
```

```
    do : out std_logic_vector(3 downto 0)
```

```
);
```

```
end raminfr;
```

```
architecture syn of raminfr is
```

```
type ram_type is array (31 downto 0)
```

```
    of std_logic_vector (3 downto 0);
```

```
signal RAM : ram_type;
```

```
begin
```

```
process (clk)
```

```
begin
```

```
    if (clk'event and clk = '1') then
```

```
        if (we = '1') then
```

```
            RAM(conv_integer(a)) <= di;
```

```
        end if;
```

```
    end if;
```

```
end process;
```

```
do <= RAM(conv_integer(a));
```

```
end syn;
```

```
...
```

Synthesizing Unit <raminfr>.

Related source file is rams\_1.vhd.

Found 128-bit single-port distributed RAM for signal <ram>.

|              |                           |      |  |
|--------------|---------------------------|------|--|
| aspect ratio | 32-word x 4-bit           |      |  |
| clock        | connected to signal <clk> | rise |  |
| write enable | connected to signal <we>  | high |  |
| address      | connected to signal <a>   |      |  |
| data in      | connected to signal <di>  |      |  |
| data out     | connected to signal <do>  |      |  |
| ram_style    | Auto                      |      |  |

INFO:Xst - For optimized device usage and improved timings, you may take advantage of available block RAM resources by registering the read address.

Summary:

inferred 1 RAM(s).

Unit <raminfr> synthesized.

=====

HDL Synthesis Report

Macro Statistics

# RAMs : 1

128-bit single-port distributed RAM : 1

=====

...

# Inferred macros in VHDL (9)

```
-- Pamięć block RAM (jednoportowa) typu read first
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_unsigned.all;

entity raminfr is
port (
  clk : in std_logic;
  we : in std_logic;
  en : in std_logic;
  addr : in std_logic_vector(4 downto 0);
  di : in std_logic_vector(3 downto 0);
  do : out std_logic_vector(3 downto 0)
);
end raminfr;

architecture syn of raminfr is
type ram_type is array (31 downto 0) of std_logic_vector (3 downto 0);
signal RAM : ram_type;
begin
process (clk)
begin
  if clk'event and clk = '1' then
    if en = '1' then
      if we = '1' then
        RAM(conv_integer(addr)) <= di;
      end if;
      do <= RAM(conv_integer(addr));
    end if;
  end if;
end process;
end syn;
```

# Inferred macros in VHDL (10)

```
-- Pamięć block RAM (jednoportowa) typu write first
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_unsigned.all;
entity raminfr is
port (
    clk : in std_logic;
    we : in std_logic;
    en : in std_logic;
    addr : in std_logic_vector(4 downto 0);
    di : in std_logic_vector(3 downto 0);
    do : out std_logic_vector(3 downto 0));
end raminfr;
architecture syn of raminfr is
type ram_type is array (31 downto 0) of std_logic_vector (3 downto 0);
signal RAM : ram_type;
signal read_addr : std_logic_vector(4 downto 0);
begin
process (clk)
begin
    if clk'event and clk = '1' then
        if en = '1' then
            if we = '1' then
                mem(conv_integer(addr)) <= di;
            end if;
            read_addr <= addr;
        end if;
    end if;
end process;
do <= ram(conv_integer(read_addr));
end syn;
```



# Inferred macros in VHDL (11)

```
-- Pamięć block RAM (dwuportowa) z dwoma
-- zegarami typu write_first
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_unsigned.all;
entity raminfr is
port (
    clka : in std_logic;
    clk b : in std_logic;
    wea : in std_logic;
    addra : in std_logic_vector(4 downto 0);
    addr b : in std_logic_vector(4 downto 0);
    dia : in std_logic_vector(3 downto 0);
    doa : out std_logic_vector(3 downto 0);
    dob : out std_logic_vector(3 downto 0));
end raminfr;
architecture syn of raminfr is
type ram_type is array (31 downto 0)
    of std_logic_vector (3 downto 0);
signal RAM : ram_type;
signal read_addra : std_logic_vector(4 downto 0);
signal read_addr b : std_logic_vector(4 downto 0);
begin
process (clka)
begin
    if (clka'event and clka = '1') then
        if (wea = '1') then
            RAM(conv_integer(addra)) <= dia;
        end if;
        read_addra <= addra;
    end if;
end process;
```

```
process (clk b)
begin
    if (clk b'event and clk b = '1') then
        read_addr b <= addr b;
    end if;
end process;
doa <= RAM(read_addra);
dob <= RAM(read_addr b);
end syn;
```

# Inferred macros in VHDL (12)

```
-- inicjalizacja pamięci block RAM
...
type ram_type is array (0 to 63) of std_logic_vector(19 downto 0);
signal RAM : ram_type :=
( X"0200A", X"00300", X"08101", X"04000", X"08601", X"0233A", X"00300",
  X"08602", X"02310", X"0203B", X"08300", X"04002", X"08201", X"00500",
  X"04001", X"02500", X"00340", X"00241", X"04002", X"08300", X"08201",
  X"00500", X"08101", X"00602", X"04003", X"0241E", X"00301", X"00102",
  X"02122", X"02021", X"00301", X"00102", X"02222", X"04001", X"00342",
  X"0232B", X"00900", X"00302", X"00102", X"04002", X"00900", X"08201",
  X"02023", X"00303", X"02433", X"00301", X"04004", X"00301", X"00102",
  X"02137", X"02036", X"00301", X"00102", X"02237", X"04004", X"00304",
  X"04040", X"02500", X"02500", X"02500", X"0030D", X"02341", X"08201",
  X"0400D");
...
process (clk)
begin
    if rising_edge(clk) then
        if we = '1' then
            RAM(conv_integer(a)) <= di;
        end if;
        ra <= a;
    end if;
end process;
...
do <= RAM(conv_integer(ra));
```

# Instantiated macros in VHDL (1)

```
library UNISIM;
use UNISIM.vcomponents.all;
...
...
...
begin
...
...
-- LUT4: 4-input Look-Up Table with general output
-- Xilinx HDL Language Template version 6.3.1i
LUT4_inst : LUT4
generic map (INIT => X"8001")
port map (
    0 => 0,    -- LUT general output
    I0 => I0,   -- LUT input
    I1 => I1,   -- LUT input
    I2 => I2,   -- LUT input
    I3 => I3    -- LUT input
);

-- Funkcja: (I0 and I1 and I2 and I3) or (~I0 and ~I1 and ~I2 and ~I3)
```

# Instantiated macros in VHDL (2)

```
library UNISIM;
use UNISIM.vcomponents.all;
...
begin
...
-- CLKDLL: Delay Locked Loop Circuit for Virtex and Spartan-II
-- (Low frequency)
-- Xilinx HDL Language Template version 6.3.1i
CLKDLL_inst : CLKDLL
generic map (
    CLKDV_DIVIDE => 2.0, -- Divide by: 1.5,2.0,2.5,3.0,4.0,5.0,8.0 or 16.0
    DUTY_CYCLE_CORRECTION => TRUE, -- Duty cycle correction, TRUE or FALSE
    STARTUP_WAIT => FALSE) -- Delay config DONE until DLL LOCK, TRUE/FALSE
port map (
    CLK0 => CLK0,      -- 0 degree DLL CLK output
    CLK180 => CLK180,  -- 180 degree DLL CLK output
    CLK270 => CLK270,  -- 270 degree DLL CLK output
    CLK2X => CLK2X,    -- 2X DLL CLK output
    CLK90 => CLK90,    -- 90 degree DLL CLK output
    CLKDV => CLKDV,    -- Divided DLL CLK out (CLKDV_DIVIDE)
    LOCKED => LOCKED,  -- DLL LOCK status output
    CLKFB => CLKFB,    -- DLL clock feedback
    CLKIN => CLKIN,    -- Clock input (from IBUFG, BUFG or DLL)
    RST => RST         -- DLL asynchronous reset input
); -- End of CLKDLL_inst instantiation
```

# Instantiated macros in VHDL (3)

```
library UNISIM;
use UNISIM.vcomponents.all;
...
begin
...
-- STARTUP_SPARTAN3: Startup primitive for GSR, GTS or startup sequence
-- control. Virtex-II/II-Pro
-- Xilinx HDL Language Template version 6.3.1i
STARTUP_SPARTAN3_inst : STARTUP_SPARTAN3
port map (
    CLK => CLK,          -- Clock input for start-up sequence
    GSR => GSR_PORT,     -- Global Set/Reset input (GSR cannot be used for the port name)
    GTS => GTS_PORT      -- Global 3-state input (GTS cannot be used for the port name)
);      -- End of STARTUP_SPARTAN3_inst instantiation

-- KEEPER: I/O Buffer Weak Keeper
--          All FPGA, CoolRunner-II
-- Xilinx HDL Language Template version 6.3.1i
KEEPER_inst : KEEPER
port map (
    0 => 0              -- Keeper output (connect directly to top-level port)
);      -- End of KEEPER_inst instantiation
```

# Instantiated macros in VHDL (4)

```
library UNISIM;
use UNISIM.vcomponents.all;

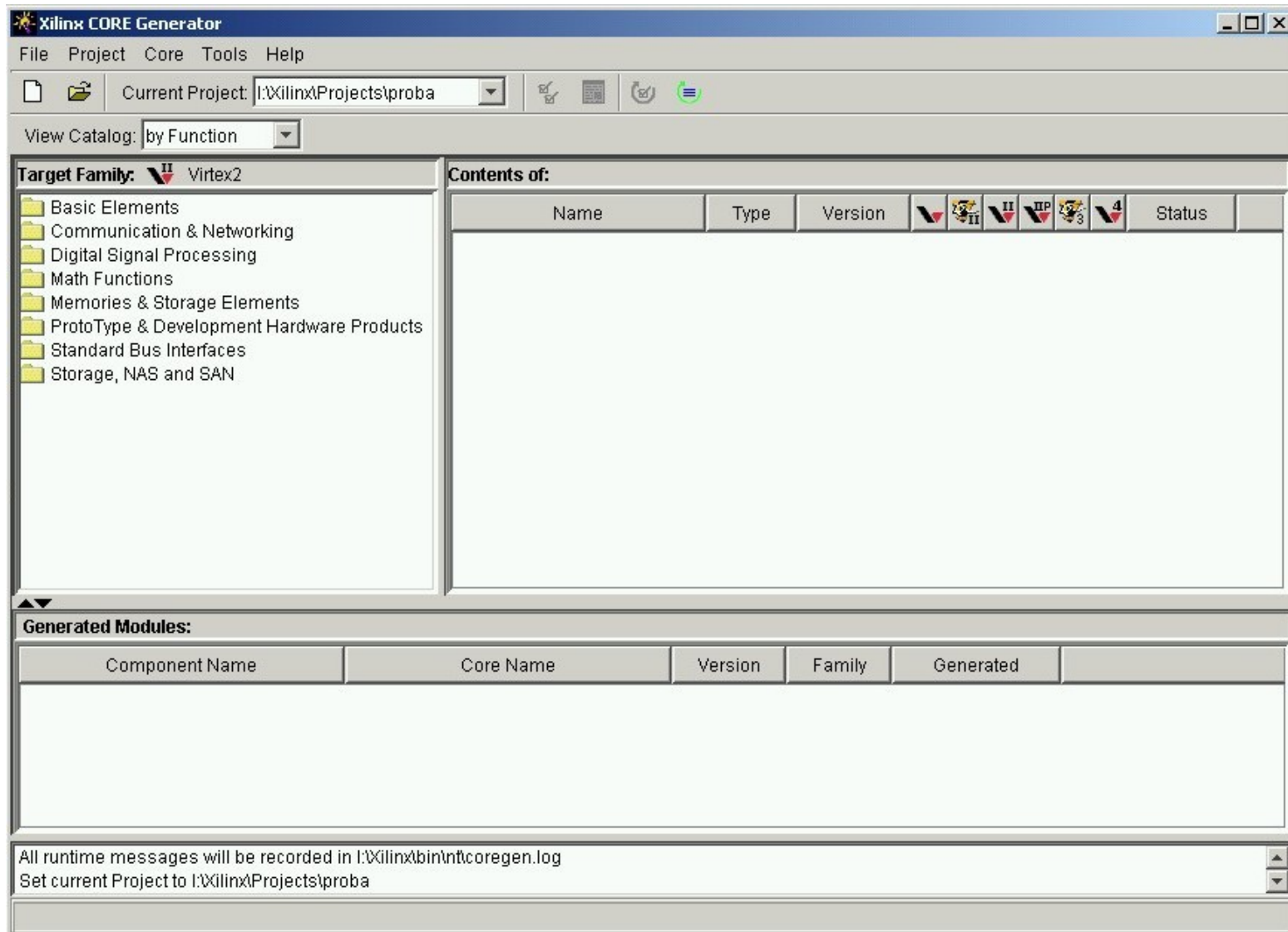
-- OFDDRCPE: Double Data Rate Output Register with Async. Clear, Async. Preset
--           and Clock Enable. Virtex-II/II-Pro, Spartan-3
-- Xilinx HDL Language Template version 6.3.1i
OFDDRCPE_inst : OFDDRCPE
port map (
    Q => Q,      -- Data output (connect directly to top-level port)
    C0 => C0,     -- 0 degree clock input
    C1 => C1,     -- 180 degree clock input
    CE => CE,     -- Clock enable input
    CLR => CLR,   -- Asynchronous reset input
    D0 => D0,     -- Posedge data input
    D1 => D1,     -- Negedge data input
    PRE => PRE    -- Asynchronous preset input
); -- End of OFDDRCPE_inst instantiation

-- ORCY: Carry-Chain OR-gate
-- Xilinx HDL Language Template version 6.3.1i
ORCY_inst : ORCY port map (
    O => O,      -- OR output signal
    CI => CI,    -- Carry input signal
    I => I       -- Data input signal
); -- End of ORCY_inst instantiation
```

## Instantiated macros in VHDL (5)

[illegible]

# Xilinx Core generator (1)





# Xilinx Core generator (2)

The screenshot displays the Xilinx CORE Generator application window. The 'Current Project' is set to 'I:\Xilinx\Projects\proba'. The 'View Catalog' is set to 'by Function'. The 'Target Family' is 'Virtex2'. The 'Contents of: Memories & Storage Elements > FIFOs' section shows a table of available FIFO cores.

| Name              | Type     | Version | LogiCORE | II | II Pro | II Pro | II Pro | II Pro | Status |
|-------------------|----------|---------|----------|----|--------|--------|--------|--------|--------|
| Asynchronous FIFO | LogiCORE | 6.1     | ◆        | ◆  | ◆      | ◆      | ◆      | ◆      |        |
| Fifo Generator    | LogiCORE | 2.0     | ◆        | ◆  | ◆      | ◆      | ◆      | ◆      |        |
| Synchronous FIFO  | LogiCORE | 5.0     | ◆        | ◆  | ◆      | ◆      | ◆      | ◆      |        |

The 'Generated Modules:' section is currently empty. The status bar at the bottom indicates that all runtime messages will be recorded in 'I:\Xilinx\bin\nt\coregen.log' and that the current project is set to 'I:\Xilinx\Projects\proba'. A note states: 'The Synchronous FIFO LogiCORE is a drop-in module for the Virtex(TM), Virtex(TM)-E, Virtex(TM)-II, Virtex(TM)-II Pro, Spartan(TM)-II, Spartan(TM)-IIE, and Sp...'.

# Xilinx Core generator (3)

Asynchronous FIFO

Parameters Core Overview Contact Web Links

Parameters

Component Name : cross\_clk\_domain

Memory Type

☒ Block Memory

☐ Distributed Memory

Optional Flag

☒ Almost Full Flag

☐ Almost Empty Flag

Handshaking Options ...

Data Port Parameters

Input Data Width : 16 Valid Range : 1..256

FIFO Depth : 15

Data Count

☐ Write Data Count (Synchronized with Write clk)

Write Data Count Width : 2 Valid Range 1..4

☐ Read Data Count (Synchronized with Read clk)

Read Data Count Width : 2 Valid Range 1..4

Layout

☐ Create RPM

DIN FULL

ALMOST\_FULL

WR\_ACK

WR\_ERR

WR\_COUNT

DOUT

EMPTY

ALMOST\_EMPTY

RD\_ACK

RD\_ERR

RD\_COUNT

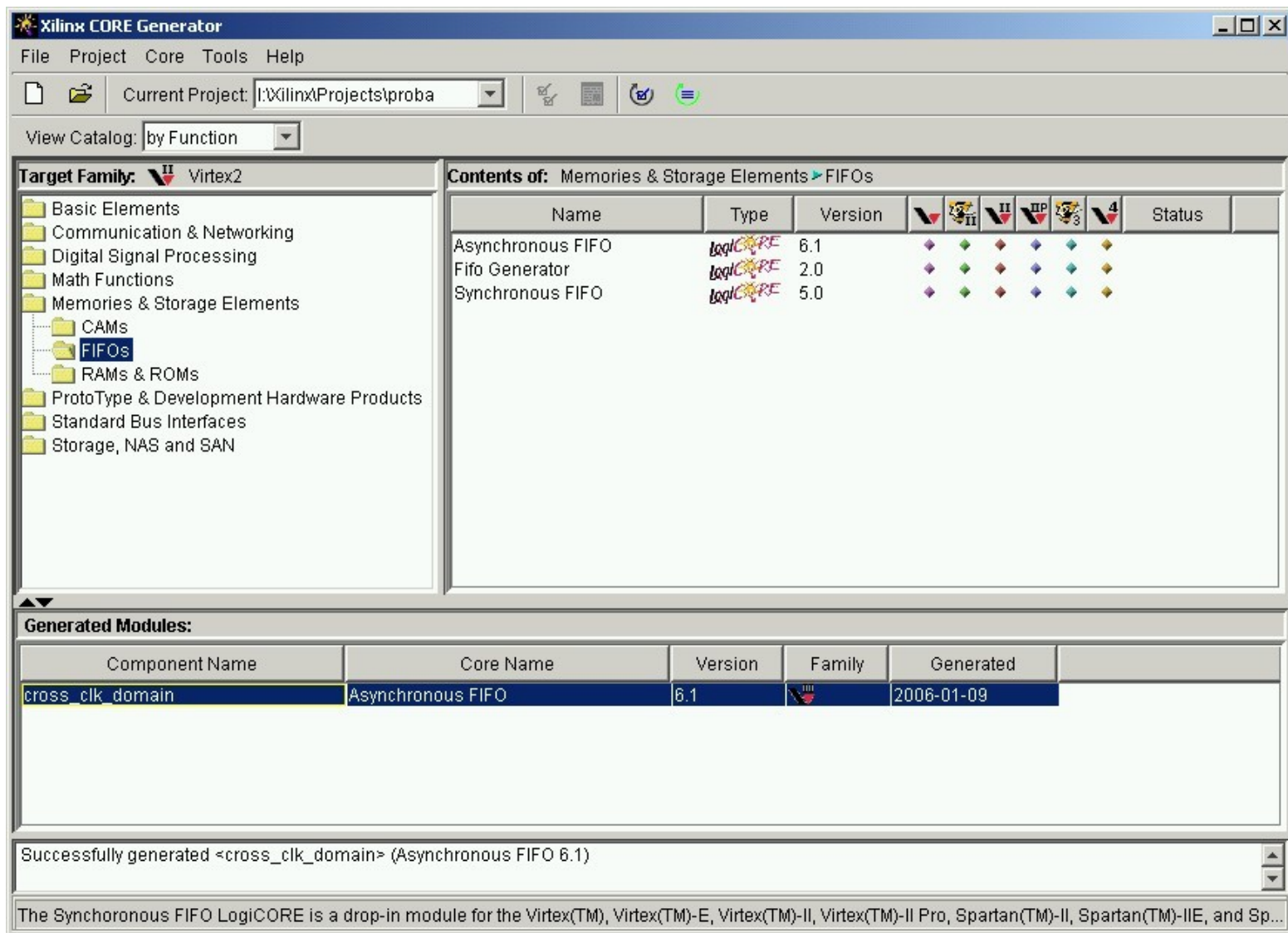
AINIT

RD\_EN

RD\_CLK

Generate Dismiss Data Sheet... Version Info... ☐ Display Core Footprint

# Xilinx Core generator (4)



# Xilinx Core generator (5)

```
# Output products list for <cross_clk_domain>
cross_clk_domain.asy
cross_clk_domain.edn
cross_clk_domain.vhd
cross_clk_domain.vho
cross_clk_domain.xco
cross_clk_domain.xcp
cross_clk_domain.sym
```

Dodatkowo czasami występują pliki: \*.coe

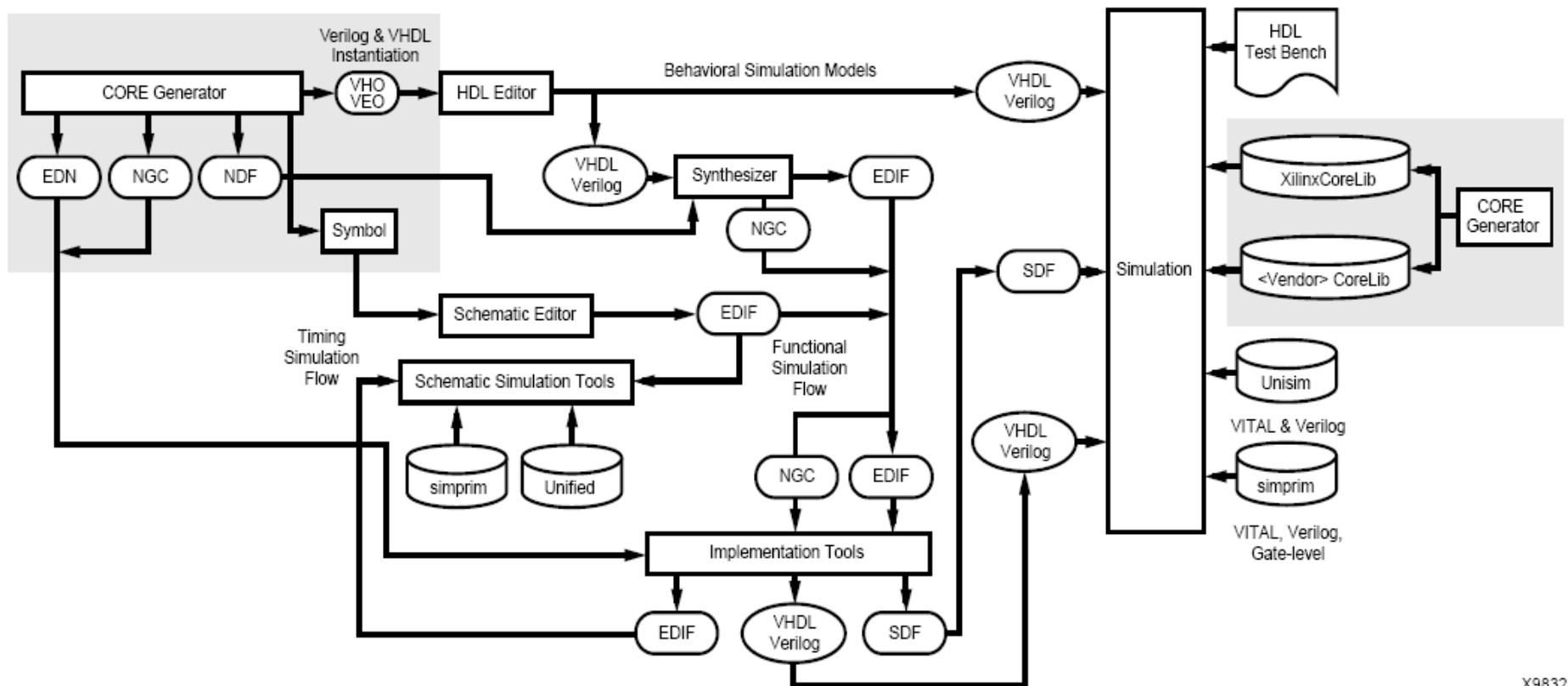
Zawierają one listę współczynników dla DSP, lub zawartość pamięci dla RAM i ROM.

Do ich edycji służy opcja w Core Generator (Tools - Memory Editor), ale także mogą być edytowane ręcznie za pomocą edytorów tekstowych.

Istnieją programy do konwersji z plików binarnych, np: bin2coe.

- **IP (Intellectual Property)**

# Xilinx Core generator (6)



X9832

# Konfiguracja układów FPGA

# Konfiguracja Spartan-3E (1)

|                                                           | Master Serial                         | SPI                                | BPI                                                                                    | Slave Parallel                                                                             | Slave Serial                                                                      | JTAG                                                                       |
|-----------------------------------------------------------|---------------------------------------|------------------------------------|----------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|----------------------------------------------------------------------------|
| M[2:0] mode pin settings                                  | <0:0:0>                               | <0:0:1>                            | <0:1:0>=Up<br><0:1:1>=Down                                                             | <1:1:0>                                                                                    | <1:1:1>                                                                           | <1:0:1>                                                                    |
| Data width                                                | Serial                                | Serial                             | Byte-wide                                                                              | Byte-wide                                                                                  | Serial                                                                            | Serial                                                                     |
| Configuration memory source                               | Xilinx <a href="#">Platform Flash</a> | Industry-standard SPI serial Flash | Industry-standard parallel NOR Flash or Xilinx parallel <a href="#">Platform Flash</a> | Any source via microcontroller, CPU, Xilinx parallel <a href="#">Platform Flash</a> , etc. | Any source via microcontroller, CPU, Xilinx <a href="#">Platform Flash</a> , etc. | Any source via microcontroller, CPU, <a href="#">System ACE™ CF</a> , etc. |
| Clock source                                              | Internal oscillator                   | Internal oscillator                | Internal oscillator                                                                    | External clock on CCLK pin                                                                 | External clock on CCLK pin                                                        | External clock on TCK pin                                                  |
| Total I/O pins borrowed during configuration              | 8                                     | 13                                 | 46                                                                                     | 21                                                                                         | 8                                                                                 | 0                                                                          |
| Configuration mode for downstream daisy-chained FPGAs     | Slave Serial                          | Slave Serial                       | Slave Parallel                                                                         | Slave Parallel or Memory Mapped                                                            | Slave Serial                                                                      | JTAG                                                                       |
| Stand-alone FPGA applications (no external download host) | ✓                                     | ✓                                  | ✓                                                                                      | Possible using XCFxxP Platform Flash, which optionally generates CCLK                      | Possible using XCFxxP Platform Flash, which optionally generates CCLK             |                                                                            |
| Uses low-cost, industry-standard Flash                    |                                       | ✓                                  | ✓                                                                                      |                                                                                            |                                                                                   |                                                                            |
| Supports optional MultiBoot, multi-configuration mode     |                                       |                                    | ✓                                                                                      |                                                                                            |                                                                                   |                                                                            |

# Konfiguracja Spartan-3E (2)

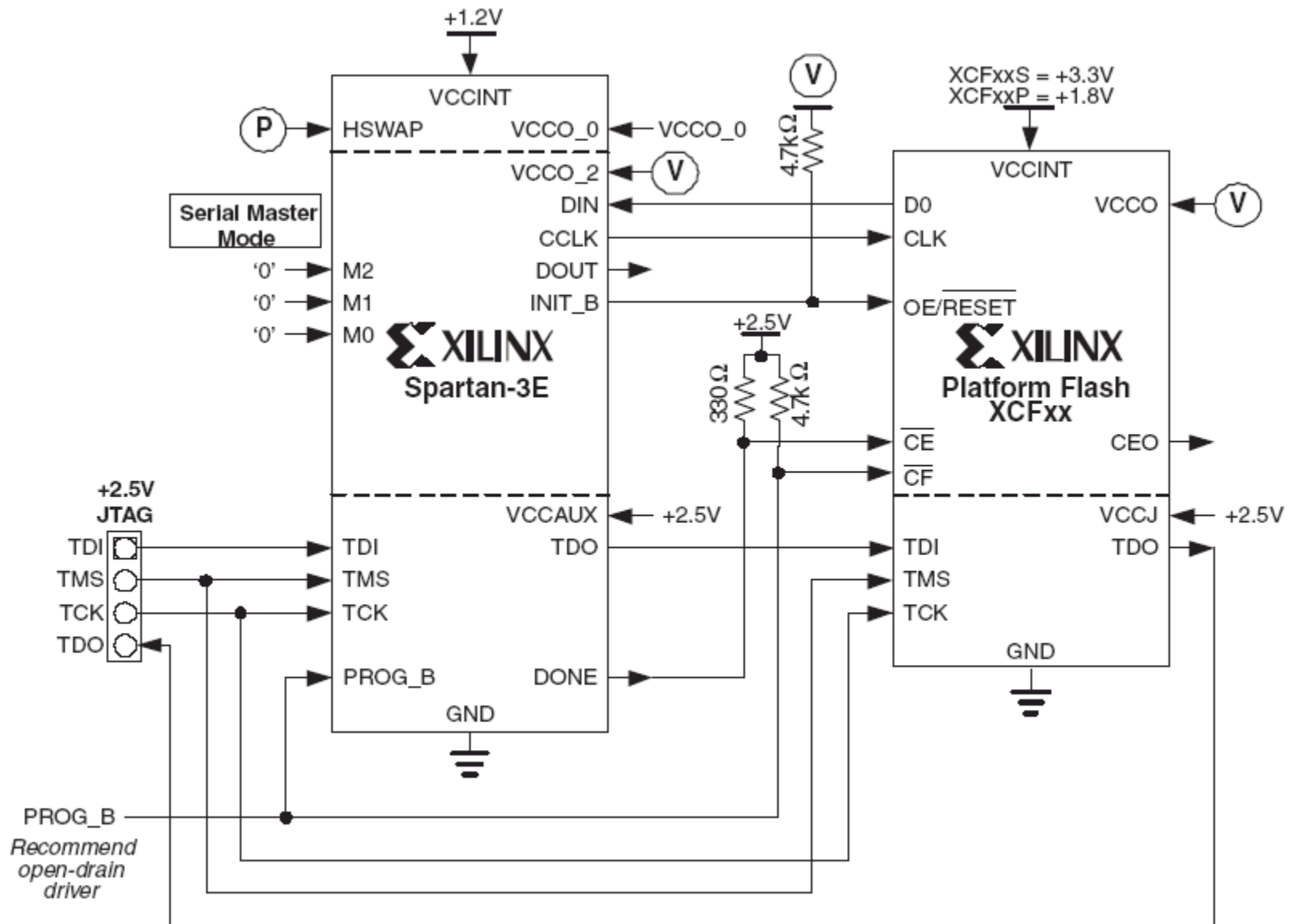
| Spartan-3E FPGA | Number of Configuration Bits |
|-----------------|------------------------------|
| XC3S100E        | 581,344                      |
| XC3S250E        | 1,353,728                    |
| XC3S500E        | 2,270,208                    |
| XC3S1200E       | 3,837,184                    |
| XC3S1600E       | 5,964,672                    |

| HSWAP Value | Description                                                                                                                                                                              |
|-------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 0           | Pull-up resistors connect to the associated $V_{CCO}$ supply for all user-I/O or dual-purpose I/O pins during configuration. Pull-up resistors are active until configuration completes. |
| 1           | Pull-up resistors disabled during configuration. All user-I/O or dual-purpose I/O pins are in a high-impedance state.                                                                    |

| Pin Name | FPGA Direction               | Description                                                                                                                                                                                                                                                                                                                                                   | During Configuration                                                                                                                                          | After Configuration                                                                           |
|----------|------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| DONE     | Open-drain bidirectional I/O | <b>FPGA Configuration Done.</b> Low during configuration. Goes High when FPGA successfully completes configuration. Requires external 330 $\Omega$ pull-up resistor to 2.5V.                                                                                                                                                                                  | Connects to PROM's chip-enable (CE) input. Enables PROM during configuration. Disables PROM after configuration.                                              | Pulled High via external pull-up. When High, indicates that the FPGA successfully configured. |
| PROG_B   | Input                        | <b>Program FPGA.</b> Active Low. When asserted Low for 300 ns or longer, forces the FPGA to restart its configuration process by clearing configuration memory and resetting the DONE and INIT_B pins once PROG_B returns High. Requires external 4.7 k $\Omega$ pull-up resistor to 2.5V. If driving externally, use an open-drain or open-collector driver. | Must be High during configuration to allow configuration to start. Connects to PROM's CF pin, allowing JTAG PROM programming algorithm to reprogram the FPGA. | Drive PROG_B Low and release to reprogram FPGA.                                               |



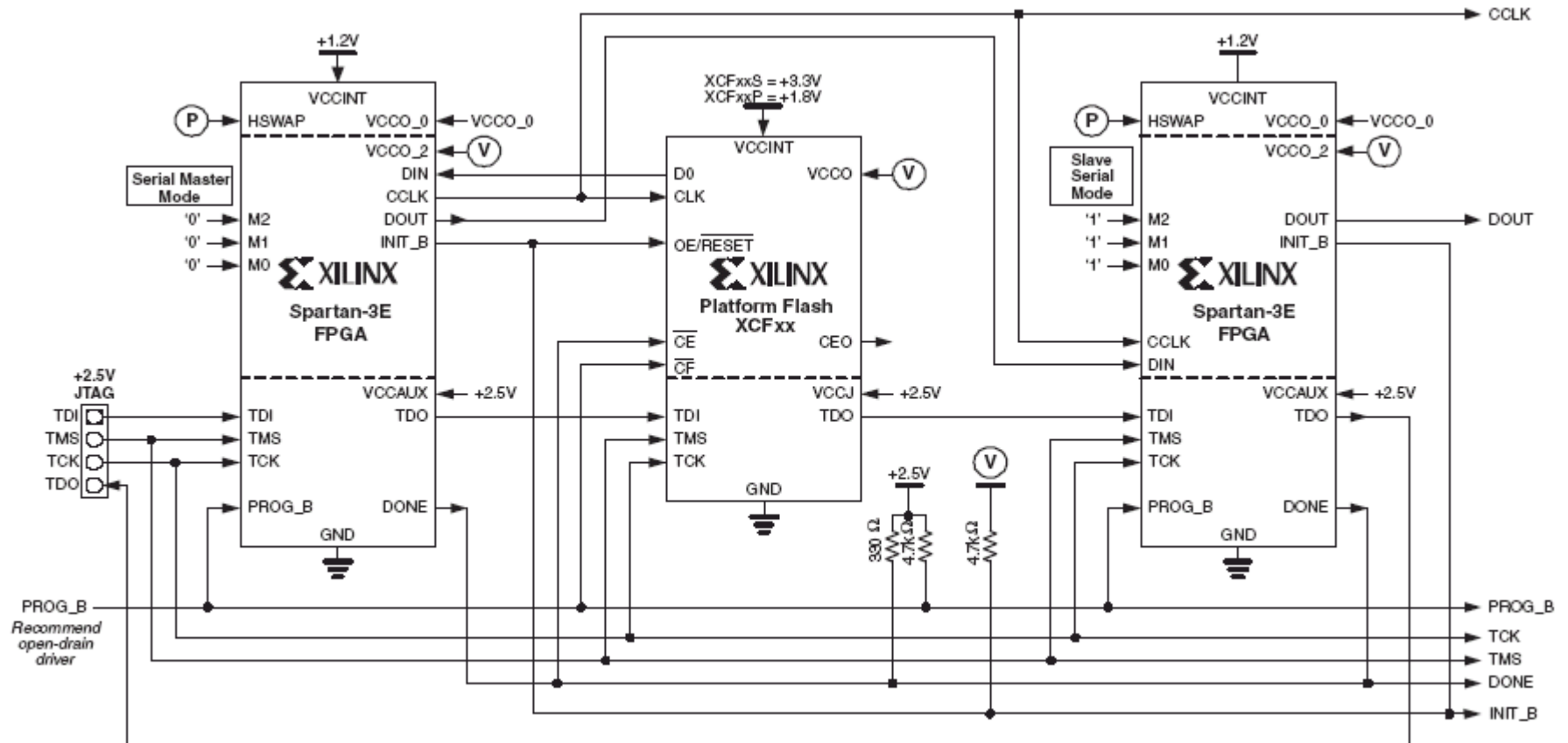
# Konfiguracja Spartan-3E (3) – master serial



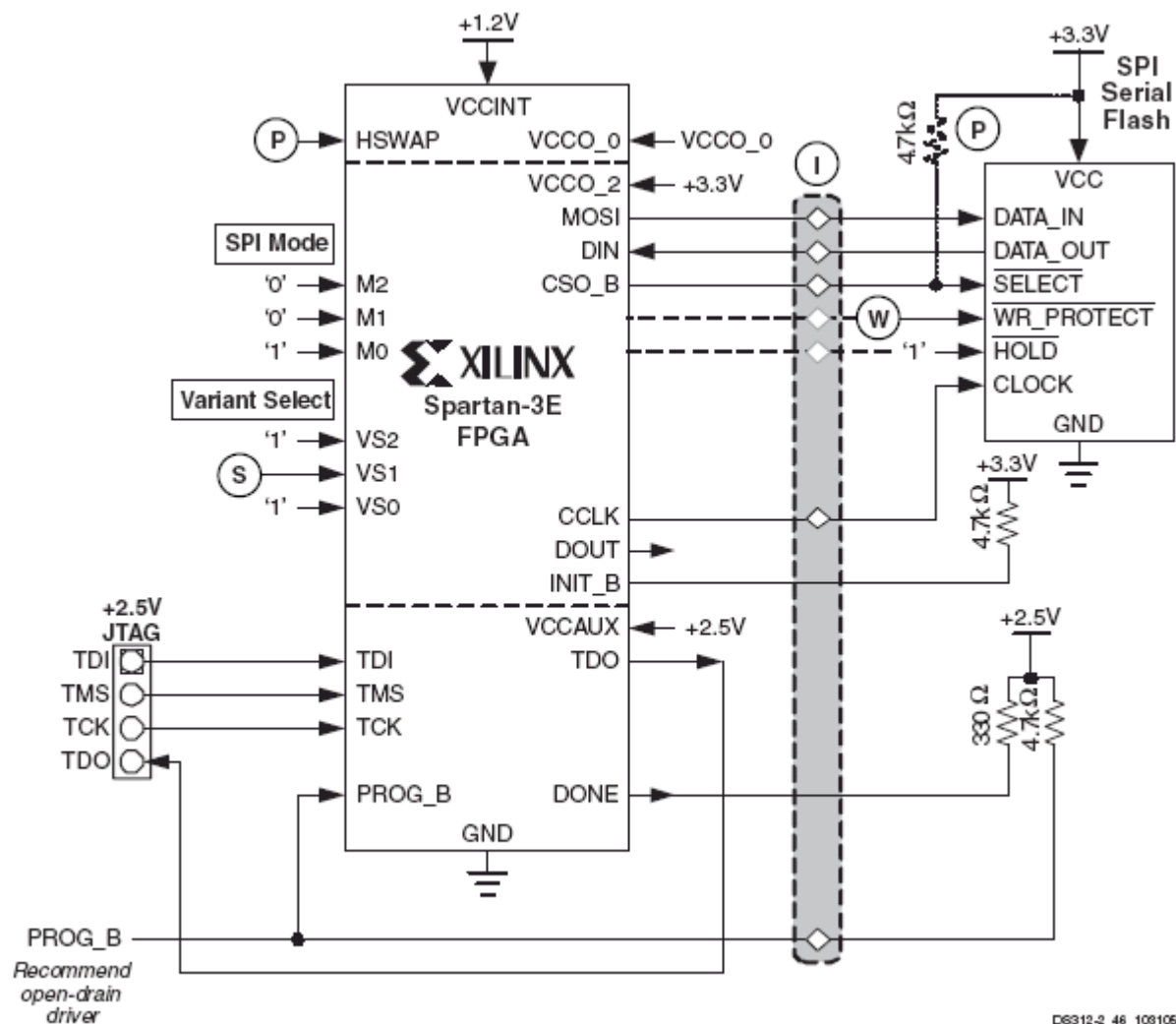
# Konfiguracja Spartan-3E (4) – master serial

| Pin Name     | FPGA Direction               | Description                                                                                                                                                                                                                                                                                | During Configuration                                                                                                                                                                                                                                                                                   | After Configuration                                        |
|--------------|------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------|
| HSWAP<br>(P) | Input                        | <b>User I/O Pull-Up Control.</b> When Low during configuration, enables pull-up resistors in all I/O pins to respective I/O bank V <sub>CCO</sub> input.<br>0: Pull-ups during configuration<br>1: No pull-ups                                                                             | Drive at valid logic level throughout configuration.                                                                                                                                                                                                                                                   | User I/O                                                   |
| M[2:0]       | Input                        | <b>Mode Select.</b> Selects the FPGA configuration mode. See <b>Design Considerations for the HSWAP, M[2:0], and VS[2:0] Pins.</b>                                                                                                                                                         | M2 = 0, M1 = 0, M0 = 0. Sampled when INIT_B goes High.                                                                                                                                                                                                                                                 | User I/O                                                   |
| DIN          | Input                        | <b>Serial Data Input.</b>                                                                                                                                                                                                                                                                  | Receives serial data from PROM's D0 output.                                                                                                                                                                                                                                                            | User I/O                                                   |
| CCLK         | Output                       | <b>Configuration Clock.</b> Generated by FPGA internal oscillator. Frequency controlled by <b>ConfigRate</b> bitstream generator option. If CCLK PCB trace is long or has multiple connections, terminate this output to maintain signal integrity. See <b>CCLK Design Considerations.</b> | Drives PROM's CLK clock input.                                                                                                                                                                                                                                                                         | User I/O                                                   |
| DOUT         | Output                       | <b>Serial Data Output.</b>                                                                                                                                                                                                                                                                 | Actively drives. Not used in single-FPGA designs. In a daisy-chain configuration, this pin connects to DIN input of the next FPGA in the chain.                                                                                                                                                        | User I/O                                                   |
| INIT_B       | Open-drain bidirectional I/O | <b>Initialization Indicator.</b> Active Low. Goes Low at start of configuration during Initialization memory clearing process. Released at end of memory clearing, when mode select pins are sampled. Requires external 4.7 kΩ pull-up resistor to V <sub>CCO_2</sub> .                    | Connects to PROM's OE/RESET input. FPGA clears PROM's address counter at start of configuration, enables outputs during configuration. PROM also holds FPGA in Initialization state until PROM reaches Power-On Reset (POR) state. If CRC error detected during configuration, FPGA drives INIT_B Low. | User I/O. If unused in the application, drive INIT_B High. |

# Konfiguracja Spartan-3E (5) – daisy chain



# Konfiguracja Spartan-3E (6) – SPI



# Konfiguracja Spartan-3E (7) – SPI

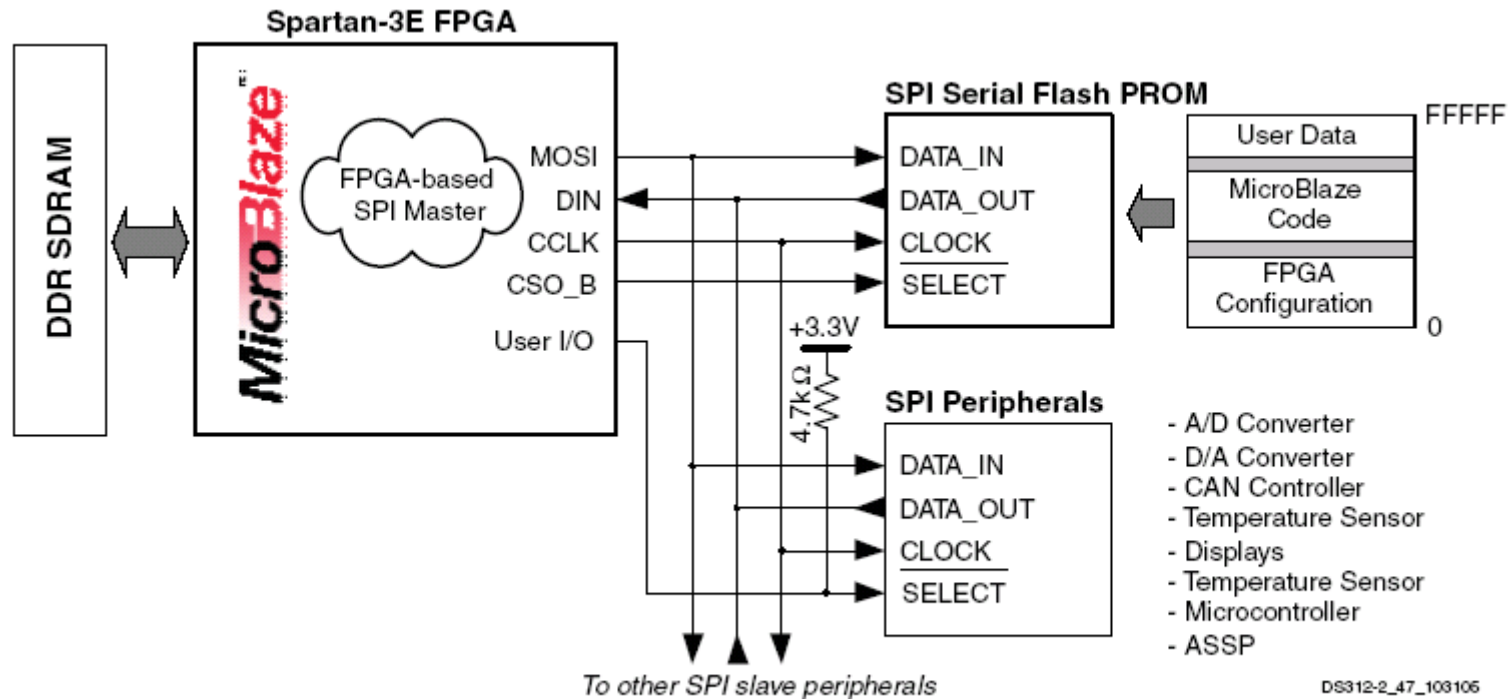
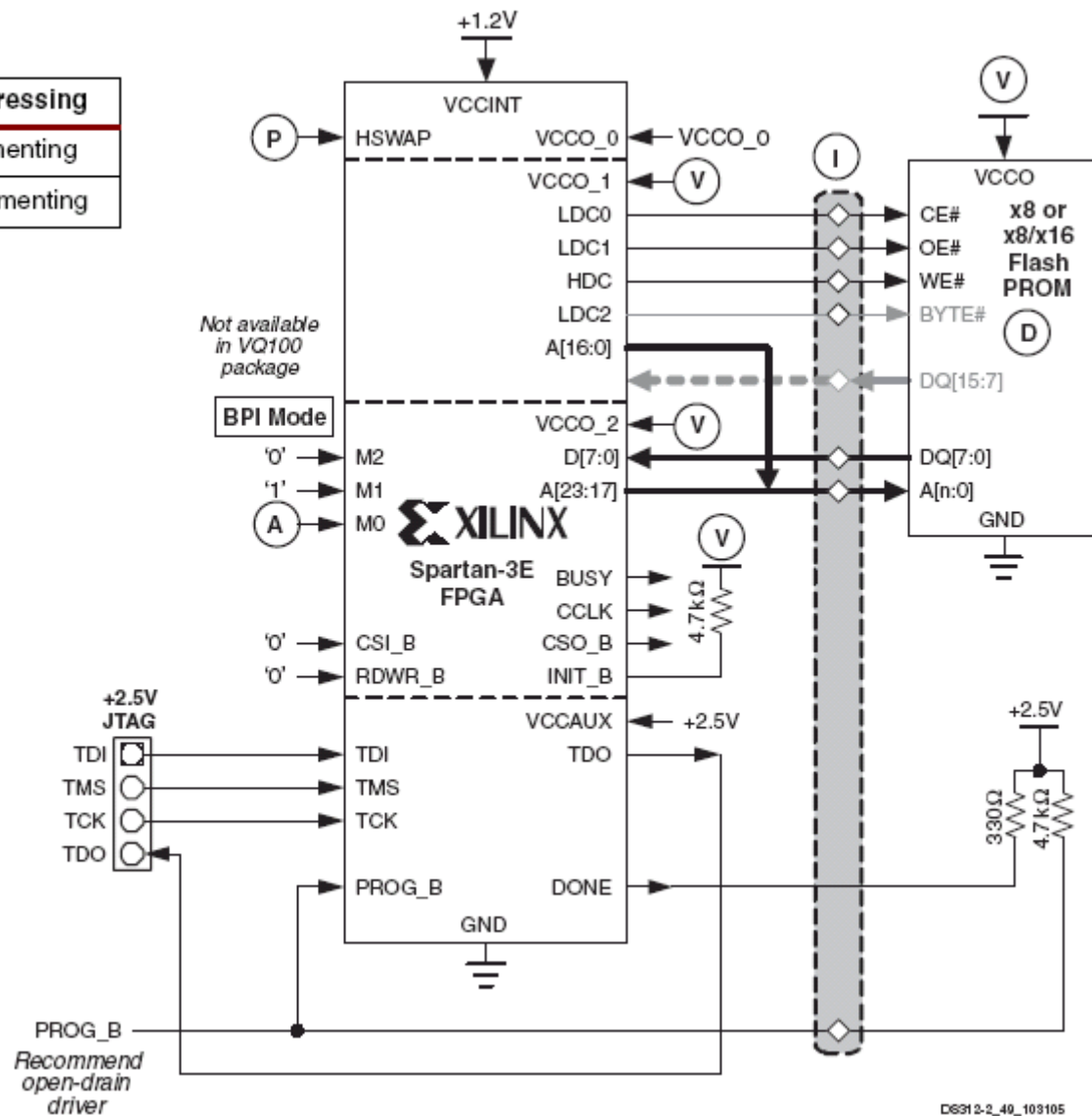


Figure 57: Using the SPI Flash Interface After Configuration

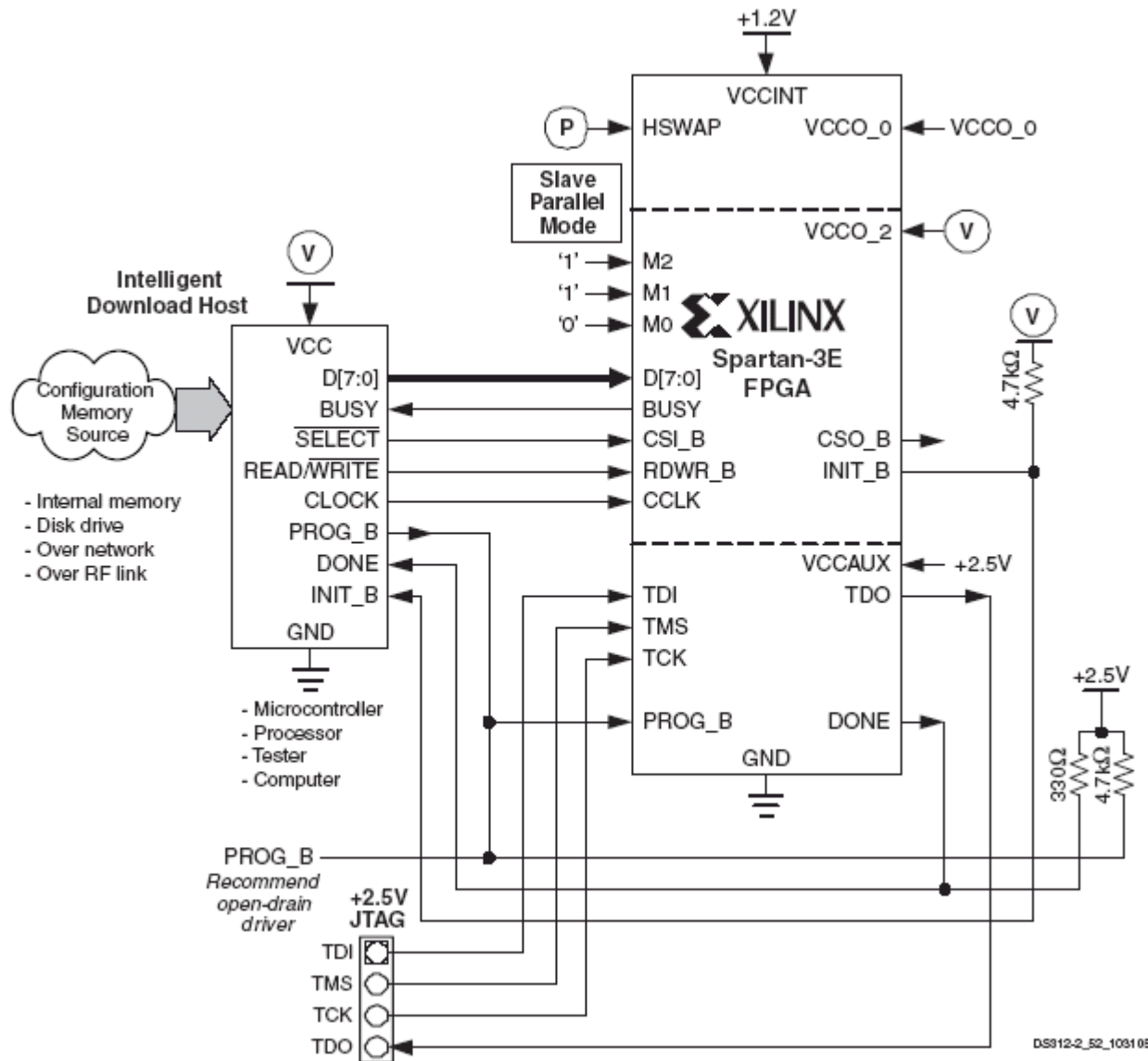
# Konfiguracja Spartan-3E (8) – BPI

| M2 | M1 | M0 | Start Address | Addressing   |
|----|----|----|---------------|--------------|
| 0  | 1  | 0  | 0             | Incrementing |
|    |    | 1  | 0xFF_FFFF     | Decrementing |

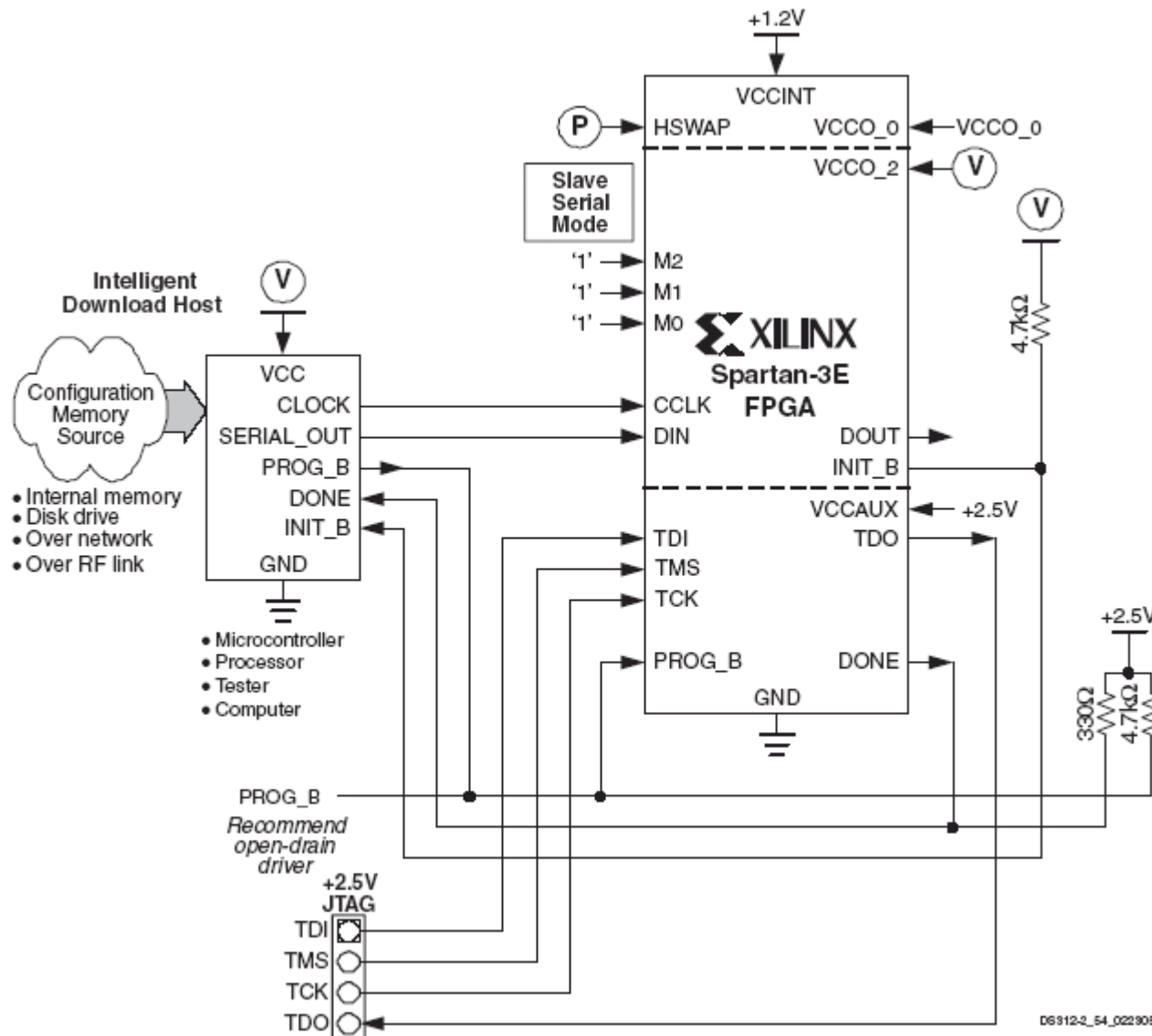


DG31 2-2\_49\_103105

# Konfiguracja Spartan-3E (9) – Slave Parallel

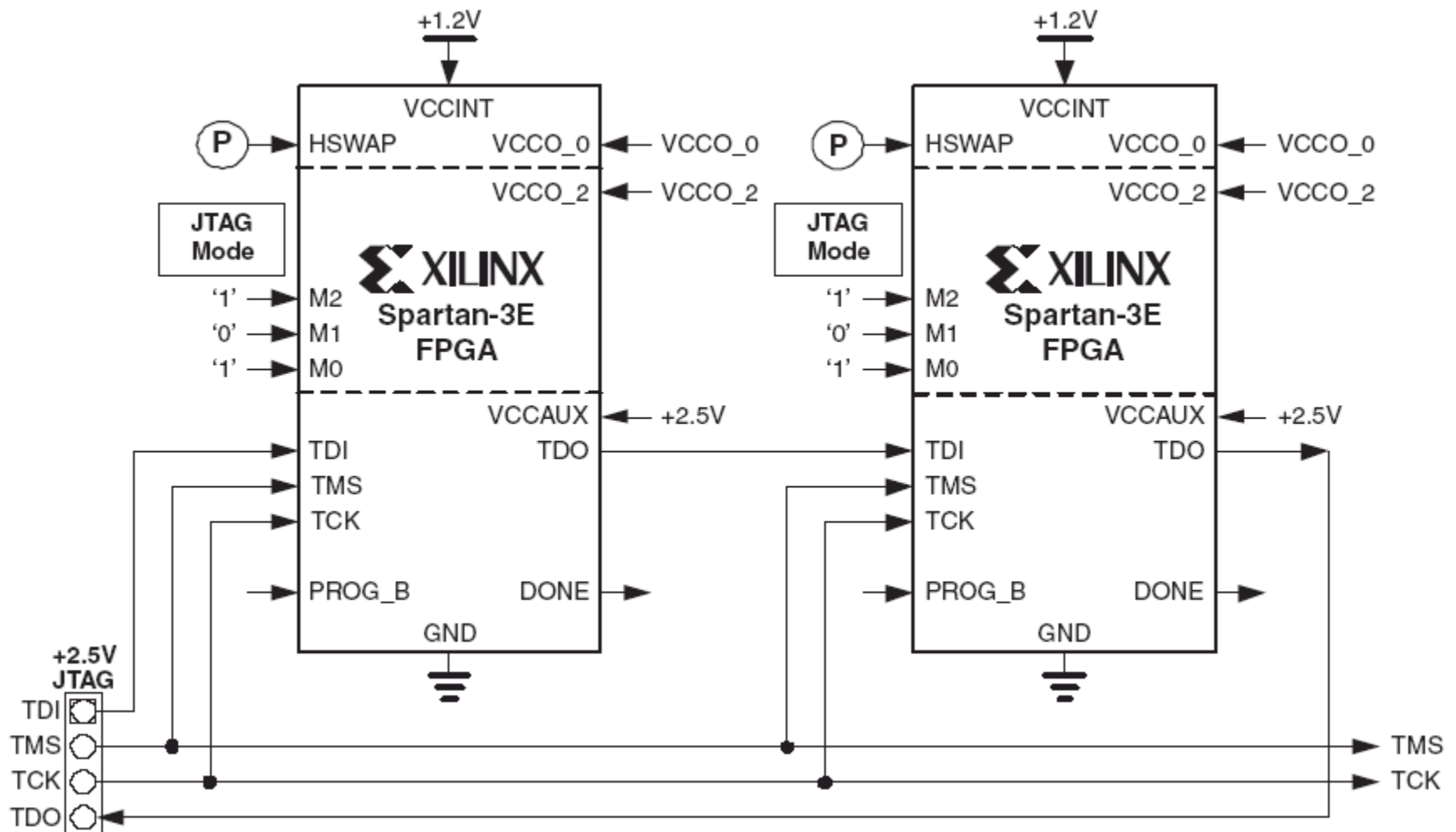


# Konfiguracja Spartan-3E (10) – Slave Serial

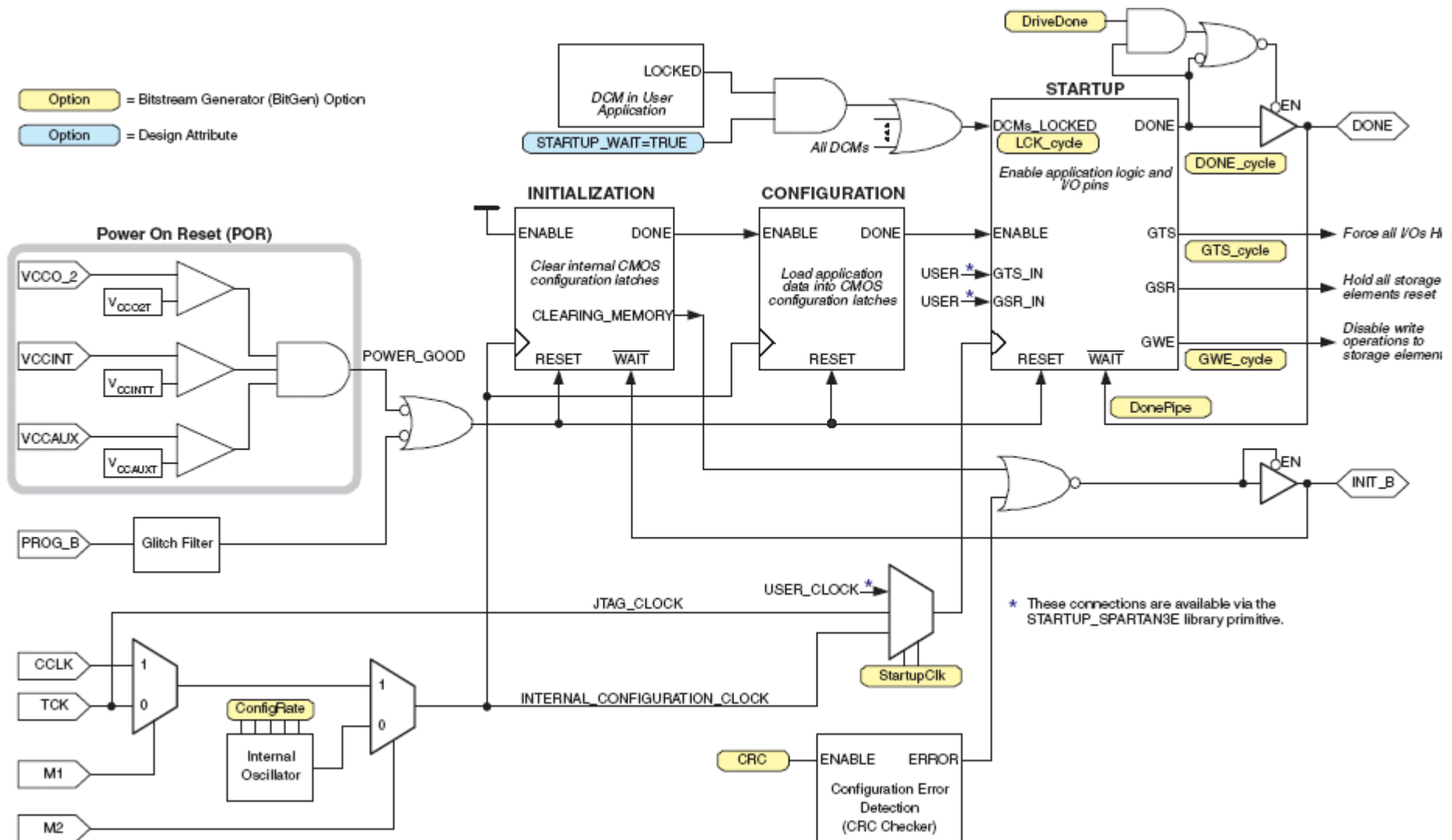




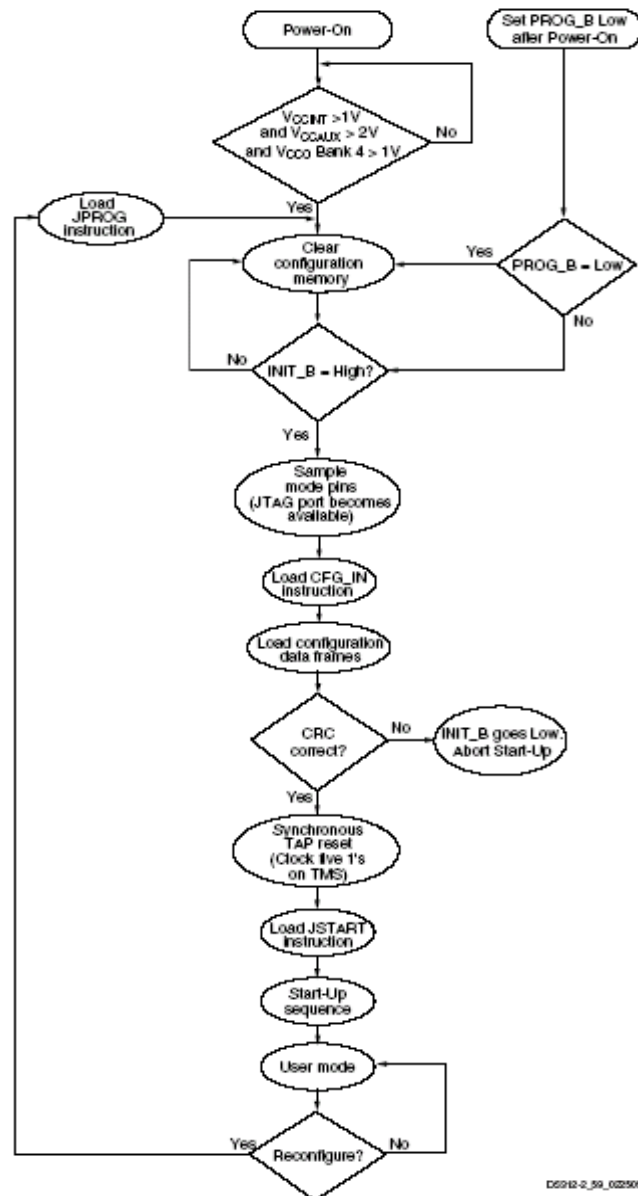
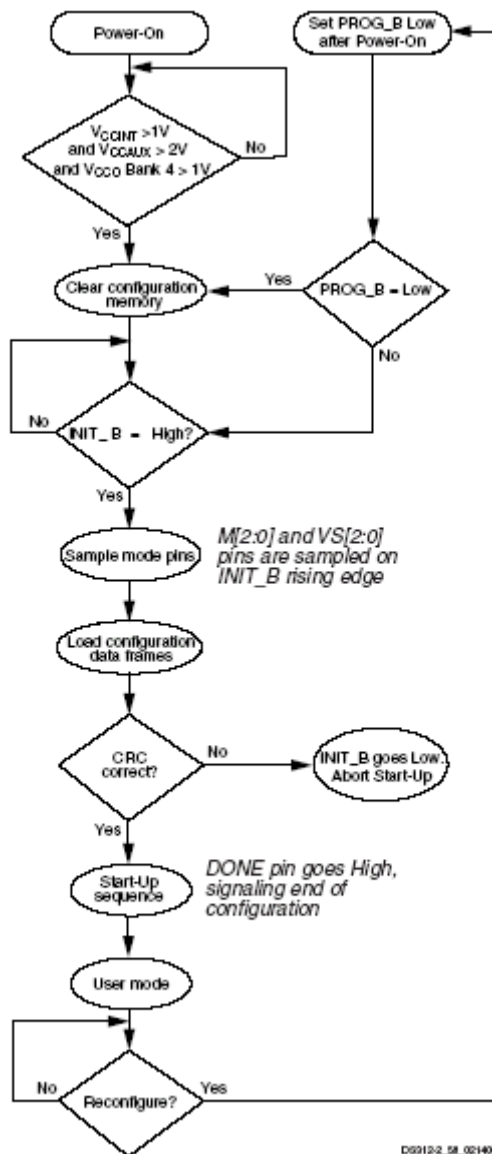
# Konfiguracja Spartan-3E (11) – JTAG



# Konfiguracja Spartan-3E (12) – startup seq.



# Konfiguracja Spartan-3E (13) – startup seq.



# Konfiguracja Spartan-3E (14) – startup seq.

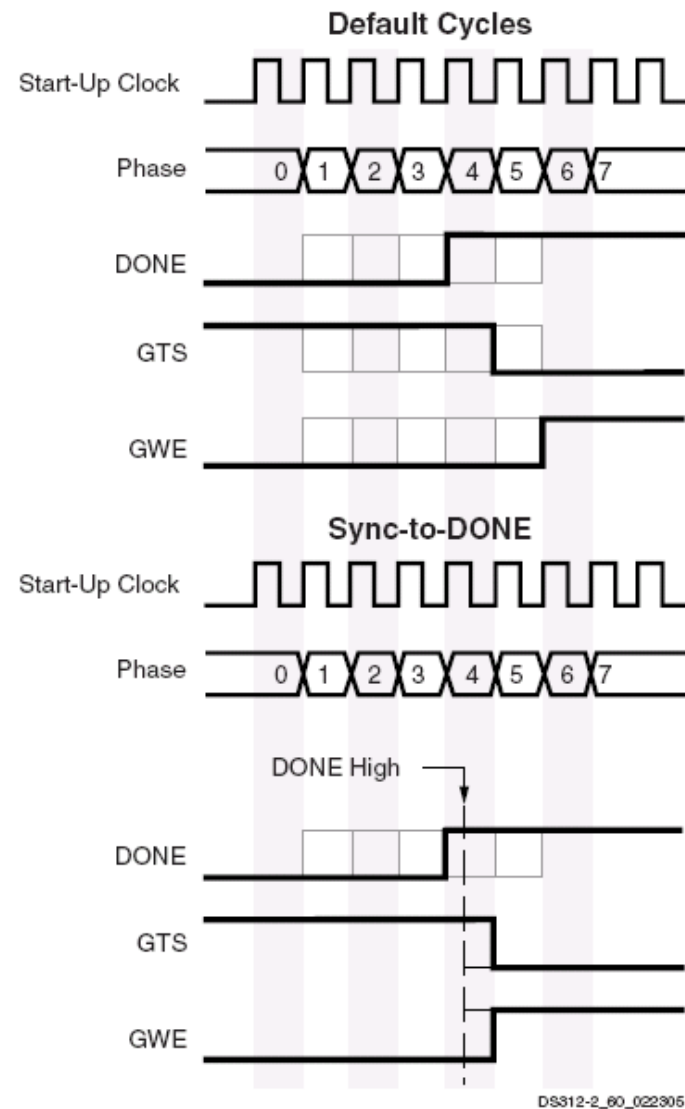
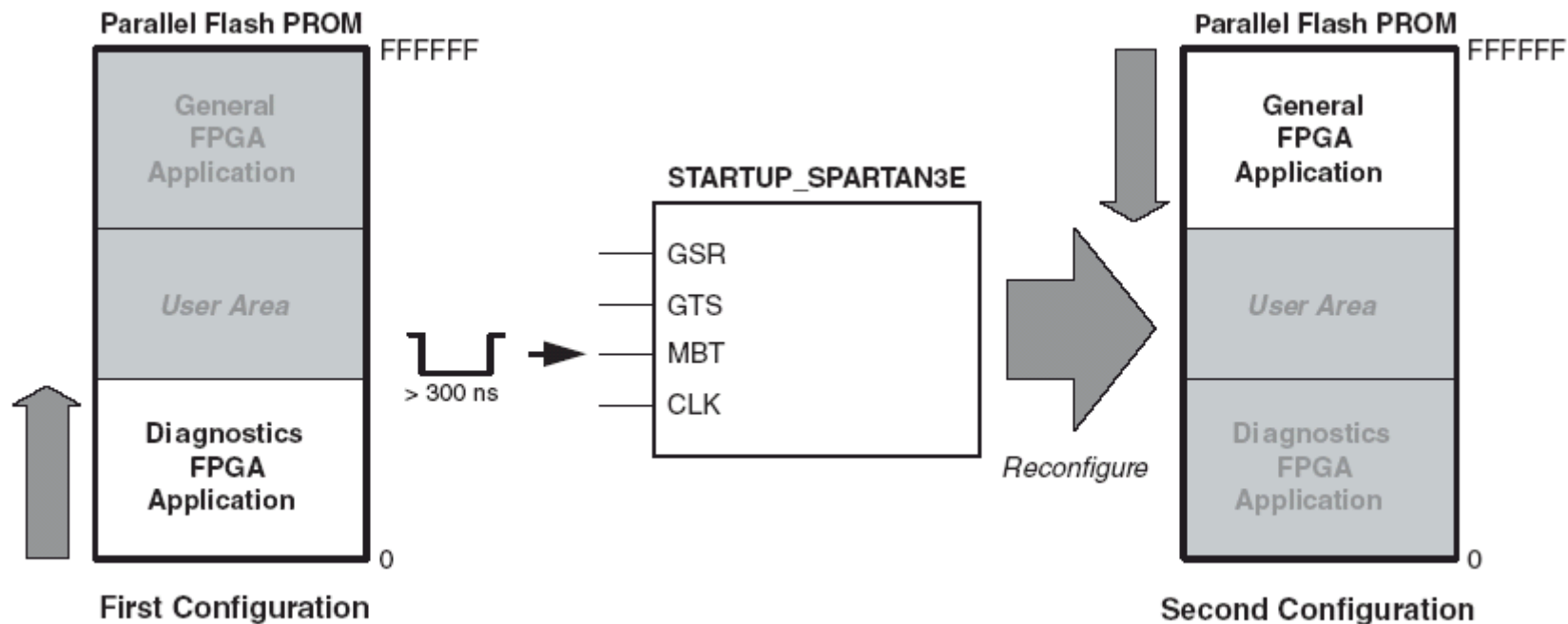


Figure 70: Default Start-Up Sequence

# Konfiguracja Spartan-3E (15) – multiboot



# Projektowanie z uwzględnieniem ograniczeń

# Constraints - rodzaje

- Attributes
- CPLD Fitter
- DLL/DCM Constraints
- Grouping Constraints
- Initialization Directives
- Logical and Physical Constraints
- Mapping Directives
- Modular Design Constraints
- Placement Constraints
- Routing Directives
- Synthesis Constraints
- Timing Constraints

# Constraints - edycja

- Schematic Editor
- Atrybuty (VHDL, Verilog, ABEL)
- UCF/NCF
- XCF
- Constraints Editor
- PCF
- Floorplanner
- PACE
- FPGA Editor
- Command Line



# Constraints – DLL/DCM

- CLK\_FEEDBACK
- CLKIN\_PERIOD
- DUTY\_CYCLE\_CORRECTION
- CLKDV\_DIVIDE
- CLKOUT\_PHASE\_SHIFT
- FAST
- CLKFX\_DIVIDE
- DESKEW\_ADJUST
- HIGH\_FREQUENCY
- CLKFX\_MULTIPLY
- DFS\_FREQUENCY\_MODE
- PHASE\_SHIFT
- CLKIN\_DIVIDE\_BY\_2
- DLL\_FREQUENCY\_MODE
- STARTUP\_WAIT

# Constraints – grupujące

Grupy predefiniowane:

| Keyword     | Description                                                                                                                                                                                                                                       |
|-------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| CPUS        | PPC405 in Virtex2p.                                                                                                                                                                                                                               |
| FFS         | <ul style="list-style-type: none"><li>- All CLB and IOB edge-triggered flip-flops.</li><li>- Shift Register LUTs in Virtex and derived.</li><li>- Dual-data-rate registers in Virtex2 and derived (includes both flip-flops in the DDR)</li></ul> |
| HSIOS       | GT and GT10 in Virtex2p.                                                                                                                                                                                                                          |
| LATCHES     | All CLB and IOB level-sensitive latches.                                                                                                                                                                                                          |
| MULTS       | Multipliers, both sync and async, in Virtex2 and derived.                                                                                                                                                                                         |
| PADS        | All I/O pads (typically inferred from top level HDL ports).                                                                                                                                                                                       |
| RAMS        | <ul style="list-style-type: none"><li>- All CLB LUT RAMs, both single- and dual-port (includes both ports of dual-port).</li><li>- All block RAMs, both single-and dual-port (includes both ports of dual-port)</li></ul>                         |
| BRAMS_PORTA | Port A of all dual-port block RAMs                                                                                                                                                                                                                |
| BRAMS_PORTB | Port B of all dual-port block RAMs                                                                                                                                                                                                                |

# Constraints – grupujące

- COMPGRP
- PIN
- TIMEGRP
- TNM
- TNM\_NET
- TPSYNC
- TPTHURU

Przykłady:

```
PIN "module.pin" LOC="location";
TIMEGRP "big_group"="small_group" "medium_group";
TIMEGRP "group1"="group2" "group3" EXCEPT "group4" "group5";
TIMEGRP "group1"=RISING ffs;
TIMEGRP "this_group"=ffs EXCEPT ffs("a*");
TIMEGRP "some_ffs"=ffs("a*:b?:c*d");
PIN "pin_name" TNM="FLOPS";
INST "instance_name" TNM=FLOPS;
INST "instance_name" TNM=RAMS identifier;
NET "logic_latch" TPSYNC=latch;
NET "MYNET" TPTHURU="ABC";
TIMESPEC "Tspath1"=FROM "A" THRU "ABC" TO "B" 30;
```

# Constraints – mapping

AREA\_GROUP, IOBDELAY, RLOC\_ORIGIN, BEL,  
IOSTANDARD, RLOC\_RANGE, BLKNM, KEEP, SAVE,  
NET, FLAG, DCI\_VALUE, KEEPER, SLEW, DOUBLE,  
MAP, U\_SET, DRIVE, NODELAY, USE\_RLOC, FAST,  
OPTIMIZE, WRITE\_MODE, HBLKNM, PULLDOWN,  
WRITE\_MODE\_A, HU\_SET, PULLUP,  
WRITE\_MODE\_B, IOB, RLOC, XBLKNM

# Constraints – modular design

- INST / AREA\_GROUP (UCF)
- COMPGRP / COMP (PCF)
- AREA\_GROUP / RANGE (UCF)
- COMPGRP / LOCATE (PCF)
- PIN / LOC (UCF)
- COMP / LOCATE (PCF)
- NET / TPSYNC (UCF)
- MODULE / RESYNTHESIZE (UCF)
- COMPGRP / LOCATE (PCF)
- PROHIBIT (PCF)

# Constraints – placement

- AREA\_GROUP
- BEL
- CONFIG
- LOC
- LOCATE
- OPT\_EFFORT
- PROHIBIT
- RLOC
- RLOC\_ORIGIN
- RLOC\_RANGE
- USE\_RLOC

```
INST "/top-12/fdrd" LOC=CLB_R1C5;
```

```
INST "instance_name" LOC=P13;
```

```
INST "instance_name" LOC=RAMB16_X0Y6;
```

```
INST "instance_name" LOC=MULT18X18_X0Y6;
```

```
INST "/Virtex/design/FF1" RLOC=R4C4;
```

```
INST "/$1I87/elemA" RLOC=r0cl.S0;
```

# Constraints - routing

- **AREA\_GROUP**
- **CONFIG\_MODE:**

**CONFIG CONFIG\_MODE=string;**

S\_SERIAL = Slave Serial Mode, M\_SERIAL = Master Serial Mode (The default value),  
S\_SELECTMAP = Slave SelectMAP Mode, M\_SELECTMAP = Master SelectMAP Mode,  
B\_SCAN = Boundary Scan Mode,  
S\_SELECTMAP+READBACK = Slave SelectMAP Mode with Persist set to support Readback and Reconfiguration.  
M\_SELECTMAP+READBACK = Master SelectMAP Mode with Persist set to support Readback and Reconfiguration,  
B\_SCAN+READBACK = Boundary Scan Mode with Persist set to support Readback and Reconfiguration,  
S\_SELECTMAP32+READBACK, S\_SELECTMAP32

- **LOCK\_PINS:**

**INST "XSYM1" LOCK\_PINS;**

**INST I\_894 LOCK\_PINS=I3:A1, I2:A4;**

- **OPT\_EFFORT:**

**INST "\$1I678/adder" OPT\_EFFORT=HIGH;**

(dotyczy makr i hierarchii, może być: Default (Low),  
Lowest, Low, Normal, High, Highest)

- **USELOWSKEWLINES:**

**NET "\$1I87/\$1N6745" USELOWSKEWLINES;**

# Constraints - routing

```
INST "X" AREA_GROUP=groupname;  
AREA_GROUP "groupname" RANGE=range;  
AREA_GROUP "groupname" COMPRESSION=percent;  
AREA_GROUP "groupname" IMPLEMENT={FORCE|AUTO};  
AREA_GROUP "groupname" GROUP={OPEN|CLOSED};  
AREA_GROUP "groupname" PLACE={OPEN|CLOSED};  
AREA_GROUP "groupname" MODE={RECONFIG};
```

```
RANGE=SLICE_Xm1Yn1:SLICE_xm2Yn2  
RANGE=TBUF_Xm1Yn1:TBUF_Xm2Yn2  
RANGE=MULT18X18_Xm1Yn1:MULT18X18_Xm2Yn2  
RANGE=RAMB16_Xm1Yn1:RAMB16_Xm2Yn2
```

```
INST "state_machine_X" AREA_GROUP=group1;  
AREA_GROUP "group1" COMPRESSION=0;  
AREA_GROUP "group1" RANGE=CLB_R1C1:CLB_R10C10;  
AREA_GROUP "group1" RANGE=TBUF_X6Y0:TBUF_X10Y22;
```

```
INST "I$1" AREA_GROUP=group2;  
INST "I$2" AREA_GROUP=group2;
```



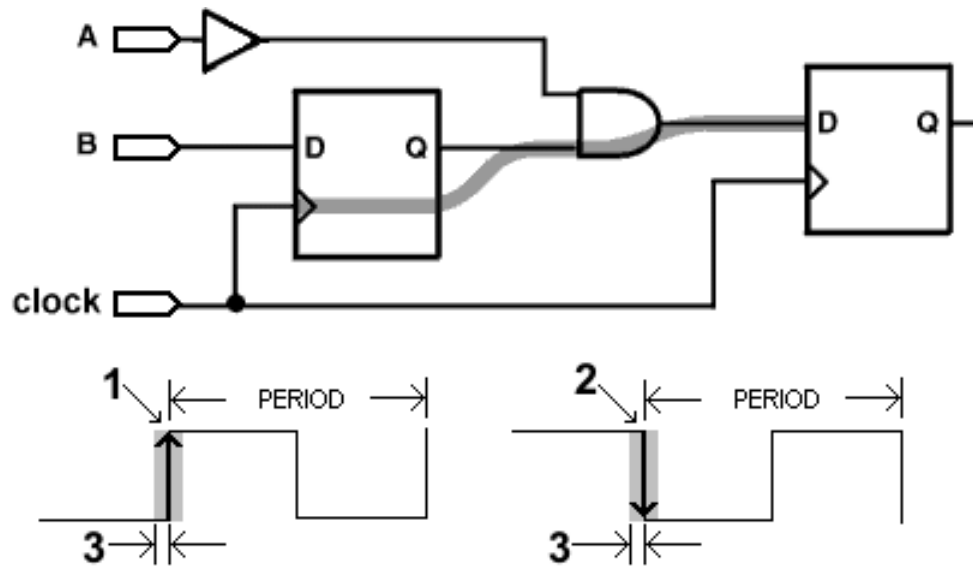
# Constraints – synthesis

**BOX\_TYPE, IOSTANDARD, READ\_CORES, BUFFER\_TYPE, KEEP, REGISTER\_BALANCING, BUFG (CPLD), KEEP\_HIERARCHY, REGISTER\_DUPLICATION, BUFGCE, LOC, REGISTER\_POWERUP, BUS\_DELIMITER, LUT\_MAP, RESOURCE\_SHARING, CLK\_FEEDBACK, MAP, RESYNTHESIZE, CLOCK\_BUFFER, MAX\_FANOUT, RLOC, CLOCK\_SIGNAL, MOVE\_FIRST\_STAGE, ROM\_EXTRACT, DECODER\_EXTRACT, MOVE\_LAST\_STAGE, ROM\_STYLE, DUTY\_CYCLE, MULT\_STYLE, SHIFT\_EXTRACT, ENUM\_ENCODING, MUX\_EXTRACT, SHREG\_EXTRACT, EQUIVALENT\_REGISTER\_REMOVAL, MUX\_STYLE, SLEW, FSM\_ENCODING, OPT\_LEVEL, SLICE\_PACKING, FSM\_EXTRACT, OPT\_MODE, SLICE\_UTILIZATION\_RATIO, FSM\_FFTYPE, PARALLEL\_CASE, SLICE\_UTILIZATION\_RATIO\_MAXMARGIN, FULL\_CASE, PERIOD, TIG, HIERARCHY\_SEPARATOR, PRIORITY\_EXTRACT, TRANSLATE\_OFF and TRANSLATE\_ON, INCREMENTAL\_SYNTHESIS, RAM\_EXTRACT, USELOWSKEWLINES, IOB, RAM\_STYLE, XOR\_COLLAPSE**

# Constraints – timing (1)

- ASYNC\_REG
- OFFSET
- TIMESPEC
- DISABLE
- ONESHOT
- TNM
- DROP\_SPEC
- PERIOD
- TNM\_NET
- DUTY\_CYCLE
- PRIORITY
- TPSYNC
- ENABLE
- TEMPERATURE
- TPTHRU
- FROM-THRU-TO
- TIG
- TSidentifier
- FROM-TO
- TIMEGRP
- VOLTAGE
- MAXSKEW

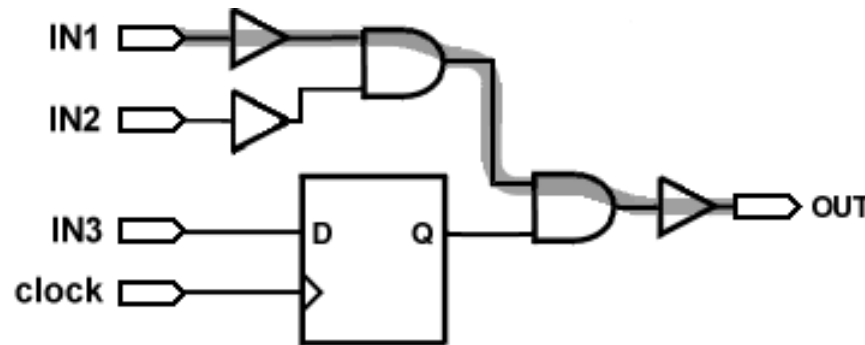
# Constraints – timing – CLK PERIOD (2)



```
NET "clk" TNM_NET = "clk";
```

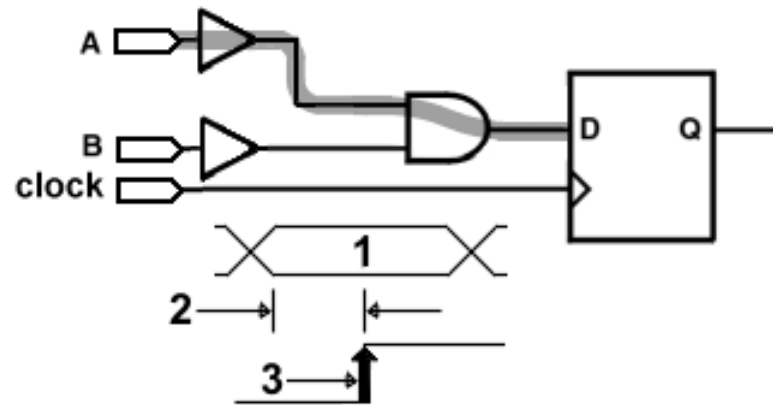
```
TIMESPEC "TS_clk" = PERIOD "clk" 20 ns HIGH 50 % INPUT_JITTER 100 ps;
```

# Constraints – timing – PAD-TO-PAD (3)



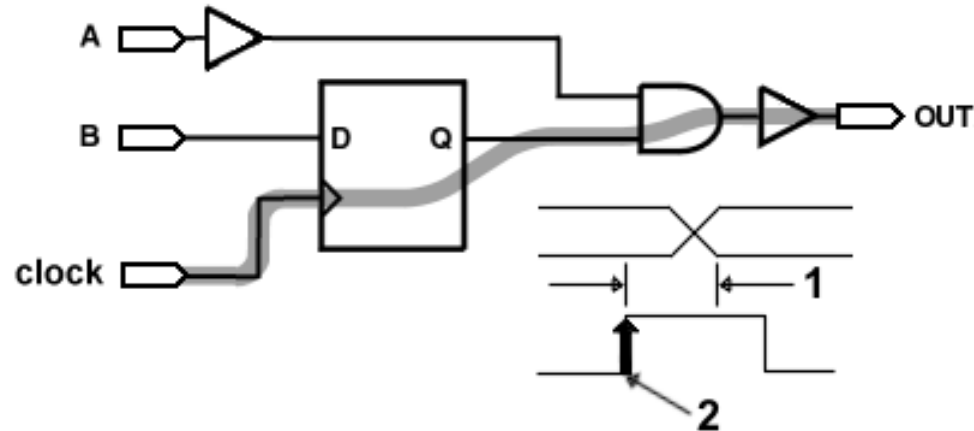
**TIMESPEC "TS\_P2P" = FROM "PADS" TO "PADS" 26 ns;**

# Constraints – timing – PAD-TO-SETUP (4)



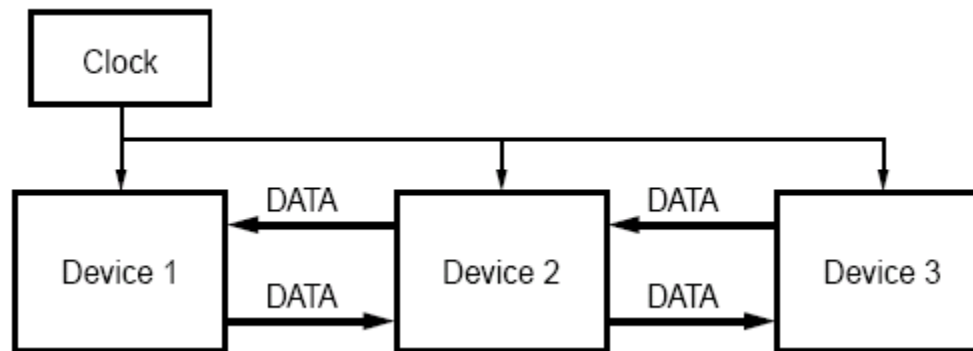
**NET "cnt\_load" OFFSET = IN 14 ns BEFORE "clk";**

# Constraints – timing – CLK-TO-PAD (5)

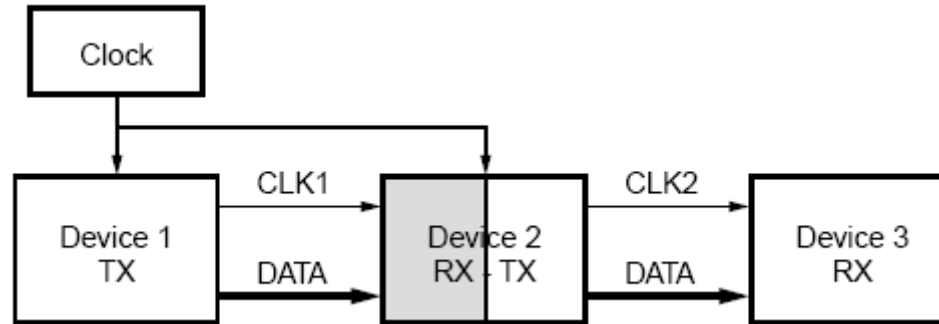


**NET "cnt<1>" OFFSET = OUT 23 ns AFTER "clk";**

# System-Synchronous timing



# Source-Synchronous timing



1. Create Period

```
NET CLK TNM_NET = CLK_GRP;  
TIMESPEC "TS_CLK" = PERIOD "CLK_GRP" 10 ns INPUT_JITTER 1;
```

2. Create Groups

```
INST DATA_IN[*] TNM = DATA_IN;  
TIMEGRP FF_RISING = RISING CLK_GRP;  
TIMEGRP FF_FALLING = FALLING CLK_GRP;
```

3. Create OFFSET constraint

```
TIMEGRP DATA_IN OFFSET IN = 1 VALID 3 BEFORE CLK TIMEGRP FF_RISING;  
TIMEGRP DATA_IN OFFSET IN = -4 VALID 3 BEFORE CLK TIMEGRP FF_FALLING;
```

4. Create SYSTEM\_JITTER constraint.

```
SYSTEM_JITTER=0.456 ns;
```



# Constraints – timing (6)

```
INST "instance_name" ASYNC_REG = {TRUE|FALSE};
```

Przypisanie do rejestru informacji że wejście może być asynchroniczne w stosunku do zegara (zabrania generowania wyjścia 'X' podczas symulacji gdy wystąpi niewłaściwy czas setup/hold).

```
OFFSET = {IN|OUT} "offset_time" [units] [VALID <datavalid time>]  
{BEFORE|AFTER} "clk_name" [TIMEGRP "group_name"] {HIGH | LOW};
```

```
NET "DATA_IN" OFFSET=IN 20.0 BEFORE "CLK_SYS";  
NET "DATA_IN" OFFSET=IN 30.0 AFTER "CLK_SYS";  
NET "Q_OUT" OFFSET=OUT 35.0 AFTER "CLK_SYS";  
NET "Q_OUT" OFFSET=OUT 15.0 BEFORE "CLK_SYS";
```

- OFFSET – ograniczenie na NET, VALID pozwala na deklarację czasu ważności danych (niezerowy Hold Time) – domyślnie VALID = OFFSET.

W przypadku grupy wejść/wyjść o takich samych ograniczeniach czasowych:

```
TIMEGRP "name" OFFSET={IN | OUT} offset_time [units] {BEFORE | AFTER}  
"clk_net" [TIMEGRP "reggroup"];
```

- DROP\_SPEC pozwala na skasowanie ograniczenia:  
TIMESPEC "TS67"=DROP\_SPEC;

# Constraints – timing DDR out (7)

# Create main PERIOD constraint.

# This is required in order to pass the phase keyword to each of the derived clocks (clk0, clk90, clk270). Note that the

# HIGH keyword indicates all transitions are with respect to the rising clock edge of clk\_p.

```
NET "clk_p" TNM_NET = "CLK";
```

```
TIMESPEC "TS_CLK" = PERIOD "CLK" 8.0 ns HIGH 50%;
```

# Create separate timing groups based off each clock domain

```
NET "clk0" TNM = "CLK0_GRP_DDR" ;
```

```
NET "clk90" TNM = "CLK90_GRP" ;
```

```
NET "clk270" TNM = "CLK270_GRP" ;
```

# clk0 contains both rising and falling clock edges. Therefore break the clk0 group (CLK0\_GRP\_DDR) into two timing groups – required to constrain each group separately.

```
TIMEGRP "CLK0_GRP_ALL" = RISING "CLK0_GRP_DDR" ; # clk0 registers
```

```
TIMEGRP "CLK180_GRP" = FALLING "CLK0_GRP_DDR" ; # ~clk0 registers
```

# The new clk0 group (CLK0\_GRP\_ALL) contains both the tx\_data ports that should be constrained,  
# but also the test\_port signals that are slower.

# Add these slower signals to their own group, and remove them from the main clk0 group.

```
INST "test_port" TNM = "TEST_GRP" ;
```

```
TIMEGRP "CLK0_GRP" = "CLK0_GRP_ALL" EXCEPT "TEST_GRP" ;
```

# Now that all groups are created, generate the OFFSET constraints.

# Recall that the OFFSET constraints must be manually adjusted to account for the clock phase.

# In this design, clock cycle = 8.0ns (as defined by the PERIOD).

```
OFFSET = OUT 6.0 ns AFTER "clk_p" TIMEGRP "CLK0_GRP" ; #6.0ns
```

```
OFFSET = OUT 8.0 ns AFTER "clk_p" TIMEGRP "CLK90_GRP" ;
```

#6.0ns +  $\frac{1}{4}$  clock cycle

```
OFFSET = OUT 10.0 ns AFTER "clk_p" TIMEGRP "CLK180_GRP";
```

#6.0ns +  $\frac{1}{2}$  clock cycle

```
OFFSET = OUT 12.0 ns AFTER "clk_p" TIMEGRP "CLK270_GRP";
```

#6.0ns +  $\frac{3}{4}$  clock cycle

```
OFFSET = OUT 10.0 ns AFTER "clk_p" TIMEGRP "TEST_GRP" ; #10.0ns
```

# Constraints – timing DDR in (8)

**# Create main PERIOD constraint.**

**NET clk\_p TNM\_NET = CLK;**

**TIMESPEC TS\_CLK = PERIOD CLK 8.0 ns HIGH 50%;**

**# Create separate timing groups based off each clock domain**

**NET clk0 TNM = CLK0\_GRP\_DDR;**

**TIMEGRP CLK0\_GRP = RISING CLK0\_GRP\_DDR ;      # clk0 registers**

**TIMEGRP CLK180\_GRP = FALLING CLK0\_GRP\_DDR ; # ~clk0 registers**

**# Now that all groups are created, generate the OFFSET constraints.**

**# Recall that the OFFSET constraints must be manually adjusted to**

**# account for the clock phase.**

**OFFSET = IN 2.0 ns BEFORE clk\_p TIMEGRP CLK0\_GRP ; # 2.0ns**

**OFFSET = IN -2.0 ns BEFORE clk\_p TIMEGRP CLK180\_GRP ;**

**# 2.0ns – ½ clock cycle**

# Constraints – timing (9)

| Delay Symbol Name | Path Type                                                        | Default  |
|-------------------|------------------------------------------------------------------|----------|
| reg_sr_q          | Asynchronous Set/Reset to output propagation delay               | Disabled |
| reg_sr_clk        | Synchronous Set/Reset to clock setup and hold checks             | Enabled  |
| lat_d_q           | Data to output transparent latch delay                           | Disabled |
| ram_we_o          | RAM write enable to output propagation delay                     | Enabled  |
| tbuf_t_o          | TBUF 3-state to output propagation delay                         | Enabled  |
| tbuf_i_o          | TBUF input to output propagation delay                           | Enabled  |
| io_pad_i          | IO pad to input propagation delay                                | Enabled  |
| io_t_pad          | IO 3-state to pad propagation delay                              | Enabled  |
| io_o_1            | IO output to input propagation delay. Disabled for 3-stated IOBs | Enabled  |
| io_o_pad          | IO output to pad propagation delay                               | Enabled  |

```
ENABLE=delay_symbol_name;  
TIMEGRP name ENABLE=delay_symbol_name;  
DISABLE=tbuf_i_o;
```

Timing ignore:

```
NET "RESET" TIG=TS_fast, TS_even_faster;  
NET "net_name" TIG = TS43;  
NET "net_name" TIG;  
PIN "ff_inst.RST" TIG=TS_1;  
INST "instance_name" TIG=TS_2;
```

# Constraints – timing (10)

## Przykłady:

```
TIMESPEC "TSname"=FROM "group1" TO "group2" value {DATAPATHONLY};
TIMESPEC "TSidentifier"=FROM "source_group" THRU
"thru_pt1"...[THRU "thru_pt2"...] TO "destination_group" value
[Units]{DATAPATHONLY};
TIMESPEC "TSidentifier" =FROM "source_group" TO "dest_group" value units;
TIMESPEC "TSidentifier" =FROM "source_group" value units;
TIMESPEC "TSidentifier" =TO "dest_group" value units;
TIMESPEC "TSID1" = FROM "gr1" TO "gr2" 50;
TIMESPEC "TSMAN" = FROM "here" TO "there" TSID1 /2
TIMESPEC "TS_THIS" =FROM FFS TO RAMS 35;
TIMESPEC "TS_THAT" =FROM PADS TO LATCHES 35;
```

```
NET "$1I3245/$SIG_6" MAXSKEW=3 ns;
```

```
normal_timespec_syntax PRIORITY integer;
```

```
TIMESPEC "TS01"=FROM "GROUPA" TO "GROUPB" 40 PRIORITY 4;
```

```
SYSTEM_JITTER= value ns;
```

```
TEMPERATURE=value [C |F| K];
```

```
TEMPERATURE=25 C;
```

```
VOLTAGE=value [V];
```

```
VOLTAGE=5;
```

# Constraints – timing – period (11)

```
TIMESPEC "TSidentifier"=PERIOD "TNM_reference" period {HIGH | LOW}  
[high_or_low_time] INPUT_JITTER value;  
TIMESPEC "TS_master"=PERIOD "master_clk" 50 HIGH 30 INPUT_JITTER 50;  
TIMESPEC "TSidentifier"=PERIOD "timegroup_name" "TSidentifier"  
[* or /] factor PHASE [+|-] phase_value [units];
```

- Period for primary clock:

```
TIMESPEC "TS01" = PERIOD "clk0" 10.0 ns;
```

- Period for clock phase-shifted forward by 180 degrees:

```
TIMESPEC "TS02" = PERIOD "clk180" TS01 PHASE + 5.0 ns;
```

- Period for clock phase-shifted backward by 90 degrees:

```
TIMESPEC "TS03" = PERIOD "clk90" TS01 PHASE - 2.5 ns;
```

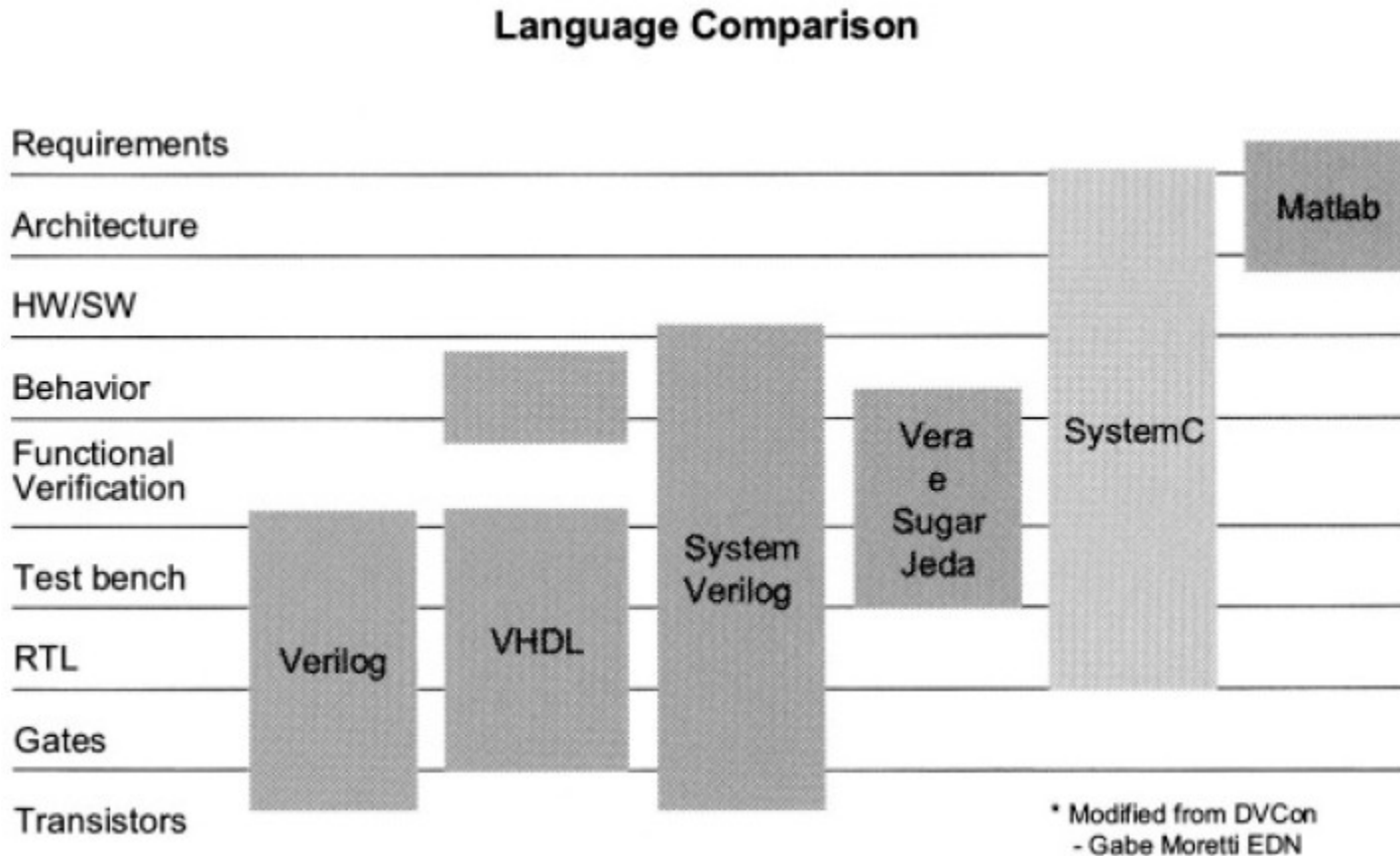
- Period for clock doubled and phase-shifted forward by 180 degrees (which is 90 degrees relative to TS01):

```
TIMESPEC "TS04" = PERIOD "clk180" TS01 / 2 PHASE + 2.5 ns;
```

If there are paths that are static in nature, you can use TIG to eliminate the paths from timing consideration in Place and Route (PAR) and TRCE. If there are paths that require faster or slower specifications than the global requirements, you can create fast or slow exceptions for those paths. If multi-cycle paths exist, identify and constrain them.

# Wprowadzenie do SystemC

# SystemC - porównanie





# Moduły

Deklaracja:

```
SC_MODULE(transmit) {
```

Porty modułów:

```
SC_MODULE(fifo) {  
    sc_in<bool> load;  
    sc_in<bool> read;  
    sc_inout<int> data;  
    sc_out<bool> full;  
    sc_out<bool> empty;  
    //rest of module not shown  
}
```



# Moduły - konstruktor

```
SC_CTOR(module_name)
: Initialization // OPTIONAL
{
    Subdesign_Allocation
    Subdesign_Connectivity
    Process_Registration
    Miscellaneous_Setup
}
```

Zadania konstruktorów:

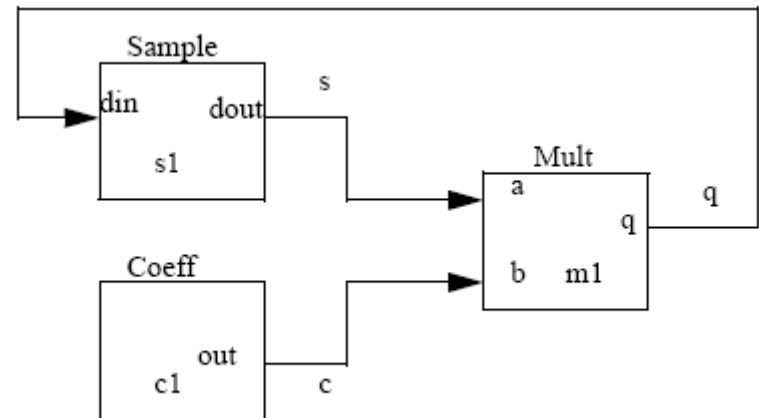
- Alokacja i inicjalizacja modułów podrzędnych (w hierarchii).
- Tworzenie połączeń z portami modułów podrzędnych.
- Rejestracja procesów (w jądrze SystemC w celu modelowania współbieżności).
- Sterowanie czułością procesów.
- Inne zadania – zdefiniowane przez użytkownika.

# Moduły - połączenia pozycyjne

```
// filter.h
#include "systemc.h"
#include "mult.h"
#include "coeff.h"
#include "sample.h"
SC_MODULE(filter) {
    sample *s1;
    coeff *c1;
    mult *m1;
    sc_signal<sc_uint<32>> q, s, c;
    SC_CTOR(filter) {
        s1 = new sample ("s1");
        (*s1)(q, s);
        c1 = new coeff ("c1");
        (*c1)(c);
        m1 = new mult ("m1");
        (*m1)(s, c, q);
    }
}
```

Tworzenie nowej instancji modułu „sample” o nazwie s1.

Podłączenie lokalnych sygnałów q,s odpowiednio do portów din, dout modułu s1.



# Moduły - połączenia nazywane

Przykład 1:

```
SC_MODULE(filter) {
    sample *s1;
    coeff *c1;
    mult *m1;
    sc_signal<sc_uint<32> > q, s,
        c;
    SC_CTOR(filter) {
        s1 = new sample ("s1");
        s1->din(q);
        s1->dout(s);
        c1 = new coeff ("c1");
        c1->out(c);
        m1 = new mult ("m1");
        m1->a(s);
        m1->b(c);
        m1->q(q);
    }
}
```

Przykład 2:

```
int sc_main(int argc, char* argv[])
{
    sc_signal<sc_uint<8> > dout1, dout2, dcnt;
    sc_signal<sc_logic > rst;

    counter_tb CNTR_TB("counter_tb"); // Instantiate
        Test Bench
    CNTR_TB.cnt(dcnt);

    ror DUT("ror"); // Instantiate Device Under Test
    DUT.din(dcnt);
    DUT.dout1(dout1);
    DUT.dout2(dout2);
    DUT.reset(rst);

    // Run simulation:
    rst=SC_LOGIC_1;
    sc_start(15, SC_NS);
    rst=SC_LOGIC_0;
    sc_start();
}
```

# Moduły – funkcja `sc_main`

SystemC posiada własną wersję funkcji `main()` i z niej uruchamia funkcję `sc_main` zdefiniowaną przez użytkownika (kopiując parametry wywołania):

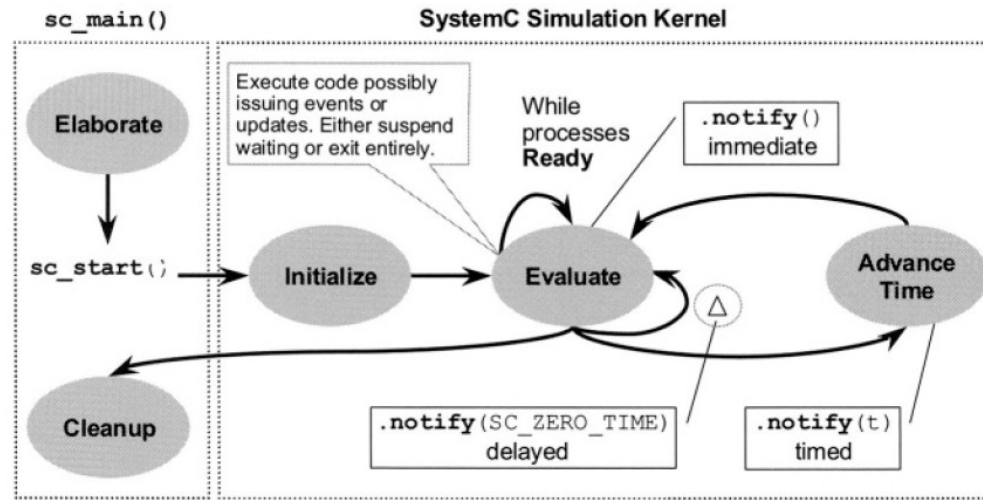
```
int sc_main(int argc, char* argv[]) {  
    // ELABORATION  
    sc_start();      // SIMULATION  
    // POST-PROCESSING  
    return EXIT_CODE; // Zero indicates success  
}
```

W fazie elaboracji tworzone są instancje obiektów takich jak jednostki projektowe, zegary, kanały. Tworzona jest hierarchia połączeń pomiędzy modułami.

Oprócz tego rejestrowane są procesy.

Moduły są zazwyczaj włączane do projektu w plikach nagłówkowych \*.h.

# Jądro symulatora



Etapy działania symulatora:

- **Elaborate**

`sc_start(1)`:

- **Initialize**
- **Evaluate**
  - Advance time
    - Delta cycles
- **Cleanup**

# Procesy

Procesy są głównymi jednostkami wykonawczymi w SystemC. Są one uruchamiane w celu emulacji zachowania urządzenia lub systemu. Są trzy rodzaje procesów:

- **Methods**
- **Threads**
- **Clocked Threads**

# Procesy c.d.

- Procesy mogą zachowywać się jak funkcje (wykonanie i powrót).
- Procesy mogą także być uruchamiane na początku symulacji i pracować do końca symulacji bądź być przerywane przez oczekiwanie na warunek (np. zbocze zegara).
- Procesy nie są hierarchiczne – tj. nie można uruchamiać jednego procesu bezpośrednio z innego (można to robić pośrednio - poprzez generowanie sygnałów wyzwalających).
- Procesy posiadają listę czułości (listę sygnałów których zmiany powodują uruchomienie procesu).
- Zmiana wartości sygnału nazywa się zdarzeniem na sygnale (event on signal).



# SC\_METHOD

- Proces SC\_METHOD jest uruchamiany tylko wtedy, gdy nastąpi jakakolwiek zmiana sygnałów na liście czułości procesu (event).
- Następnie proces wykonuje swoje zadania (zmienia stany sygnałów) i w skończonym (i najlepiej krótkim) czasie kończy działanie przekazując sterowanie do jądra SystemC (proces czeka na następny event).
- Instrukcja wait() wewnątrz procesu SC\_METHOD jest niedozwolona.

**Przykład (przerzutnik D):**

```
SC_MODULE(dff) {
    sc_in<bool> clock;
    sc_in<bool> din;
    sc_out<bool> dout;

    void proc1() {
        dout.write(din.read());
    }

    SC_CTOR(dff) {
        SC_METHOD(proc1);
        sensitive << clock.pos();
    }
};
```

# SC\_METHOD – c.d.

Przykład (przerzutnik D z asynchronicznym resetem):

```
SC_MODULE(dffr) {
    sc_in<bool> clock;
    sc_in<bool> din;
    sc_in<bool> reset;
    sc_out<bool> dout;

    void proc1() {
        if (reset.read())
            dout.write(0);
        else
            if (clock.event() && clock.read())
                dout.write(din.read());
    }

    SC_CTOR(dffr) {
        SC_METHOD(proc1);
        sensitive << clock.pos() << reset;
    }
};
```

# SC\_METHOD – czułość dynamiczna

W procesach można modyfikować warunki wyzwalania w sposób dynamiczny za pomocą instrukcji `next_trigger`. Przykłady:

```
next_trigger(time);
next_trigger(event);
next_trigger(event1 | event2);    // dowolne ze zdarzeń
next_trigger(event1 & event2);    // wszystkie zdarzenia
next_trigger(timeout, event);    // wersje z timeoutem
next_trigger(timeout, event1 | event2);
next_trigger(timeout, event1 & event2);
next_trigger();    // powrót do statycznych ustawień czułości
```

Wszystkie procesy są wyzwalane w chwili  $t=0$ .  
Aby tego uniknąć należy użyć funkcji `dont_initialize()`:

```
...
SC_METHOD(test);
sensitive(fill_request);
dont_initialize();
...
```

# SC\_THREAD

- Proces SC\_THREAD jest uruchamiany jednorazowo przez symulator.
- Następnie proces wykonuje swoje zadania, a sterowanie do symulatora może oddać albo poprzez return (ostateczne zakończenie procesu) albo poprzez wykonanie instrukcji wait (czasem pośrednio – np. poprzez blokujący read lub write).
- Wait powoduje przejście procesu do stanu zawieszenia. W zależności od wersji instrukcji wait wznowienie procesu może nastąpić pod różnymi warunkami:  
`wait(time);`  
`wait(event);`  
`wait(event1 | event2);`  
`wait(event1 & event2);`  
`wait(timeout, event);`  
`wait(timeout, event1 | event2);`  
`wait(timeout, event1 & event2);`  
`wait();`

Sposób wykrywania zakończenia instrukcji wait poprzez timeout:

```
sc_event ok_event, error_event;  
wait(t_MAX_DELAY, ok_event | error_event);  
if (timed_out()) break; // wystąpił time out
```

# SC\_THREAD - zdarzenia

Sposób generowania zdarzeń (event):

```
event_name.notify(); // immediate notification  
event_name.notify(SC_ZERO_TIME) ; // next delta-cycle notification  
event_name.notify(time); // timed notification
```

Przykład modułu z procesem SC\_THREAD:

```
SC_MODULE(simple_example) {  
    SC_CTOR(simple_example) {  
        SC_THREAD(my_process); }  
    void my_process(void); };
```

Przykład uruchomienia symulatora:

```
int sc_main(int argc, char* argv[]) { // args unused  
    simple_example my_instance("my_instance");  
    sc_start(10, SC_SEC);  
    // sc_start(); // forever  
    return 0;    // simulation return code  
}
```

# Typ `sc_string` (literały)

`sc_string name("0base[sign]number[e[+|-]exp]");`

| prefix | znaczenie               | Sc_numrep |
|--------|-------------------------|-----------|
| 0d     | Decimal                 | SC_DEC    |
| 0b     | Binary                  | SC_BIN    |
| 0bus   | Binary unsigned         | SC_BIN_US |
| 0bsm   | Binary signed magnitude | SC_BIN_SM |
| 0o     | Octal                   | SC_OCT    |
| 0ous   | Octal unsigned          | SC_OCT_US |
| 0osm   | Octal signed magnitude  | SC_OCT_SM |
| 0x     | Hex                     | SC_HEX    |
| 0xus   | Hex unsigned            | SC_HEX_US |
| 0xsm   | Hex signed magnitude    | SC_HEX_SM |
| 0csd   | Canonical signed digit  | SC_CSD    |

## Przykłady:

```
sc_string a ("0d13"); // decimal 13
```

```
a = sc_string ("0b101110") ; // binary of decimal 44
```

# Typy w SystemC

- Zalecane jest wykorzystywanie typów języka C++
- Typ odpowiadający typowi integer z języka VHDL (potrzebny np. do syntezy):  
`sc_int<LENGTH> nazwa...;`  
`sc_uint<LENGTH> nazwa...;`  
`sc_int` - integer ze znakiem o długości `LENGTH` bitów,  
`sc_uint` - integer bez znaku o długości `LENGTH` bitów.
- Typ integer o większej szerokości niż 64 bity:  
`sc_bigint<BITWIDTH> nazwa...;`  
`sc_biguint<BITWIDTH> nazwa...;`
- Typ bitowy (i wektorowy typ bitowy):  
`sc_bit nazwa...;`  
`sc_bv<BITWIDTH> nazwa...;`
- Typ logiczny (i wektorowy typ logiczny) – posiada 4 możliwe wartości (podobny do `std_logic`):  
`sc_logic nazwa[,nazwa]...;`  
`sc_lv<BITWIDTH> nazwa[,nazwa]...;`  
możliwe wartości: `SC_LOGIC_1`, `SC_LOGIC_0`, `SC_LOGIC_X`, `SC_LOGIC_Z`  
zapis w łańcuchach: `sc_lv<8> dat_x ("ZZ01XZ1Z");`

# Typy w SystemC – typ stałoprzecinkowy

Należy zdefiniować przed `#include <systemc.h>`:

```
#define SC_INCLUDE_FX
```

```
sc_fixed<WL,IWL[,QUANT[,OVFLW[,NBITS]> NAME...;
sc_ufixed<WL,IWL[,QUANT[,OVFLW[,NBITS]> NAME...;
sc_fixed_fast<WL,IWL[,QUANT[,OVFLW[,NBITS]> NAME...;
sc_ufixed_fast<WL,IWL[,QUANT[,OVFLW[,NBITS]> NAME...;
sc_fix_fast NAME(WL,IWL[,QUANT[,OVFLW[,NBITS]))...;
sc_ufix_fast NAME(WL,IWL[,QUANT[,OVFLW[,NBITS]))...;
sc_fixed_fast NAME(WL,IWL[,QUANT[,OVFLW[,NBITS]))...;
sc_ufixed_fast NAME(WL,IWL[,QUANT[,OVFLW[,NBITS]))...;
WL - Word length (długość słowa)
IWL - Integer Word length (długość części całkowitej)
QUANT - Tryb kwantyzacji (default: SC_TRN)
OVFLW - Tryb obsługi przepełnienia (default: SC_WRAP)
NBITS - liczba bitów nasycenia
```

| Nazwa       | Tryb obsługi przepełnienia |
|-------------|----------------------------|
| SC_SAT      | Nasycenie do +/- max.      |
| SC_SAT_ZERO | Zerowanie w nasyceniu      |
| SC_SAT_SYM  | Symetryczne nasycenie      |
| SC_WRAP     | Wraparound (default)       |
| SC_WRAP_SYM | Symetryczny wraparound     |

| Nazwa          | Tryb kwantyzacji                                                             |
|----------------|------------------------------------------------------------------------------|
| SC_RND         | Zaokrąglenie (do $+\infty$ przy równym dystansie)                            |
| SC_RND_ZERO    | Zaokrąglenie (do 0 przy równym dystansie)                                    |
| SC_RND_MIN_INF | Zaokrąglenie (do $-\infty$ przy równym dystansie)                            |
| SC_RND_INF     | Zaokrąglenie (do $\pm\infty$ zależnie od znaku liczby przy równym dystansie) |
| SC_RND_CONV    | Zbieżne zaokrąglenie                                                         |
| SC_TRN         | Obcięcie (default)                                                           |
| SC_TRN_ZERO    | Obcięcie do zera                                                             |



# Typy w SystemC – opis czasu

Możliwe jednostki czasu:

**SC\_SEC, SC\_MS, SC\_US** (mikrosekundy),  
**SC\_NS, SC\_PS, SC\_FS**

Typ `sc_time`:

`sc_time name...; // bez inicjalizacji`  
`sc_time name (wartość, jednostka)...; // z inicjalizacją`

Przykłady użycia:

```
sc_time t_PERIOD(20, SC_NS) ;  
sc_time t_TIMEOUT(10, SC_MS) ;  
sc_time t_X, t_CUR, t_LAST;  
t_X = (t_CUR-t_LAST) ;  
if (t_X > t_MAX) { error ("Timing error"); }
```

Odczyt czasu symulacji:

```
cout << sc_time_stamp() << endl;
```